

NiceGUI的中文入门教程

NiceGUI的中文入门教程

0 前言

1 环境准备

1.1 运行环境

1.1.1 基础环境初始化

1.1.2 NiceGUI运行环境

1.2 开发工具

1.3 自托管文档【可选】

2 入门基础

2.1 认识NiceGUI

2.1.1 NiceGUI的 Hello World!

2.2 NiceGUI的基本结构

2.2.1 图形界面的基础概念

2.2.1.1 控件

2.2.1.2 布局

2.2.1.3 交互

2.2.2 NiceGUI与基础概念的对应

2.3 NiceGUI中不得不学的功能

2.3.1 文本控件

2.3.1.1 `ui.label`

2.3.1.2 `ui.link`

2.3.1.3 `ui.element`

2.3.1.4 `ui.markdown` 和 `ui.html` (2025.03.23更新)

2.3.2 常用控件

2.3.2.1 `ui.button`

2.3.2.2 `ui.input`

2.3.2.3 `ui.slider` 和 `ui.knob`

2.3.3 多媒体控件

2.3.3.1 `ui.image` 和 `ui.interactive_image`

2.3.3.2 `ui.icon` 和 `ui.avatar`

2.3.3.3 `ui.audio` 和 `ui.video`

2.3.4 图形布局

2.3.4.1 `ui.row`、`ui.column` 和 `ui.grid`

2.3.4.2 `ui.space`

2.3.4.3 `ui.separator`

2.3.4.4 `ui.menu` 和 `ui.context_menu`

2.3.4.5 `ui.tooltip`

2.3.4.6 `ui.notify`

2.3.5 属性绑定和数据控件

2.3.5.1 `binding`

2.3.5.2 `ui.linear_progress` 和 `ui.circular_progress`

2.3.5.3 `ui.spinner`

2.3.5.4 `ui.code`

2.3.6 外观美化

2.3.6.1 需要记住具体名字的 `props`、`classes`、`style` 方法

2.3.6.2 好记一点的 `tailwindcss` 属性和 `Tailwind` 对象

2.3.6.3 暗黑模式 `ui.dark_mode`

2.3.6.4 设置主题颜色 `ui.colors`

2.3.7 事件和执行

2.3.7.1 通用事件 (2025.05.23更新)

2.3.7.2 常用事件 `app.on_*`

2.3.7.3 定时器 `ui.timer` 和 `app.timer`

2.3.7.4 UI更新和可刷新方法 `ui.refreshable` (2025.05.04更新)

2.3.7.5 运行JavaScript代码 `ui.run_javascript`

2.3.8 网站页面

2.3.8.1 `ui.page`

2.3.8.2 页面布局

2.3.8.3 `ui.navigate`

2.3.8.4 `ui.download`

2.3.8.5 `ui.fullscreen` (2025.02.05更新)

2.3.9 运行配置

2.3.9.1 `ui.run`

2.3.9.2 `native mode`

3 高阶技巧

3.1 `with` 的技巧

3.2 `slot`的技巧

3.3 TailWindCSS的技巧

3.4 自定义控件

3.4.1 通过继承NiceGUI现有控件来创建新控件

3.4.2 使用Quasar的标签定义新控件

3.4.3 使用VUE自定义新控件 (2025.01.08更新)

3.5 for 循环的技巧

3.5.1 用 for 创建多个有规律的控件

3.5.2 与lambda表达式组合使用时的的问题

3.5.3 更好的 for 循环

3.6 绑定的技巧

3.6.1 绑定到字典

3.6.2 绑定到全局变量

3.6.3 性能优化

3.7 app.storage 的技巧

3.8 修改指定元素的技巧

3.8.1 ui.query

3.8.2 ui.teleport

3.8.3 ElementFilter

3.8.3.1 初始化方法

3.8.3.2 within 方法和 not_within 方法

3.8.3.3 exclude 方法

3.8.3.4 传送控件到匹配结果

3.8.3.5 总结

3.9 其他布局

3.9.1 ui.list

3.9.2 ui.splitter

3.9.3 ui.tabs

3.9.4 ui.scroll_area

3.9.5 ui.skeleton

3.9.6 ui.carousel

3.9.7 ui.expansion

3.9.8 ui.pagination

3.9.9 ui stepper

3.9.10 ui.timeline

3.9.11 ui.notification

3.9.12 ui.dialog

3.9.13 ui.menu 补充

3.9.14 `ui.tooltip` 补充 (2025.01.21更新)

3.9.15 `ui.slide_item` (2025.03.04更新)

3.10 其他常用控件

3.10.1 `ui.dropdown_button`

3.10.2 `ui.button_group`

3.10.3 `ui.badge`

3.10.4 `ui.chip`

3.10.5 `ui.radio`

3.10.6 `ui.toggle`

3.10.7 `ui.select`

3.10.8 `ui.checkbox`

3.10.9 `ui.switch`

3.10.10 `ui.range`

3.10.11 `ui.joystick`

3.10.12 `ui.textarea`

3.10.13 `ui.number`

3.10.14 `ui.color_picker`

3.10.15 `ui.color_input`

3.10.16 `ui.date`

3.10.17 `ui.time`

3.10.18 `ui.upload`

3.10.19 `ui.rating` (2025.03.04更新)

3.11 多媒体控件的使用技巧

3.11.1 `ui.interactive_image` 的交互技巧

3.12 `ui.add_*` 和 `app.add_*` 的技巧

3.12.1 `app.add_static_file` 和 `app.add_static_files`

3.12.2 `app.add_media_file` 和 `app.add_media_files`

3.12.3 `ui.add_head_html` 和 `ui.add_body_html`

3.12.4 `ui.add_css` 、 `ui.add_scss` 、 `ui.add_sass`

3.13 `ui.keyboard` 的事件处理技巧

3.14 其他数据展示控件

3.14.1 `ui.table`

3.14.2 `ui.tree`

3.14.3 `ui.log`

3.14.4 `ui.editor`

3.14.5 `ui.codemirror`

3.14.6 `ui.json_editor`

- 3.14.7 `ui.scene`
- 3.14.8 `ui.aggrid`
- 3.14.9 `ui.highchart`
- 3.14.10 `ui.echart`
- 3.14.11 `ui.pyplot`
- 3.14.12 `ui.matplotlib`
- 3.14.13 `ui.line_plot`
- 3.14.14 `ui.plotly`
- 3.14.15 `ui.leaflet`
- 3.14.16 `ui.mermaid` (2025.03.23更新)
- 3.15 多任务与异步的技巧 (2025.01.08更新)
 - 3.15.1 异步 (2025.01.08更新)
 - 3.15.1.1 异步支持
 - 3.15.1.2 `ui.clipboard` 和剪贴板
 - 3.15.2 后台任务 (2025.01.08更新)
- 3.16 版本亮点 (随版本更新)
 - 2.9.0版本新增: `app.timer` ——app级别的定时器
 - 2.10.0版本新增: `ui.clipboard.read_image` ——从剪贴板读取图片
 - 2.11.0版本新增: `ui.fullscreen` ——让浏览器全屏更简单
 - 2.12.0版本新增: `ui.rating` 和 `ui.slide_item` ——打分和滑动“解锁”

前言

对于Python语言的使用者来说，NiceGUI是一款优秀的WebUI、GUI框架，只需学习一定量的前端知识，就能使用NiceGUI快速搭建出美观的UI界面。但是，由于官方作者不提供系统性的入门、中文教程，很多中文初学者望而却步。于是，本教程应运而生。

本教程旨在用中文提供官方文档没有的系统性入门教程，并将部分社区讨论问题汉化、简化，方便中文学习者尽快上手并解决常见难题。虽然教程的名字叫入门教程，但本教程并没有停留在翻译官网文档的阶段，能够解决常见问题的高阶技巧也有。对于官方提供的各个控件的详细API，本教程并不会照本宣科，而是在提供思路之后，由读者自行查阅。正所谓“授人以鱼不如授人以渔”，掌握方法比掌握结果更有效。

1 环境准备

本章主要介绍运行和开发NiceGUI程序的环境准备，包括虚拟环境的建立、开发工具的选择、如何自托管文档。

1.1 运行环境

为了保证最佳开发体验，所有的环境准备优先使用Windows系统，使用Linux、Mac的话，请自己根据系统变通。

首先，需要准备Python解释器和pip。其中，Python解释器是运行后续代码、工具的基础，只需到官网安装最新稳定版（当前为3.12.*）即可；pip是Python官方的包管理工具，安装解释器时务必勾选此选项，如果没有勾选或者想要后续单独安装pip，可以到pypi官方查看安装方法，这里不在赘述。

环境管理工具有pdm和poetry，使用以下命令全部安装：

```
1 pip install pdm poetry
```

环境管理工具是快捷管理Python运行环境的工具，可以创建出独立的Python运行环境，各个运行环境内安装的软件包不会干扰其他环境，也不会影响到默认的Python环境。

pdm是一款国人创建的环境管理工具，语法简单，操作方便，因此教程采用此工具。

poetry也是一款环境管理工具，是NiceGUI官方仓库采用的，可以基于官方源代码自己编译安装包，也用于后续自托管文档，因此需要安装。

但是，poetry的检查依赖速度比pdm慢太多，故这里采用pdm作为学习工具，如果读者有能力，可以只使用poetry。

1.1.1 基础环境初始化

先在纯英文、无空格、无特殊字符的路径下创建纯英文、无空格、无特殊字符的空白文件夹，进入该文件夹后，右键，在此处打开终端或者命令行，运行以下命令：

```
1 pdm init
```

会看到以下输出：

```
1 PS E:\PSF\git\chinese_guide_of_nicegui_for_beginner> pdm init
2 Creating a pyproject.toml for PDM...
3 Please enter the Python interpreter to use
4 0. cpython@3.12 (D:\Programs\Python\Python312\python.EXE)
5 1. cpython@3.11 (D:\mingw64\bin\python3.11.exe)
```

```
6 2. cpython@3.11 (D:\mingw64\bin\python3.exe)
7 3. cpython@3.11 (D:\mingw64\bin\python.exe)
8 4. cpython@3.11
(C:\Users\peter\AppData\Local\pdm\pdm\python\cpython@3.11.5\python.exe)
9 Please select (0): 0
10 Virtualenv is created successfully at
E:\PSF\git\chinese_guide_of_nicegui_for_beginner\.venv
11 Project name (chinese_guide_of_nicegui_for_beginner):
12 Project version (0.1.0):
13 Do you want to build this project for distribution(such as wheel)?
14 If yes, it will be installed by default when running `pdm install`. [y/n] (n):
15 License(SPDX name) (MIT):
16 Author name (): Peter
17 Author email (): peter@linux
18 Python requires('*' to allow any) (==3.12.*):
19 Project is initialized successfully
20 PS E:\PSF\git\chinese_guide_of_nicegui_for_beginner>
```

过程为交互式，需要自己输入之后回车才能继续，不输入直接回车则采用默认。

Please select (0): 为选择Python版本，pdm会自动识别当前电脑安装的所有Python解释器，部分工具（如gcc）也会自带Python解释器，需要正确选择自己安装的、可以直接运行pip命令的Python解释器，这一步根据实际情况选择，一般默认第一个，可以直接回车。

Project name (chinese_guide_of_nicegui_for_beginner): 为设置项目名称，通常为当前文件夹名字，可以自己输入来修改。这个不会修改当前文件夹名字，只会影响项目描述文件中的项目名称和后续创建的源代码文件夹名称。这里可以直接回车。

Project version (0.1.0): 为设置当前项目的版本号，该版本号符合语义化规则，不懂或者不想了解的可以直接回车。

If yes, it will be installed by default when running pdm install. [y/n] (n): 这一步是问你要不要将项目构建成分发包（可以用pip安装的），如果选择 y，使用 pdm install 就会默认安装项目。这里选择不创建分发包，所以直接回车。

License(SPDX name) (MIT): 、 Author name (): 、 Author email (): 分别是许可协议、作者名字、作者邮箱地址，了解、知道的可以修改，不清楚或者不想写的可以直接回车。

`Python requires('*' to allow any) (==3.12.*)`: 为Python的版本要求, 如果后续要用低版本或者高版本Python运行, 这里需要修改, 否则此项目会限制Python的版本。这个后续可以自己了解, 这里直接回车即可。

至此, 基础环境已经准备完毕, 可以得到以下目录结构:

```
1  chinese_guide_of_nicegui_for_beginner
2  ├── .venv
3  │   ├── Lib
4  │   │   └── site-packages
5  │   └── Scripts
6  ├── src
7  │   └── chinese_guide_of_nicegui_for_beginner
8  └── tests
```

`./src/chinese_guide_of_nicegui_for_beginner/` 下存放项目的源代码, 后续的代码操作(创建修改)均在此目录。此外, 项目根目录下有 `pyproject.toml` (项目描述文件) 和 `README.md` (自述文件), 本教程不涉及手动修改。

1.1.2 NiceGUI运行环境

基础环境初始化完毕之后, 项目还不能直接运行基于NiceGUI框架的代码, 因为基础环境还没有安装NiceGUI。因此, 需要在项目根目录下使用以下命令安装:

```
1  pdm add nicegui
```

如果运行环境是全局环境或者想用pip安装, 可以执行下面的命令安装:

```
1  pip install nicegui
```

安装过程取决于网速, 耐心等待。

如果后续项目中需要使用其他库, 可以使用 `pdm add {pip安装命令中库的名字}` 来添加到项目环境中。

对于调试使用NiceGUI的程序, 通常在native mode下比较方便, 因此, 建议安装 `pywebview` 来增加native mode的支持, 命令是:

```
1 pdm add pywebview
```

如果运行环境是全局环境或者想用pip安装，可以执行下面的命令安装：

```
1 pip install nicegui[native]
2 #或者下面这条
3 pip install pywebview
```

对于没有安装过Microsoft Edge WebView2或者版本较旧的Windows系统，建议访问 [Microsoft Edge WebView2下载](#) 安装最新版本。

1.2 开发工具

[VSCode](#)或者[PyCharm](#)，其中VSCode比PyCharm轻量，但需要手动安装Python插件，而PyCharm自带插件，操作简单。这里推荐使用VSCode，比较流畅，如果是使用PyCharm，后续操作根据VSCode对应即可。

对于VSCode，建议安装以下插件：

```
1 ms-python.python
2 ms-python.autopep8
3 ms-python.black-formatter
```

1.3 自托管文档【可选】

因为官网文档是可交互的，只有连接到官网才能操作。但是，部分地区的访问官网存在网络不佳的情况，如果需要自托管官网文档，可以执行以下步骤。

首先要安装git，具体可以看[Git - 安装 Git](#)。

安装完成后，额外找一个空白文件夹，打开终端，执行以下命令：

```
1 git clone https://github.com/zauberzeug/nicegui.git
```

没有git，可以打开以下链接：

```
1 https://github.com/zauberzeug/nicegui/archive/refs/heads/main.zip
```

下载压缩包之后解压，结果一样。对于GitHub难以访问的问题，可以自行搜索GitHub加速的方法。

使用以下命令进入NiceGUI的源代码文件夹，并执行安装操作：

```
1 cd nicegui
2 poetry install
```

等安装完成之后（poetry的install时间会比较长），执行以下命令，会开启一个服务器托管官网文档，并自动调用浏览器打开自托管的官网：

```
1 poetry run python .\main.py
```

注意：如果不使用poetry创建的虚拟环境运行，而是使用全局Python运行的话，首先要安装NiceGUI库，然后根据提示安装缺失的库（自托管文件需要的部分依赖不会随NiceGUI自动安装），并且在源码根目录执行 `python main.py`。

关掉终端就可以关闭服务器，下次运行这条命令就可以直接开启自托管的官网文档，无需再次安装。

如果官方有源代码更新，后续将源代码解压覆盖之后，执行一次安装操作即可更新。

2 入门基础

本章主要介绍NiceGUI的基础知识，系统性了解NiceGUI的基本结构，在自学NiceGUI、查阅官网文档时有方向。

2.1 认识NiceGUI

NiceGUI是一个开源的Python库，可以搭建运行在浏览器的图形界面，也就是WebUI，也可以理解为和网站一样。NiceGUI的学习过程并不难，但不意味着你可以零基础入门。对于开发NiceGUI的开发者而言，Python的基础是必须的；如果掌握Python不常用语法和用法更好，后续在使用NiceGUI的过程中，可以很方便理解一些为了达成效果而使用的骚操作。尽管大部分Python的学习者是零基础入门，除了转全栈、爬虫的开发者，后续学习过程中很少接触前端（HTML、CSS、JavaScript），但还是在使用NiceGUI之前，要有学习前端的心理准备。NiceGUI为了方便Python工程师快速搭建图形界面，专注于

Python代码本身，做了不少前端细节的隐藏。不过，默认样式不一定符合预期效果，为了达成效果，还是需要了解一些前端知识，才能让界面完全符合心意。

NiceGUI的底层使用了FastAPI作为运行服务器，Quasar作为前端框架，支持TailWindCSS的CSS语法，因为Quasar的内部使用VUE搭建，NiceGUI也不可避免地用到了VUE的语法。因此，对于想要用好NiceGUI的开发者，在后续使用NiceGUI的过程中，需要对上述提到的项目有一定了解。想要对后端部分定制、修改的，需要了解FastAPI以及其基于的其他组件；想要让界面美化、随心的，需要掌握Quasar、TailWindCSS基础；对于有能力和需求完全定制界面的，则需要掌握VUE基础。

相关链接：

NiceGUI官方文档：<https://nicegui.io/documentation>

FastAPI官方文档：<https://fastapi.tiangolo.com/zh/>

Quasar官方文档：<https://quasar.dev/docs>

TailWindCSS官方文档：<https://tailwindcss.com/>

VUE官方文档：<https://cn.vuejs.org/guide/introduction>

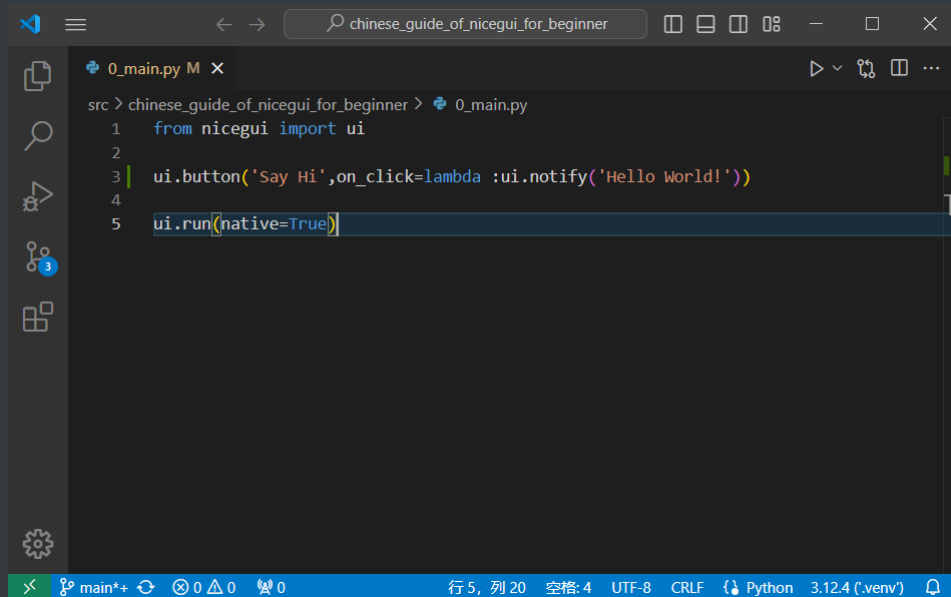
2.1.1 NiceGUI的 Hello World!

如果你能看到这里，证明你有学好NiceGUI的基础和能力，并不畏惧上一节提到的那么多基础知识。那么，就用一段简单的Hello World代码开启NiceGUI的入门之旅。

使用VSCode在项目文件夹下的 `src\chinese_guide_of_nicegui_for_beginner` 内，新建以 `.py` 为后缀的Python代码文件，复制以下代码到代码文件中并保存：

```
1 from nicegui import ui
2
3 ui.button('Say Hi',on_click=lambda :ui.notify('Hello World!'))
4
5 ui.run(native=True)
```

注意vscode的右下角，务必确保使用的是 `venv` 下的Python解释器，如果不是，点击右下角 `3.12.4{'.venv'}` 对应位置，选择输入解释器路径-查找，选择 `.venv\Scripts\` 下的 `python.exe`。

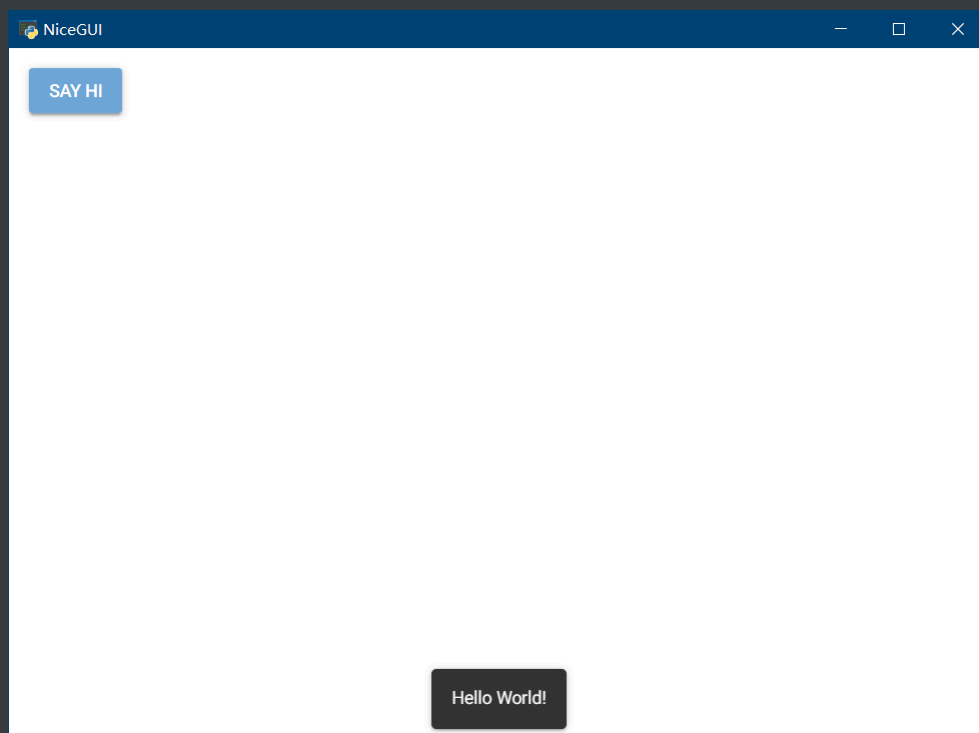


The image shows a Visual Studio Code (VS Code) editor window. The top bar displays the search icon, a search bar containing 'chinese_guide_of_nicegui_for_beginner', and window management icons. The editor has a single file open, '0_main.py', with the following Python code:

```
src > chinese_guide_of_nicegui_for_beginner > 0_main.py
1  from nicegui import ui
2
3  ui.button('Say Hi',on_click=lambda :ui.notify('Hello World!'))
4
5  ui.run(native=True)
```

The status bar at the bottom indicates the file is 'main*.py', uses 'UTF-8' encoding, 'CRLF' line endings, and is running 'Python 3.12.4 (.venv)'.

点击VSCode右上角的运行按钮（三角形）， 就可以看到一个窗口弹出， 点击 SAY HI 按钮， 就能看到窗口底部弹出的 Hello World! 。



从示例可以看到， 基于NiceGUI的Python程序寥寥三行代码， 除去导入语句和必不可少的 `ui.run` ， 主体只有一行代码， 就能实现一个完整的交互过程， 足见NiceGUI的强大、简洁。后续教程中很多例子都可以做到几行代码实现不错的效果， 这也是NiceGUI大受欢迎的原因。

2.2 NiceGUI的基本结构

2.2.1 图形界面的基础概念

在正式学习NiceGUI之前，需要先对图形界面有个基础的理解。

一般来说，搭建图形界面需要理解三个概念：控件、布局、交互。

2.2.1.1 控件

控件是搭建图形界面的基本元素，就像是盖房子用的砖、门、窗等最小搭建单位。控件通常是图形界面框架提供、直接可用的。如果使用过程中发现基本元素不够，可以结合布局功能，用基本元素组合出新的控件。

2.2.1.2 布局

布局是排布控件的方式，就像是房屋的基本框架。用砖可以铺地，也可以垒墙，对于砖而言，墙或是地，就是布局。控件是横向排列还是竖向排列，是像网格一样一一对应，还是大控件套着小控件，都是由布局控制。大部分图形程序框架提供的布局类似，除了基本的几种布局之外，部分图形程序框架还提供额外的组合布局。

2.2.1.3 交互

交互是图形界面的重中之重，也是一个程序最难的部分。论难度的话，前面的控件和布局的学习只是对照文档，按图索骥，交互则需要身经百战，不断积累经验。

事件机制是目前大部分图形界面采用的交互反馈机制，也就是基于特定的事件触发，执行对应的函数。微软的winform中采用的消息机制，Qt的信号与槽，现代网页开发中的event事件监听，都可以理解为事件机制，只是对于winform和Qt而言，他们框架内的事件分别叫做消息和信号而已。

除了事件机制，美化也是交互的一部分。大部分现代图形界面框架。如Qt、WPF以及一系列基于网页开发的图形界面框架，支持CSS或者类似语法的美化功能，让图形界面变得更加美观，也让控件的动画效果更加丰富，这个极大提升了用户的使用体验。

此外，基于图形界面框架的特性，后端的处理逻辑以及数据的传递也是交互的一部分。在函数内，对于控件的控制，如何做到符合要求，毕竟有的框架、编程语言不支持没有定义或者声明函数就调用，而有的语言不支持声明函数。如果需要对控件显示的文本与另一个控件的文本一致，如何处理数据同步过程也需要技巧。

2.2.2 NiceGUI与基础概念的对应

对图形界面有基础的理解之后，下面就可以根据NiceGUI与基础概念的对应，进一步理解NiceGUI的设计理念。

在 Hello World! 示例中，使用了导入语句 `from nicegui import ui` 导入了 `ui`，顾名思义，`ui` 就是用户界面，这也是NiceGUI调用控件的模块，也可以调用布局。具体的控件和布局用法可以参考下一节NiceGUI中不得不学的功能，下一节将选取一些NiceGUI中常用、不好理解的功能重点讲解。

因为NiceGUI是基于Quasar这个Web框架做的Python调用绑定，因此，NiceGUI的交互部分，很大程度与Web结合。在Web设计中，基于CSS实现的美化效果，基于JavaScript的event做的事件响应，都能在NiceGUI中看到。所以，如果用好NiceGUI，对Web三件套HTML、CSS、JavaScript的学习不可避免。此外，因为NiceGUI与Quasar这个Web框架的深度集成的关系，Quasar中的属性、slot、事件也需要掌握，才能让交互设计更加得心应手。

关于美化，下一节中的外观美化将会详细介绍，也可以查阅对应的官方文档。事件的学习，可以参阅下一节的事件和执行，也可以查阅对应的官方文档。

2.3 NiceGUI中不得不学的功能

以下是官网文档对于NiceGUI提供的功能做了大致的划分，本教程将会对每个部分中不好掌握、需要重点学习的控件、功能进行剖析：

1. 文本控件: https://nicegui.io/documentation/section_text_elements
2. 常用控件: https://nicegui.io/documentation/section_controls
3. 多媒体控件: https://nicegui.io/documentation/section_audiovisual_elements
4. 数据控件: https://nicegui.io/documentation/section_data_elements
5. 属性绑定: https://nicegui.io/documentation/section_binding_properties
6. 图形布局: https://nicegui.io/documentation/section_page_layout
7. 外观美化: https://nicegui.io/documentation/section_styling_appearance
8. 事件和执行: https://nicegui.io/documentation/section_action_events
9. 网站页面: https://nicegui.io/documentation/section_pages_routing
10. 部署与配置: https://nicegui.io/documentation/section_configuration_deployment

这部分对于官方内容的解析并不会完全覆盖，主要讲经常用到的参数、属性、方法，对于某些隐藏参数和不常用的属性方法，会在后面用到的时候详细介绍，这里最多提一嘴。

2.3.1 文本控件

文本类控件主要是一些静态展示可复制文本的控件，是构成网页显示效果的主要控件。

2.3.1.1 ui.label

文本标签，用法很简单，通过传入一个字符串类型的参数 `text`，让网页显示字符串内的文字。注意，虽然参数支持多行文字，但是输出只能一行，需要原样输出多行文字的话，可以使用下面介绍的 `ui.html`，将 `tag` 参数设置为 `'pre'`。

```
1 from nicegui import ui
2
3 ui.label(text='some label')
4
5 ui.run(native=True)
```

2.3.1.2 ui.link

超链接，一种点击之后跳转到指定地址的文本元素。可以传入 `text`、`target`、`new_tab` 三个参数。代码如下：

```
1 from nicegui import ui
2
3 ui.link(text='NiceGUI on GitHub', target= 'https://github.com/zauberzeug/nicegui',
4         new_tab=False)
5
6 ui.run(native=True)
```

`text` 参数，字符串类型，表示超链接显示什么文字。

`target` 参数，字符串类型、`page function` 类型、`ui.element` 类型，表示超链接跳转到什么位置，这里只介绍字符串类型用法，其他类型可以自行探索或者后续需要用到时候补充。字符串类型参数表示超链接跳转的url地址，可以使用协议开头的完整地址，也可以使用省略主机的绝对路径、相对路径。

`new_tab` 参数，布尔类型，默认为 `False`，表示要不要在新建标签页中打开超链接。

2.3.1.3 ui.element

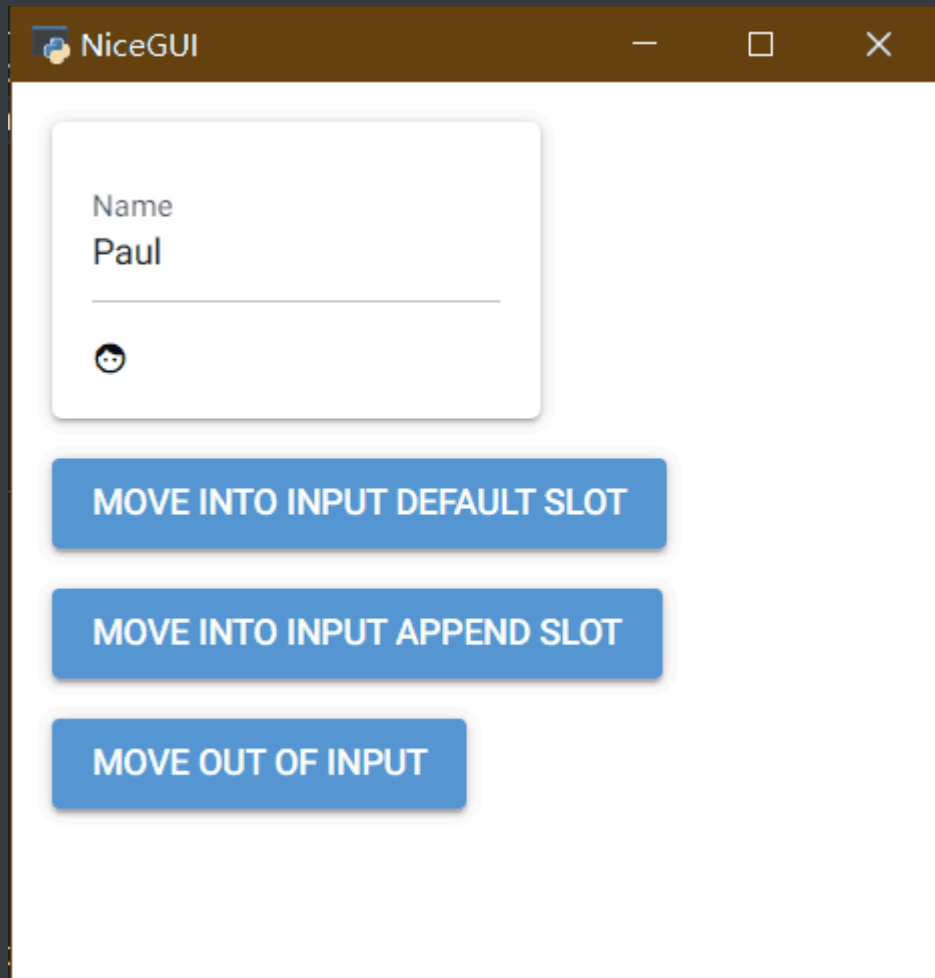
通用元素，也是NiceGUI大部分界面控件的基类。很多控件都是通过继承这个类来调用自定义标签、JavaScript代码实现。通过继承实现自定义控件、修改默认风格属于高级用法，这里只说基本用法。

`tag` 参数，字符串类型，默认为 `'div'`，表示生成的元素用什么HTML标签，实际使用时可以根据需要修改为其他HTML标签或者Quasar标签。代码如下：

```
1 from nicegui import ui
2
3 with ui.element('div').classes('p-2 bg-blue-100'):
4     ui.label('inside a colored div')
5
6 ui.run(native=True)
```

`move` 方法，将控件移动到指定控件之内，默认为 `'default'` slot，也可以给 `target_slot` 参数传入 slot name来指定slot。代码如下：

```
1 from nicegui import ui
2
3 with ui.card() as card:
4     name = ui.input('Name', value='Paul')
5     name.add_slot('append')
6     icon = ui.icon('face')
7
8 ui.button('Move into input default slot', on_click=lambda: icon.move(name))
9 ui.button('Move into input append slot', on_click=lambda: icon.move(name,
10 target_slot='append'))
11
12 ui.button('Move out of input', on_click=lambda: icon.move(card))
13
14 ui.run(native=True)
```



2.3.1.4 ui.markdown 和 ui.html (2025.03.23更新)

与 `ui.label` 类似，`ui.markdown` 和 `ui.html`，都可以用来展示文本，只是后两者支持markdown语法和HTML语法，因为markdown语法支持一部分HTML的标签，可以看到放在 `ui.markdown` 里的HTML标签也能被解析。以下是三种控件解析同一内容的代码：

```
1 from nicegui import ui
2
3 content = '''
4 This is **Markdown**.
5 This is <u>emphasized</u>
6 '''
7 ui.label(content)
8 ui.markdown(content)
9 ui.html(content)
10
11 ui.run(native=True)
```

此外，`ui.html` 还支持传入字符串类型参数 `tag` 给基类 `ui.element`，用于修改生成 `ui.html` 用的标签，比如：

```
1 from nicegui import ui
2
3 ui.html('This is <u>emphasized</u>.', tag='em')
4
5 ui.run(native=True)
```

如果觉得在指定 `ui.html` 使用的标签时，参数没有智能提示不太方便，可以使用 `html` 模块代替。大部分HTML标签都被写成了该模块的成员，直接访问对应的成员，就可以生成对应的HTML标签：

```
1 from nicegui import ui,html
2
3 ui.html('This is <u>emphasized</u>.', tag='em')
4
5 with html.em('This is '):
6     html.u('emphasized')
7     html.em('.')
8
9 ui.link(text='NiceGUI on GitHub', target= 'https://github.com/zauberzeug/nicegui',
10        new_tab=False)
11
12 html.a('NiceGUI on GitHub').props('href="https://github.com/zauberzeug/nicegui"')
13
14 ui.run(native=True)
```

2.3.2 常用控件

常用控件主要是一些支持点击、输入、拖动等交互功能的控件。

2.3.2.1 `ui.button`

按钮作是网页交互设计中最常见的基本元素，在移动互联网没有普及之前，使用鼠标点击为主要交互方式的年代，除了用于跳转网页的超链接，按钮就是网页中用的最多的可交互元素。在NiceGUI中，按钮控件可以传入位置参数 `text`，关键字参数 `on_click`、`color` 和 `icon`。

以下代码就是一个定义了基本交互的按钮，点击会弹出一个通知提示：

```
1 from nicegui import ui
2
3 ui.button('Click me!', color='green', icon='thumb_up', on_click=lambda: ui.notify('You
  clicked me!'))
4
5 ui.run(native=True)
```

`text` 参数，字符串类型，表示显示在按钮上的文字，如果是英文的话，默认全部大写。该参数默认只支持字符串类型，但是整数和小数可以直接使用，其他类型需要先转换为字符串类型才能传入。

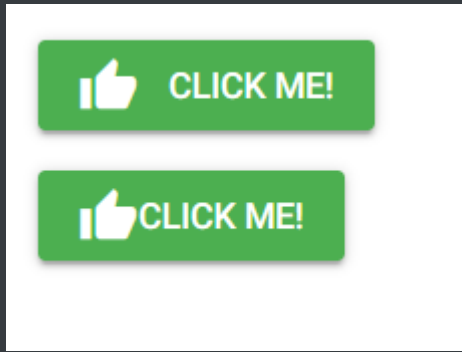
`color` 参数，字符串类型或者 `None`，表示按钮的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让按钮变成默认颜色），默认为 `'primary'`，即和主题颜色一致。

`icon` 参数，字符串类型，表示按钮额外显示的图标，支持传入字符串类型的图标名，具体名字会在 `ui.icon` 一节中介绍，这里不做详细介绍。

`on_click` 参数，可调用类型，表示点击按钮调用的函数，可以使用lambda表达式，也可以使用函数名。

如果觉得对按钮传入参数来自定义按钮内容的方法太死板，也可以使用以下语法，使用上下文管理器关键字 `with` 来进入按钮的default slot（默认插槽），随意组合按钮内的内容：

```
1 from nicegui import ui
2
3 with ui.button(color='green', on_click=lambda: ui.notify('You clicked me!')):
4     ui.icon('thumb_up')
5     ui.label('Click me!')
6
7 ui.run(native=True)
```



对于 `on_click` 参数、后续会涉及到的'on'开头的参数、'on'开头的方法和 `on` 方法（函数名就是 `on`）里的 `callback` 参数或者 `handler` 参数，均为可调用类型参数，既可以在创建控件时定义lambda表达式，也可以提前定义。对于复杂一点逻辑操作，应该定义函数而不是lambda表达式，比如：

```
1 from nicegui import ui
2
3 test = lambda :...
4 #如果要执行的操作比较多、复杂，应该定义函数
5 def test():
6     pass
7 #在控件中传参可以直接使用test
8 ui.button('Click me!',on_click=test)
9
10 ui.run(native=True)
```

对于 `ui.button` 等控件而言，除了支持通过传参创建响应动作，还支持调用对应的'on'开头方法（比如 `on_click`）创建，这个方法极大提高了响应动作的灵活性，上面的例子就可以借助这个方法调整函数定义与按钮创建的先后顺序，代码如下：

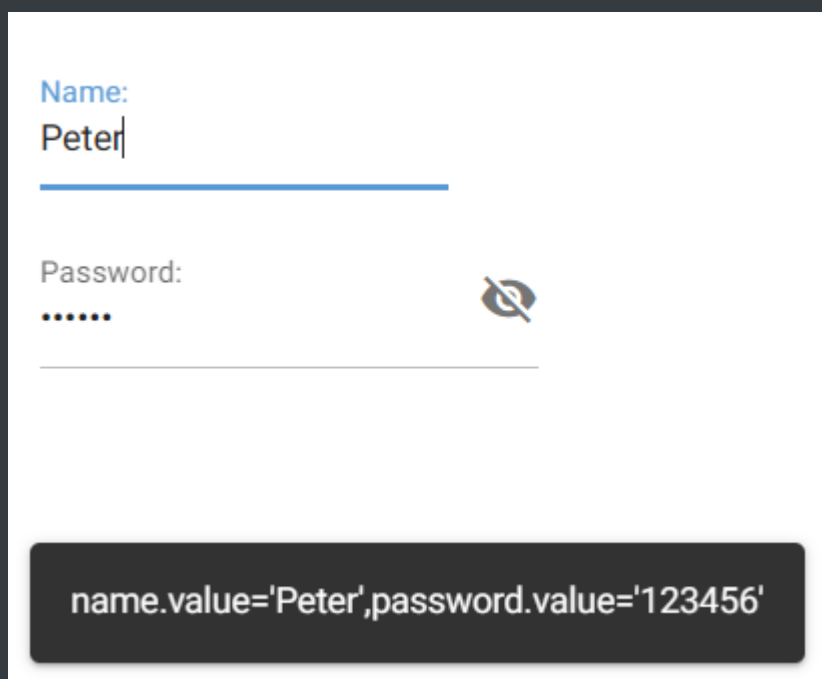
```
1 from nicegui import ui
2
3 button = ui.button('Click me!')
4 def test():
5     pass
6 button.on_click(test)
7
8 ui.run(native=True)
```


2.3.2.2 ui.input

输入框，大概是网页中仅次于按钮和超链接，用得最多的控件。HTML中输入框的变体很多，在NiceGUI中，输入框的参数也很多，基于输入框扩展的控件也多，使用输入框的逻辑设计、注意事项一样很多。不过，在这一节，针对输入框的学习并不会那么深入，只要能掌握常用的参数和基础的方法，那些疑难点会放到进阶和需要的时候细讲，以免一时不好理解而记混。

以下代码定义了一个名字输入框和密码输入框，并在输入名字的时候，自动弹出通知显示名字和密码：

```
1 from nicegui import ui
2
3 name = ui.input(label='Name:', value='Peter', on_change=lambda
    :ui.notify(f'{name.value=},{password.value=}'))
4 password = ui.input(label='Password:', value='123456', password=True,
    password_toggle_button=True)
5
6 ui.run(native=True)
```



`label` 参数，字符串类型，直译的话是标签，表示显示在输入框上方的文本，但不是输入的文本，如果输入的内容是空的，点击输入的之前会显示在输入框内，点击之后会移动到输入框上方。

`value` 参数，字符串类型，表示输入框内的内容，也就是输入框的值。对于后续介绍的以及其他支持交互输入的控件，都会有这么一个参数、属性、相关方法，来设置控件的值，这一点上，NiceGUI的设计倒是很统一。

`on_change` 参数，可调用类型，表示输入框的值变化时执行的函数。

`password` 参数，布尔类型，表示输入框是否设置为密码输入框，如果设置为 `True`，输入的内容将不显示明文，转而显示统一的密码符号。

`password_toggle_button` 参数，布尔类型，表示输入框内是否显示密码按钮，密码按钮可以切换输入框内的密码、明文状态。

以下参数用得比较少或者有一定使用难度，学有余力的读者可以继续学习：

`placeholder` 参数，字符串类型，表示输入框的占位符，即输入框没有输入之前，显示什么内容。与标签不太一样的是，占位符是输入框获得焦点时候显示的，标签是输入框没有获得焦点时候显示的。

`autocomplete` 参数，字符串列表类型，表示在输入框输入内容时候，搜索这个字符串列表，来自动提示、补全要输入的内容，按Tab键可以补全内容。

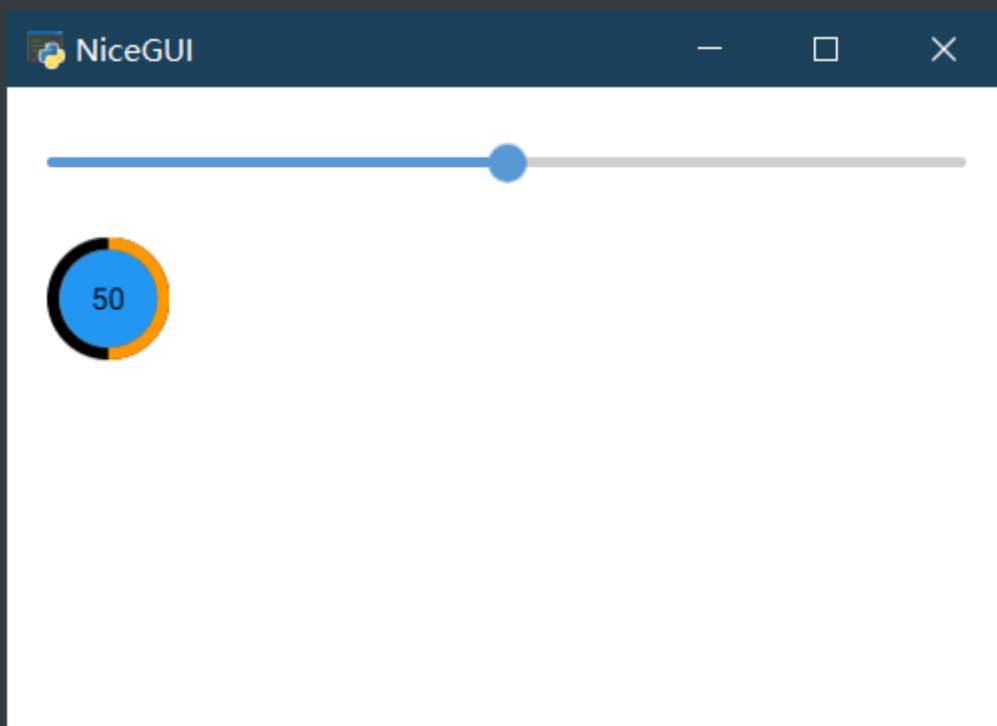
`validation` 参数，可调用类型、字典类型或者 `None`，表示验证输入的内容是否有效。如果传入可调用类型参数，该参数返回错误信息表示内容无效，返回 `None` 表示内容有效。如果传入字典类型参数，则字典的键（key）表示错误信息，字典的值（value）返回 `True` 表示内容有效，返回 `False` 则表示内容无效并输出错误信息。默认值为 `None`，表示不验证输入的内容。

2.3.2.3 `ui.slider` 和 `ui.knob`

`ui.slider` 是滑动条，这个比较简单，参数也不多，按理说不用专门讲一下，自学即可。至于 `ui.knob` ——旋钮，这个的样式参数比较多，也有很大的自定义自由度，值得一讲。不过，这些都不是这里要专门放在一起介绍的原因，具体原因是什么，这里先卖个关子，后面再做解释。

先看一段代码：

```
1 from nicegui import ui
2
3 slider = ui.slider(min=0, max=100, step=1, value=50)
4 ui.knob(min=0, max=100, step=1, value=50,
5         color='orange', center_color='blue', track_color='black',
6         size='3.5em',
7         show_value=True
8         ).bind_value(slider, 'value')
9
10 ui.run(native=True)
```



可以看到，`ui.knob` 的前四个参数和 `ui.slider` 的一样，都是浮点类型，分别代表最小值、最大值、每次调整的最小步长、当前值。

在 `ui.knob` 中，有 `color`、`center_color`、`track_color` 三种参数可以修改颜色，分别代表旋钮边缘的颜色、旋钮中间的颜色、旋钮边缘没有覆盖之前的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）、`None`（即让按钮变成默认颜色）或者 `'primary'`（跟随主题颜色）。

`size` 参数是旋钮的整体大小，字符串类型，采用CSS语法的大小表示方式。

`show_value` 参数，布尔类型，是否在旋钮中间显示当前值。

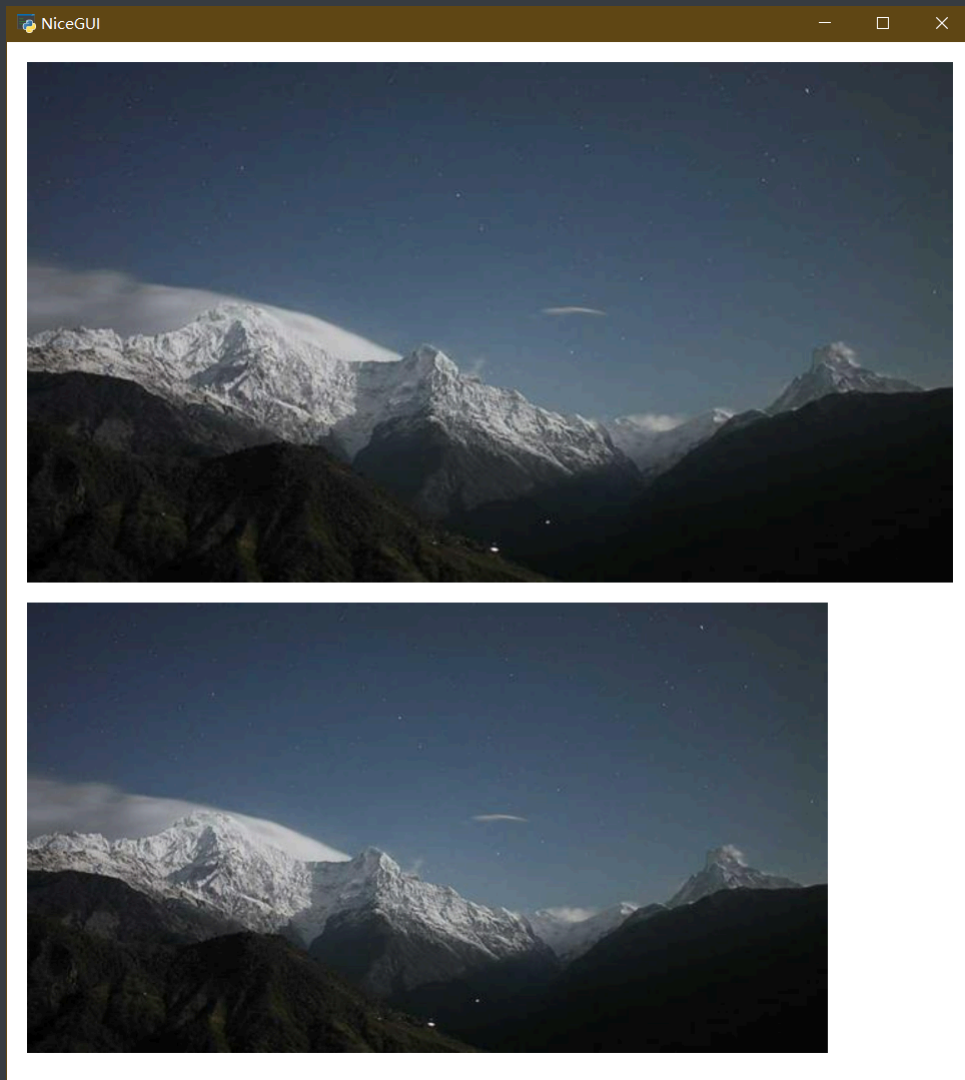
2.3.3 多媒体控件

2.3.3.1 `ui.image` 和 `ui.interactive_image`

点开一个网页，最抓人眼球的是什么内容？没错，是图片。既然用NiceGUI设计网页，没有图片元素怎么行？在NiceGUI中，有两种显示图片的控件：`ui.image` 和 `ui.interactive_image`。前者可以简单理解为显示图片的简单标签，后者是基于前者扩展了很多交互功能的plus版本。两者的第一个参数都是 `source`，支持字符串类型的本地图像路径、网络图像路径，或者base64编码的图像本身，这个没什么难点，这里不做细讲，接下来要重点讲的是 `ui.interactive_image` 的其他参数的用法，因为这个控件有时候比看似简单的 `ui.image` 更加好用。

先看一段代码：

```
1 from nicegui import ui
2
3 src = 'https://picsum.photos/id/377/640/360'
4 img = ui.image(src)
5 img2 = ui.interactive_image(src)
6
7 ui.run(native=True)
```



可以看到，同样的图片地址，都是不传入其他参数的情况下，即使可用空间大于图片大小，`ui.interactive_image` 也不会随着页面大小而缩放图片，始终保持图片的原始大小，这个有别于 `ui.image` 的特性，可以在日后想要保持图片真实大小时使用。

除了 `source` 参数外，`ui.interactive_image` 还有以下参数：

`content` 参数，字符串类型，表示覆盖在图片之上的SVG内容，SVG的画布大小就是图片的大小。当然，不太理解SVG的话也没关系，后面用到会详细介绍，也可以专门找一下资料。这里可以简单理解SVG为一种用定义描述的几何图形，这种图形不会因为缩放变成马赛克，因为它是基于定义绘制的。

`size` 参数，元组类型（宽度，高度），表示如果 `source` 没有设置的话，这就是默认图形的尺寸。这个对于绘制上面的SVG内容来说比较重要，因为这个尺寸就是画布的大小。对于想要交互创建SVG内容的操作，指定画布大小很重要。

`on_mouse` 参数，可调类型，表示触发鼠标事件之后要执行的操作，默认包含一个鼠标事件的参数，参数字典内的 `image_x` 和 `image_y` 的值是以像素为单位表示的鼠标交互位置。

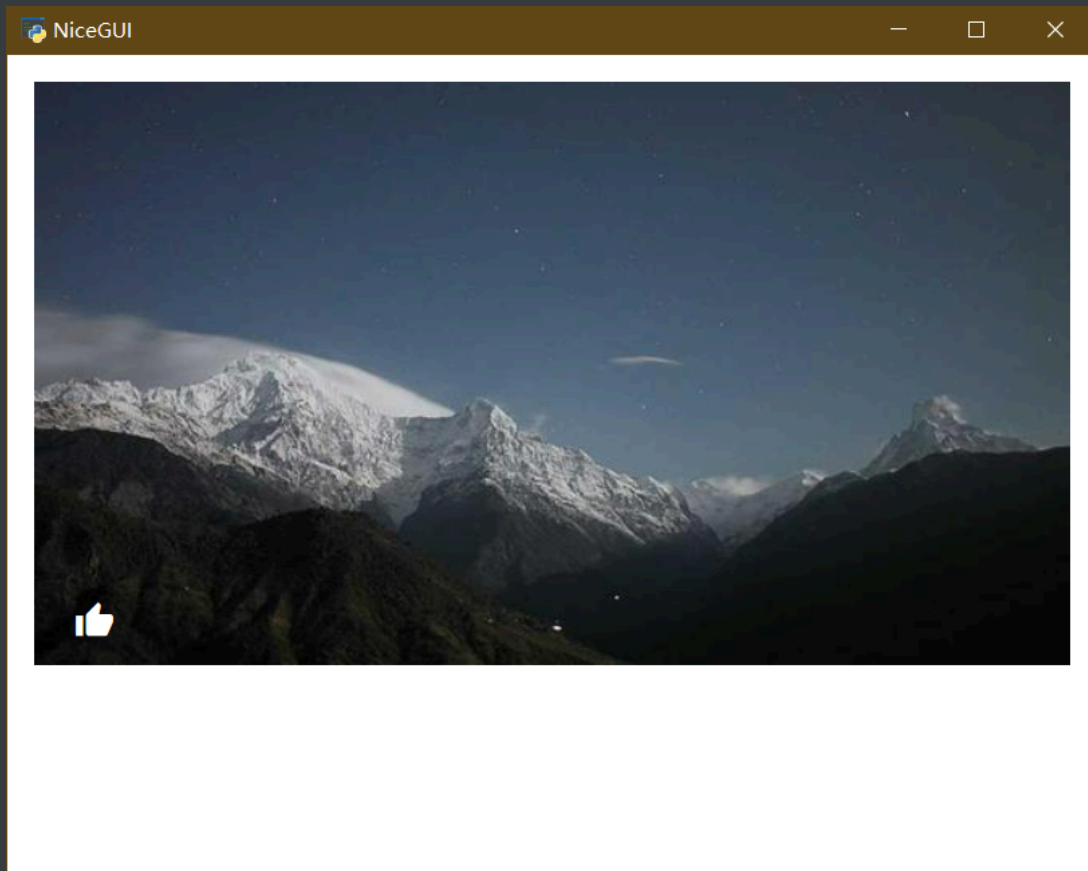
`events` 参数，字符串列表，表示JavaScript订阅的事件，默认订阅点击事件，即 `['click']`，也可以自己指定要订阅的事件内容。

`cross` 参数，字符串类型或者布尔类型，表示要不要显示十字线来指示鼠标位置，默认为 `False`。如果为 `True` 或者表示颜色的字符串，就会显示指定颜色（即字符串表示的颜色）的十字线。

除了上面的参数，还有一个 `force_reload` 方法，可以强制重新载入图片。这个方法两者图片控件都有，对于某些时候网络不好造成图片加载失败、支持刷新随机图片的接口，这个方法还是很实用的。

对于图片控件，一样可以使用 `with` 嵌入其他内容。比如，下面的代码就嵌入了一个按钮，实现了点击图片和按钮有不同的通知内容：

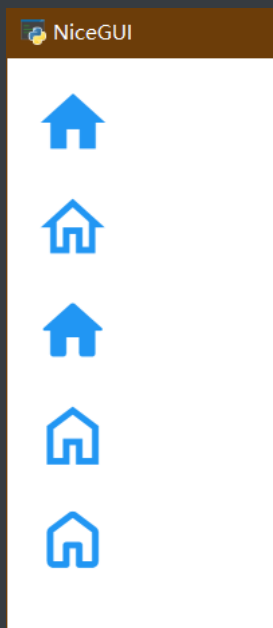
```
1  from nicegui import ui
2
3  src = 'https://picsum.photos/id/377/640/360'
4  with ui.interactive_image(src,on_mouse=lambda :ui.notify('You clicked
   interactive_image.')):
5      ui.button(on_click=lambda: ui.notify('You clicked button.'), icon='thumb_up')\
6          .props('flat color=white').classes('absolute bottom-0 left-0 m-2')
7
8  ui.run(native=True)
```



2.3.3.2 ui.icon 和 ui.avatar

在 [ui.button](#) 一节中挖了一个有关 `ui.icon` 的坑，现在，终于到了填坑的时候。先看一个简单的示例，了解一下 `ui.icon` 控件的用法：

```
1 from nicegui import ui
2
3 ui.icon(name='home', color='blue', size='xl')
4 ui.icon(name='o_home', color='blue', size='xl')
5 ui.icon(name='r_home', color='blue', size='xl')
6 ui.icon(name='sym_o_home', color='blue', size='xl')
7 ui.icon(name='sym_r_home', color='blue', size='xl')
8
9 ui.run(native=True)
```



`ui.icon` 的参数不多，就三个：

`name` 参数，字符串类型，表示图标字体中的图标名，`ui.icon` 通过给定的图标名，从字体中加载图标，默认支持Material Icons图标字体。也可以自己添加其他图标字体，并结合对应图标字体的用法加载。这一部分需要根据具体情况灵活变通，这里不做详解。

`color` 参数，字符串类型，表示图标的颜色。

`size` 参数，字符串类型，表示图标的大小。

理解参数不难，但是，给出的示例代码并没有看上去那么简单。这个时候，聪明的读者已经发现了猫腻：几个图标看上去很像，但不完全一样。再仔细看的话就会发现，它们的后缀都是一样的，只有前缀不同。没错，这组图标本质上都是一个，只是名字的前缀有特定的风格含义，需要特别注意一下。

无前缀代表实心填充风格，`'o_'`前缀表示轮廓线风格，`'r_'`前缀表示圆角风格，`'s_'`前缀表示锐化风格，`'sym_o_'`前缀表示轮廓线符号，`'sym_r_'`前缀表示圆角符号，`'sym_s_'`前缀表示锐化符号。

这个时候，有的读者就要头疼了，前缀就这么几个，还好记，可图标那么多，每个图标名字怎么记得住，有没有方便查询的网站？有，不过国内没法直接访问，那就是官网：

Material Icons查询网站（非中国大陆地区）：<https://fonts.google.com/icons?icon.set=Material+Icons>

当然，国内也有很多渠道查询，为了避免广告嫌疑，这里就不具体指明了。

不过，只是提供一个404网站，教程就失去了意义，这里还有一个备用方法，那就是小工具：


```

'sym_s_'], 'window_open': ['sym_o_', 'sym_r_', 'sym_s_'], 'window_sensor':
['sym_o_', 'sym_r_', 'sym_s_'], 'work_alert': ['sym_o_', 'sym_r_', 'sym_s_'],
'work_update': ['sym_o_', 'sym_r_', 'sym_s_'], 'wounds_injuries': ['sym_o_',
'sym_r_', 'sym_s_'], 'wrist': ['sym_o_', 'sym_r_', 'sym_s_'], 'your_trips':
['sym_o_', 'sym_r_', 'sym_s_'], 'youtube_activity': ['sym_o_', 'sym_r_', 'sym_s_'],
'zone_person_alert': ['sym_o_', 'sym_r_', 'sym_s_'], 'zone_person_idle': ['sym_o_',
'sym_r_', 'sym_s_'], 'zone_person_urgent': ['sym_o_', 'sym_r_', 'sym_s_']]
5
6 icon_name_list = list(icon_name_dict.keys())
7 icon_name_dict_typed = {'':[], 'o_':[], 'r_':[], 's_':[], 'sym_o_':[], 'sym_r_':
[], 'sym_s_':[],}
8 for i in icon_name_list:
9     for k in icon_name_dict[i]:
10         icon_name_dict_typed[k].append(i)
11
12 type_name_map = {'': 'Filled',
13                  'o_': 'Outline',
14                  'r_': 'Round',
15                  's_': 'Sharp',
16                  'sym_o_': 'Outline symbol',
17                  'sym_r_': 'Round symbol',
18                  'sym_s_': 'Sharp symbol',}
19
20 current_name = ''
21 current_type = ''
22
23 def all_show(icon_name = None, icon_type = None):
24     with ui.card():
25         ui.label(icon_name+':')
26         with ui.column():
27             with ui.row() as row1:
28                 label1= ui.label('This name has no filled icon.')
29             with ui.row() as row2:
30                 label2= ui.label('This name has no styled icon.')
31             with ui.row() as row3:
32                 label3= ui.label('This name has no symbol icon.')
33             for prefix in icon_type:
34                 if prefix in ['o_', 'r_', 's_']:

```

```

35         label2.set_text('Styled:')
36         with row2,ui.icon(prefix+icon_name,
size='3em').on('click',lambda :(ui.clipboard.write(prefix+icon_name),ui.notify('name
copied!'))):
37             ui.tooltip(prefix+icon_name).tailwind.font_size('sm')
38         elif prefix in ['sym_o_', 'sym_r_', 'sym_s_']:
39             label3.set_text('Symbol:')
40             with row3,ui.icon(prefix+icon_name,
size='3em').on('click',lambda :(ui.clipboard.write(prefix+icon_name),ui.notify('name
copied!'))):
41                 ui.tooltip(prefix+icon_name).tailwind.font_size('sm')
42         else:
43             label1.set_text('Filled:')
44             with row1,ui.icon(prefix+icon_name,
size='3em').on('click',lambda :(ui.clipboard.write(prefix+icon_name),ui.notify('name
copied!'))):
45                 ui.tooltip(prefix+icon_name).tailwind.font_size('sm')
46
47 def bigger_show(icon_name = None,icon_type = None):
48     with ui.card():
49         with ui.column():
50             ui.label(icon_name+f' of {type_name_map[icon_type]}'+':')
51             with ui.icon(icon_type+icon_name, size='10em').on('click',lambda :
(ui.clipboard.write(icon_type+icon_name),ui.notify('name copied!'))):
52                 ui.tooltip(icon_type+icon_name).tailwind.font_size('sm')
53
54 def title():
55     with ui.row():
56         ui.link('all_types','/')
57         ui.link('bigger_type','/bigger')
58         ui.link('bigger_type_fisrt','/bigger_first')
59
60 @ui.page('/',title='all types')
61 def page_index():
62     title()
63     icon_name = ui.select(icon_name_list, value= current_name if (current_name in
icon_name_list) else icon_name_list[0] ,label='icon name:',
with_input=True).classes('w-64')

```

```

64
65     @ui.refreshable
66     def show():
67
68         all_show(icon_name=icon_name.value,icon_type=icon_name_dict[icon_name.value])
69         show()
70         icon_name.on_value_change(show.refresh)
71         icon_name.on_value_change(lambda
72 e:globals().update({'current_name':e.sender.value}))
73
74 @ui.page('/bigger',title='bigger type')
75 def page_bigger():
76     title()
77     icon_name = ui.select(icon_name_list, value = current_name if (current_name in
78 icon_name_list) else icon_name_list[0],label='icon name:',
79 with_input=True).classes('w-64')
80
81 @ui.refreshable
82 def show():
83     current_type_list = icon_name_dict[icon_name.value]
84     icon_type_name = ui.select({ i:type_name_map[i] for i in current_type_list},
85 value= current_type if(current_type in current_type_list) else current_type_list[0]
86 , label='icon type name:',with_input=True).classes('w-64')
87     icon_type_name.bind_value_to(globals(),'current_type')
88
89 @ui.refreshable
90 def inner_show():
91     bigger_show(icon_name=icon_name.value,icon_type=icon_type_name.value)
92
93     inner_show()
94     icon_type_name.on_value_change(inner_show.refresh)
95     show()
96     icon_name.on_value_change(show.refresh)
97     icon_name.on_value_change(lambda
98 e:globals().update({'current_name':e.sender.value}))
99
100 @ui.page('/bigger_first',title='bigger_type_fisrt')
101 def page_bigger_first():

```



```
95     title()
96     icon_type_name = ui.select(type_name_map, value=current_type,label='icon type
name:', with_input=True).classes('w-64')
97     icon_type_name.bind_value_to(globals(),'current_type')
98
99     @ui.refreshable
100     def show():
101         current_name_list = icon_name_dict_typed[icon_type_name.value]
102         icon_name = ui.select(current_name_list, value= current_name if current_name
in current_name_list else current_name_list[0], label='icon
name:',with_input=True).classes('w-64')
103
104     @ui.refreshable
105     def inner_show():
106         bigger_show(icon_name = icon_name.value, icon_type=
icon_type_name.value)
107
108     inner_show()
109     icon_name.on_value_change(inner_show.refresh)
110     icon_name.on_value_change(lambda
e:globals().update({'current_name':e.sender.value}))
111     show()
112     icon_type_name.on_value_change(show.refresh)
113
114 ui.run(native=True,title='Material Icons Search')
115
```

[all_types](#)[bigger_type](#)[bigger_type_fisrt](#)

icon name:

flag



flag:

Filled:

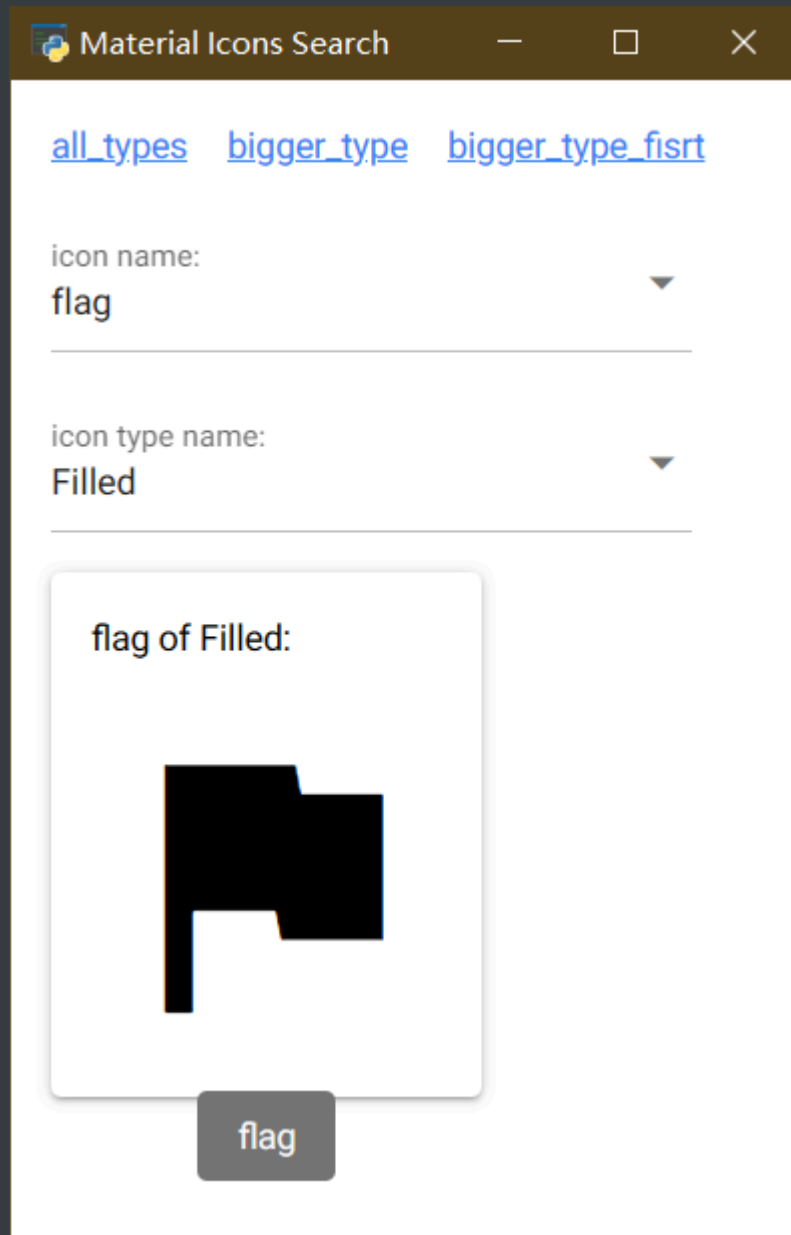


Styled:



Symbol:





可以在选择框里选择或者输入名字，得到该名字家族的全部图标，如果有的名字不支持全部风格的图标，会有英文提示。也可以点第二、第三个链接，查看对应图标的放大细节。点击图标，会自动复制全名。

也可以使用下面的 `icons` 模块，将代码存为 `icons.py`，放到同目录下，使用 `from icons import Filled,filled,Outline,outline,Round,round,Sharp,sharp,Outline_symbol,outline_symbol,Round_symbol,round_symbol,Sharp_symbol,sharp_symbol` 来导入使用。首字母大写的是字典，语法为 `Filled['10k']`，首字母小写的是函数，语法为 `filled('10k')`。这两种使用方法都会返回准确的图标名字，在VSCode或者其他IDE中会弹出提示。

```
1 from __future__ import annotations
2 from typing import TYPE_CHECKING,Literal
3
```



```
'sym_s_weather_snowy', 'web_traffic': 'sym_s_web_traffic', 'weight': 'sym_s_weight',  
'width': 'sym_s_width', 'wifi_add': 'sym_s_wifi_add', 'wifi_calling_1':  
'sym_s_wifi_calling_1', 'wifi_calling_2': 'sym_s_wifi_calling_2', 'wifi_home':  
'sym_s_wifi_home', 'wifi_notification': 'sym_s_wifi_notification', 'wifi_proxy':  
'sym_s_wifi_proxy', 'window_closed': 'sym_s_window_closed', 'window_open':  
'sym_s_window_open', 'window_sensor': 'sym_s_window_sensor', 'work_alert':  
'sym_s_work_alert', 'work_update': 'sym_s_work_update', 'wounds_injuries':  
'sym_s_wounds_injuries', 'wrist': 'sym_s_wrist', 'your_trips': 'sym_s_your_trips',  
'youtube_activity': 'sym_s_youtube_activity', 'zone_person_alert':  
'sym_s_zone_person_alert', 'zone_person_idle': 'sym_s_zone_person_idle',  
'zone_person_urgent': 'sym_s_zone_person_urgent'}
```

29

```
30 def filled(name:_FILLED)->str:
```

```
31     if name in Filled.keys():
```

```
32         return Filled[name]
```

```
33     else:
```

```
34         raise Exception('Name is illegal')
```

35

```
36 def outline(name:_OUTLINE)->str:
```

```
37     if name in Outline.keys():
```

```
38         return Outline[name]
```

```
39     else:
```

```
40         raise Exception('Name is illegal')
```

41

```
42 def round(name:_ROUND)->str:
```

```
43     if name in Round.keys():
```

```
44         return Round[name]
```

```
45     else:
```

```
46         raise Exception('Name is illegal')
```

47

```
48 def sharp(name:_SHARP)->str:
```

```
49     if name in Sharp.keys():
```

```
50         return Sharp[name]
```

```
51     else:
```

```
52         raise Exception('Name is illegal')
```

53

```
54 def outline_symbol(name:_OUTLINE_SYMBOL)->str:
```

```
55     if name in Outline_symbol.keys():
```

```

56         return Outline_symbol[name]
57     else:
58         raise Exception('Name is illegal')
59
60 def round_symbol(name:_ROUND_SYMBOL)->str:
61     if name in Round_symbol.keys():
62         return Round_symbol[name]
63     else:
64         raise Exception('Name is illegal')
65
66 def sharp_symbol(name:_SHARP_SYMBOL)->str:
67     if name in Sharp_symbol.keys():
68         return Sharp_symbol[name]
69     else:
70         raise Exception('Name is illegal')

```

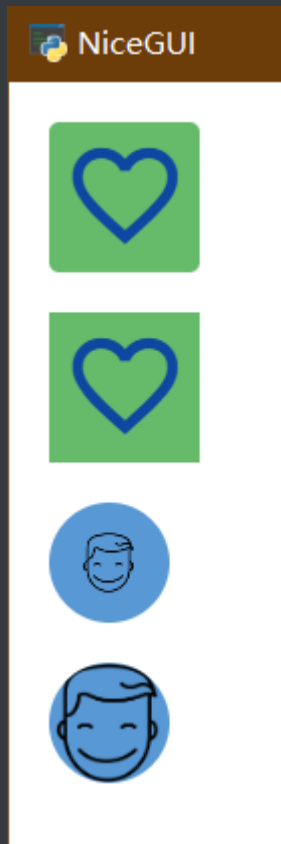
这一节到这里还没结束，还有一个控件要一并讲解。之所以放在一起讲，是因为这个控件主要参数和 `ui.icon` 一样，实际应用也不少，那就是头像控件——`ui.avatar`。

按照惯例先看代码：

```

1  from nicegui import ui
2
3  ui.avatar(icon='favorite_border', color='green-5', text_color='blue-10',
4           size='60px', font_size='50px', square=False, rounded=True)
5  ui.avatar(icon='favorite_border', color='green-5', text_color='blue-10',
6           size='60px', font_size='50px', square=True, rounded=False)
7  # 使用图片文件当做头像
8  url = 'https://nicegui.io/logo_square.png'
9  ui.avatar(f'img:{url}')
10 with ui.avatar():
11     ui.image(url)
12
13 ui.run(native=True)

```



从参数上看，`icon` 就是 `ui.icon` 的 `name`；颜色这里不太一样，`color` 和 `text_color` 分别代表头像的背景颜色、图标颜色；`size` 是头像大小，`font_size` 是图标大小。此外，还有两个布尔类型的参数决定头像框的形状：`square` 是方形，`rounded` 是圆角方形，如果两个都是 `False`，头像框就是圆形。

除了使用图标字体外，头像当然可以使用图片。一种是用以 `img:` 开头的图片地址，另一种是在 `default slot` 中加入图片控件。要注意的是，两种方法呈现的图片不一样，后者使用的是 `ui.image` 的样式，实际图片显示会有差别。

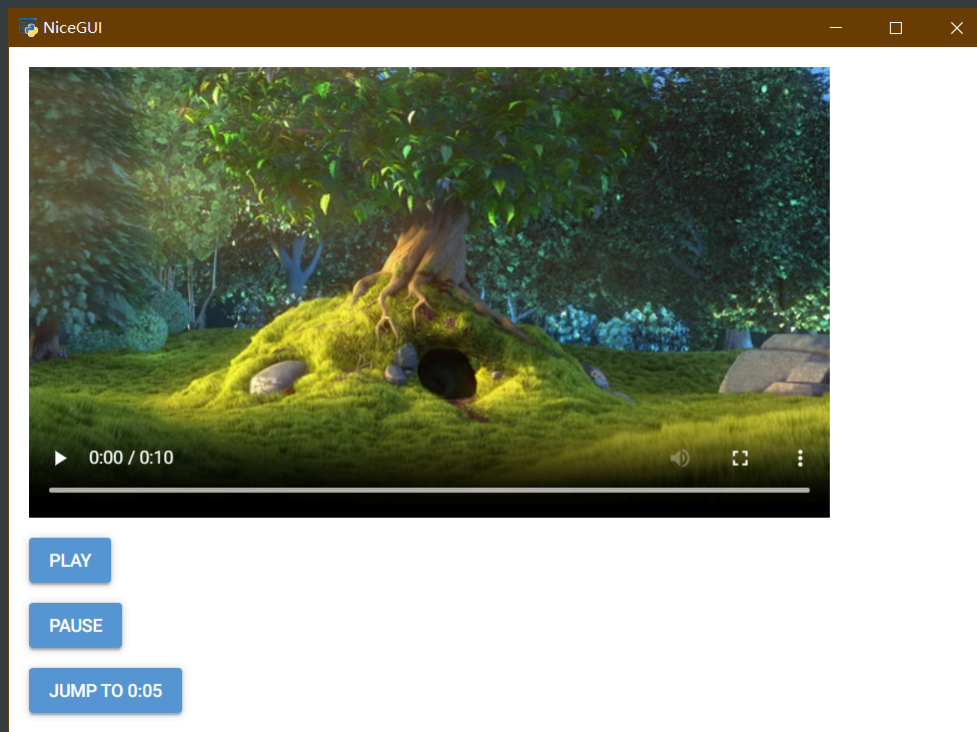
2.3.3.3 `ui.audio` 和 `ui.video`

`ui.audio` 和 `ui.video` 从本质上讲是一样的，参数、方法基本一致，所以，只需学会其中一个的用法，另一个就是改个名字的事儿。

顾名思义，`ui.audio` 和 `ui.video`，分别代表着音频、视频控件，对于需要在网页播放音频、视频的情况，这两个控件再合适不过。

先看示例代码：

```
1 from nicegui import ui
2
3 v = ui.video(src='https://test-
  videos.co.uk/vids/bigbuckbunny/mp4/h264/360/Big_Buck_Bunny_360_10s_1MB.mp4',
4           controls=True, autoplay=False, muted=False, loop=False)
5 ui.button('Play', on_click=v.play)
6 ui.button('Pause', on_click=v.pause)
7 ui.button('Jump to 0:05', on_click=lambda: v.seek(5))
8
9 ui.run(native=True)
```



控件通过字符串类型参数 `src` 传入视频地址，四个布尔类型参数 `controls`、`autoplay`、`muted`、`loop` 分别代表着是否显示内置工具栏、是否自动开始播放、是否静音、是否循环播放。

当然，除了上面代码里用按钮在外面控制播放暂停，还可以监听播放、暂停、播放结束的事件，执行特定的操作。


```
1 from nicegui import ui
2
3 v = ui.video(src='https://test-
    videos.co.uk/vids/bigbuckbunny/mp4/h264/360/Big_Buck_Bunny_360_10s_1MB.mp4',
4             controls=True, autoplay=False, muted=False, loop=False)
5 v.on('play', lambda _: ui.notify('Started'))
6 v.on('pause', lambda _: ui.notify('Paused'))
7 v.on('ended', lambda _: ui.notify('Completed'))
8
9 ui.run(native=True)
```

2.3.4 图形布局

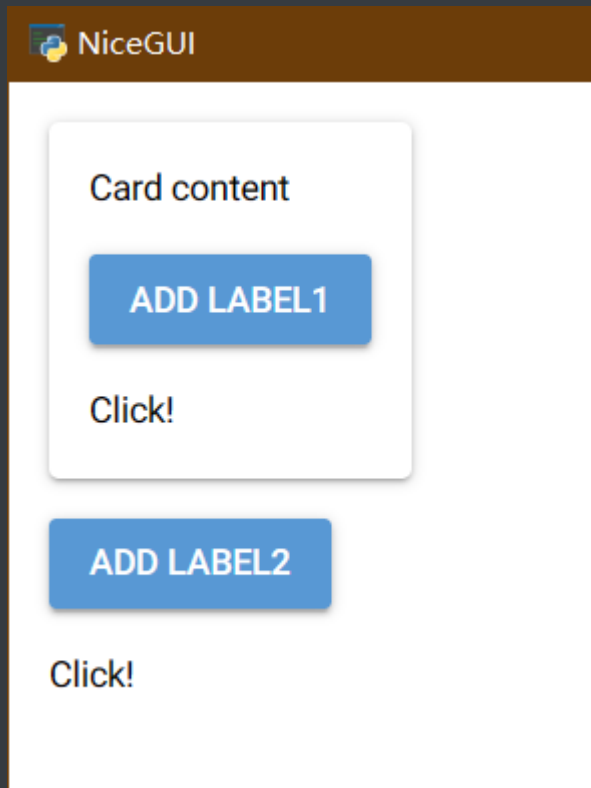
盖房子不是把砖简单地垒砌，做图形界面一样不是简单地添加控件，美观、简洁的布局除了让图形界面整体风格更加整齐，也让用户使用时更加方便、快捷，这才是图形界面的意义。在掌握了一定数量的常用控件之后，在继续深入学习其他控件之前，是时候了解一下NiceGUI支持的布局控件了。

在NiceGUI中，有一个概念非常重要，那就是上下文。前面代码中使用的 `with` ——Python的上下文管理器关键字，在NiceGUI中，就是进入某个控件的上下文，也就意味着后续添加控件的操作都是在这个控件的上下文内。直观感受的话，就是后续添加的控件都在这个控件内部。

以代码为例：

```
1 from nicegui import ui
2
3 with ui.card():
4     ui.label('Card content')
5     ui.button('Add label1', on_click=lambda: ui.label('Click!'))
6 ui.button('Add label2', on_click=lambda: ui.label('Click!'))
7
8 ui.run(native=True)
```

'Add label1'按钮在 `ui.card` 内部，它添加的 `ui.label` 就在`ui.card`内。'Add label2'按钮在 `ui.card` 外，它添加的 `ui.label` 就在 `ui.card` 外。所以点击两个按钮的效果是这样的：



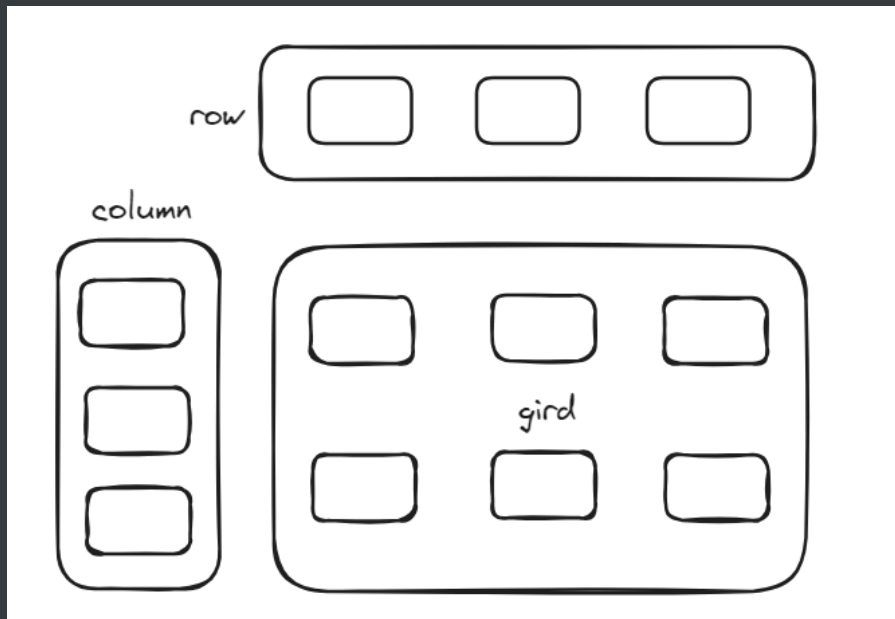
2.3.4.1 `ui.row`、`ui.column` 和 `ui.grid`

在图形界面的布局设计中，最常用也是最基本的布局，就是行布局、列布局和网格布局，几乎所有的现代GUI框架都提供了这三种布局。说来神奇，哪怕不用框架提供的其他布局，只是这三种布局相互组合，也能构成了用户在程序里看到的各种控件排布，真可谓“三生万物”。

`ui.row`（行）布局，即所有控件排成一行。

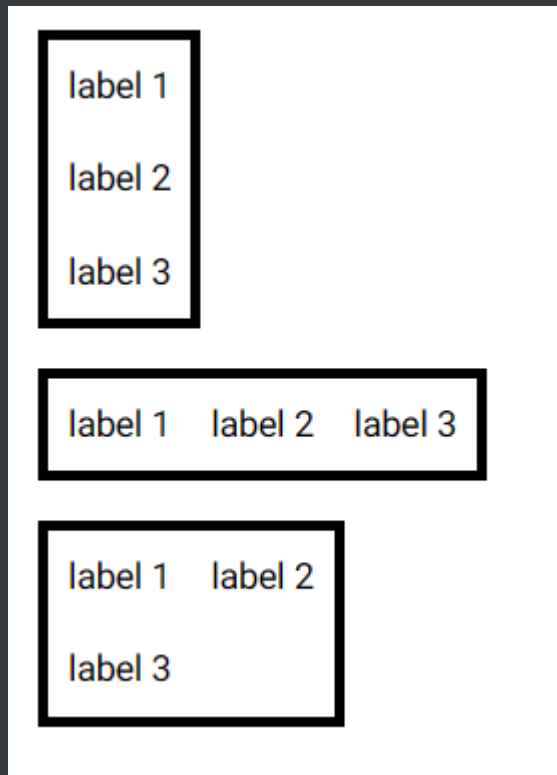
`ui.column`（列）布局，即所有控件排成一行。

`ui.grid`（网格）布局，即所有控件按照给定的网格排布（几行几列），依次占据每个单元格，就和表格一样。



在NiceGUI中，想要让指定的控件按照给定的布局排布，就要在布局的上下文内创建控件，因此，这三个常用布局的代码就要这样写：

```
1  from nicegui import ui
2
3  with ui.column().classes('border-4 border-black p-2'):
4      ui.label('label 1')
5      ui.label('label 2')
6      ui.label('label 3')
7
8  with ui.row().classes('border-4 border-black p-2'):
9      ui.label('label 1')
10     ui.label('label 2')
11     ui.label('label 3')
12
13  with ui.grid(columns=2,rows=2).classes('border-4 border-black p-2'):
14      ui.label('label 1')
15      ui.label('label 2')
16      ui.label('label 3')
17
18  ui.run(native=True)
```



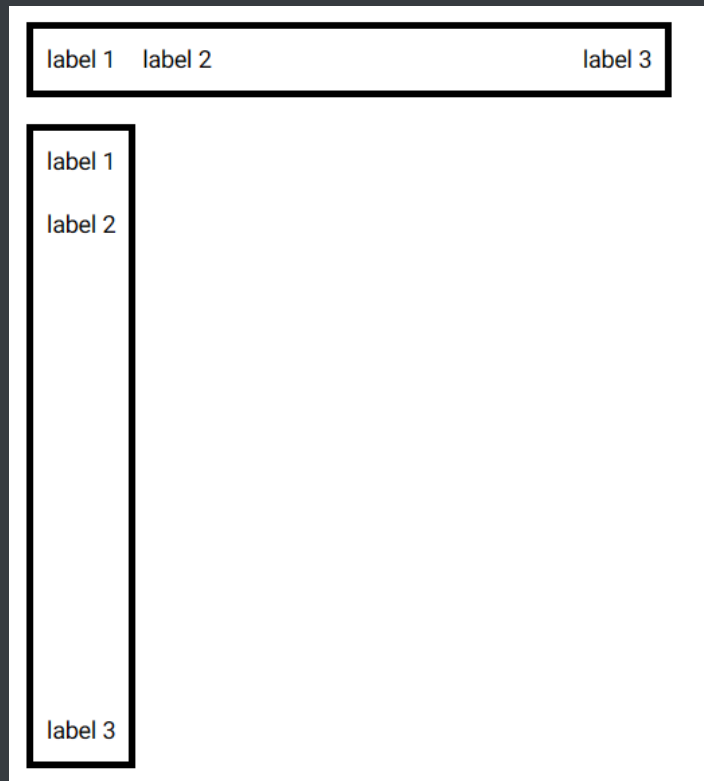
2.3.4.2 ui.space

在布局控件的时候，如果想要一些控件放在末尾，可按照布局控件的规则，内部的控件总是像排队一样一个挨着一个，似乎需要调整CSS才行。其实，有一种不用CSS的方法，那就是使用 `ui.space`。

`ui.space` 可以默认填充浮动布局的可用空间，让布局不会因为默认的排列规则而显得末尾空空如也。

```
1 from nicegui import ui
2
3 with ui.row().classes('w-96 border-4 border-black p-2'):
4     ui.label('label 1')
5     ui.label('label 2')
6     ui.space()
7     ui.label('label 3')
8
9 with ui.column().classes('h-96 border-4 border-black p-2'):
10     ui.label('label 1')
11     ui.label('label 2')
12     ui.space()
13     ui.label('label 3')
14
```

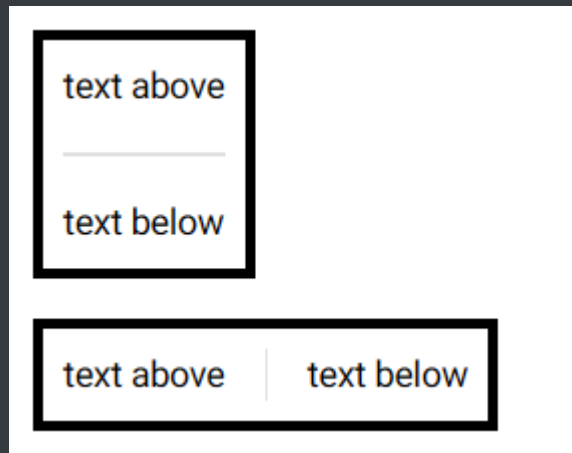
```
15 ui.run(native=True)
```



2.3.4.3 ui.separator

用 `ui.space` 可以填充空白，但是有读者担心，目的是达到了，能不能画条线在末尾的控件前，用来区分一下这个位置的控件与其他位置控件的区别？没问题，`ui.separator` 就是用来干这个的，不管是水平还是垂直，都可以。代码如下：

```
1 from nicegui import ui
2
3 with ui.column().classes('border-4 border-black p-2'):
4     ui.label('text above')
5     ui.separator()
6     ui.label('text below')
7
8 with ui.row().classes('border-4 border-black p-2'):
9     ui.label('text above')
10    ui.separator().props('vertical')
11    ui.label('text below')
12
13 ui.run(native=True)
```



2.3.4.4 ui.menu 和 ui.context_menu

有用过图形程序的读者看到前面的内容，肯定联想到，`ui.separator` 好像菜单里面用来分割退出功能与其他功能的分割符，NiceGUI有没有菜单？当然也有，普通的点击菜单 `ui.menu` 和右键菜单 `ui.context_menu`。两个菜单的语法一致，只是触发方式不同。菜单的内部一般使用 `ui.menu_item` 当菜单项，一般可以当做按钮处理，因为它们的触发操作都是传给点击事件。

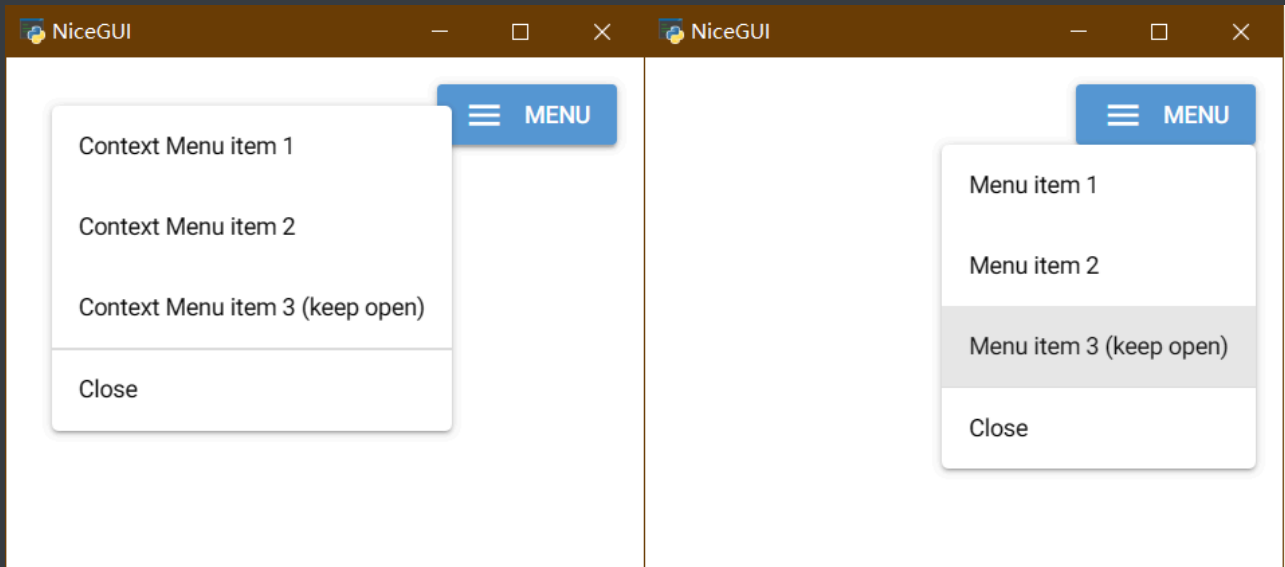
下面的代码给'menu'按钮创建点击菜单和右键菜单，可以分别用鼠标左击和右击查看弹出的菜单的异同，除了菜单内文字为了作出区分专门修改了之外，菜单的布局和样式一模一样，因此两种菜单可以很方便复用相同的内容：

```
1 from nicegui import ui
2
3 with ui.row().classes('w-full items-center'):
4     result = ui.label().classes('mr-auto')
5     with ui.button(text='menu', icon='menu'):
6         with ui.context_menu() as menu1:
7             ui.menu_item('Context Menu item 1', lambda: result.set_text('Selected
            item 1'))
8             ui.menu_item('Context Menu item 2', lambda: result.set_text('Selected
            item 2'))
9             ui.menu_item('Context Menu item 3 (keep open)',
10                          lambda: result.set_text('Selected item 3'),
            auto_close=False)
11             ui.separator()
12             ui.menu_item('Close', menu1.close)
13         with ui.menu() as menu2:
14             ui.menu_item('Menu item 1', lambda: result.set_text('Selected item 1'))
```

```

15         ui.menu_item('Menu item 2', lambda: result.set_text('Selected item 2'))
16         ui.menu_item('Menu item 3 (keep open)',
17                       lambda: result.set_text('Selected item 3'),
18                       auto_close=False)
19         ui.separator()
20         ui.menu_item('Close', menu2.close)
21
22 ui.run(native=True)

```



2.3.4.5 ui.tooltip

`ui.tooltip` 可以给控件添加一种光标悬停之后弹出的提示。一般是在给定控件的上下文添加，也可以用给定控件的 `tooltip` 方法来添加，代码如下：

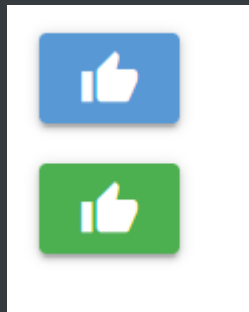
```

1  from nicegui import ui
2
3  with ui.button(icon='thumb_up'):
4      ui.tooltip('I like this')
5
6  button = ui.button(icon='thumb_up')
7  button.tooltip('I like this')
8
9  ui.run(native=True)

```

但是要注意，如果想要美化 `ui.tooltip`，只能给上下文添加的 `ui.tooltip` 美化，如果是用 `tooltip` 方法添加的，则不能美化。因为 `tooltip` 方法返回的是控件本身，而不是 `tooltip`。可以看到，下面代码的 `tooltip` 方法后添加的美化，在按钮本身生效了，实际上按钮的 `tooltip` 并没有改变颜色：

```
1 from nicegui import ui
2
3 with ui.button(icon='thumb_up'):
4     ui.tooltip('I like this').classes('bg-green')
5
6 button = ui.button(icon='thumb_up')
7 button.tooltip('I like this').classes('bg-green')
8
9 ui.run(native=True)
```



2.3.4.6 ui.notify

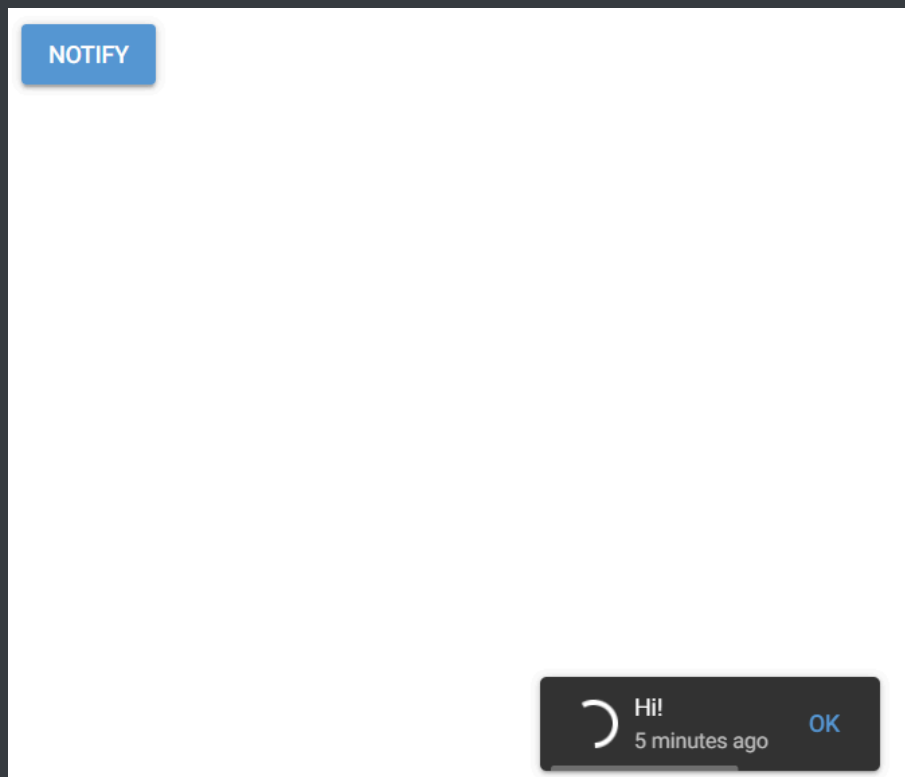
通知提示控件，在前面 `ui.button` 中已经用过，虽然简单传递一个字符串参数的用法很好用，但是到这一节结束都不说一下的话，实在是对不起读者。毕竟，这个控件的参数可不止看上去那么少，隐藏在其中的参数也很有用。

代码如下：

```
1 from nicegui import ui
2
3 ui.button('Notify',
4         on_click=lambda: ui.notify(
5             message='Hi!',
6             position='bottom',
7             close_button='OK',
8             type=None,
```



```
9         color='black-5',
10         multi_line=False,
11         progress=True,
12         caption='5 minutes ago',
13         timeout=5*1000,
14         spinner= True
15     ))
16
17 ui.run(native=True)
```



参数有点多，但不必害怕，习惯用法只需记住第一个参数，用起来也简单，其他参数仅作参考。

`message` 参数，字符串类型，信息文本，显示在通知中的主要内容。

`position` 参数，字符串类型，通知出现的位置，有 "top-left"、"top-right"、"bottom-left"、"bottom-right"、"top"、"bottom"、"left"、"right"、"center" 可选，默认为 "bottom"。

`close_button` 参数，字符串类型或者布尔型，是否显示关闭按钮，如果是字符串类型，关闭按钮的文字就是给定的文字，默认为 `False`。

`type` 参数, 字符串类型, 通知的类型, 有 "positive"、 "negative"、 "warning"、 "info"、 "ongoing", 默认为 `None`, 不是其中的任何一种。

`color` 参数, 字符串类型, 通知的背景颜色。

`multi_line` 参数, 布尔类型, 是否让通知内容以多行格式显示。

`progress` 参数, 布尔类型, 是否显示通知消失的进度条。

`caption` 参数, 字符串类型, 显示在信息文本下的说明文字。

`timeout` 参数, 整数型, 通知自动消失的时间, 单位毫秒, 为0就是不消失, 但是要确保 `close_button` 不是 `False`, 否则通知没法正常消除, 影响用户体验。

`spinner` 参数, 布尔类型, 是否显示转盘动画, 默认为 `False`。

2.3.5 属性绑定和数据控件

2.3.5.1 binding

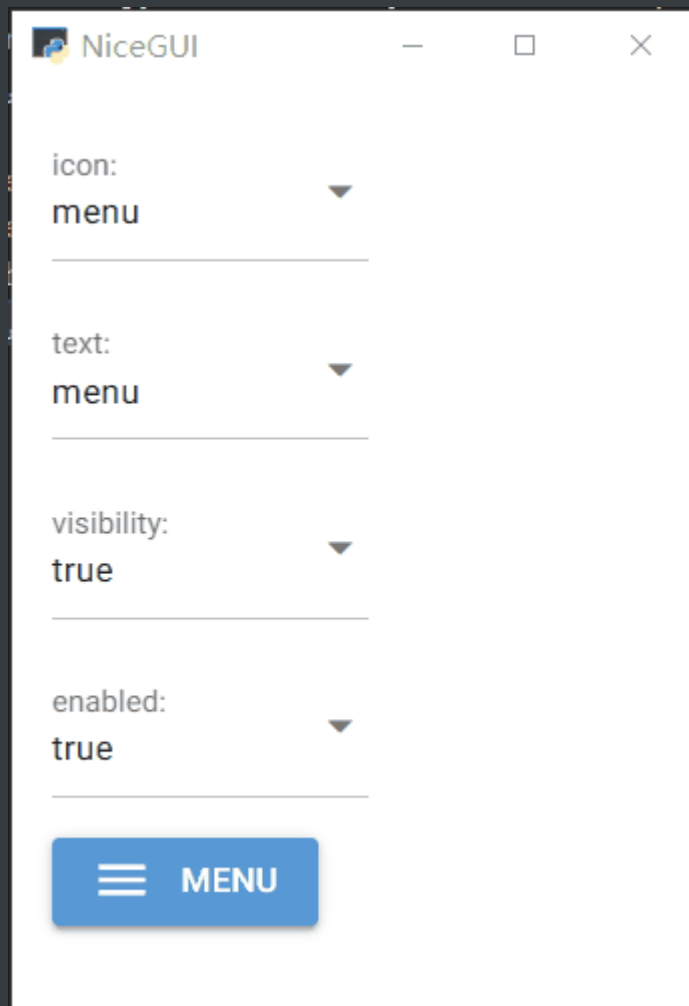
在 [ui.slider](#) 和 [ui.knob](#) 一节中, 为了让 `ui.slider` 和 `ui.knob` 联动, 使用了 `bind_value` 方法来实现, 现在是时候详细了解了。

对于大部分控件来说, 都有几个可以 `bind` (绑定) 的属性, 以 `ui.button` 为例, 有 `icon`、 `text`、 `visibility`、 `enabled` 几个属性可以绑定。在下面的代码中, 就将 `ui.button` 这四个属性绑定到 `ui.select` 上, 可以手动选择每个 `ui.select` 的值, 让按钮实时发生变化:

```

1 from nicegui import ui
2
3 icon = ui.select(['home', 'menu'], value='menu', label='icon:').classes('w-32')
4 text = ui.select(['home', 'menu'], value='menu', label='text:').classes('w-32')
5 visibility = ui.select([True, False], value=True, label='visibility:').classes('w-32')
6 enabled = ui.select([True, False], value=True, label='enabled:').classes('w-32')
7 ui.button()\
8     .bind_icon(icon, 'value')\
9     .bind_text(text, 'value')\
10    .bind_visibility(visibility, 'value')\
11    .bind_enabled(enabled, 'value')
12
13 ui.run(native=True)

```



`bind_{属性名}` 方法有四个参数：

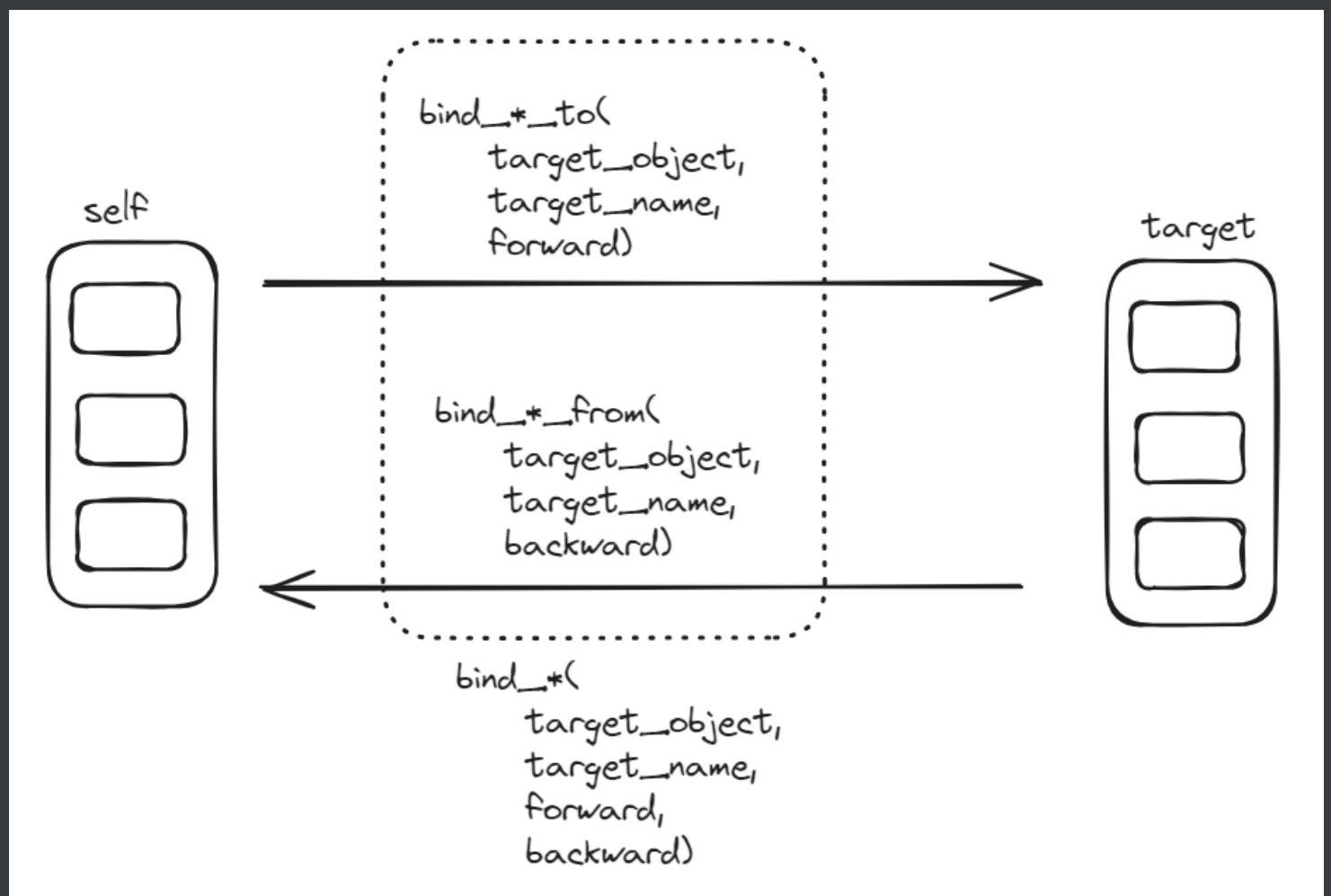
`target_object` 参数，任意对象，是被绑定的对象。

`target_name` 参数，字符串类型，是被绑定对象的属性名，以被绑定的 `ui.select` 为例，它的值就对应 `value` 属性。

`forward` 参数和 `backward` 参数，可调用类型，是正向绑定和反向绑定的转换函数。因为绑定是双向的，除了在目标值改变的时候同步过来，本身的值改变还要同步过去，是两个方向的操作。两个参数的默认值都是 `lambda x:x`，意思是双向传递过程不改变值，如果有需要，可以修改这两个参数，在传递的时候实现数据的转换和映射的改变。

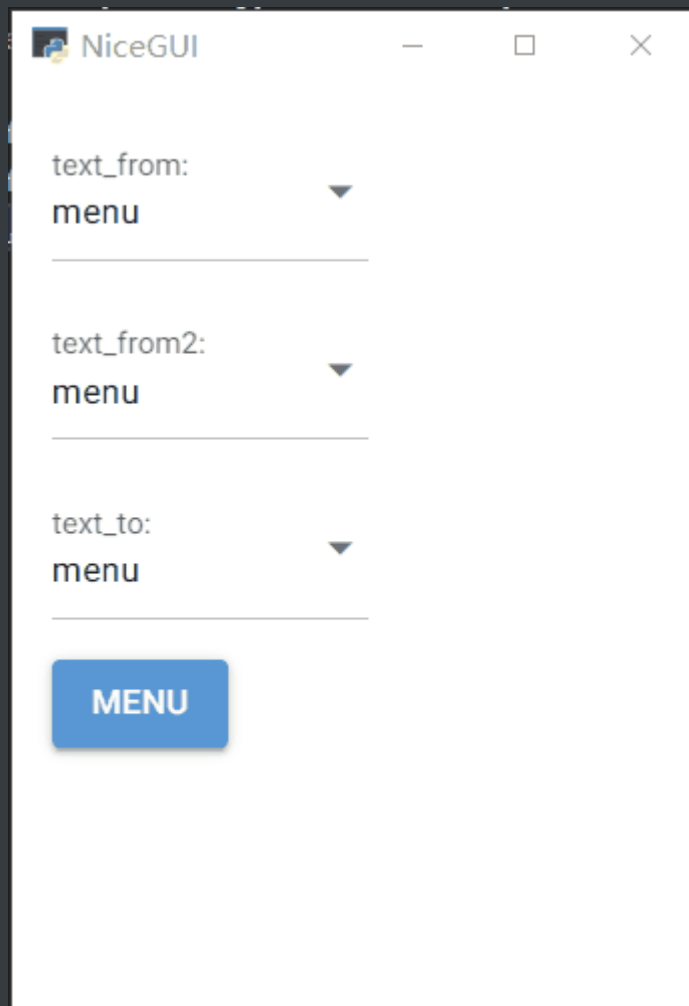
要注意的是，`bind_{属性名}` 方法返回的是控件本身，因此可以实现连续bind的操作，不需要每次bind都要写一次控件对象，NiceGUI的这个特性在实际开发中非常方便。

除了 `bind_{属性名}` 方法，对于同一属性，还有 `bind_{属性名}_to` 方法和 `bind_{属性名}_from` 方法。相比之下，后两者方法就像是把前者的两个方向拆分，分别对应着正向和反向，参数自然分别去掉了不需要的反向绑定 `backward` 参数和正向绑定 `forward` 参数。而NiceGUI实际上内部的实现，是 `bind_{属性名}` 方法执行了 `bind_{属性名}_to` 方法和 `bind_{属性名}_from` 方法，因此，三者的关系更像下面这张图：



`bind_{属性名}_to` 方法和 `bind_{属性名}_from` 方法的特性，可以参照下面代码的演示：

```
1 from nicegui import ui
2
3 text_from = ui.select(['home', 'menu'], value='menu', label='text_from:').classes('w-32')
4 text_from2 = ui.select(['home', 'menu'], value='menu', label='text_from2:').classes('w-32')
5 text_to = ui.select(['home', 'menu'], value='menu', label='text_to:').classes('w-32')
6 ui.button()\
7     .bind_text_from(text_from, 'value')\
8     .bind_text_from(text_from2, 'value')\
9     .bind_text_to(text_to, 'value')
10
11 ui.run(native=True)
```



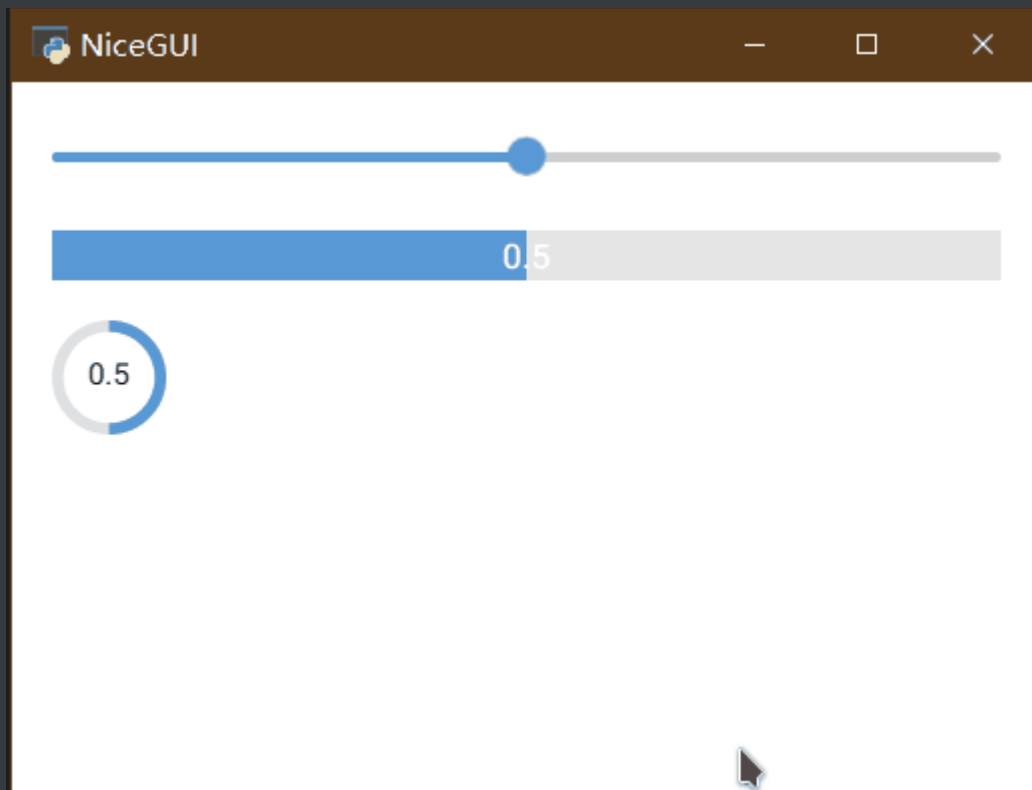
代码中做了两个反向绑定和一个正向绑定到 `ui.button`。实际操作的话就会发现，因为两个反向绑定的控件都是单向绑定，任意操作其中一个反向绑定的控件，都只会影响到 `ui.button` 和正向绑定的控件，不会影响到另一个反向绑定的控件。对于正向绑定的控件，只有 `ui.button` 的 `text` 发生变化之后，正向绑定的控件才会发生变化，只是操作正向绑定的控件，`ui.button` 和两个反向绑定的控件不会受影响。

2.3.5.2 `ui.linear_progress` 和 `ui.circular_progress`

前面 `ui.slider` 和 `ui.knob` 介绍过滑动条和旋钮，肯定有聪明的读者想用这两个控件做条形进度条和环形进度条，但是，这一节的标题已经表明，有专门的进度条控件，使用滑动条和旋钮充当进度条就未免有些画蛇添足。更关键的是，滑动条和旋钮接受用户输入，如果想避免用户输入影响数据展示，还要将控件禁用，具体样式又要做诸多调整，很不方便。然而，进度条控件没有这些弊端，使用起来更加简单。显然，这一节要介绍的进度条控件更适合展示进度。

`ui.linear_progress` 和 `ui.circular_progress` 分别是线性进度条和环形进度条，样子看上去就像滑动条和旋钮，但与滑动条和旋钮不同，这两个控件不支持交互输入，只接受数据输入，所以，要想实现简单的交互演示，需要用到上一节的 `bind` 方法，这里用的是 `bind_value_from`。当然，如果有其他控件作为该控件的下游，还可以使用 `bind_value_to` 串联下去。

```
1 from nicegui import ui
2
3 slider = ui.slider(min=0, max=1, step=0.01, value=0.5)
4 ui.linear_progress().bind_value_from(slider, 'value')
5 ui.circular_progress().bind_value_from(slider, 'value')
6
7 ui.run(native=True)
```

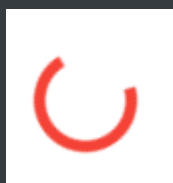


2.3.5.3 ui.spinner

spinner直译的话是陀螺，在实际使用中，其实是加载中常见的动画图标，这里就称之为加载动画控件。

先看代码：

```
1 from nicegui import ui
2
3 ui.spinner(type='default',size='xl',color='red',thickness=5)
4
5 ui.run(native=True)
```



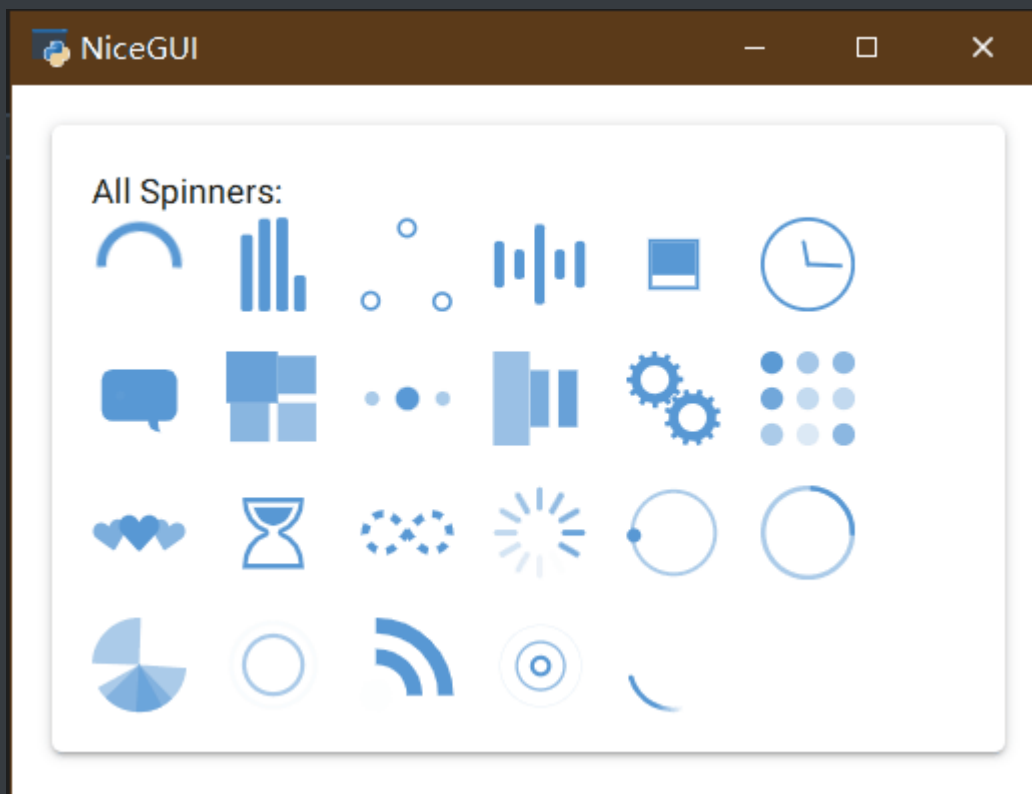
`ui.spinner` 有 `type`、`size`、`color`、`thickness` 四个参数，分别是字符串类型的动画类型、字符串类型的整体尺寸大小、字符串类型的颜色和浮点类型的厚度（这个只有 `type` 为 `default` 时生效）。

下面的代码展示了所有动画类型的 `ui.spinner`，读者可以根据需要选择合适的 `ui.spinner`：

```

1 from nicegui import ui
2
3 spinner_type = ['default', 'audio', 'ball', 'bars', 'box', 'clock',
4                 'comment', 'cube', 'dots', 'facebook', 'gears', 'grid',
5                 'hearts', 'hourglass', 'infinity', 'ios', 'orbit', 'oval',
6                 'pie', 'puff', 'radio', 'rings', 'tail']
7
8 with ui.card().classes('w-96'), ui.label('All Spinners:'), ui.row():
9     for i in spinner_type:
10         with ui.element():
11             ui.spinner(i, size='lg')
12             ui.tooltip(i)
13
14 ui.run(native=True)

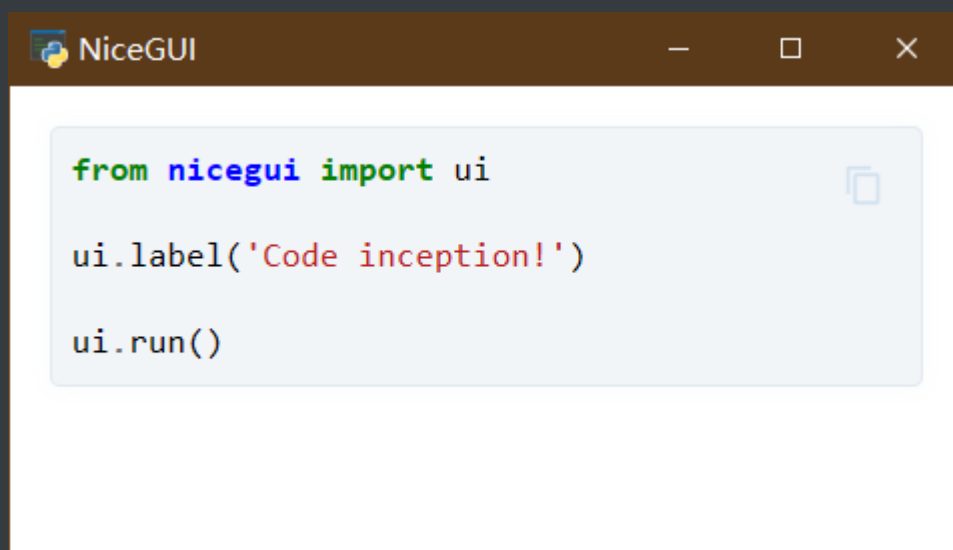
```



2.3.5.4 ui.code

如果想要在网页上显示有语法高亮的代码块，除了截图之外，惯用的做法是将代码放到代码块里。在NiceGUI中，`ui.code` 就是用来展示代码的。`ui.code` 有两个字符串参数，`content` 参数是代码主体，`language` 参数是代码的语言类型。


```
1 from nicegui import ui
2
3 code = '''
4     from nicegui import ui
5
6     ui.label('Code inception!')
7
8     ui.run()
9 '''
10 ui.code(content=code, language='python').classes('w-full')
11
12 ui.run(native=True)
```



用于数据展示的控件还有很多，比如：表格 `ui.table`，树形图 `ui.tree`，基于AG Grid的 `ui.aggrid`，用于绘制图表 `ui.echart`、`ui.highchart`、`ui.pyplot`、`ui.line_plot`、`ui.plotly`，展示3D模型的 `ui.scene` 等。因为用法稍微有点复杂，有能力的读者可以自己了解或者等基础学完之后学习后续的进阶教程，这里就不做讲解了。

2.3.6 外观美化

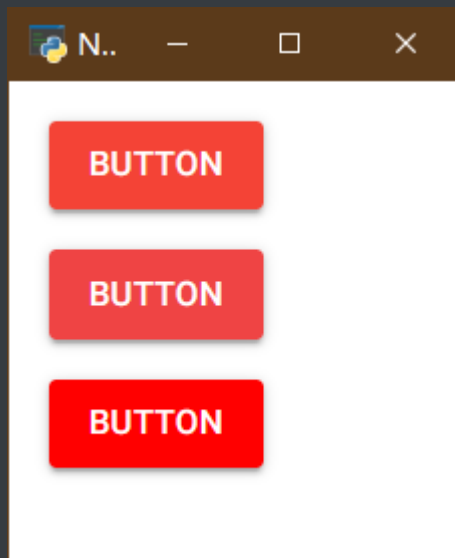
哪怕控件的默认样式好看，总有人觉得厌烦，调整颜色、大小、排列方式，或者一点小小的位移，都能让人眼前一亮。可是，对于前端来说的改变很小，需要涉及到的经验、代码很多，不去深入学习的话似乎很难达到效果。好在有NiceGUI，它的前端样式可以使用TailWindCSS，提前定义的一系列样式省去不少麻烦。当然，它也提供了多种自定义的接口，也没有因为降低难度而限制了前端的自由。

2.3.6.1 需要记住具体名字的 props 、 classes 、 style 方法

在掌握便利的工具之前，先辛苦一下，学点需要基础知识、但以后会很有用的方法。

先看代码：

```
1 from nicegui import ui
2
3 ui.button('button').props('color="red"')
4 ui.button('button').classes('!bg-red-500')
5 ui.button('button').style('background-color:red !important')
6
7 ui.run(native=True)
```



代码中，使用了三种方法将按钮颜色设置为红色。下面将分别说明三种方法的用法：

`props` 方法，设置的是HTML标签的属性，对于NiceGUI基于的Quasar来说，标签属性有不少是决定显示效果的，因此，在NiceGUI中，可以用这个方法设置对应底层组件的属性。就代码案例而言，具体属性名可以参考<https://quasar.dev/vue-components/button>，属性值参考 <https://quasar.dev/style/sass-scss-variables>。

`classes` 方法，设置的是HTML标签的类（class）名，NiceGUI支持TailWindCSS，因此，`classes` 方法可以使用TailWindCSS预定义的类，也可以使用自定义的CSS类。就代码案例而言，具体类名可以参考<https://tailwindcss.com/docs/background-color>。

`style` 方法，设置的是HTML标签的CSS属性，可以直接给组件设置CSS属性，如果有一定前端基础的话，可以使用该方法充分自定义，没有相关基础也没关系，上面的两个方法提供了丰富的预定义功能，学习难度也不高。就代码案例而言，具体语法可以参考 <https://developer.mozilla.org/en-US/docs/Web/CSS/background-color>。

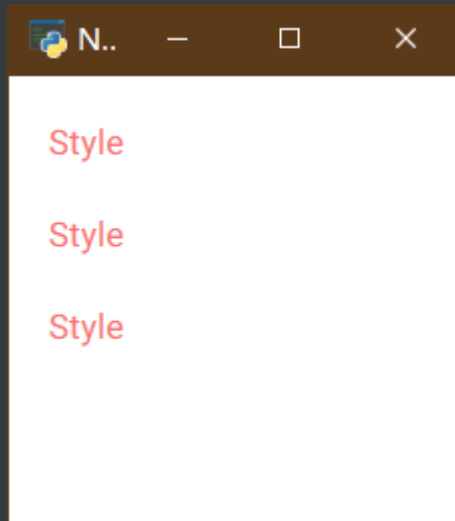
2.3.6.2 好记一点的 `tailwindcss` 属性和 `Tailwind` 对象

TailWindCSS官网提供了搜索功能，但总归需要一定的时间学习，更别说网络不好或者没网的时候，没法打开官网搜索。当然，也有方法自托管TailWindCSS的文档，但有一定难度，也不符合NiceGUI的宗旨。

因此，NiceGUI后续推出了 `tailwindcss` 属性和 `Tailwind` 对象。无需死记硬背TailWindCSS，也不需要反复查询官网，直接使用 `tailwindcss` 属性或者使用 `Tailwind` 对象，会有自动提示，只要有一点英语基础，很容易找到对应的TailWindCSS类名。

比如：

```
1 from nicegui import ui
2 from nicegui.tailwind import Tailwind
3
4 #设定标签的字体颜色为红色
5 label = ui.label('Style')
6 red_style = Tailwind().text_color('red-400')
7 red_style.apply(label)
8 #上述代码可以简化为一行，但是调用tailwind必须放到最后。
9 #因为tailwind的函数返回的是tailwind对象，不是element对象。
10 #调用完tailwind之后没法继续调用基于element的方法。
11 ui.label('Style').tailwind.text_color('red-400')
12 #以上方法等同于设定class为text-red-400
13 ui.label('Style').classes('text-red-400')
14
15 ui.run(native=True)
```



在开发工具中调用控件的 `tailwindcss` 属性或者创建 `Tailwind()` 对象之后，再输入 `.`，就会弹出子方法提示，选择对应的子方法之后，再输入引号，会进一步弹出参数的提示。

2.3.6.3 暗黑模式 `ui.dark_mode`

现在很多网站支持暗黑模式，方便访问者光线不佳时浏览网页，让网页没那么刺眼。NiceGUI中，也有一个控件可以快速切换暗黑模式，实现同样的效果，那就是 `ui.dark_mode`。

以下代码用 `ui.dark_mode` 创建了一个暗黑模式的控件，默认值为 `False`，并且创建了一个模式切换的响应函数，会在模式切换时弹出通知，通过点击下方的两个按钮，可以直接启用、禁用暗黑模式：

```
1 from nicegui import ui
2
3 dark = ui.dark_mode(value=False, on_change=lambda :ui.notify(f'Dark mode is
    {dark.value}.'))
4 ui.label('Switch mode:')
5 ui.button('Dark', on_click=dark.enable)
6 ui.button('Light', on_click=dark.disable)
7
8 ui.run(native=True)
```

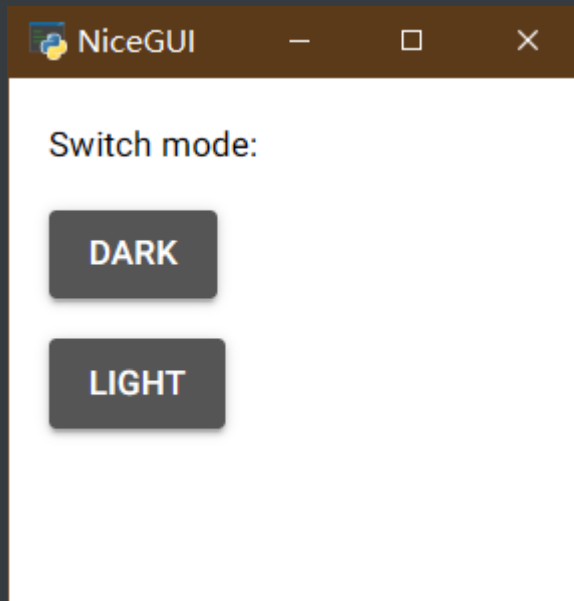
2.3.6.4 设置主题颜色 `ui.colors`

前面讲了如何给单个控件设置颜色的方法，肯定有读者担心了，如果想要修改主题颜色，还需要每个控件修改一次或者添加一下主题类，会不会很麻烦？其实，这个操作也有简单的方法，`ui.colors` 可以直接设置整个程序的主题颜色，直接影响所有控件。因为Quasar提供了一系列预先定义好的类名，这些名字用在NiceGUI的控件颜色中，通过 `ui.colors` 可以修改这些类名的具体值。类名可以参考<https://q>

uasar.dev/style/theme-builder。

以下面的代码为例，`primary` 就是各个控件主要的颜色，通过 `ui.colors` 将 `primary` 修改为 `#555`，上一节中的按钮默认颜色就变成灰色了：

```
1  from nicegui import ui
2
3  dark = ui.dark_mode(value=False, on_change=lambda: ui.notify(
4      f'Dark mode is {dark.value}.'))
5  ui.label('Switch mode:')
6  ui.button('Dark', on_click=dark.enable)
7  ui.button('Light', on_click=dark.disable)
8
9  main_color = dict(primary='#5898d4',
10                    secondary='#26a69a',
11                    accent='#9c27b0',
12                    dark='#1d1d1d',
13                    positive='#21ba45',
14                    negative='#c10015',
15                    info='#31ccec',
16                    warning='#f2c037')
17  ui.colors().props.update(main_color)
18  #或者简单修改其中一个类别
19  ui.colors(primary='#555')
20
21  ui.run(native=True)
```



2.3.7 事件和执行

2.3.7.1 通用事件 (2025.05.23更新)

大部分控件都有预定义事件监听，比如，`ui.button` 的 `on_click` 点击事件监听，在传参或者调用方法时定义。除了这种已经定义的事件监听，每个控件还支持通过 `on` 方法创建任意事件监听，比如使用 `on` 方法创建点击事件监听，也可以创建鼠标进入、离开的事件监听。正如下面的代码所示：

```
1 from nicegui import ui
2
3 ui.button('A',on_click=lambda: ui.notify('You clicked the button A.'))
4 ui.button('B').on('click',lambda: ui.notify('You clicked the button B.'))
5 ui.button('C').on('mouseenter',lambda: ui.notify('You entered the button C.'))
6 ui.button('D').on('mouseleave',lambda: ui.notify('You left the button D.'))
7
8 ui.run(native=True)
```

以下内容为随 NiceGUI 2.18.0 版本更新增加的 `on` 方法详解。

`on` 方法支持以下参数：

- `type` 参数，字符串类型，表示监听什么事件。
- `handler` 参数，可调用类型，表示服务器端的Python响应函数。响应函数接收一个表示事件对象的 `events.GenericEventArguments` 类型参数，该参数包含一个 `args` 属性。
- `args` 参数，`None` 或者元素为字符串的序列或者元素为序列（元素为字符串）的单元素序列，

表示客户端的哪些参数及其值会在执行响应函数时，会传给响应函数接收参数的 `args` 属性（字典形式）。如果为 `None` 的话，表示将客户端所有的参数传入响应函数接收参数的 `args` 属性。比如，可以检查客户端响应事件时，有没有按下其他功能键：

```
1 from nicegui import ui
2
3 button = ui.button('click')
4 button.on(
5     type='click',
6     handler=lambda e: ui.notify(f'hello {e}'),
7     args=['ctrlKey', 'shiftKey', 'altKey'],
8     #或者[['ctrlKey', 'shiftKey', 'altKey']]
9 )
10
11 ui.run(native=True)
```

- `throttle` 参数，浮点类型，表示事件之间的发生间隔，小于该间隔的事件不会重复处理（默认第一个和最后一个都会处理），该参数默认为 `0.0`。从此参数开始，只能通过关键字传入。
- `leading_events` 参数，布尔类型，事件发生间隔内的第一个事件发生时是否立即执行响应函数，默认为 `True`。
- `trailing_events` 参数，布尔类型，事件发生间隔内的最后一个事件发生后是否也要执行响应函数，默认为 `True`。
- `js_handler` 参数，字符串类型，表示客户端的JavaScript响应函数，默认为 `'(...args) => emit(...args)'`。注意，如果JavaScript响应函数内不使用 `emit` 且与 `handler` 参数同时定义的话，`handler` 参数表示的响应函数不会执行。而JavaScript响应函数内使用的 `emit` 方法，会把传给该方法的参数，传给 `handler` 参数表示的响应函数的接收参数的 `args` 属性。

以下为示例代码：

```

1  from nicegui import ui
2
3  button = ui.button('click')
4  button.on(
5      type='click',
6      handler=lambda e: ui.notify(f'hello {e.args}'),
7      js_handler='(e) => emit(123)'
8  )
9
10 ui.run(native=True)

```

注意，`handler` 参数与 `js_handler` 参数同时使用的规则始于 NiceGUI 2.18.0 版本，之前版本只允许使用二者其一，不允许同时使用。

2.3.7.2 常用事件 `app.on_*`

如果想要针对浏览器端的连接、关闭，服务器端的启动、关闭，添加事件监听，就要导入 `app` 模块。`app.on_connect`、`app.on_disconnect` 可以监听浏览器端的连接、关闭；`app.on_startup`、`app.on_shutdown` 可以监听服务器端的启动、关闭。代码如下：

```

1  from nicegui import ui, app
2
3  app.on_startup(lambda :print('started!'))
4  app.on_shutdown(lambda :print('shuttingdown!'))
5  app.on_connect(lambda :print('connected!'))
6  app.on_disconnect(lambda :print('disconnected!'))
7
8  ui.run(native=True)

```

2.3.7.3 定时器 `ui.timer` 和 `app.timer`

定时器可以根据给定的时间间隔，周期性执行指定函数。`ui.timer` 有四个参数：浮点类型的时间间隔 `interval`、可调用类型的执行操作 `callback`、布尔类型的是否激活 `active`、布尔类型的是否运行一次 `once`。

代码如下：

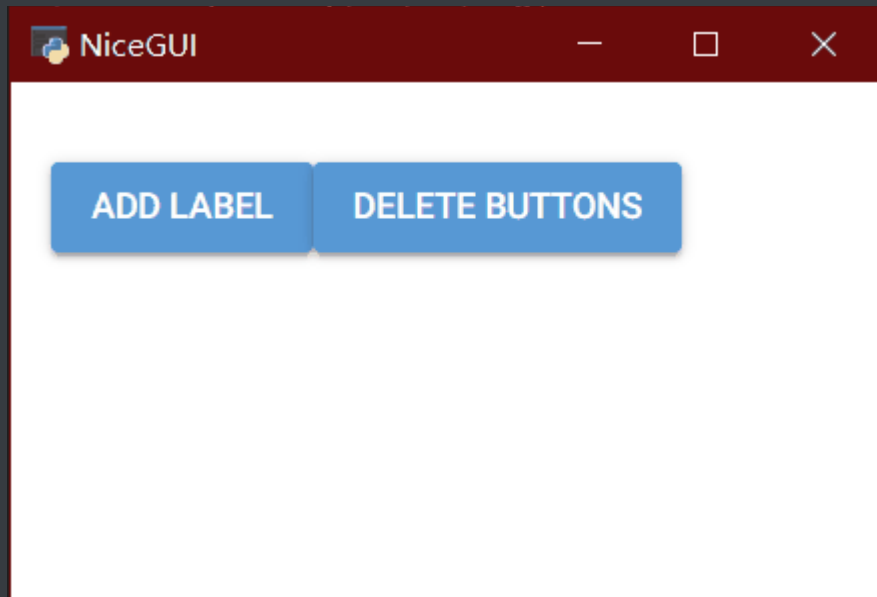

```
1 from nicegui import ui
2
3 ui.timer(interval=6.0,callback=lambda :ui.notify('Timer.'),active=True,once=False)
4
5 ui.run(native=True)
```

以下的内容按理来说属于进阶部分，就算不学习，也能满足基础开发需要。但是官方将此部分内容与基础部分放在一起解释，为了方便有相关需求的读者学习，特地将该部分内容与基础合并，并且进阶部分也会同步增加。不理解、不需要此功能的读者可以暂时忽略，等学进阶部分时再学也可以。

NiceGUI官方在2.9.0版本新增了 `app.timer` 定时器，虽然用法上和 `ui.timer` 一样，但其归属于 `app` 而不是 `ui`，还是有所区别的。

为了理解区别，需要先运行以下示例代码：

```
1 from nicegui import app, ui
2
3 counter = {'value': 0}
4 label = ui.element()
5 timer = None
6
7 with ui.element() as buttons:
8     button1 = ui.button('add label')
9     button2 = ui.button('delete buttons')
10
11 def add_label():
12     timer = ui.timer(1,lambda :counter.update(value=counter['value']+1))
13     with label:
14         ui.label().bind_text_from(counter, 'value', lambda value: f'Count: {value}')
15
16 def delete_buttons():
17     buttons.clear()
18
19 button1.on_click(add_label)
20 button2.on_click(delete_buttons)
21
22 ui.run(native=True)
```



示例代码源于NiceGUI官方仓库的一个问题，这里稍微简化了一下。问题作者想要让按钮创建一个定时更新显示内容的定时器，然后用另一个按钮删掉创建定时器的按钮。就是这样听起来很简单的操作，结果在删掉按钮时，工作定时器好像被一并“删掉”了。导致删掉按钮之后，原本应该继续执行的显示更新操作随之停止了。

听起来很奇怪，像是一个问题，其实不是，一开始就没有必要让按钮创建定时器。定时器可以在按钮的响应函数之外创建，按钮只需启动（`activate`）、停止（`deactivate`）定时器即可。因为定时器（`ui.timer`）会自动关联创建定时器的UI组件，一般做法是在`auto-index`页创建定时器，定时器关联了`auto-index`页，而`auto-index`页一般不会被删掉（也不能删掉，会出问题），所以使用定时器不会出问题。如果是其他UI组件创建了定时器，删掉创建定时器的UI组件，同时会一并删掉定时器，这也就是问题的原因。

上面的示例代码更换成常规用法也可以，解决方法也不难，不过，NiceGUI官方还是为此增加了一个独立于UI组件的定时器——`app.timer`，既是对此问题的解决方案，也是对后续有类似需求的功能实现。

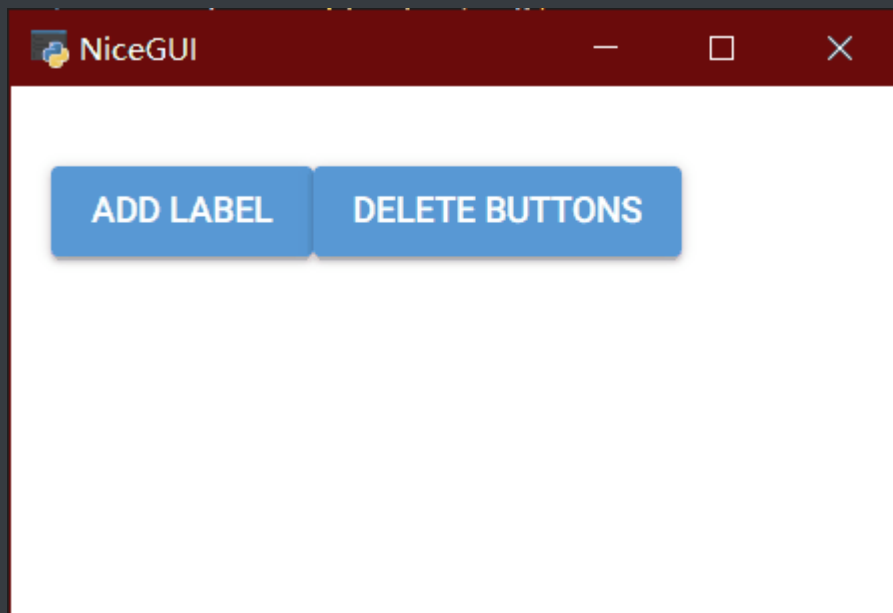
那么，上面的代码在基本不动的前提下，只需将 `ui.timer` 换成 `app.timer` 即可：

```
1 from nicegui import app, ui
2
3 counter = {'value': 0}
4 label = ui.element()
5 timer = None
6
7 with ui.element() as buttons:
```

```

8     button1 = ui.button('add label')
9     button2 = ui.button('delete buttons')
10
11 def add_label():
12     timer = app.timer(1,lambda :counter.update(value=counter['value']+1))
13     with label:
14         ui.label().bind_text_from(counter, 'value', lambda value: f'Count: {value}')
15
16 def delete_buttons():
17     buttons.clear()
18
19 button1.on_click(add_label)
20 button2.on_click(delete_buttons)
21
22 ui.run(native=True)

```



2.3.7.4 UI更新和可刷新方法 `ui.refreshable` (2025.05.04更新)

一般来说，当修改控件的属性时，NiceGUI会尝试更新控件的显示状态，比如标签的文本、输入框的内容、控件的 `style / classes / props` 属性等。当然，如果某些修改不会触发UI刷新，那就要调用控件的 `update` 方法来更新控件显示。如果需要更新的控件比较多，也可以使用 `ui` 的 `update` 方法，该方法可以传入多个控件，一次性完成每个控件的更新，不用重复调用每个控件 `update` 方法的代码。

以下的代码为例，如果不调用 `update` 方法，加载动画控件不会因为显示为新的大小：

```
1 from nicegui import ui
2
3 spinner = ui.spinner(type='default',size='xl',color='red',thickness=5)
4
5 def change_size():
6     spinner.props.update({'size':'100px'})
7     spinner.update()
8     # 或者用ui.update(spinner)
9
10 ui.button('change size',on_click=change_size)
11
12 ui.run(native=True)
```

但是，`update` 方法有一定局限性，只能刷新控件自身，如果控件下嵌套其他控件，该方法不能确保子控件一并刷新。因此，可刷新方法 `ui.refreshable` 就派上用场了。

以下面代码为例：

```
1 from nicegui import ui
2
3 a = ['A','A']
4
5 def row_ui():
6     with ui.row() as row:
7         for i in a:
8             ui.label(i)
9     return row
10
11 row = row_ui()
12
13 def clicked():
14     a.append('A')
15     row.update()
16
17 ui.button('A+1',on_click=clicked)
18
19 ui.run(native=True)
```

定义了一个字符串列表 `a`，用于存储字符串。函数 `row_ui` 会根据 `a` 的内容创建相同数目的 `ui.label`。然而，按钮 'A+1' 要执行的函数里，会执行一次给 `a` 追加一个 'A'，也就是说，每点一次按钮，`a` 里的 'A' 就会多一个。可是，即便函数 `clicked` 里已经添加了 `row.update()` 来刷新 `ui.row`，每次点击按钮，屏幕上的 `ui.label` 并不会增加。

问题出在哪里？稍安勿躁，先修改一下代码，用 `@ui.refreshable` 装饰一下函数 `row_ui`，然后将函数 `clicked` 里的 `row.update()` 换成 `row_ui.refresh()`，再看看按钮点击的效果：

```
1 from nicegui import ui
2
3 a = ['A', 'A']
4
5 @ui.refreshable
6 def row_ui():
7     with ui.row():
8         for i in a:
9             ui.label(i)
10
11 row_ui()
12
13 def clicked():
14     a.append('A')
15     row_ui.refresh()
16
17 ui.button('A+1', on_click=clicked)
18
19 ui.run(native=True)
```

这一次，结果总算正确了，点一次按钮，屏幕上的 `ui.label` 增加一个。

为什么？

可刷新方法 `ui.refreshable` 装饰的函数，会多一个 `refresh` 方法，调用这个方法会触发被装饰函数的重新执行，相当于这一部分的UI全部重新绘制，并不会像 `update` 方法一样只是触发刷新。触发刷新只能检测到与控件相关联的事件，示例代码里定义的列表 `a`，没有使用绑定方法与控件关联，只有执行迭代的时候才有不同，因此 `ui.row` 的刷新并不能实现预期效果。

一般来说，为了快捷创建有规律的控件而使用迭代方法，需要根据数据变化刷新显示，都要用 `ui.refreshable` 来重新绘制。当然，没有控件数量变化的时候也有需要使用 `ui.refreshable` 来重新绘制的情况，读者在遇到时见机行事。毕竟重新绘制比触发刷新的性能开销大，非必要情况，还是不要制造需要重新绘制的情况。

也许有的读者会说，每次增加一个 `ui.label` 就要手动调用一次 `refresh` 方法，代码看上去很“啰嗦”。这个时候，可以尝试一下 `ui.state`。这个控件没有界面，但它却是 `ui.refreshable` 的最佳搭档。

假定 `ui.refreshable` 装饰的函数是一个新的控件——计数器，函数内定义的控件就是被包装的控件（包括显示当前数字的标签和负责加一、归零的按钮）。按照前面的写法，代码是这样的：

```
1 from nicegui import ui
2
3 num = 0
4 @ui.refreshable
5 def counter():
6     with ui.row():
7         ui.label(str(num))
8         ui.button('+1', on_click=lambda :
9             (globals().update({'num':num+1}), counter.refresh()))
10        ui.button('0', on_click=lambda :
11            (globals().update({'num':0}), counter.refresh()))
12
13 counter()
14
15 ui.run(native=True)
```

好像出了什么问题，明明说前面的代码看上去很“啰嗦”，怎么这里的写法还是那个味道？另外，为什么要在函数的外面定义 `num = 0`？

不管是加一还是归零，每次计数变化，都需要调用 `refresh` 方法，所以两个按钮除了做本职工作之外，还要负责计数器的刷新。

为了让计数可以存下来，必须要由一个变量存储这个值，所以，这个变量只能在函数外面（在函数内定义不行，在类内部可以）。

如果想要在这个组合控件内定义一个只由内部控件控制的变量来存储当前的计数值，那就只能用类，但类不能被 `ui.refreshable` 装饰。倒是可以在类内实现一系列方法来实现这个组合控件，但难度和复杂度都会上升，不符合教程前半部分的简洁宗旨。

好了，不卖关子了，是时候让 `ui.state` 登场了。该控件需要放在被 `ui.refreshable` 装饰的函数内，创建控件时需要给该控件传入一个值（也可以是对象）。该控件会返回一个包含代表该值的变量、设置该值的方法的二元素元组（即（变量，方法））。当调用该方法设置该值时，如果值有变化（值是对象的话需要重新创建，不然只能手动调用上层函数的 `refresh` 方法），该控件会自动调用上层函数的 `refresh` 方法，来避免使用该值的控件没有刷新。

一般来说，该控件的实际代码是这样的：`num,set_num = ui.state(0)`。

因此，用了 `ui.state` 之后，计数器可以这样写：

```
1 from nicegui import ui
2
3 @ui.refreshable
4 def counter():
5     num,set_num = ui.state(0)
6     with ui.row():
7         ui.label(str(num))
8         ui.button('+1',on_click=lambda :set_num(num+1))
9         ui.button('0',on_click=lambda :set_num(0))
10
11 counter()
12
13 ui.run(native=True)
```

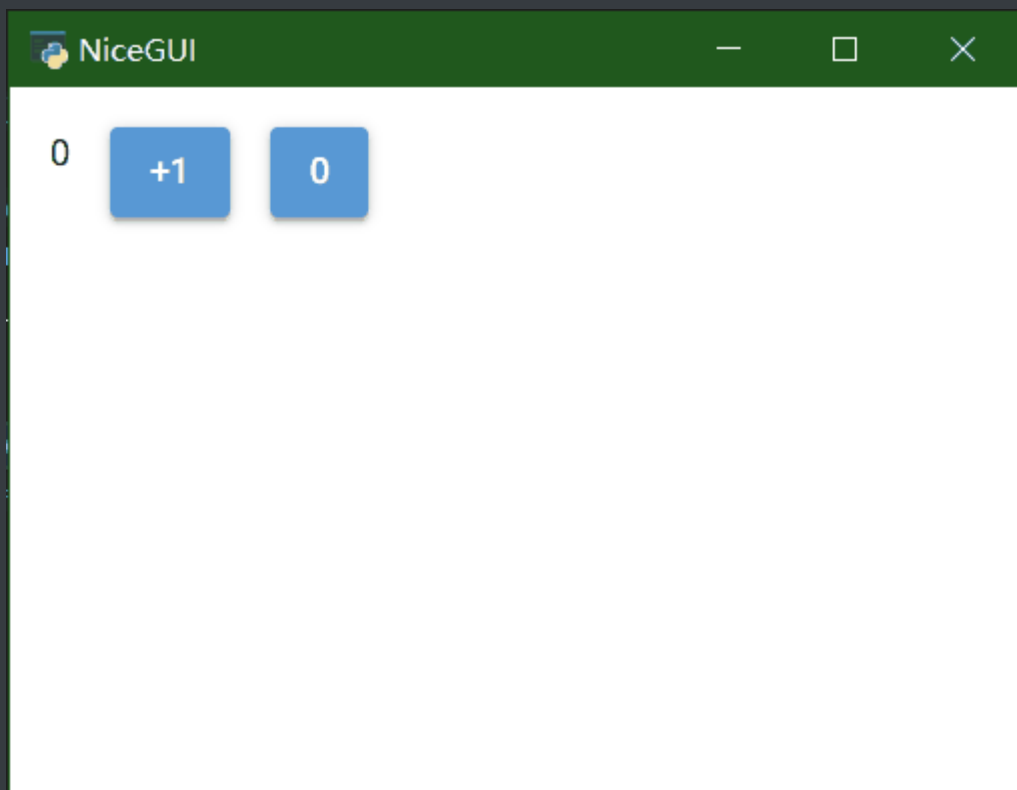
当然，为了验证 `ui.state` 有没有真的调用 `refresh` 方法，可以添加一个动态创建同等数量标签的代码：

```
1 from nicegui import ui
2
3 @ui.refreshable
4 def counter():
5     num,set_num = ui.state(0)
6     with ui.row():
```

```

7         ui.label(str(num))
8         ui.button('+1',on_click=lambda :set_num(num+1))
9         ui.button('0',on_click=lambda :set_num(0))
10     if num:
11         with ui.row():
12             for _ in range(num):
13                 ui.label('A')
14
15 counter()
16
17 ui.run(native=True)

```



可刷新方法 `ui.refreshable` 的学习到这里还没结束，但接下来的内容涉及到后面才会讲到 `ui.page`，如果不是回过头学习的读者，可以先加个书签，跳过这部分，等理解了 `ui.page` 之后再回来。

如果读者已经理解了 `ui.page`，那么接下来的部分将会解决未来可能遇到的问题，也是一个非常重要的知识点，那就是可刷新方法的作用域。

学习 `ui.page` 时，有个概念非常重要，那就是page页面。已知不使用 `ui.page` 装饰函数的话，控件都是显示在auto-index的页面下，那么此时这些控件的作用域都是全局的，在私有的page页面中，也可以访问全局的控件。对于可刷新方法来说，效果是相同的。

比如下面的例子：

```
1 from nicegui import ui
2 from datetime import datetime
3
4 @ui.refreshable
5 def timer():
6     ui.label(f'Time: {datetime.now()}')
7
8 ui.link('timer1', 'timer1')
9 ui.link('timer2', 'timer2')
10
11 @ui.page('/timer1', title='timer1')
12 def timer_1():
13     ui.label('timer1')
14     ui.link('index', '/')
15     timer()
16     ui.button('refresh', on_click=timer.refresh)
17
18 @ui.page('/timer2', title='timer2')
19 def timer_2():
20     ui.label('timer2')
21     ui.link('index', '/')
22     timer()
23     ui.button('refresh', on_click=timer.refresh)
24
25 ui.run()
```

虽然创建控件是在不同的页面上下文中，但点击其他页面（如 `/timer2`）中的刷新按钮，所有页面中的时间都会刷新。

当然，如果想要让可刷新方法制造的控件独立，那就要改变定义可刷新方法的位置，其位置决定了可刷新方法所属的作用域。

因此，如果在页面创建时定义可刷新方法，这样得到控件就不再是全局生效了：

```
1 from nicegui import ui
```

```

2  from datetime import datetime
3
4  def timer_func():
5      ui.label(f'Time: {datetime.now()}')
6
7  ui.link('timer1', 'timer1')
8  ui.link('timer2', 'timer2')
9
10 @ui.page('/timer1', title='timer1')
11 def timer_1():
12     ui.label('timer1')
13     ui.link('index', '/')
14     timer = ui.refreshable(timer_func)
15     timer()
16     ui.button('refresh', on_click=timer.refresh)
17
18 @ui.page('/timer2', title='timer2')
19 def timer_2():
20     ui.label('timer2')
21     ui.link('index', '/')
22     timer = ui.refreshable(timer_func)
23     timer()
24     ui.button('refresh', on_click=timer.refresh)
25
26 ui.run()

```

如果是在类内定义可刷新方法（修饰类的实例方法推荐使用 `ui.refreshable_method` 装饰器，但使用 `ui.refreshable` 装饰器不影响结果），因为不同实例之间存在隔离性，调用实例对象的可刷新方法的 `refresh` 方法不会互相影响（只能调用实例对象的可刷新方法的 `refresh` 方法）。因此，创建实例对象时的位置，就是可刷新方法所属的作用域，并非取决于类的定义位置：

```

1  from nicegui import ui
2  from datetime import datetime
3
4  class Clock:
5      @ui.refreshable_method
6      def timer(self):
7          ui.label(f'Time: {datetime.now()}')

```

```

8         ui.button('refresh',on_click=self.timer.refresh)
9
10    ui.link('timer1','timer1')
11    ui.link('timer2','timer2')
12    c = Clock()
13    c.timer()
14    c.timer()
15    Clock().timer()
16
17    @ui.page('/timer1',title='timer1')
18    def timer_1():
19        ui.label('timer1')
20        ui.link('index','/')
21        c.timer()
22
23    @ui.page('/timer2',title='timer2')
24    def timer_2():
25        ui.label('timer2')
26        ui.link('index','/')
27        Clock().timer()
28        Clock().timer()
29
30    ui.run()

```

可以看到，如果是执行 `c.timer()`，因为 `c` 的定义位置在全局作用域，因此生成的控件是同步刷新。对于私有页面中的 `Clock().timer()`，创建实例对象的位置是局部作用域，所以不会因为可刷新方法是在同一个类内定义而受影响，依然是独立的。当然，可刷新方法是类的实例方法，因此不同实例之间也不会互相影响。

如果可刷新方法是类的类方法（只能使用类方法装饰器装饰可刷新方法，静态方法也是一样的要求和表现），那么，类的定义位置决定了可刷新方法的作用域：

```

1    from nicegui import ui
2    from datetime import datetime
3
4    class Clock:
5        @classmethod
6        @ui.refreshable

```

```

7     def timer(cls):
8         ui.label(f'Time: {datetime.now()}')
9         ui.button('refresh',on_click=cls.timer.refresh)
10
11 ui.link('timer1','timer1')
12 ui.link('timer2','timer2')
13 c = Clock()
14 c.timer()
15 c.timer()
16 Clock().timer()
17
18 @ui.page('/timer1',title='timer1')
19 def timer_1():
20     ui.label('timer1')
21     ui.link('index','/')
22     c.timer()
23
24 @ui.page('/timer2',title='timer2')
25 def timer_2():
26     ui.label('timer2')
27     ui.link('index','/')
28     Clock().timer()
29     Clock().timer()
30
31 ui.run()

```

下面来一个直观的表格总结一下上面的内容：

函数类型	可刷新方法的作用域取决于
普通函数	定义可刷新方法时的位置
实例方法	创建对象时的位置（因为实例的隔离性）
类方法、静态方法	定义类时的位置

还记得本节前面介绍的 `ui.state` 吗？它可以在可刷新方法内部存储数值，让可刷新方法表现得更像一个控件。因此，如果可刷新方法内部使用了 `ui.state`，与其相关的控件，就会有所不同。`ui.state` 存储的数值只有在调用其返回的方法时更新，直接使用可刷新方法的 `refresh` 方法，不会更新数值。所以，基于普通函数创建的可刷新方法，`ui.state` 的作用域则属于函数内部：

```
1  from nicegui import ui
2  from datetime import datetime
3
4  @ui.refreshable
5  def timer():
6      time, set_time = ui.state('')
7      ui.label(f'Time: {time}')
8      ui.button('refresh', on_click=lambda :set_time(datetime.now()))
9
10 ui.link('timer1', 'timer1')
11 ui.link('timer2', 'timer2')
12 timer()
13 timer()
14
15 @ui.page('/timer1', title='timer1')
16 def timer_1():
17     ui.label('timer1')
18     ui.link('index', '/')
19     timer()
20
21 @ui.page('/timer2', title='timer2')
22 def timer_2():
23     ui.label('timer2')
24     ui.link('index', '/')
25     timer()
26
27 ui.run()
```

但是需要注意，不同于 `auto-index` 页面属于全局作用域，程序运行之后，只执行一次创建控件的代码。私有页面内的变量属于局部作用域，每次刷新都是重新执行一次 `ui.page` 修饰的函数。而每次执行可刷新方法，创建出来的 `ui.state` 都是独立的。因此，私有页面内可刷新方法中的 `ui.state`，在页面刷新之后会被重置，无法继续存储数值，这一表现并非 bug。

如果是基于实例方法创建的可刷新方法，`ui.state` 的作用域则会表现得有点诡异：

```
1  from nicegui import ui
2  from datetime import datetime
3
4  class Clock:
5      @ui.refreshable_method
6      def timer(self):
7          time, set_time = ui.state('')
8          ui.label(f'Time: {time}')
9          ui.button('refresh', on_click=lambda :set_time(datetime.now()))
10
11  ui.link('timer1', 'timer1')
12  ui.link('timer2', 'timer2')
13  c = Clock()
14  c.timer()
15  c.timer()
16  Clock().timer() # 无法操作
17
18  @ui.page('/timer1', title='timer1')
19  def timer_1():
20      ui.label('timer1')
21      ui.link('index', '/')
22      c.timer()
23
24  @ui.page('/timer2', title='timer2')
25  def timer_2():
26      ui.label('timer2')
27      ui.link('index', '/')
28      Clock().timer() # 无法操作
29      Clock().timer() # 导致auto-index的可刷新方法无法操作
30
31  ui.run()
```

可以参考代码中添加了注释的代码，这些代码执行的可刷新方法是没法正常使用的。规律很简单，那就是只有最新执行可刷新方法所属的实例对象是正常，属于该对象的可刷新方法全部可以正常使用。

如果把实例方法改成类方法或者静态方法（特点和普通函数一样），那所有的可刷新方法都可以正常使用：

```
1  from nicegui import ui
2  from datetime import datetime
3
4  class Clock:
5      @classmethod
6      @ui.refreshable_method
7      def timer(cls):
8          time, set_time = ui.state('')
9          ui.label(f'Time: {time}')
10         ui.button('refresh', on_click=lambda :set_time(datetime.now()))
11
12  ui.link('timer1', 'timer1')
13  ui.link('timer2', 'timer2')
14  c = Clock()
15  c.timer()
16  c.timer()
17  Clock().timer()
18  Clock().timer()
19
20  @ui.page('/timer1', title='timer1')
21  def timer_1():
22      ui.label('timer1')
23      ui.link('index', '/')
24      c.timer()
25
26  @ui.page('/timer2', title='timer2')
27  def timer_2():
28      ui.label('timer2')
29      ui.link('index', '/')
30      Clock().timer()
31      Clock().timer()
32
33  ui.run()
```

为什么会出现这种情况？接下来，将问题代码稍做修改，就能发现问题所在：

```

1  from nicegui import ui
2  from datetime import datetime
3
4  class Clock:
5      @ui.refreshable_method
6      def timer(self):
7          time, set_time = ui.state('')
8          ui.label(f'Time from state: {time}')
9          ui.label(f'Time from now: {datetime.now()}') # 如果触发刷新，时间会更新
10         ui.button('refresh', on_click=lambda :set_time(datetime.now()))
11
12     ui.link('timer1', 'timer1')
13     ui.link('timer2', 'timer2')
14     c = Clock()
15     c.timer()
16     c.timer()
17     Clock().timer()
18
19     @ui.page('/timer1', title='timer1')
20     def timer_1():
21         ui.label('timer1')
22         ui.link('index', '/')
23         c.timer()
24
25     @ui.page('/timer2', title='timer2')
26     def timer_2():
27         ui.label('timer2')
28         ui.link('index', '/')
29         Clock().timer()
30         Clock().timer()
31
32     ui.run()

```

通过执行代码，终于发现了规律：所有的刷新按钮都与最新实例的可刷新方法关联，而非创建这些按钮的实例。那为什么前面介绍的不使用 `ui.state` 的实例方法就没有问题？

这里就不得不说 `ui.state` 内部的实现原理，实际上，同一个可刷新方法内的 `ui.state` 共享存放数值的容器，哪怕是重复执行可刷新方法时，也是在同一个容器内添加新的数值，因为容器绑定的是可刷新方法本身。因此，哪怕多次执行可刷新方法、存在多个 `ui.state`，它们的容器都是相同的，只是每个数值之间是独立的。

对于实例方法来说，这个容器不会在创建新的实例对象时重复创建，因此，不同实例对象实际上使用了相同的 `ui.state` 容器。但是，由于实例之间的隔离性，创建新的实例并执行可刷新方法后，`ui.state` 要执行可刷新方法的 `refresh` 方法时，并非执行与实例绑定的 `refresh` 方法，而是执行 `ui.state` 内部获取的 `refresh` 方法（每次创建新的实例对象后，此方法只能获取到最新实例对象可刷新方法的 `refresh` 方法）。因此，才会出现点击其他刷新按钮，只有最新的实例对象响应。

为了解决此问题，笔者绞尽脑汁学习框架代码，终于想到了一个临时可用的补丁（也许官方以后会修复此问题，也许此问题会成为一个特性）：

```
1  # ugly patch is here
2  from nicegui import ui
3  from nicegui.functions.refreshable import RefreshableContainer, RefreshableTarget
4  import copy
5  from typing import Callable, Generic, TypeVar
6  from typing_extensions import Concatenate, ParamSpec
7  _S = TypeVar('_S')
8  _T = TypeVar('_T')
9  _P = ParamSpec('_P')
10
11 class refreshable(ui.refreshable, Generic[_P, _T]):
12     def __call__(self, *args: _P.args, **kwargs: _P.kwargs) -> _T:
13         target = RefreshableTarget(
14             container=RefreshableContainer(),
15             refreshable=copy.copy(self),
16             instance=self.instance,
17             args=args, kwargs=kwargs
18         )
19         self.targets.append(target)
20         return target.run(self.func)
21
22 class refreshable_method(Generic[_S, _P, _T], refreshable[_P, _T]):
23     def __init__(self, func: Callable[Concatenate[_S, _P], _T]) -> None:
24         """Refreshable UI methods
```

```

25
26     The `@ui.refreshable_method` decorator allows you to create methods that have
    a `refresh` method.
27     This method will automatically delete all elements created by the function
    and recreate them.
28     """
29     super().__init__(func) # type: ignore
30 ui.refreshable = refreshable
31 ui.refreshable_method = refreshable_method
32
33 # ugly patch is over

```

在程序源码开头添加上面的代码之后，原来有问题的代码就被临时修复了：

```

1 # ugly patch is here
2 from nicegui import ui
3 from nicegui.functions.refreshable import RefreshableContainer, RefreshableTarget
4 import copy
5 from typing import Callable, Generic, TypeVar
6 from typing_extensions import Concatenate, ParamSpec
7 _S = TypeVar('_S')
8 _T = TypeVar('_T')
9 _P = ParamSpec('_P')
10
11 class refreshable(ui.refreshable, Generic[_P, _T]):
12     def __call__(self, *args: _P.args, **kwargs: _P.kwargs) -> _T:
13         target = RefreshableTarget(
14             container=RefreshableContainer(),
15             refreshable=copy.copy(self),
16             instance=self.instance,
17             args=args, kwargs=kwargs
18         )
19         self.targets.append(target)
20         return target.run(self.func)
21
22 class refreshable_method(Generic[_S, _P, _T], refreshable[_P, _T]):
23     def __init__(self, func: Callable[Concatenate[_S, _P], _T]) -> None:
24         """Refreshable UI methods

```



```

61     ui.label('timer2')
62     ui.link('index', '/')
63     Clock().timer()
64     Clock().timer()
65
66 ui.run()

```

还有一种更简单的修复补丁，就是针对 `ui.state` 进行修改。因为只是使用可刷新方法的话没有问题，一旦引入 `ui.state` 才会导致问题，那就使用下面的补丁修改 `ui.state`：

```

1  # patch is here
2  from nicegui import ui
3  from nicegui.functions.refreshable import RefreshableTarget
4  from typing import Any, Callable, Tuple, cast
5
6  def state(value: Any) -> Tuple[Any, Callable[[Any], None]]:
7      """Create a state variable that automatically updates its refreshable UI
8         container.
9
10         :param value: The initial value of the state variable.
11
12         :return: A tuple containing the current value and a function to update the value.
13         """
14
15         target = cast(RefreshableTarget, RefreshableTarget.current_target)
16
17         if target.next_index >= len(target.locals):
18             target.locals.append(value)
19         else:
20             value = target.locals[target.next_index]
21
22         def set_value(new_value: Any, index=target.next_index) -> None:
23             if target.locals[index] == new_value:
24                 return
25             target.locals[index] = new_value
26             target.refreshable.refresh(_instance=target.instance)
27
28         target.next_index += 1

```



```

32 from nicegui import ui
33 from datetime import datetime
34
35 class Clock:
36     @ui.refreshable
37     def timer(self):
38         time, set_time = ui.state('')
39         ui.label(f'Time from state: {time}')
40         ui.label(f'Time from now: {datetime.now()}') # 如果触发刷新，时间会更新
41         ui.button('refresh', on_click=lambda :set_time(datetime.now()))
42
43 ui.link('timer1', 'timer1')
44 ui.link('timer2', 'timer2')
45 c = Clock()
46 c.timer()
47 c.timer()
48 Clock().timer()
49
50 @ui.page('/timer1', title='timer1')
51 def timer_1():
52     ui.label('timer1')
53     ui.link('index', '/')
54     c.timer()
55
56 @ui.page('/timer2', title='timer2')
57 def timer_2():
58     ui.label('timer2')
59     ui.link('index', '/')
60     Clock().timer()
61     Clock().timer()
62
63 ui.run()

```

最后结合 `ui.state`，总结一下可刷新方法中不同控件的状态共享情况：

可刷新方法的类别	与 <code>ui.state</code> 无关的控件	与 <code>ui.state</code> 相关的控件
普通函数 类方法 静态方法	共享情况取决于可刷新方法的定义位置（ <code>auto-index</code> 或者 <code>page</code> ）： 1, 在 <code>auto-index</code> 中定义，则全局共享 2, 在 <code>page</code> 中定义，则只在相同的 <code>page</code> 中共享	互相独立
实例方法	不同实例之间互相独立，相同实例之间共享	互相独立

2.3.7.5 运行JavaScript代码 `ui.run_javascript`

NiceGUI既然是基于Web的UI，免不了与网页三剑客HTML、CSS、JavaScript打交道。前面提到过HTML的标签，也讲过了CSS的美化语法，JavaScript如何执行却没说过。现在，终于到了与JavaScript交互的时候。

`ui.run_javascript` 可以运行任意JavaScript代码，方便在Python无法处理的时候，补充使用JavaScript的方法。当然，JavaScript毕竟是一门设计时就很仓促的语言，时至今日，哪怕ES标准正在解决JavaScript留下的烂摊子，也避免不了JavaScript糟糕的历史包袱。对于各位读者来说，非必要还是别执行JavaScript代码，但是这个执行的方法，不能不会。

```
1 from nicegui import ui
2
3 ui.button('js',on_click=lambda :ui.run_javascript(code='alert("Hello World from
  JavaScript!")',timeout=1))
4
5 ui.run(native=True)
```

字符串类型参数 `code` 就是要执行的JavaScript代码，浮点类型参数 `timeout` 是超时，有些JavaScript方法的执行时间比较长，可以修改这个参数来避免JavaScript执行触发超时而导致结果返回失败。

2.3.8 网站页面

2.3.8.1 `ui.page`

在继续下面的内容前，各位需要了解一个NiceGUI的概念——page页面。前面讲了不少控件，想要访问这些控件组成的页面，通常直接访问弹出的native mode窗口，当然，也可以直接在浏览器里输入地址。

但是，细心的读者就会发现，当用户打开多个浏览器页面访问这个地址或者多个用户打开同一地址的时候，任意一个页面进行交互，其他页面都会有反应，和印象中的网站页面不太一样，正常的网站页面不应该这样。

没错，前面的控件其实都布局在auto-index的页面下，也就是网站的根目录，即网址后面只有一个斜杠或者没有斜杠的情况（`http://127.0.0.1:8000/` 或者 `http://127.0.0.1:8000`）。前面提到过一个概念叫上下文，这里想要快速理解页面的概念的话，还是要正确划分上下文关系，后面才能更好应用页面设计。

默认情况下，没有添加 `ui.page` 装饰器的话，整个程序内的控件都在auto-index的页面下；即使添加了 `ui.page` 装饰器，没有在装饰器修饰的函数的上下文内，也会自动划分到auto-index的页面。

理解auto-index之后，现在，可以正式介绍 `ui.page` 了。

不同于auto-index的页面是多人共享，`ui.page` 产生的页面是私有的，即每个人在打开的页面交互，不会影响其他人打开的页面。

以下面的代码为例：

```
1  from nicegui import ui
2  from uuid import uuid4
3
4  @ui.page(path='/private_page',title='private_page')
5  def private_page():
6      ui.label(f'private page with ID {uuid4()}')
7
8  ui.label(f'shared auto-index page with ID {uuid4()}')
9  ui.link('private page', private_page)
10
11 ui.run(port=8000)
```

访问 `http://127.0.0.1:8000/` 时看到的uuid64和访问 `http://127.0.0.1:8000/private_page` 时看到的uuid是不一样的，`http://127.0.0.1:8000/private_page` 是 `ui.page` 产生的，属于私有页面，每次打开（可以尝试刷新）之后，`uuid4()` 都会运行一次，因此uuid会重新生成。而 `http://127.0.0.1:8000/` 下的uuid是程序启动之后就生成了，`uuid4()` 只在程序启动时运行一次，因此每次打开 `http://127.0.0.1:8000/` 显示的uuid是一样的。如果想要 `http://127.0.0.1:8000/` 下的uuid和 `http://127.0.0.1:8000/private_page` 一样，就要创建一个 `path='/'` 的 `ui.page`。

创建 `ui.page` 的方法很简单，使用Python中的装饰器，在 `@ui.page(path='/private_page',title='private_page')` 的下一行定义一个函数，函数的内部就是 `ui.page` 的上下文。

`ui.page` 默认有很多参数，这里只讲常用的 `path` 和 `title` 。

`path` 参数，字符串类型，表示这个 `ui.page` 对应的服务器地址，即 `http://127.0.0.1:8000` 后面的部分。

`title` 参数，字符串类型，表示这个 `ui.page` 的浏览器窗口标题。除了在定义 `ui.page` 时候设置标题外，还可以调用 `ui.page_title(title='title')` 来设置当前页的浏览器窗口标题。

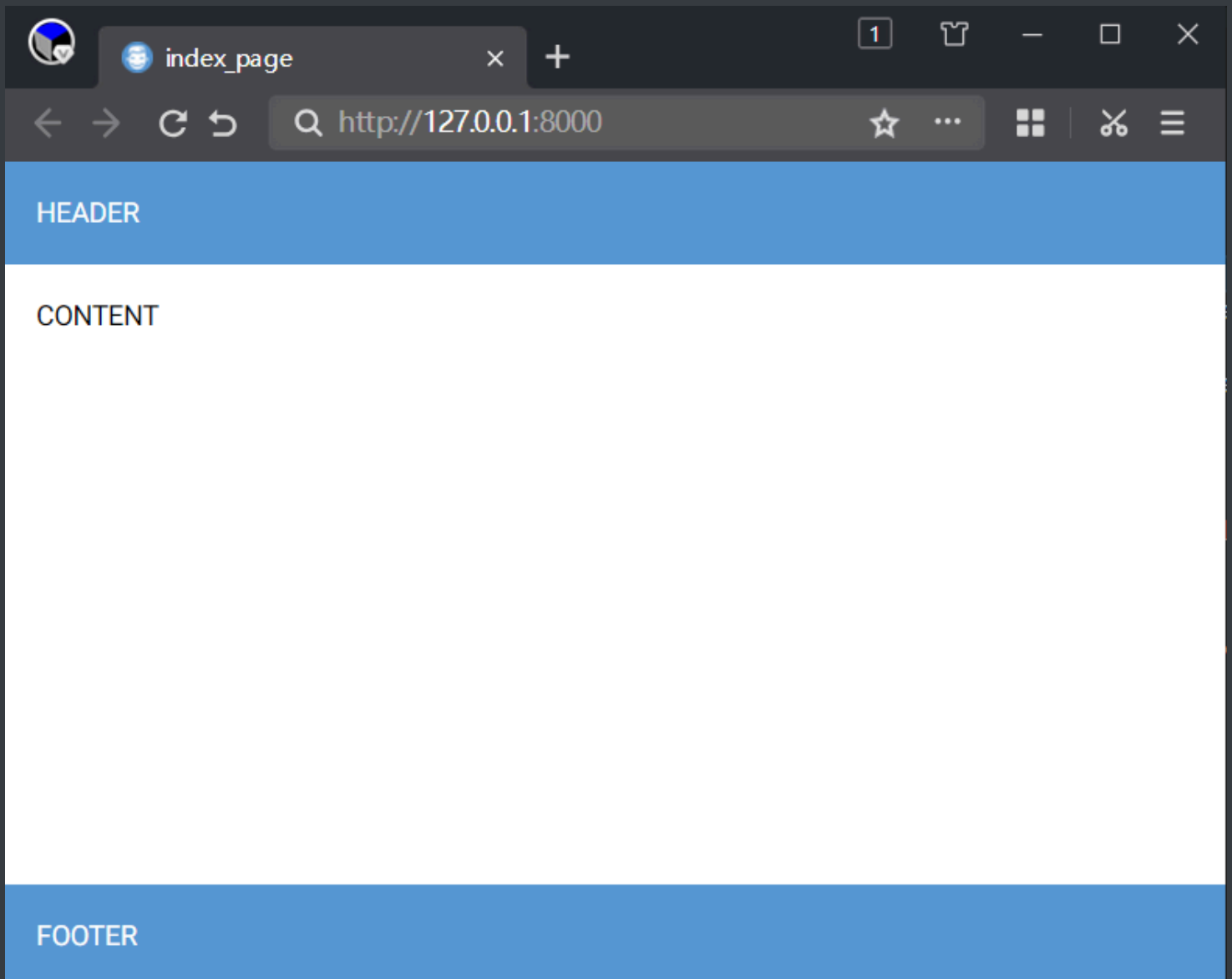
2.3.8.2 页面布局

尽管前面已经说过图形布局，这里还是有必要介绍一下页面布局。不同于图形布局能被嵌套在其他布局、控件内，页面布局的上一层只能是 `ui.page`，或者说页面布局不受上层布局影响（专指 `ui.page_sticky`，虽然放在图形布局下不报错，但图形布局并不会生效）。

`ui.header` 和 `ui.footer`：

标题和页脚，就是浮动在页面最上面和最下面的区域，就像报纸的页眉和页脚一样，不过这里使用的单词是 `header`，所以称之为标题。这两个控件的代码参数只有一处差异，不过一般不怎么用。平时用的最多的还是相同的几个参数，这里只说布尔类型参数 `value`，这个参数决定了这两个布局的显示状态，可以通过初始化传参来决定默认状态，也可以通过后续使用 `set_value` 修改这个值来显示、隐藏，使用 `set_visibility` 也能达到同样的效果，`toggle` 可以自动切换显示、隐藏。

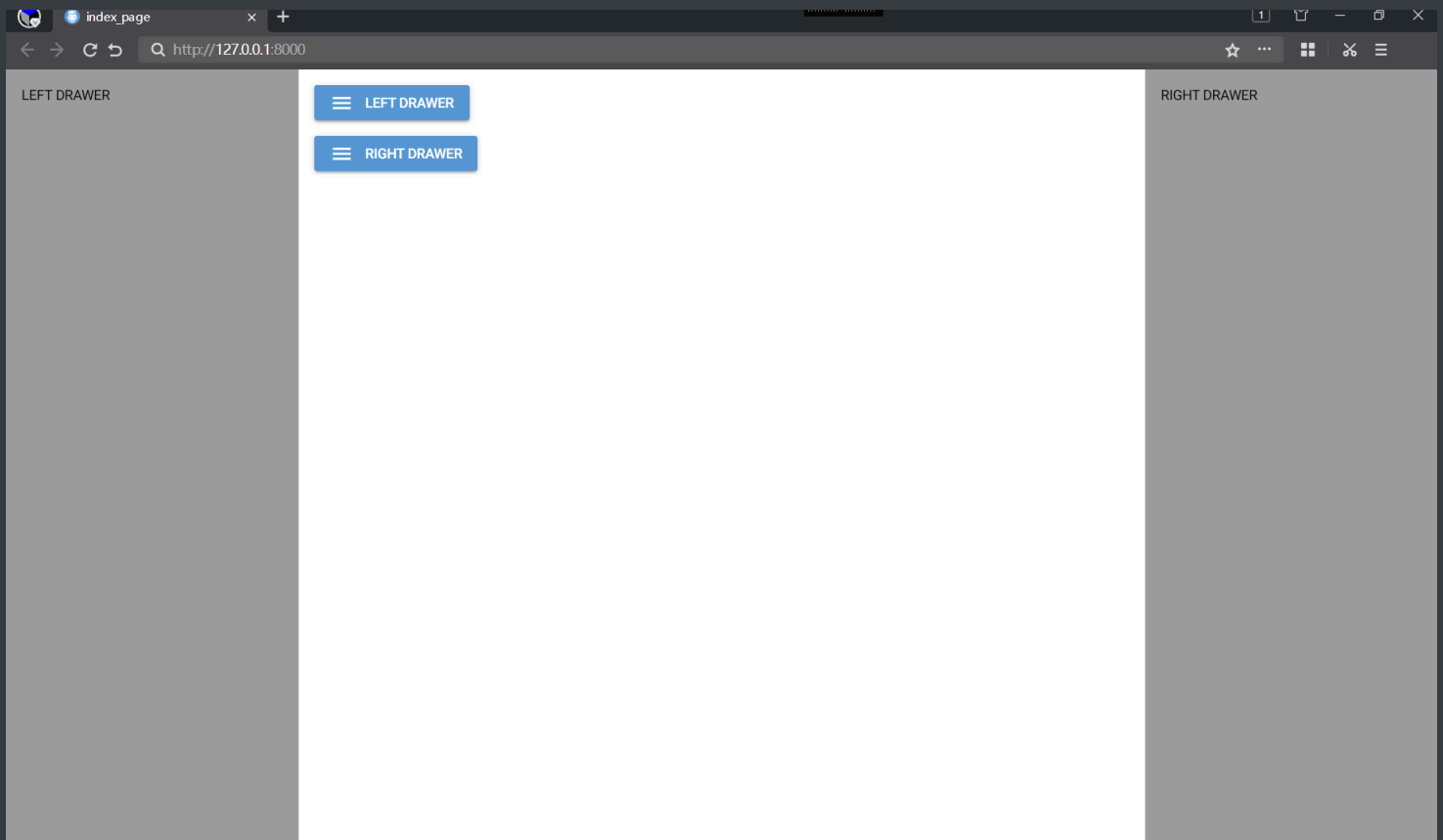
```
1 from nicegui import ui
2
3 @ui.page(path='/', title='index_page')
4 def page_index():
5     ui.label('CONTENT')
6     with ui.header(value=True, fixed=True, bordered=False, elevated=False, wrap=True,
7         add_scroll_padding=False).classes('items-center'):
8         ui.label('HEADER')
9     with ui.footer(value=True, fixed=True, bordered=False, elevated=False,
10         wrap=True):
11         ui.label('FOOTER')
```



`ui.left_drawer` 和 `ui.right_drawer` :

左右抽屉，其实这两个布局是基于 `ui.drawer` 的包装，呈现出来就是左右边栏。既然是基于同一控件的包装，参数自然是一样的。与标题和页脚一样，主要关注的还是参数 `value`，含义一样是表示控件的显示、隐藏。

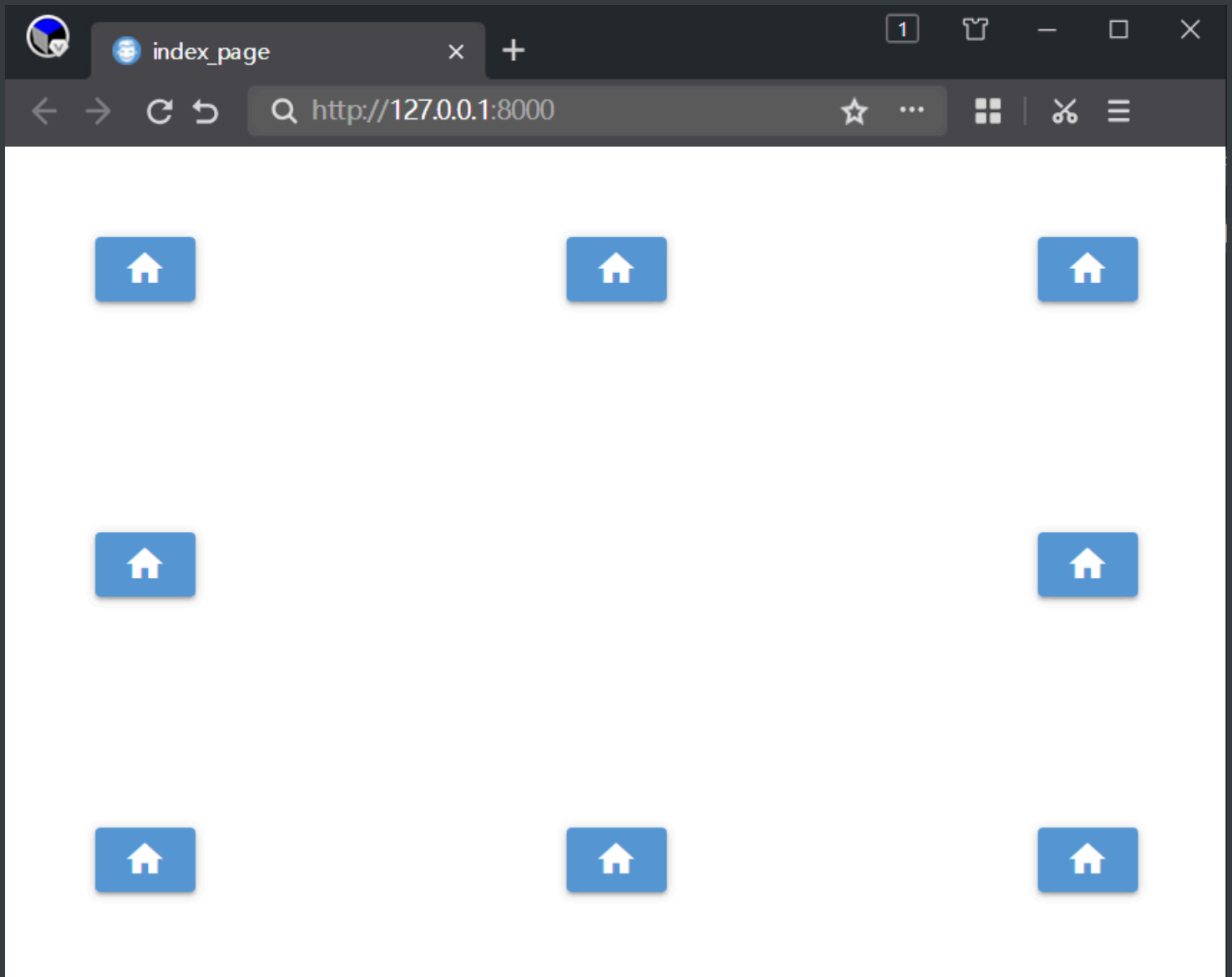
```
1 from nicegui import ui
2
3 @ui.page(path='/', title='index_page')
4 def page_index():
5     with ui.left_drawer(value=True, fixed=True, bordered=False, elevated=False,
6         top_corner=False, bottom_corner=False).classes('bg-grey') as left_drawer:
7         ui.label('LEFT DRAWER')
8     with ui.right_drawer(value=True, fixed=True, bordered=False, elevated=False,
9         top_corner=False, bottom_corner=False).classes('bg-grey') as right_drawer:
10         ui.label('RIGHT DRAWER')
11     ui.button(on_click=lambda: left_drawer.toggle(),
12         icon='menu',text='left drawer')
13     ui.button(on_click=lambda: right_drawer.toggle(),
14         icon='menu',text='right drawer')
15
16 ui.run(port=8000)
```



`ui.page_sticky` :

便签，这是从字面意义上看，实际作用也和便签类似，放在这个布局内的内容，会出现在侧边或者角落，就像贴在显示器的便签一样。 `ui.page_sticky` 的参数里，用得最多的是：字符串参数 `position` 表示位置，浮点类型参数 `x_offset` 和 `y_offset` 表示XY方向的偏移。

```
1 from nicegui import ui
2
3 @ui.page(path='/', title='index_page')
4 def page_index():
5     PageStickyPositions = ['top-right', 'top-left', 'bottom-right',
6                             'bottom-left', 'top', 'right', 'bottom', 'left']
7     for i in PageStickyPositions:
8         with ui.page_sticky(position=i, x_offset=50, y_offset=50, expand=False):
9             ui.button(icon='home')
10
11 ui.run(port=8000)
```



2.3.8.3 ui.navigate

有了 `ui.page` 之后，访问不同的地址代表不同的页面，可以使用之前的 `ui.link` 来创建指向不同页面的超链接。那么，有没有方法来跳转不同页面而不用明显地摆出超链接？

也有，那就是这一节要说的 `ui.navigate`。

`ui.navigate` 提供了四个方法，分别是 `back` 后退、`forward` 前进、`reload` 重新载入、`to` 跳转到。除了跳转到之外，其他方法都没有参数，跳转到的参数和 `ui.link` 一样，具体可以参考前面的介绍。

```

1  from nicegui import ui
2
3  with ui.row():
4      ui.button('Back', on_click=ui.navigate.back)
5      ui.button('Forward', on_click=ui.navigate.forward)
6      ui.button('Reload', on_click=ui.navigate.reload)
7      ui.button(text='baidu', icon='home',
8                on_click=lambda: ui.navigate.to('https://www.baidu.com/'))
9
10 ui.run(port=8000)

```

`ui.navigate` 的 `history` 属性表示历史记录，执行此属性的 `push` 方法可以在历史记录中添加一条历史记录并将当前地址修改为指定地址，执行此属性的 `replace` 方法则只是将当前地址修改为指定地址。这两个方法都有一个字符串类型参数 `url`，表示省略网站主机地址（即 `https://127.0.0.1:8080/` 中的 `https://127.0.0.1:8080`）之后绝对路径或者相对路径。这两个方法修改当前地址仅仅是修改地址栏的显示，并不会真的跳转，执行 `ui.navigate` 的 `reload` 方法重新载入时，才会跳转到当前地址。

```

1  from nicegui import ui
2
3  ui.button('push', on_click=lambda :ui.navigate.history.push('push'))
4  ui.button('replace', on_click=lambda :ui.navigate.history.replace('replace'))
5  ui.button('reload', on_click=ui.navigate.reload())
6
7  ui.run()

```

2.3.8.4 `ui.download`

除了用 `ui.link` 提供链接让用户点击下载文件之外，还可以调用 `ui.download` 来触发下载。

`ui.download` 有三个参数，常用的是前两个参数：字符串类型或者字节类型参数 `src` 来源，字符串类型参数 `filename` 文件名。

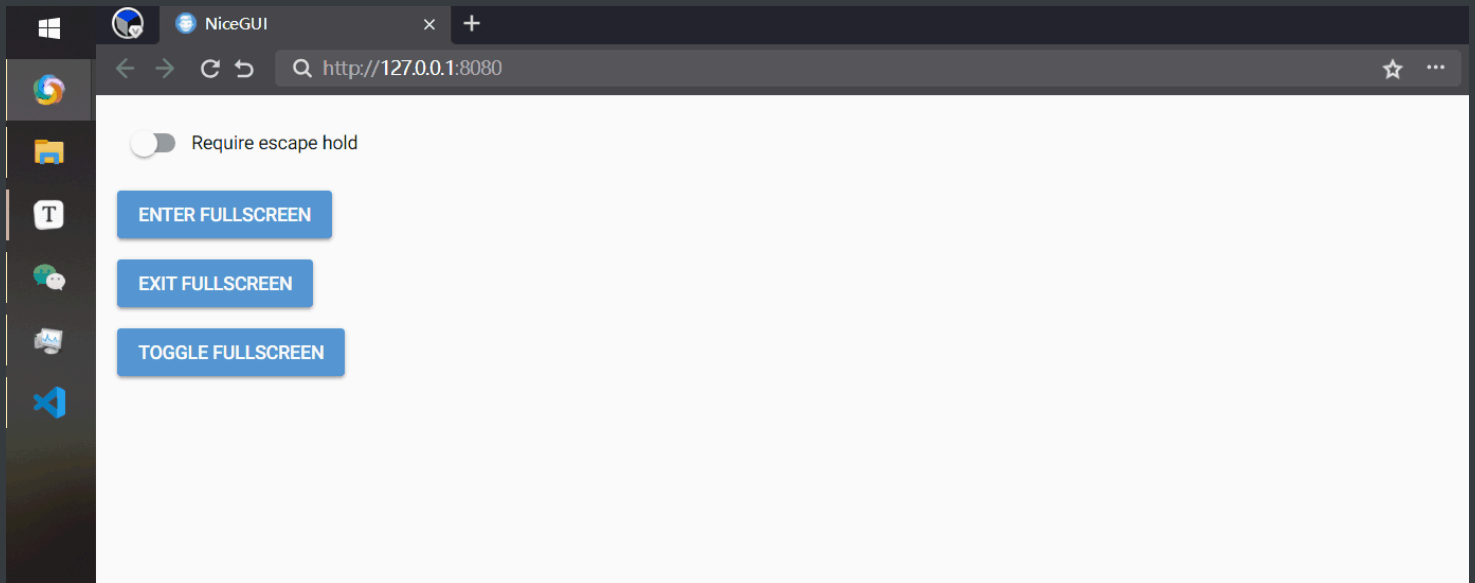
```
1 from nicegui import ui,app
2
3 app.native.settings['ALLOW_DOWNLOADS'] = True
4 ui.button('Logo', on_click=lambda:
    ui.download(src='https://nicegui.io/logo.png',filename='logo.png',media_type=''))
5 ui.button('Download', on_click=lambda: ui.download(src=b'Hello
    World',filename='hello.txt',media_type=''))
6
7 ui.run(port=8000)
```

因为 pywebview 默认禁止下载，在测试上面的代码时，需要设置 `app.native.settings['ALLOW_DOWNLOADS']` 为 `True` 来允许下载。

2.3.8.5 ui.fullscreen (2025.02.05更新)

如果想让浏览器全屏，除了使用快捷键 `f11`，用得最多的就是使用JavaScript代码。不管是哪种方法，都不如通过NiceGUI的2.11.0版本新增的 `ui.fullscreen` 进入全屏方便。先看代码：

```
1 from nicegui import ui
2
3 fullscreen = ui.fullscreen()
4
5 ui.switch('Require escape hold').bind_value_to(fullscreen, 'require_escape_hold')
6
7 ui.button('Enter Fullscreen', on_click=fullscreen.enter)
8 ui.button('Exit Fullscreen', on_click=fullscreen.exit)
9 ui.button('Toggle Fullscreen', on_click=fullscreen.toggle)
10
11 ui.run()
```



`ui.fullscreen` 支持两个参数：

- `require_escape_hold` 参数，布尔类型，表示退出全屏时是否需要长按 `esc` 键，默认为 `False`，即短按 `esc` 键即可退出。注意，只有谷歌的Chrome浏览器和其他基于chromium开发的浏览器（比如微软的Edge浏览器和大部分国产浏览器）支持长按退出全屏的操作。另外，此参数实际上对应的是 `ui.fullscreen` 的 `require_escape_hold` 属性，如果想要动态启用、禁用长按退出全屏的功能，可以在非全屏状态下修改此属性。但是，在启用时，此功能是即时生效的；如果在启用之后进入了一次全屏，禁用之后则需要刷新页面才能生效。
- `on_value_change` 参数，可调用类型，表示当 `ui.fullscreen` 的 `value` 属性变化时，执行什么操作。`ui.fullscreen` 的 `value` 属性的值实际上就是当前是否为全屏状态的布尔值，不过并不能通过直接设置此值来切换全屏状态。因为浏览器的安全设置，想要通过此控件切换全屏状态，必须要用户手动触发，即用户点击了控件之后，此控件将此值设置为其他布尔值，才能切换全屏状态。如果想要默认进入全屏状态，需要设置运行参数 `fullscreen`，比如 `ui.run(fullscreen=True)`。

此外，需要额外注意的是，此控件只能用于浏览器，后面提到的native mode不能通过此控件切换全屏。

`ui.fullscreen` 支持以下方法，可以让用户通过与控件交互之后的触发动作调用这些方法，来实现对全屏状态的控制：

- `enter` 方法，进入全屏状态。
- `exit` 方法，退出全屏状态。
- `toggle` 方法，切换全屏状态。

2.3.9 运行配置

2.3.9.1 ui.run

几乎每个例子都要用的 `ui.run` 有非常多的参数，可基础内容都快讲完也不说说参数的话，多少有点说不过去。接下来，针对 `ui.run` 里常用的几个参数说一下。

`host` 参数，字符串类型，表示整个程序监听的ip，默认情况下是 `'0.0.0.0'`，即监听全部可用的本机ip，如果是native mode下，这个值默认为 `'127.0.0.1'`，即只监听本地ip。

`port` 参数，整数类型，表示程序监听的端口。

`title` 参数，字符串类型，表示程序启动后的默认窗口标题。

`show` 参数，布尔类型，表示程序启动后是否自动唤起默认浏览器打开程序的地址，如果同时开启了native mode的话，这个参数会被强制禁用。

`native` 参数，布尔类型，表示是否启用本地窗口模式，即调用 `pywebview` 来显示NiceGUI程序，看上去就和桌面程序一样。

`reload` 参数，布尔类型，表示是否启用热重载。不知道各位读者有没有注意到，之前都没有设置这个参数，但是修改了程序并保存的话，保存都是很快就生效，不需要手动重启程序。没错，这个参数默认为 `True`，即程序修改会触发程序自动重启，让界面修改即使体现，这一点倒是方便开发的时候调试界面。不过，这个参数在非开发过程中最好设置为 `False`，以免频繁触发重启。

2.3.9.2 native mode

前面很多例子都是在native mode本地模式下调试，其实，native mode相关的设置也有很多需要注意的地方，下面就说一说常用的几个。

以代码为例：

```
1 from nicegui import app, ui
2
3 app.native.window_args['resizable'] = False
4 app.native.start_args['debug'] = True
5 app.native.settings['ALLOW_DOWNLOADS'] = True
6
7 ui.label('app running in native mode')
8 ui.button('enlarge', on_click=lambda: app.native.main_window.resize(1000, 700))
9 ui.button('title', on_click=lambda: app.native.main_window.set_title('main window'))
10
11 ui.run(native=True, window_size=(400, 300), fullscreen=False, title='main')
```

默认本地模式窗口可以自由调整窗口大小，但是有些时候界面没做大窗口的布局，不喜欢用户调整窗口大小或者最大化，可以设置 `app.native.window_args['resizable']` 为 `False`，禁用本地窗口的大小调整功能。

有能力的读者肯定试过在浏览器窗口中启用开发者调试工具来调整界面效果。但是，在本地窗口中，虽然本质上也是浏览器，但想要调试界面效果的话，还要额外开个浏览器窗口，总觉得有些麻烦，没关系，本地窗口也是可以启用开发者工具的。将 `app.native.start_args['debug']` 设置为 `True`，在启动本地窗口的同时，还会弹出一个开发者工具，和在普通浏览器里一样，方便同步调试界面。

在前面介绍过 `ui.download`，这个功能是程序化控制下载文件的。但是，有的读者可能遇到过在本地窗口下，这个功能好像失效了，以为是NiceGUI的bug。其实不然，这个是因为 `pywebview` 默认禁止下载，可以设置 `app.native.settings['ALLOW_DOWNLOADS']` 为 `True` 来允许下载。

前面讲了禁用窗口大小调整的方法，但是，如果在这种情况下，还想控制窗口大小的变化，或者在没有禁用的情况下自由控制窗口大小，有没有办法？有，使用 `app.native.main_window.resize` 方法即可，参数就是窗口的宽度和高度。

在 [ui.page](#) 一节介绍过 `ui.page_title` 方法来设置窗口标题，有没有一种方法设置窗口标题但是只针对本地窗口有效？有，`app.native.main_window.set_title` 方法只设置本地窗口标题，不会影响浏览器的窗口标题。其实，`ui.page_title` 方法里默认调用了一次 `app.native.main_window.set_title` 方法，如果没有特殊需求，只用 `ui.page_title` 方法更简单。

3 高阶技巧

NiceGUI的控件有很多，日常开发中，除了了解常用控件之外，不常用的控件也可以在学有余力的时候看看。当然，图形界面的开发不止对控件的了解，一些逻辑上的处理技巧，Python语言的特性与框架的结合，也是难免会遇到的难题。不过，不用怕，授人以鱼不如授人以渔，日常能遇到、能解决的难题，这里都有。

3.1 with 的技巧

`with` 可以嵌套使用，来实现类似HTML中 `div` 嵌套的效果，比如：

```
1 from nicegui import ui
2
3 with ui.element('div') as div1:
4     with ui.element('div') as div2:
5         ui.label('div in div')
6
7 ui.run(native=True)
```

也可以缩减一行，让代码更加紧凑：

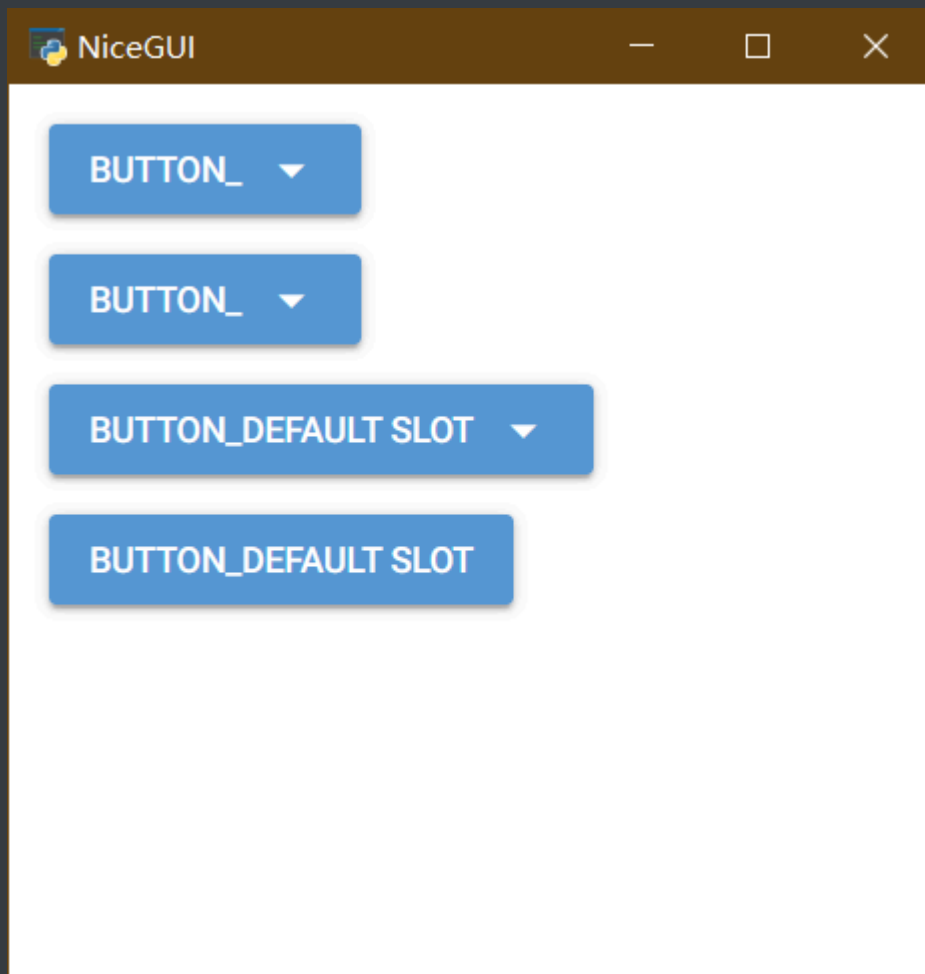
```
1 from nicegui import ui
2
3 with ui.element('div') as div1, ui.element('div') as div2:
4     ui.label('div in div')
5
6 ui.run(native=True)
```

3.2 slot的技巧

其实，所有的 `with element` 都是修改了 `element` 中名为 `'default'` 的slot。基于这个操作原理，可以借用 `add_slot` 的方法，结合 `with` 的用法，优雅、快捷地美化元素，实现复杂的布局。

比如，`ui.dropdown_button` 有两个slot，`default` 和 `label`；其中，`default` 就是默认的slot，常规方法就可以嵌入元素到弹出的下拉列表里，如果想要像修改 `ui.button` 一样修改 `ui.dropdown_button` 本身，则要修改 `ui.dropdown_button` 的 `'label'` 这个slot，代码如下：

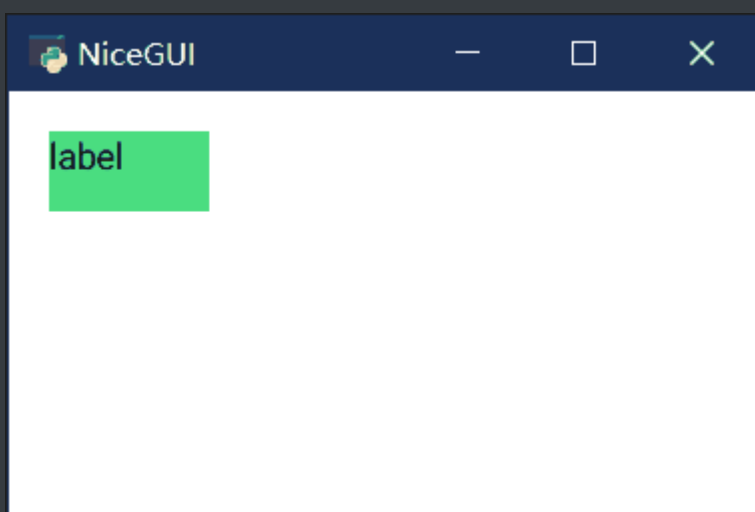
```
1 from nicegui import ui
2
3 with ui.dropdown_button('button_'):
4     ui.label('default slot')
5 #和上面的代码相同，主要是为了和下面的代码对比
6 with ui.dropdown_button('button_').add_slot('default'):
7     ui.label('default slot')
8 #修改另一个slot，可以查看不同的效果
9 with ui.dropdown_button('button_').add_slot('label'):
10     ui.label('default slot')
11 #可以对比 dropdown_button 和 button 的显示效果
12 with ui.button('button_').add_slot('default'):
13     ui.label('default slot')
14
15 ui.run(native=True)
```



3.3 TailWindCSS的技巧

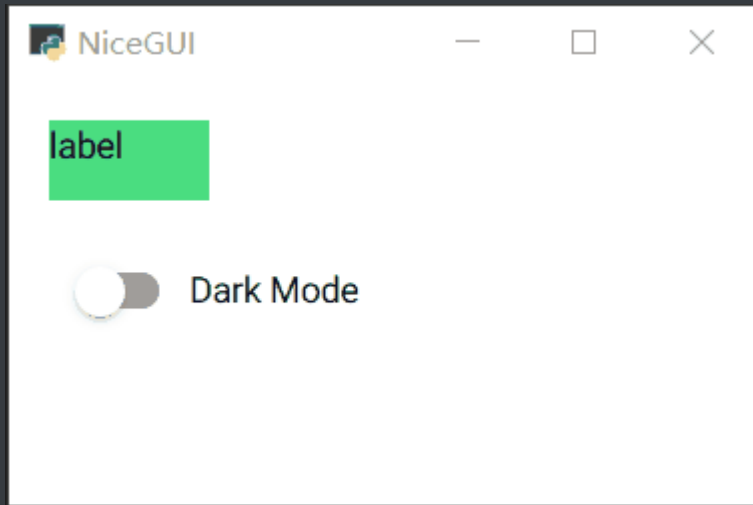
不同于CSS定义中伪类在冒号之后来定义效果，在TailWindCSS中，美化悬停（`hover`）和激活（`active`），需要放在冒号之前，冒号后紧随着要对状态应用的效果。比如，要实现标签背景颜色的悬停为红色、点击为黄色，代码如下：

```
1 from nicegui import ui
2
3 ui.label('label').classes('w-16 h-8 bg-green-400 hover:bg-red-400 active:bg-yellow-400')
4
5 ui.run(native=True)
```



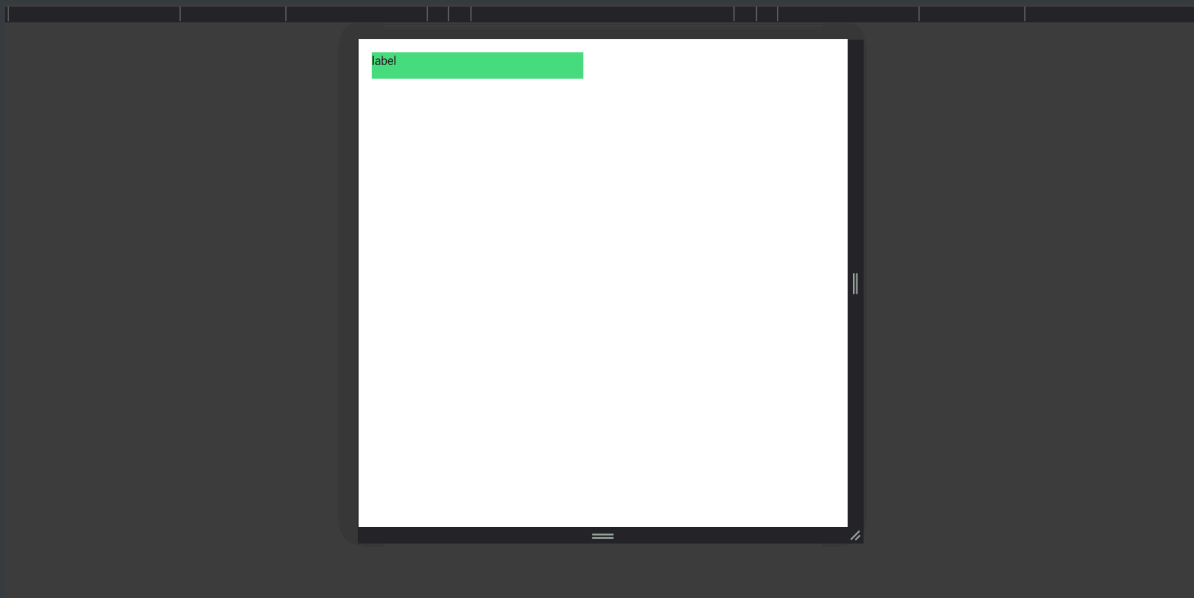
类似的，还可以实现暗黑模式（`dark`）下的颜色定义，点击 `switch` 来切换暗黑模式的开关，可以看到标签在暗黑模式下的背景颜色为红色，非暗黑模式下的背景颜色为绿色，代码如下：

```
1 from nicegui import ui
2
3 ui.label('label').classes('w-16 h-8 bg-green-400 dark:bg-red-400')
4 dark_mode = ui.dark_mode()
5 switch = ui.switch('Dark Mode', on_change=lambda :dark_mode.set_value(switch.value))
6
7 ui.run(native=True)
```



在此基础上，还有一种根据屏幕宽度调整显示的技巧，就是将冒号前的单词换成代表屏幕宽度的断点 `sm`、`md`、`lg`、`xl`、`2xl`。如果要让标签的宽度随窗口大小变化自适应，也就是小窗口宽度小一些，窗口越大，宽度越大，那么，代码可以这样写：

```
1 from nicegui import ui
2
3 ui.label('label').classes('w-64 h-8 bg-green-400 sm:w-8 md:w-16 lg:w-32')
4
5 ui.run(native=True)
```



然而，运行之后，可以看到上面的代码其实有问题，按照理解这样写是没错，但断点代表的含义是，大于这个屏幕宽度值才会应用这个样式，而且一次写这么多条，等屏幕宽度同时符合两条以上条件的时候，CSS就会处于竞争选择的状态，虽然样式上表现可能没问题，但规范要求应该明确断点范围，就好像写分段函数一样，必须明确区间。

所以，正确的根据屏幕宽度使用不同的样式应该这样写。使用 `max-*` 来表示最大到什么大小使用什么样式，使用冒号表示区间范围。于是，可以用 `sm:max-md:w-16` 来表示 `sm` 到 `md` 的范围内使用 `w-16` 的宽度样式，具体代码如下：

```
1 from nicegui import ui
2
3 ui.label('label').classes('h-8 bg-green-400 max-sm:w-8 sm:max-md:w-16 md:max-lg:w-32
   lg:w-64')
4
5 ui.run(native=True)
```

3.4 自定义控件

3.4.1 通过继承NiceGUI现有控件来创建新控件

在Python中，可以通过继承来扩展现有类的功能，这个操作对于NiceGUI同样适用。

如果想要基于 `ui.button` 实现一个可以通过点击切换颜色的按钮，可以这样做：

继承现有的控件类 `ui.button`，先在 `__init__` 内调用父类的初始化方法；然后增加 `_state` 属性，默认为 `False`，用于保存状态；最后定义点击事件的响应调用自身的 `toggle` 方法。

增加 `toggle` 方法，在方法内实现每次调用就翻转 `_state` 属性，并调用自身的 `update` 方法来更新显示。

重写 `update` 方法，先要根据 `_state` 属性设定button的显示颜色（动态更新 `color` 属性，详见Quasar提供的API），调用父类的 `update` 方法更新显示。

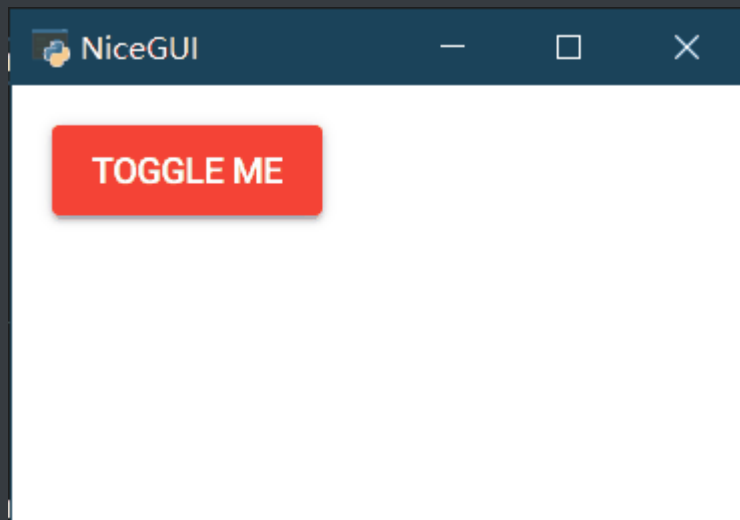
代码如下：

```
1 from nicegui import ui
2
3 class ToggleButton(ui.button):
4
5     def __init__(self, *args, **kwargs) -> None:
6         super().__init__(*args, **kwargs)
7         self._state = False
8         self.on('click', self.toggle)
```

```

9
10     def toggle(self) -> None:
11         """Toggle the button state."""
12         self._state = not self._state
13         self.update()
14
15     def update(self) -> None:
16         self.props(f'color={"green" if self._state else "red"}')
17         super().update()
18
19 ToggleButton('Toggle me')
20
21 ui.run(native=True)

```



3.4.2 使用Quasar的标签定义新控件

如果想要实现的功能比较复杂，但是Quasar提供了NiceGUI没有实现的组件，还有一种简单的方法创建新控件。

Quasar有一个浮动功能按钮[Floating Action Button](#)，但NiceGUI没有实现。浮动功能按钮在Quasar的使用代码是：

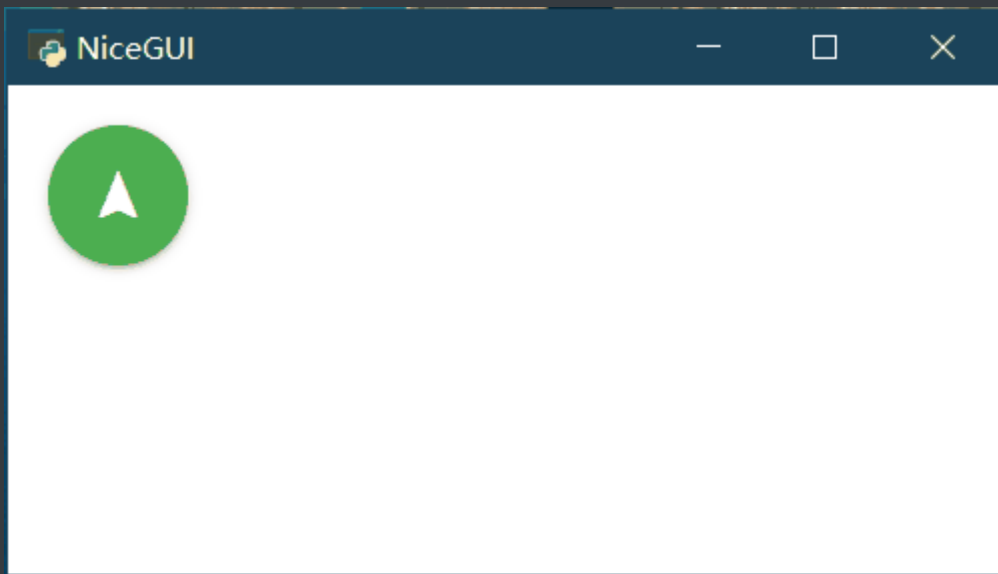
```

1 <q-fab color="green" icon="navigation" >
2     <q-fab-action color="green-5" icon="train" />
3     <q-fab-action color="green-5" icon="sailing" />
4     <q-fab-action color="green-5" icon="rocket" />
5 </q-fab>

```


对应地，将HTML标签嵌套关系转换为Python代码，`q-fab` 标签就变成了 `ui.element('q-fab')`，代码如下：

```
1 from nicegui import ui
2
3 with ui.element('q-fab').props('icon=navigation color=green'):
4     ui.element('q-fab-action').props('icon=train color=green-5').on('click', lambda:
5         ui.notify('train'))
6     ui.element('q-fab-action').props('icon=sailing color=green-5').on('click', lambda:
7         ui.notify('boat'))
8     ui.element('q-fab-action').props('icon=rocket color=green-5').on('click', lambda:
9         ui.notify('rocket'))
10
11 ui.run(native=True)
```



3.4.3 使用VUE自定义新控件（2025.01.08更新）

如果有VUE基础，可以结合VUE创建新的控件，在VUE中定义界面和部分交互，比在Python中更自由。由于笔者不擅长VUE，以下来自[官方示例](#)的代码就不做完全详细的解释，只简单说一下思路。

`counter.js` 内容为：

```
1 // NOTE: Make sure to reload the browser with cache disabled after making changes to
2   this file.
3 export default {
4   template: `
```

```

4   <button @click="handle_click">
5       <strong>{% raw %}{{title}}: {{value}}{% endraw %}</strong>
6   </button>`,
7   data() {
8       return {
9           value: 0,
10      };
11  },
12  methods: {
13      handle_click() {
14          this.value += 1;
15          this.$emit("change", {value:this.value});
16      },
17      reset() {
18          this.value = 0;
19      },
20  },
21  props: {
22      title: String,
23  },
24  };

```

counter.py 内容为:

```

1  from typing import Callable, Optional
2  from nicegui.element import Element
3
4  class Counter(Element, component='counter.js'):
5      def __init__(self, title: str, *, on_change: Optional[Callable] = None) -> None:
6          super().__init__()
7          self.props['title'] = title
8          self.on('change', on_change)
9      def reset(self) -> None:
10         self.run_method('reset')

```

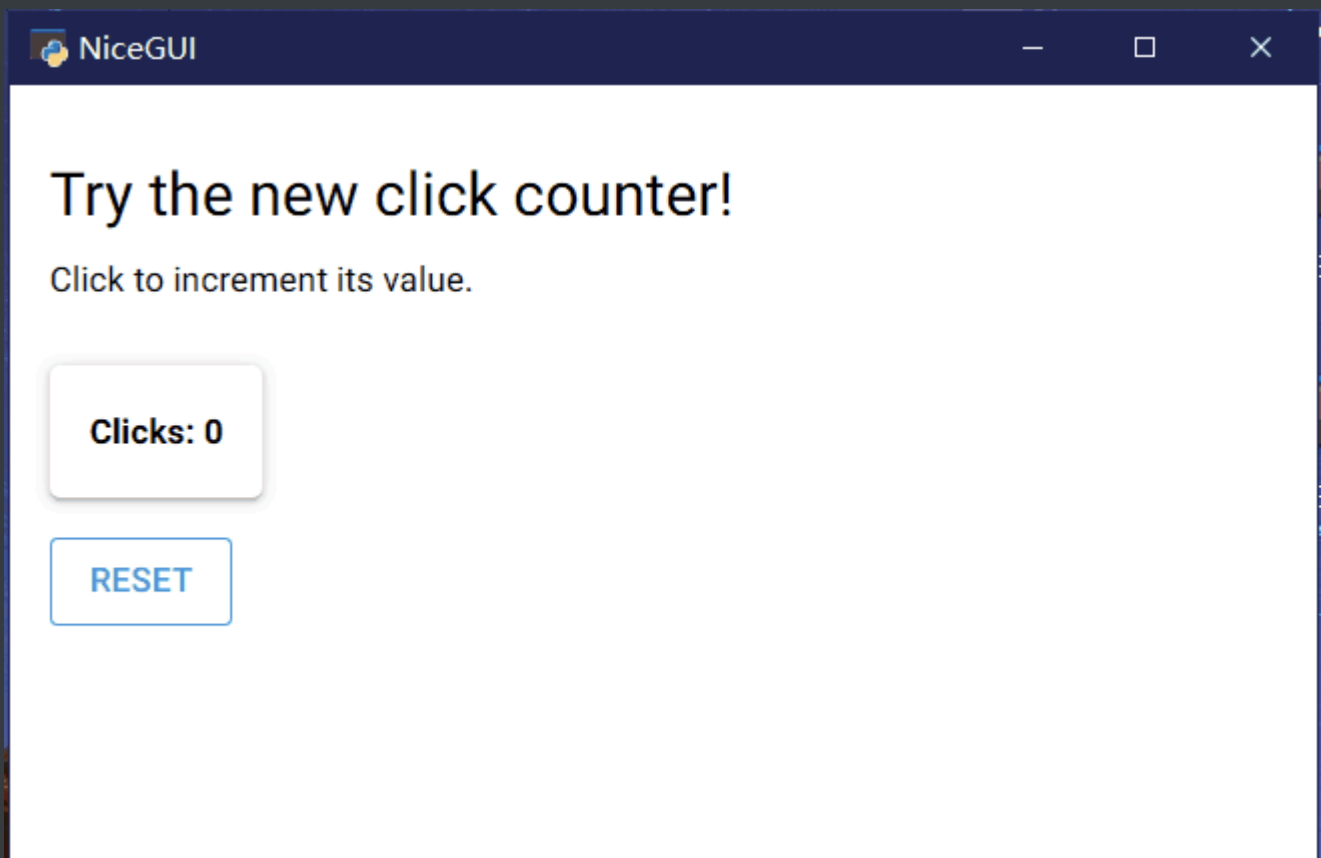
测试代码为:

```

1  from counter import Counter
2  from nicegui import ui
3
4  ui.markdown('''
5  ##### Try the new click counter!
6
7  Click to increment its value.
8  ''')
9  with ui.card():
10     counter = Counter('Clicks', on_change=lambda e: ui.notify(f'The value changed to {e.args['value']}.'))
11
12  ui.button('Reset', on_click=counter.reset).props('small outline')
13
14  ui.run(native=True)

```

自定义控件的核心在 `counter.js` 文件中，由VUE暴露需要用到的属性和JavaScript方法，在 `counter.py` 文件中通过 `props` 属性接收和设置暴露的属性，使用 `run_method` 方法执行暴露出的JavaScript方法。如果在 `counter.js` 文件中发射（`$emit`）了事件，还可以在 `counter.py` 文件中使用 `on` 方法响应对应的事件。

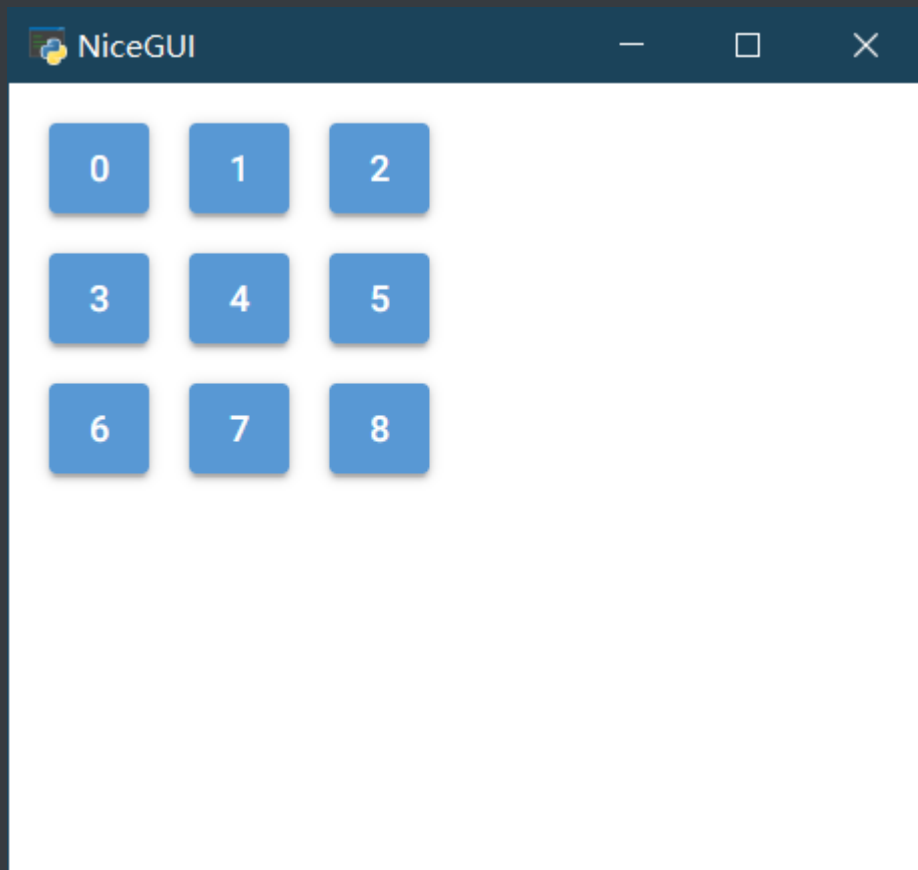


3.5 for 循环的技巧

3.5.1 用 for 创建多个有规律的控件

有时候，要创建多个外观一致或者有规律的控件，一个一个写代码或者复制粘贴的话，就不太 pythonic 了。在 Python 中，可以使用 for 来遍历迭代，同样可以使用 for 来创建多个外观一致或者有规律的控件。

```
1 from nicegui import ui
2
3 with ui.grid(rows=3,columns=3):
4     for i in range(9):
5         ui.button(i)
6
7 ui.run(native=True)
```



3.5.2 与lambda表达式组合使用时的问题

除了要创建一样的控件，还要给每个控件添加事件响应的话，每次都写一遍函数定义未免大材小用，更何况同名函数会出现覆盖，让函数名动态变化又没那么简单。这个时候Python的匿名函数——lambda表达式就派上用场了。lambda表达式可以创建语句简单的匿名函数，不必担心函数名重复的情况。比如，在下面的代码中，通过使用lambda表达式，让按钮的点击操作变成弹出一条通知。

```
1 from nicegui import ui
2
3 with ui.grid(rows=3,columns=3):
4     for i in range(9):
5         ui.button(i,on_click=lambda :ui.notify(i))
6
7 ui.run(native=True)
```

不过，事情并没有看上去那么简单，当写完代码开始执行的时候，才发现每个按钮的点击结果都一样，都是弹出内容为 8 的通知，这是为何？

原来，使用lambda表达式执行的 `ui.notify(i)`，因为表达式没有绑定默认值，实际上绑定到了动态的 `i` 上，按钮的 `on_click` 的定义不是第一时间执行，而是在完成定义之后响应用户的操作。最终，当 `for` 完成遍历之后，动态的 `i` 已经被赋值为 8，因此按钮的响应操作中的 `i` 都被统一修改了。为了避免这种情况，需要修改一下lambda表达式，添加一个参数并绑定默认值：

```
1 from nicegui import ui
2
3 with ui.grid(rows=3,columns=3):
4     for i in range(9):
5         ui.button(i,on_click=lambda i=i:ui.notify(i))
6
7 ui.run(native=True)
```

修改之后的 `lambda i=i:ui.notify(i)` 中，`i=i` 的意思是lambda表达式里的 `i` 变成了函数的参数 `i`，而这个 `i` 绑定到了外部的 `i` 当时值。

当然，实际代码中不建议这样写，太容易混淆了（怕被裁员倒是可以这样做）。

上面的代码可以再修改一下，让可读性变得更好：

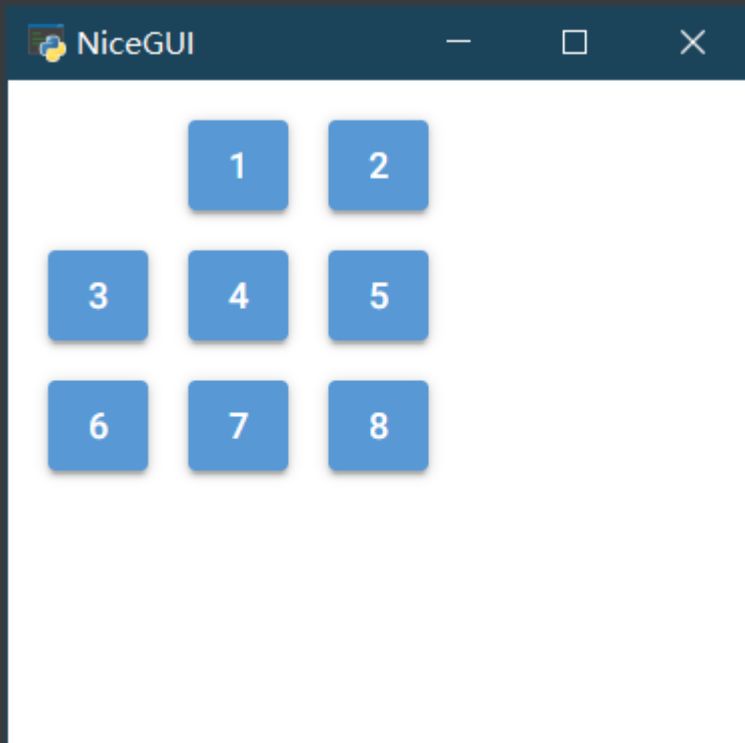
```
1 from nicegui import ui
2
3 with ui.grid(rows=3, columns=3):
4     for i in range(9):
5         ui.button(i, on_click=lambda x=i: ui.notify(x))
6
7 ui.run(native=True)
```

3.5.3 更好的 for 循环

为了确保批量生成之后还能访问每个控件，最好将批量生成的控件存储到列表里（不建议使用元组，没法修改；字典非必要也别用，字典的结构有点复杂，除非是列表没法实现需求）。

以下面的代码为例，使用 `buttons` 创建一个列表，在列表中用列表生成式来创建多个控件。后续如果需要修改某一个控件，就可以通过 `buttons` 来访问任意一个控件，这里是将第一个按钮隐藏。

```
1 from nicegui import ui
2
3 with ui.grid(rows=3, columns=3):
4     buttons = [ui.button(i, on_click=lambda x=i: ui.notify(x)) for i in range(9)]
5
6 buttons[0].tailwind.visibility('invisible')
7
8 ui.run(native=True)
```

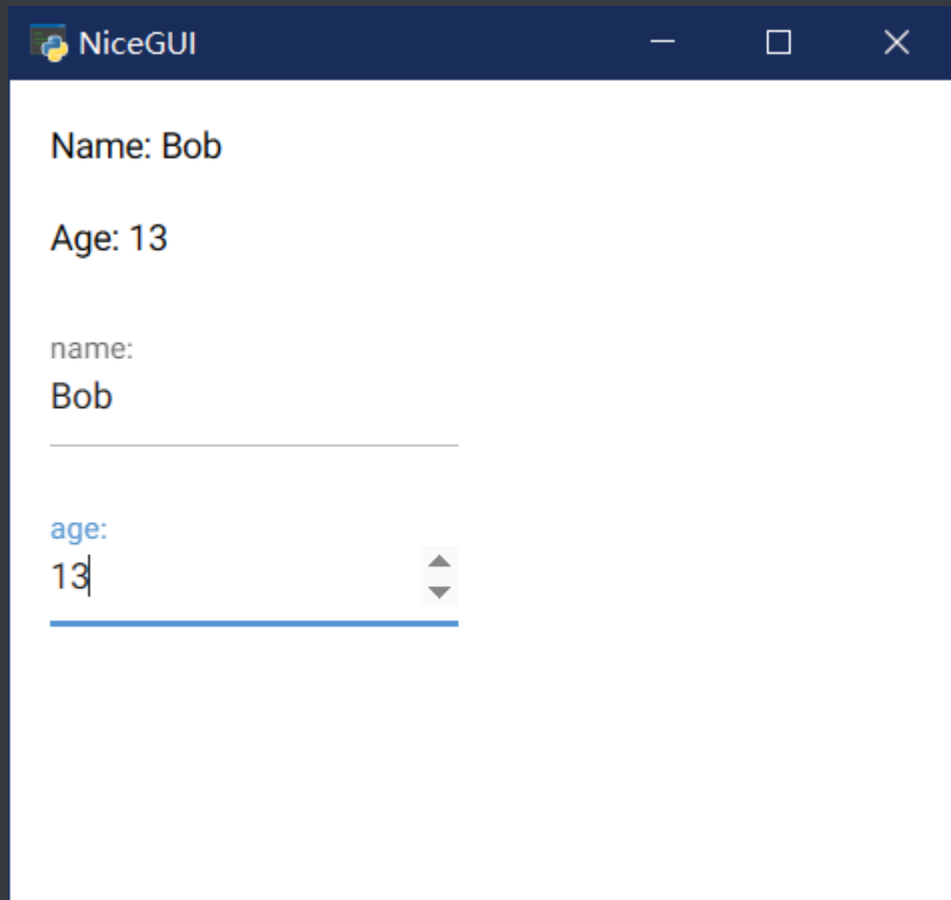


3.6 绑定的技巧

3.6.1 绑定到字典

在入门基础里提到的绑定只介绍如何绑定两个控件，其实，绑定除了绑定另一个控件，还支持绑定字典。绑定控件时，`target_object` 是控件对象，这里则换成字典对象；`target_name` 是控件对象的属性名，这里则换成字典的key，于是，就有了以下代码：

```
1 from nicegui import ui
2
3 data = {'name': 'Bob', 'age': 17}
4
5 ui.label().bind_text_from(data, 'name', backward=lambda n: f'Name: {n}')
6 ui.label().bind_text_from(data, 'age', backward=lambda a: f'Age: {a}')
7
8 ui.input(label='name:').bind_value(data, 'name')
9 ui.number(label='age:').bind_value(data, 'age', forward=lambda x: int(x))
10
11 ui.run(native=True)
```



要注意的是，`ui.number` 的值输出为小数，如果不增加 `forward=lambda x:int(x)` 的话，`data['age']` 会被修改为小数，而不是整数。同理，`ui.input` 的值输出为字符串，如果字典输入不是字符串的话，在输出时需要转换。

3.6.2 绑定到全局变量

还是上一节的代码，假如有人说：“字典还是有点复杂，能不能绑定到一个简单的变量上？”

怎么办？

也就是说，`data`字典没有了，取而代之的是：

```
1 name = 'Bob'
2 age = 17
```

其实也简单，只要将 `data` 换成 `globals()` 即可：


```
1 from nicegui import ui
2
3 name = 'Bob'
4 age = 17
5
6 ui.label().bind_text_from(globals(), 'name', backward=lambda n: f'Name: {n}')
7 ui.label().bind_text_from(globals(), 'age', backward=lambda a: f'Age: {a}')
8
9 ui.input(label='name:').bind_value(globals(), 'name')
10 ui.number(label='age:').bind_value(globals(), 'age', forward=lambda x: int(x))
11
12 ui.run(native=True)
```

任何在Python文件内定义的全局变量，都会成为全局变量字典的一个键值，可以使用 `globals()` 访问全局变量字典。

3.6.3 性能优化

在NiceGUI中有两种类型的绑定：

1. "Bindable properties"（可绑定属性）会自动检测写入访问并触发值变动传播。大多数 NiceGUI 元素使用这种可绑定属性，例如 `ui.input` 的 `value` 或 `ui.label` 的 `text`。基本上所有带有 `bind()` 方法的属性都支持这种类型的绑定。
2. 另一种绑定"active links"（活动链接）不会自动检测写入访问并触发值变动传播。如果将标签文本绑定到字典或自定义数据模型的属性，NiceGUI 的绑定模块则需要主动检查值是否发生变化。这个主动检查是通过每 0.1 秒运行一次 `refresh_loop()` 来完成。主动检查间隔可以通过设置 `ui.run()` 的参数 `binding_refresh_interval` 来修改。

可绑定属性非常高效，只要值不变，就不会产生任何性能开销（相对而言比较小而已）。但活动链接需要每秒检查所有绑定值10次。这可能会消耗较多的性能，尤其是活动链接的绑定关系非常复杂、非常多的时候。

因为不能让主线程阻塞太久，所以如果太多主动检查导致运行 `refresh_loop()` 的耗时过长，程序会发出警告。当然，可以配置阈值 `binding.MAX_PROPAGATION_TIME`（默认为 0.01 秒）来消除警告。但是，这个警告是有意义的，是在告诉开发者性能可能存在问题。比如，CPU在更新绑定花费太长时间的话，主线程就没法做别的事情，程序界面会因此卡住。

为了避免性能出问题，需要将活动链接改为可绑定属性之间的绑定，需要使用 `binding.BindableProperty()` 来创建可绑定属性。于是，基于第一小节的代码，将字典改为数据类，在数据类中定义两个可绑定属性，控件的绑定改为与数据类对象的绑定。代码如下：

```
1  from nicegui import ui, binding
2
3  class data_base:
4      name = binding.BindableProperty()
5      age = binding.BindableProperty()
6      def __init__(self) -> None:
7          self.name = 'Bob'
8          self.age = 17
9
10 data = data_base()
11
12 ui.label().bind_text_from(data, 'name', backward=lambda n: f'Name: {n}')
13 ui.label().bind_text_from(data, 'age', backward=lambda a: f'Age: {a}')
14
15 ui.input(label='name:').bind_value(data, 'name')
16 ui.number(label='age:').bind_value(data, 'age', forward=lambda x: int(x))
17
18 ui.run(native=True)
```

因为代码中的绑定数量很少，因此差异不大，如果将绑定数量放大百倍，就能看出两种绑定的性能差异。

3.7 app.storage 的技巧

有时候，网页上不同页面、用户需要存储、共享特定数据，依靠自己编程实现的话确实麻烦。好在 NiceGUI 提供了一种简单有效的数据存储功能，那就是 `app.storage`（存储）。存储有 5 个子字典，分别对应着不同的空间，有不同的应用范围：

- `app.storage.tab`：存储在服务器的内存中，此字典对于每个选项卡、会话都是唯一的，可以存储任意对象。需要注意的是，在实现 <https://github.com/zauberzeug/nicegui/discussions/2841> 之前，重启服务器会导致此字典的数据丢失。对于使用复制选项卡功能（右键选项卡点复制）创建的新选项卡，二者的 `tab_id`（`ui.context.client.tab_id`）是相同的，因此，复制的选项卡与原选项卡共享此字

典。此外，此字典只能在仅在 `ui.page` 中使用，并且需要等待客户端建立连接（确保读写此字典的操作在异步函数内的 `await ui.context.client.connected()` 之后）。

- `app.storage.client`：该字典也存储在服务器的内存中，对于每个客户端连接都是唯一的，并且可以存储任意对象。当页面重新加载或用户导航到另一个页面时，数据将被销毁。不同于能在服务器上保存数据好几天的 `app.storage.tab`，`app.storage.client` 更适合缓存频繁使用、一次性的数据。比如，需要动态更新的数据或者数据库连接，但希望在用户离开页面或关闭浏览器时立即销毁。同样的，这个字典只能在 `ui.page` 中使用。
- `app.storage.user`：存储在服务器磁盘中，每个字典都与浏览器cookie中保存的唯一标识符相关联，换句话说，此字典对于每个用户都是唯一的，并与浏览器的其他选项卡共享。可以通过存储在 `app.storage.browser['id']` 的标识符识别用户、会话。同样的，这个字典只能在 `ui.page` 中使用。此外，这个字典需要设置 `ui.run()` 的 `storage_secret` 参数来签名浏览器会话cookie。
- `app.storage.general`：该字典也存储在服务器磁盘中，提供了所有用户都可以访问的共享存储空间。
- `app.storage.browser`：与前几个字典不同，此字典直接存储为浏览器会话cookie，在同一用户的所有浏览器选项卡之间共享。虽然很多方面看起来很像 `app.storage.user`，不过，`app.storage.user` 因为其在减少数据负载、增强安全性和提供更大存储容量方面的优势，在实际使用中比 `app.storage.browser` 更受欢迎。默认情况下，NiceGUI会在 `app.storage.browser['id']` 中为每个浏览器会话保留一个唯一标识符。同样的，这个字典只能在 `ui.page` 中使用。此外，这个字典需要设置 `ui.run()` 的 `storage_secret` 参数来签名浏览器会话cookie。

如果因为上述介绍看起来不够直观，而在选用存储字典时候头疼，可以参考下面的对比表格快速选用（表示是，表示否）：

存储的子字典	tab	client	user	general	browser
存储位置	服务器内存	服务器内存	服务器磁盘	服务器磁盘	浏览器
是否在不同选项卡之间共享					
是否在不同浏览器客户端之间共享					
是否在服务器重启后保留数据					
是否在页面重载后保留数据					
是否只能用在ui.page内					

存储的子字典	tab	client	user	general	browser
是否需要客户端建立连接	✓	✗	✗	✗	✗
是否只能在响应之前写入	✗	✗	✗	✗	✓
是否要求数据可序列化	✗	✗	✓	✓	✓
是否需要设置 storage_secret 参数	✗	✗	✓	✗	✓

下面是个使用存储字典的简单例子：

```

1 from nicegui import app, ui
2
3 @ui.page('/')
4 def index():
5     app.storage.user['count'] = app.storage.user.get('count', 0) + 1
6     with ui.row():
7         ui.label('your own page visits:')
8         ui.label().bind_text_from(app.storage.user, 'count')
9
10 ui.run(storage_secret='private_key')
```

默认数据是以无缩进的JSON格式存储在 `app.storage.user` 和 `app.storage.general` 中，可以将 `app.storage.user.indent`、`app.storage.general.indent` 设置为 `True` 来让对应存储字典的数据采用2个空格的缩进格式。

3.8 修改指定元素的技巧

在CSS中，有个非常重要的概念叫选择器。

每一条css样式定义由两部分组成，形式如下：

```
1 选择器{样式}
```

在 `{` 之前的部分就是“选择器”。“选择器”指明了 `{样式}` 中的“样式”的作用对象，也就是“样式”作用于网页中的哪些元素。

选择器有一套自己的语法规则，通过合理设置选择器的规则，可以很精准地选择指定元素。

NiceGUI简化了不少CSS上的操作，但不代表不需要CSS的基础。如果读者掌握了CSS的选择器，与 `ui.query` 和 `ui.teleport` 结合使用，那就如同得到了屠龙宝刀，操作界面布局、美化界面将更加得心应手。

注意，前两小节要求读者具备CSS选择器基础，没有相应基础的读者可以搁置前两小节，直接看第三小节。

3.8.1 `ui.query`

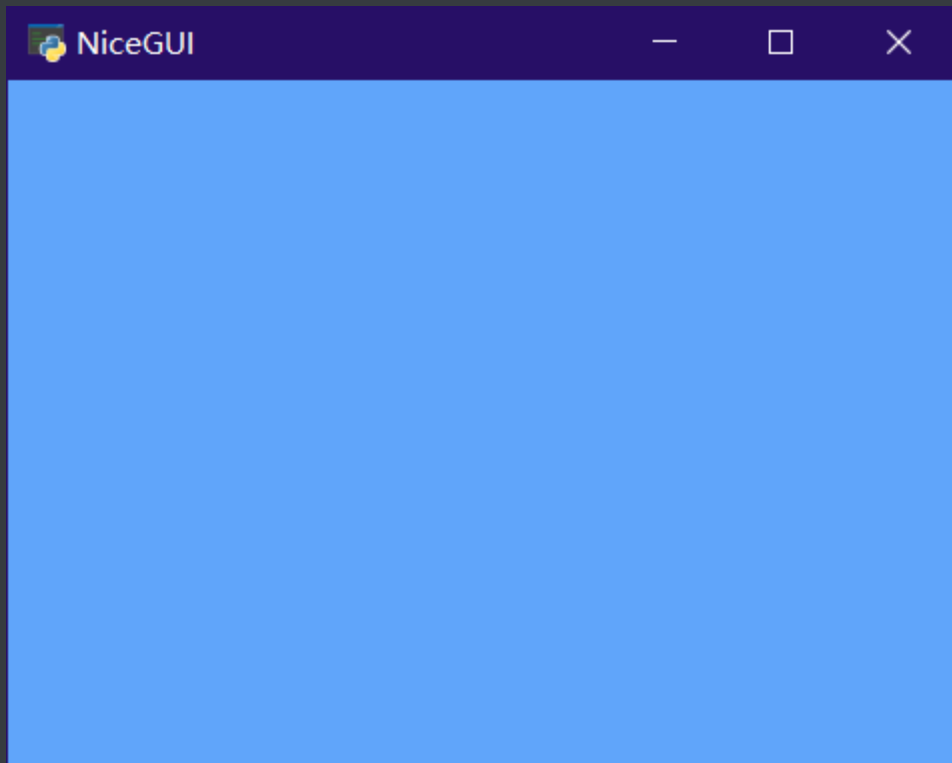
前面讲过如何美化控件，即在控件定义时使用 `props`、`classes`、`style` 等方法美化控件，也可以在控件定义好之后，通过给定的变量名调用相应方法。但是，如果想要美化的控件、元素根本就不是定义出来的，而是框架带出来的，想要美化就有点麻烦。当然，直接修改内置样式、源码很直观，但麻烦。要是有种方法能让想要修改的内容就像被定义为变量一样，后续直接使用，那就方便不少。正巧，`ui.query` 就有这样的功能。

注意，`ui.query` 的 `props` 方法修改的是HTML元素的属性（`attribute`），而不是 `ui.element` 或者 Quasar组件的属性（`props`）。

`ui.query` 只有一个字符串类型参数 `selector`，顾名思义，就是前面提到的选择器。通过给 `ui.query` 传入选择器语法，`ui.query` 将返回CSS选择器能够选择的元素，后续可以直接对该元素执行样式美化的方法。

下面的代码就是使用 `ui.query` 选择了 `body`（网页的主体），并设置 `body` 的背景颜色：

```
1 from nicegui import ui
2
3 body = ui.query(selector='body')
4 body.classes('bg-blue-400')
5
6 ui.run(native=True)
```



`ui.query` 的用法很简单，难点在于确定CSS选择器的写法，这一部分属于CSS基础知识，这里就不再赘述，有能力的读者可以抽时间深入学习CSS选择器的语法。

3.8.2 `ui.teleport`

肯定有读者在学了 `ui.query` 美化指定元素之后，突发奇想，想要给指定元素内部添加控件，比如，下面的代码：

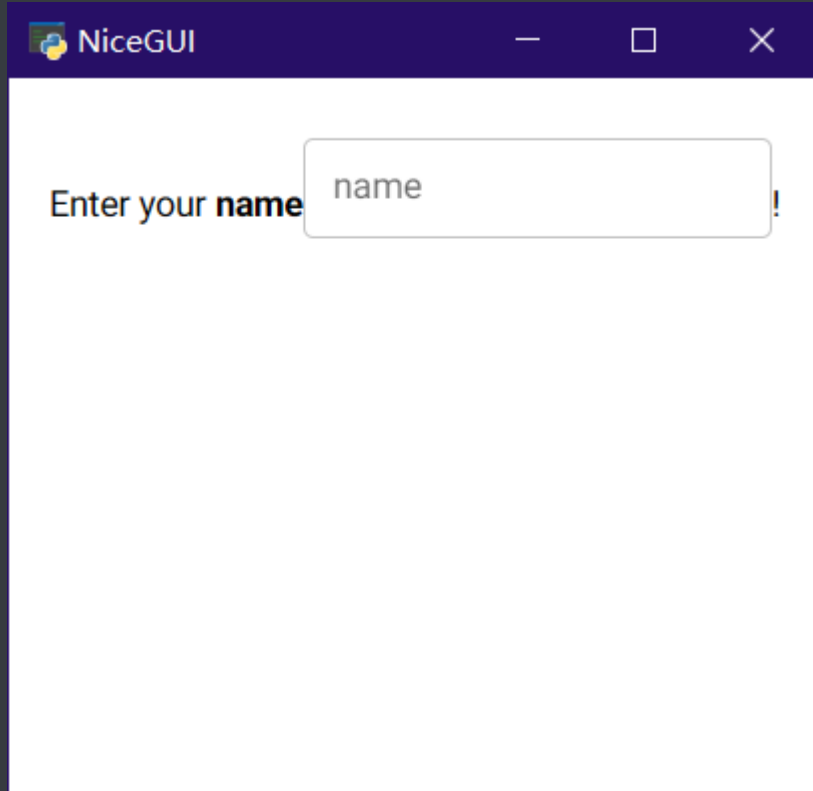
```
1 from nicegui import ui
2
3 markdown = ui.markdown('Enter your **name**!')
4 with ui.query(f'#c{markdown.id} strong'):
5     ui.input('name').classes('inline-flex').props('dense outlined')
6
7 ui.run(native=True)
```

然而，这段代码并不能成功运行，因为 `ui.query` 并不支持 `add_slot`。如果想要实现类似效果，只需将 `ui.query` 换成 `ui.teleport` 即可，不过传递的参数名不是 `selector`，而是 `to`：

```

1  from nicegui import ui
2
3  markdown = ui.markdown('Enter your **name**!')
4  with ui.teleport(to=f'#c{markdown.id} strong'):
5      ui.input('name').classes('inline-flex').props('dense outlined')
6
7  ui.run(native=True)

```



`ui.teleport` 就是这样一个基于CSS选择器语法将任意控件传送至指定位置的控件。

3.8.3 ElementFilter

暂时不会CSS选择器语法的读者也不用着急，尽管CSS选择器语法很强大，但在Python中不够直观，想要快速确定选择器还要去网页中开启调试模式。好在NiceGUI提供了另一种不需要CSS选择器的定位指定元素工具，那就是 `ElementFilter`。

`ElementFilter` 和 `ui` 模块同级，使用 `from nicegui import ElementFilter` 来导入。

`ElementFilter` 的功能等于 `ui.query` 加 `ui.teleport`，既能设置指定元素的样式，又能将控件传送到指定位置。但与 `ui.query` 和 `ui.teleport` 使用CSS选择器语法不同，`ElementFilter` 的筛选方式更pythonic，更直观，更契合Python编程习惯。

以下代码是用于匹配的模板内容，以下面的代码为例，分别看看 `ElementFilter` 不同参数、方法的用途：

```
1 from nicegui import ui,ElementFilter
2
3 with ui.card():
4     ui.button('button A')
5     ui.label('label A_A')
6     ui.label('label A_B')
7
8 with ui.card():
9     ui.button('button B')
10    ui.label('label B_A')
11    ui.label('label B_B')
12
13 ui.run(native=True)
```

3.8.3.1 初始化方法

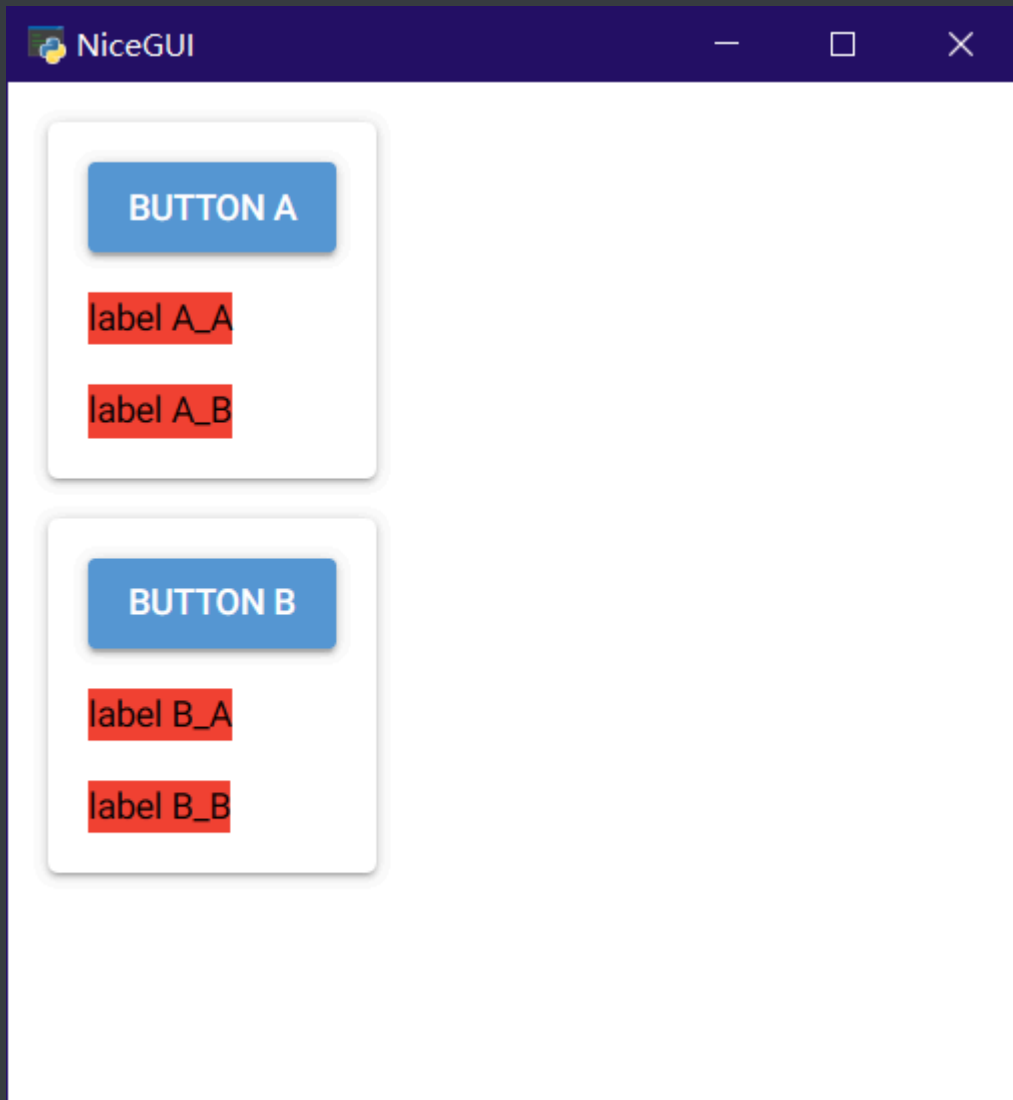
`ElementFilter` 是一个类，需要初始化为对象实例才能使用。`ElementFilter` 的初始化方法有四个参数，分别是 `kind` 、 `marker` 、 `content` 、 `local_scope` 。

`kind` 参数，NiceGUI的 `ui` 类型，表示筛选什么类型的控件。比如，在下面的代码中，传入的参数是 `ui.label` ， `ElementFilter` 就会筛选 `ui.label` ，这样给 `ElementFilter` 对象设置背景颜色为红色的时候，页面内所有的 `ui.label` 的背景颜色就相应变成红色。

```
1 from nicegui import ui,ElementFilter
2
3 with ui.card():
4     ui.button('button A')
5     ui.label('label A_A')
6     ui.label('label A_B')
7
8 with ui.card():
9     ui.button('button B')
10    ui.label('label B_A')
11    ui.label('label B_B')
12
```



```
13 ElementFilter(kind=ui.label).classes('bg-red')
14
15 ui.run(native=True)
```



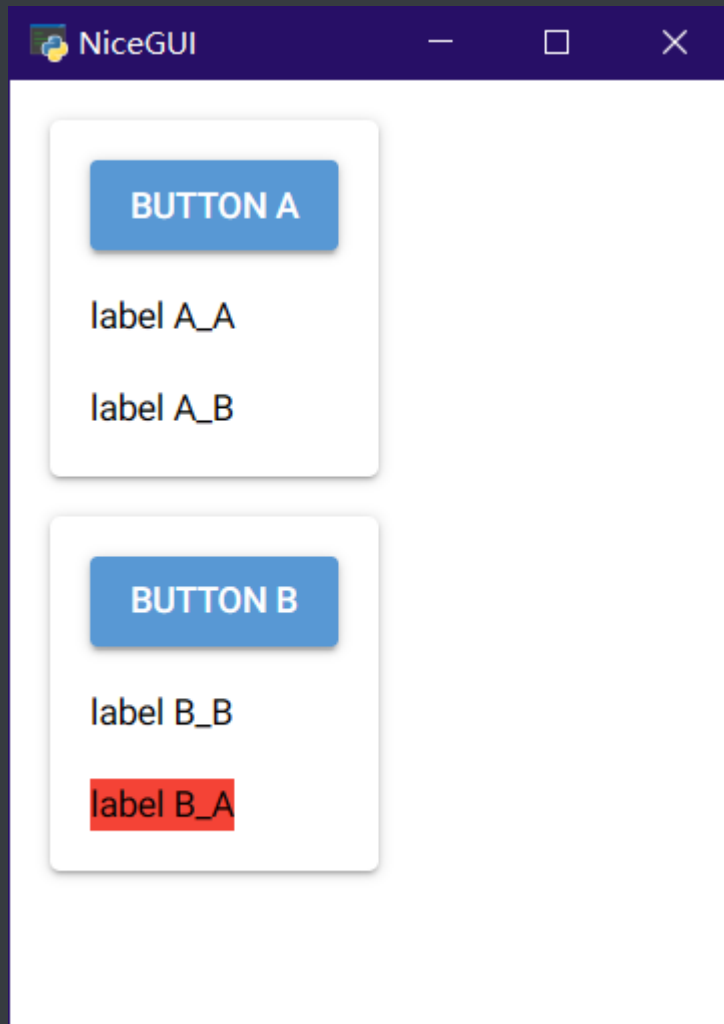
`marker` 参数，字符串类型或者字符串列表类型，表示筛选包含指定marker或者指定marker列表的对象。

在此，需要额外介绍一下控件的 `mark` 方法，也就是如何给控件添加marker。对于每一个控件，都可以通过 `mark` 方法定义一组marker，用于 `ElementFilter` 的筛选。`mark` 方法的参数是一个支持解包、分解的字符串类型参数 `markers`。也就是说，传入 `'A'`、`'A','B','AB'`、`'B A BA'`、`'A','B BA'` 都是可以的。本质上说，`mark` 方法就是将传入的字符串转换为该对象的 `_markers` 列表。对于 `'A','B','AB'` 这样多个字符串，该方法会转化为 `['B','A','AB']` 这样的列表来使用。对于 `'B A BA'` 这样用空格划分的字符串，该方法会自动以空格为分隔符分解为 `['B','A','BA']` 这样的列表来使用。当然，两种方法混用也没问题，`'A','B BA'` 这样的多个字符串，则会转化为 `['A','B','BA']` 这样的列表。注意，虽然 `mark` 方法支持串联、重复使用，但最好不要这样做，因为后执行的 `mark` 方法结果会覆盖先前 `mark` 方法的结果，如果是想清除之前的marker，倒是可以重复执行。

说完给控件添加marker，下面回归正题，说说如何筛选。marker 参数和 mark 方法的 markers 参数类似，只不过 marker 参数没有解包过程，想要传入多个字符串，只能使用字符串列表。与 mark 方法的宽松不同，marker 参数的要求比较严格，要么是纯字符串，带空格的会自动划分、转化为列表，要么是无空格的字符串组成列表，不支持正确解析内含带空格的字符串列表，所以，只有以下格式才是正确的用法：'A'、['A','B','AB']、'B A BA'。

代码示例如下：

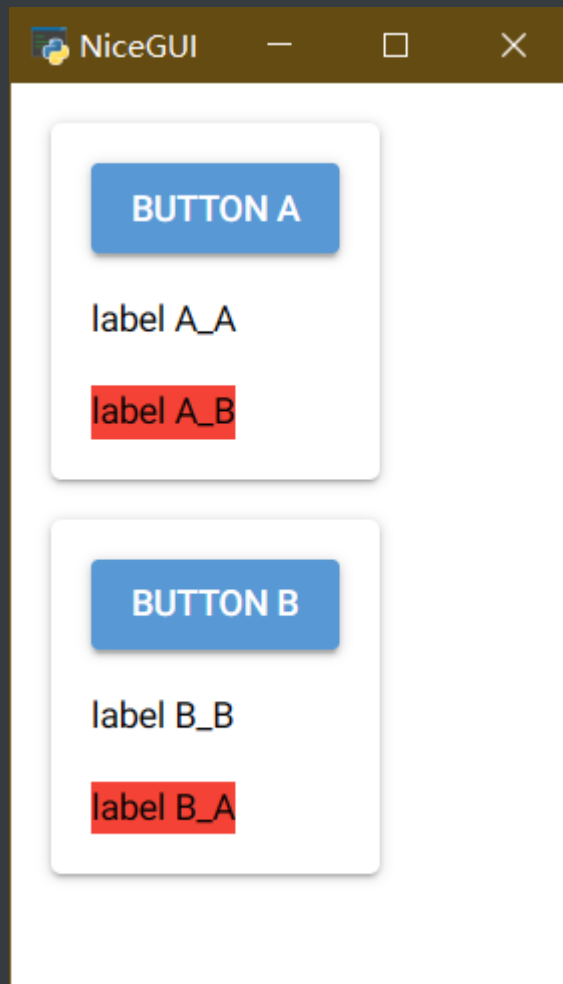
```
1 from nicegui import ui,ElementFilter
2
3 with ui.card():
4     ui.button('button A')
5     ui.label('label A_A').mark('A')
6     ui.label('label A_B').mark('A','B','AB')
7
8 with ui.card():
9     ui.button('button B')
10    ui.label('label B_B').mark('B')
11    ui.label('label B_A').mark('B A BA')
12
13 ElementFilter(marker='BA').classes('bg-red')
14 #ElementFilter(marker='A B').classes('bg-red')
15 #ElementFilter(marker=['A','B']).classes('bg-red')
16
17 ui.run(native=True)
```



`content` 参数, 字符串类型或者字符串列表类型, 表示筛选包含指定内容的对象。筛选范围包括对象的 `value`、`text`、`label`、`icon`、`placeholder` 等文本属性。匹配要求完全包含指定字符串或者字符串列表。

```
1 from nicegui import ui, ElementFilter
2
3 with ui.card():
4     ui.button('button A')
5     ui.label('label A_A').mark('A')
6     ui.label('label A_B').mark('A', 'B', 'AB')
7
8 with ui.card():
9     ui.button('button B')
10    ui.label('label B_B').mark('B')
11    ui.label('label B_A').mark('B A BA')
12
13 ElementFilter(content=['B', 'A']).classes('bg-red')
```

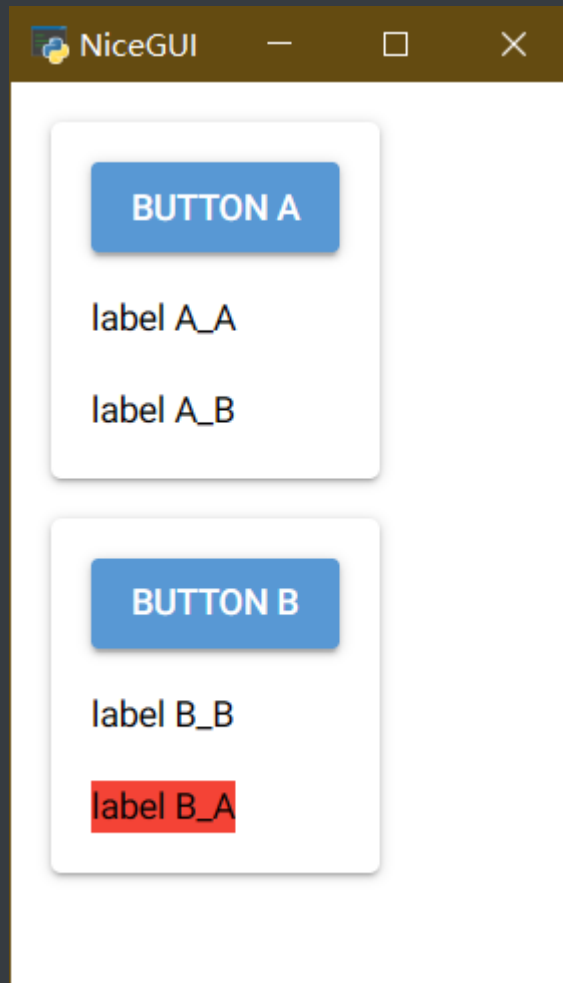
```
14
15 ui.run(native=True)
```



`local_scope` 参数，布尔类型，表示 `ElementFilter` 匹配当前范围还是全局，默认为 `False`，即匹配全局。如果设置为 `True`，则只匹配当前上下文。可以看以下代码，修改了缩进并将此参数设置为 `True`，`ElementFilter` 对象就只能匹配同一缩进内的控件：

```
1 from nicegui import ui, ElementFilter
2
3 with ui.card():
4     ui.button('button A')
5     ui.label('label A_A').mark('A')
6     ui.label('label A_B').mark('A', 'B', 'AB')
7
8 with ui.card():
9     ui.button('button B')
10    ui.label('label B_B').mark('B')
11    ui.label('label B_A').mark('B A BA')
```

```
12     ElementFilter(content=['B', 'A'], local_scope=True).classes('bg-red')
13
14 ui.run(native=True)
```



3.8.3.2 within 方法和 not_within 方法

顾名思义，这两个方法就是在 `ElementFilter` 初始化参数的筛选范围内进一步筛选指定的父级对象，得到在指定的父级对象上下文之内、不在指定的父级对象上下文之内的对象。对 `within` 方法而言，会得到符合该方法匹配条件的对象。对 `not_within` 方法而言，会排除符合该方法匹配条件的对象

两个方法的参数都一样，都是三个，分别是 `kind`、`marker`、`instance`。

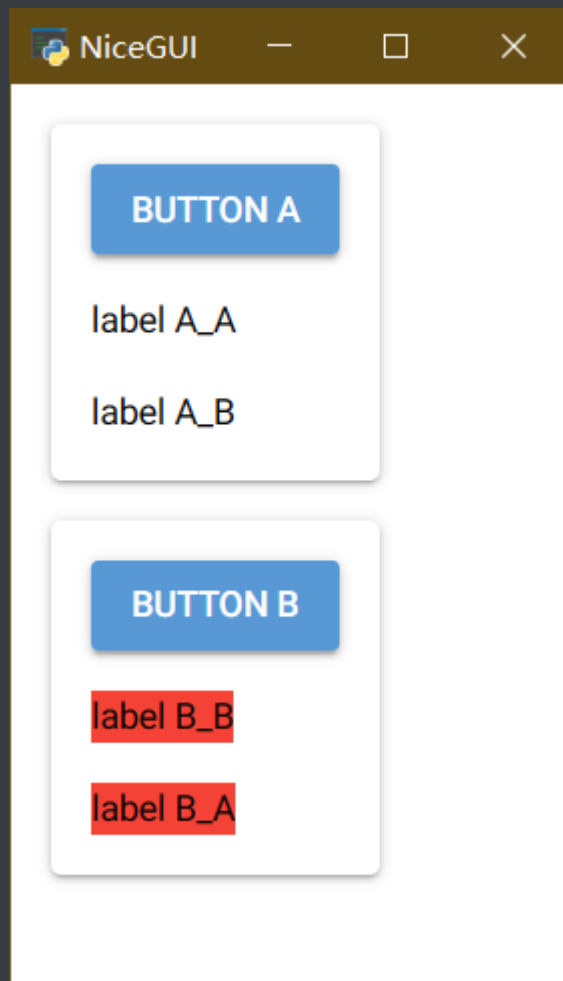
`kind` 和 `marker` 与初始化方法的参数一样，这里不再赘述。只是，这里的 `marker` 不支持字符串列表。

`instance` 参数，对象或者对象列表，指定具体对象的范围内是否筛选。以 `within` 方法为例，给此参数传递具体对象，`ElementFilter` 将只筛选在该对象之内的 `ui.label`：

```

1  from nicegui import ui, ElementFilter
2
3  with ui.card() as card1:
4      ui.button('button A')
5      ui.label('label A_A').mark('A')
6      ui.label('label A_B').mark('A', 'B', 'AB')
7
8  with ui.card() as card2:
9      ui.button('button B')
10     ui.label('label B_B').mark('B')
11     ui.label('label B_A').mark('B A BA')
12
13  ElementFilter(kind=ui.label).within(instance=card2).classes('bg-red')
14
15  ui.run(native=True)

```



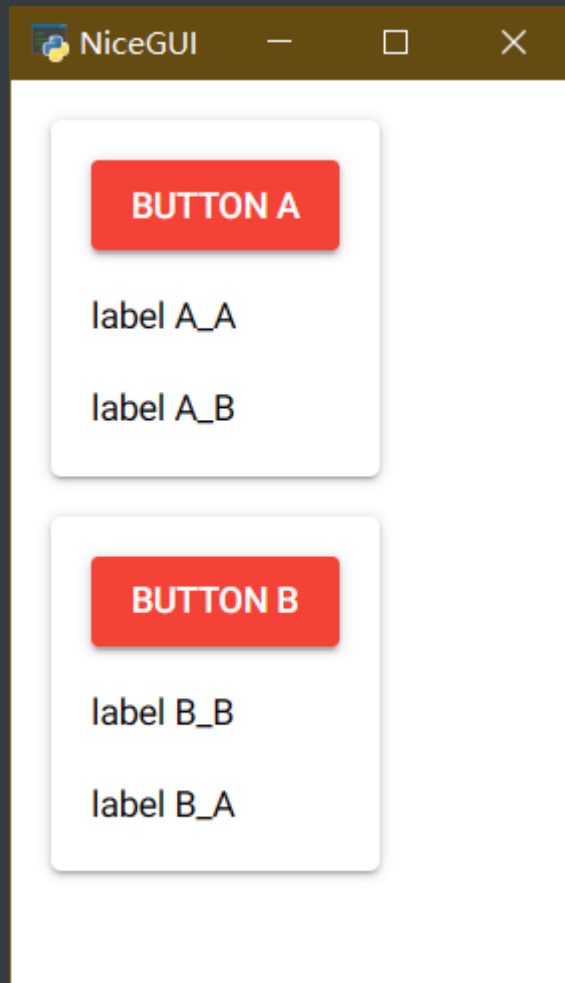
这两个方法支持串联调用，不过串联就和传递列表给参数一样，是扩展了对应筛选条件的内部列表。对于这两种筛选条件的内部列表，匹配规则是不一样的：对于 `within` 方法，筛选则是要求列表内元素全部匹配；对于 `not_within` 方法，筛选则是要求列表内元素任意一个匹配。

3.8.3.3 `exclude` 方法

该方法是在 `ElementFilter` 初始化参数的筛选范围内进一步排除指定的对象。

该方法有三个参数，`kind`、`marker`、`content`，同初始化方法的参数一样，这里简单说一下示例代码，不做详解。不过，该方法的三个参数不支持传入列表，`marker` 也不支持根据空格自动划分字符串，这一点需要注意。

```
1 from nicegui import ui, ElementFilter
2 from nicegui.elements.mixins.text_element import TextElement
3
4 with ui.card() as card1:
5     ui.button('button A')
6     ui.label('label A_A').mark('A')
7     ui.label('label A_B').mark('A', 'B', 'AB')
8
9 with ui.card() as card2:
10     ui.button('button B')
11     ui.label('label B_B').mark('B')
12     ui.label('label B_A').mark('B A BA')
13
14 ElementFilter(kind=TextElement).exclude(kind=ui.label).classes('bg-red')
15
16 ui.run(native=True)
```



`ui.label` 和 `ui.button` 都继承了 `TextElement`，因此匹配 `TextElement` 会同时匹配到这两种控件，因此，在 `exclude` 方法中指定 `kind` 为 `ui.label` 之后，匹配结果就排除了 `ui.label`，只有 `ui.button` 的颜色变成红色。

3.8.3.4 传送控件到匹配结果

对于 `ElementFilter`，想要传送控件到结果也很简单，只需遍历 `ElementFilter` 对象，就能获取匹配结果。

如下面代码所示，使用 `for` 遍历 `ElementFilter` 对象，使用 `with` 进入每个元素的上下文，就和正常添加控件到对应 `slot` 一样：

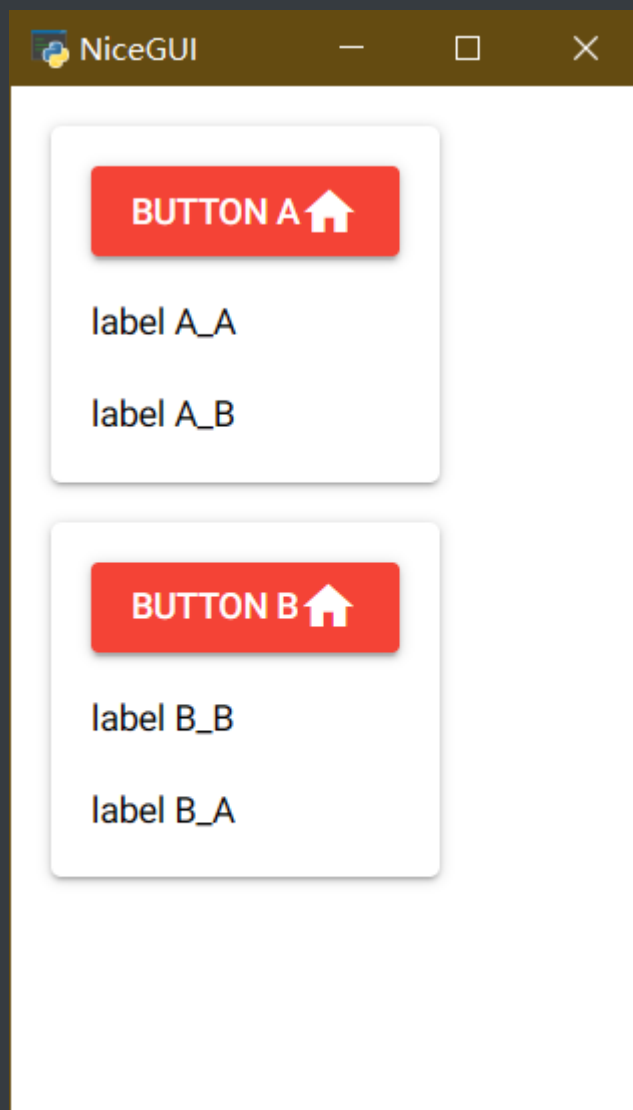
```
1 from nicegui import ui, ElementFilter
2 from nicegui.elements.mixins.text_element import TextElement
3
4 with ui.card() as card1:
5     ui.button('button A')
6     ui.label('label A_A').mark('A')
```



```

7     ui.label('label A_B').mark('A','B','AB')
8
9     with ui.card() as card2:
10         ui.button('button B')
11         ui.label('label B_B').mark('B')
12         ui.label('label B_A').mark('B A BA')
13
14     for ele in ElementFilter(kind=TextElement).exclude(kind=ui.label).classes('bg-red'):
15         with ele:
16             ui.icon('home')
17
18     ui.run(native=True)

```



3.8.3.5 总结

ElementFilter 的方法、参数不多，但用法不统一，要是组合使用，需要一些时间思考其匹配模式。而有的读者看到文字太多就头疼，没关系，这里将上面的内容简化为一个表格方便查阅。详细看过一遍文字教程之后，后续开发中再次遇到，可以快速参阅表格来确定匹配模式。

对应参数的匹配模式：

ElementFilter的方法	__init__	within	not_within	exclude
kind 参数	任意一个	全部匹配	任意一个	任意一个
content 参数	全部匹配	无此参数	无此参数	任意一个
instance 参数	无此参数	全部匹配	任意一个	无此参数
marker 参数	全部匹配	全部匹配	任意一个	任意一个

Match type for parameters in ElementFilter 's method:

ElementFilter's method	__init__	within	not_within	exclude
parameter kind	any/or	all/and	any/or	any/or
parameter content	all/and	----	----	any/or
parameter instance	----	all/and	any/or	----
parameter marker	all/and	all/and	any/or	any/or

另外，对于NiceGUI2.1版本的 ElementFilter 部分方法参数不支持列表传入，这里特地补丁了一份模块文件，有需要的读者可以自行替换，文件的具体路径为 .venv\Lib\site-packages\nicegui\element_filter.py ，如果是全局环境的Python，路径为 {Python执可执行文件所在目录}\Lib\site-packages\nicegui\element_filter.py

```
1  from __future__ import annotations
2
3  from typing import Generic, Iterator, List, Optional, Type, TypeVar, Union, overload
4
5  from typing_extensions import Self
6
7  from .context import context
```

```

8  from .element import Element
9  from .elements.mixins.content_element import ContentElement
10 from .elements.mixins.source_element import SourceElement
11 from .elements.mixins.text_element import TextElement
12 from .elements.notification import Notification
13 from .elements.select import Select
14
15 T = TypeVar('T', bound=Element)
16
17
18 class ElementFilter(Generic[T]):
19     DEFAULT_LOCAL_SCOPE = False
20
21     @overload
22     def __init__(self: ElementFilter[Element], *,
23                 marker: Union[str, List[str], None] = None,
24                 content: Union[str, List[str], None] = None,
25                 local_scope: bool = DEFAULT_LOCAL_SCOPE,
26                 ) -> None:
27         ...
28
29     @overload
30     def __init__(self, *,
31                 kind: Union[Type[T], List[Type[T]], None] = None,
32                 marker: Union[str, List[str], None] = None,
33                 content: Union[str, List[str], None] = None,
34                 local_scope: bool = DEFAULT_LOCAL_SCOPE,
35                 ) -> None:
36         ...
37
38     def __init__(self, *,
39                 kind: Union[Type[T], List[Type[T]], None] = None,
40                 marker: Union[str, List[str], None] = None,
41                 content: Union[str, List[str], None] = None,
42                 local_scope: bool = DEFAULT_LOCAL_SCOPE,
43                 ) -> None:
44         """ElementFilter
45

```



```

72         :param content: filter for elements which contain ``content`` in one of
their content attributes like ``.text``, ``.value``, ``.source``, ...; can be a
single string or a list of strings which all must match
73         :param local_scope: if `True`, only elements within the current scope are
returned; by default the whole page is searched (this default behavior can be
changed with ``ElementFilter.DEFAULT_LOCAL_SCOPE = True``)
74         """
75         self._kind = kind if isinstance(kind, list) else ([kind] if kind else [])
76         self._markers = marker.split() if isinstance(marker, str) else [word for
single_marker in (marker or []) for word in single_marker.split()]
77         self._contents = [content] if isinstance(content, str) else content or []
78
79         self._within_kinds: List[Type[Element]] = []
80         self._within_instances: List[Element] = []
81         self._within_markers: List[str] = []
82
83         self._not_within_kinds: List[Type[Element]] = []
84         self._not_within_instances: List[Element] = []
85         self._not_within_markers: List[str] = []
86
87         self._exclude_kinds: List[Type[Element]] = []
88         self._exclude_markers: List[str] = []
89         self._exclude_content: List[str] = []
90
91         self._scope = context.slot.parent if local_scope else context.client.layout
92
93     def __iter__(self) -> Iterator[T]:
94         for element in self._scope.descendants():
95             if self._kind and not isinstance(element, tuple(self._kind)):
96                 continue
97             if self._exclude_kinds and isinstance(element,
tuple(self._exclude_kinds)):
98                 continue
99
100             if any(marker not in element._markers for marker in self._markers):
101                 continue
102             if any(marker in element._markers for marker in self._exclude_markers):
103                 continue

```

```

104
105         if self._contents or self._exclude_content:
106             element_contents = [content for content in (
107                 element.props.get('text'),
108                 element.props.get('label'),
109                 element.props.get('icon'),
110                 element.props.get('placeholder'),
111                 element.props.get('value'),
112                 element.text if isinstance(element, TextElement) else None,
113                 element.content if isinstance(element, ContentElement) else
None,
114                 element.source if isinstance(element, SourceElement) else None,
115             ) if content]
116             if isinstance(element, Notification):
117                 element_contents.append(element.message)
118             if isinstance(element, Select):
119                 options = {option['value']: option['label'] for option in
element.props.get('options', [])}
120                 element_contents.append(options.get(element.value, ''))
121                 if element.is_showing_popup:
122                     element_contents.extend(options.values())
123                 if any(all(needle not in str(haystack) for haystack in
element_contents) for needle in self._contents):
124                     continue
125                 if any(needle in str(haystack) for haystack in element_contents for
needle in self._exclude_content):
126                     continue
127
128             ancestors = set(element.ancestors())
129             if self._within_instances and not
ancestors.issuperset(self._within_instances):
130                 continue
131             if self._not_within_instances and not
ancestors.isdisjoint(self._not_within_instances):
132                 continue
133             if self._within_kinds and not all(any(isinstance(ancestor, kind) for
ancestor in ancestors) for kind in self._within_kinds):
134                 continue

```

```

135         if self._not_within_kinds and any(isinstance(ancestor,
tuple(self._not_within_kinds)) for ancestor in ancestors):
136             continue
137         ancestor_markers = {marker for ancestor in ancestors for marker in
ancestor._markers}
138         if self._within_markers and not
ancestor_markers.issuperset(self._within_markers):
139             continue
140         if self._not_within_markers and not
ancestor_markers.isdisjoint(self._not_within_markers):
141             continue
142
143         yield element # type: ignore
144
145     def within(self, *,
146               kind: Union[Element, List[Element], None] = None,
147               marker: Union[str, List[str], None] = None,
148               instance: Union[Element, List[Element], None] = None,
149               ) -> Self:
150         """Filter elements which have a specific match in the parent hierarchy."""
151         if kind is not None:
152             if isinstance(kind, list):
153                 for every_kind in kind:
154                     assert issubclass(every_kind, Element)
155                     self._within_kinds.extend(kind)
156             else:
157                 assert issubclass(kind, Element)
158                 self._within_kinds.append(kind)
159         if marker is not None:
160             markers = marker.split() if isinstance(marker, str) else [word for
single_marker in marker for word in single_marker.split()]
161             self._within_markers.extend(markers)
162         if instance is not None:
163             self._within_instances.extend(instance if isinstance(instance, list)
else [instance])
164         return self
165
166     def exclude(self, *,

```

```

167         kind: Union[Element, List[Element], None] = None,
168         marker: Union[str, List[str], None] = None,
169         content: Union[str, List[str], None] = None,
170     ) -> Self:
171         """Exclude elements with specific element type, marker or content."""
172         if kind is not None:
173             if isinstance(kind, list):
174                 for every_kind in kind:
175                     assert issubclass(every_kind, Element)
176                     self._exclude_kinds.extend(kind)
177             else:
178                 assert issubclass(kind, Element)
179                 self._exclude_kinds.append(kind)
180         if marker is not None:
181             markers = marker.split() if isinstance(marker, str) else [word for
single_marker in marker for word in single_marker.split()]
182             self._exclude_markers.extend(markers)
183         if content is not None:
184             self._exclude_content.extend([content] if isinstance(content, str) else
content)
185         return self
186
187     def not_within(self, *,
188                   kind: Union[Element, List[Element], None] = None,
189                   marker: Union[str, List[str], None] = None,
190                   instance: Union[Element, List[Element], None] = None,
191                   ) -> Self:
192         """Exclude elements which have a parent of a specific type or marker."""
193         if kind is not None:
194             if isinstance(kind, list):
195                 for every_kind in kind:
196                     assert issubclass(every_kind, Element)
197                     self._not_within_kinds.extend(kind)
198             else:
199                 assert issubclass(kind, Element)
200                 self._not_within_kinds.append(kind)
201         if marker is not None:

```



```

202         markers = marker.split() if isinstance(marker, str) else [word for
single_marker in marker for word in single_marker.split()]
203         self._not_within_markers.extend(markers)
204         if instance is not None:
205             self._not_within_instances.extend(instance if isinstance(instance, list)
else [instance])
206         return self
207
208     def classes(self, add: Optional[str] = None, *, remove: Optional[str] = None,
replace: Optional[str] = None) -> Self:
209         """Apply, remove, or replace HTML classes.
210
211         This allows modifying the look of the element or its layout using `Tailwind
<https://tailwindcss.com/>`_ or `Quasar <https://quasar.dev/>`_ classes.
212
213         Removing or replacing classes can be helpful if predefined classes are not
desired.
214
215         :param add: whitespace-delimited string of classes
216         :param remove: whitespace-delimited string of classes to remove from the
element
217         :param replace: whitespace-delimited string of classes to use instead of
existing ones
218         """
219         for element in self:
220             element.classes(add, remove=remove, replace=replace)
221         return self
222
223     def style(self, add: Optional[str] = None, *, remove: Optional[str] = None,
replace: Optional[str] = None) -> Self:
224         """Apply, remove, or replace CSS definitions.
225
226         Removing or replacing styles can be helpful if the predefined style is not
desired.
227
228         :param add: semicolon-separated list of styles to add to the element
229         :param remove: semicolon-separated list of styles to remove from the element

```

```

230         :param replace: semicolon-separated list of styles to use instead of
existing ones
231         """
232         for element in self:
233             element.style(add, remove=remove, replace=replace)
234         return self
235
236     def props(self, add: Optional[str] = None, *, remove: Optional[str] = None) ->
Self:
237         """Add or remove props.
238
239         This allows modifying the look of the element or its layout using `Quasar
<https://quasar.dev/>`_ props.
240
241         Since props are simply applied as HTML attributes, they can be used with any
HTML element.
242
243         Boolean properties are assumed ``True`` if no value is specified.
244
245         :param add: whitespace-delimited list of either boolean values or key=value
pair to add
246         :param remove: whitespace-delimited list of property keys to remove
247         """
248         for element in self:
249             element.props(add, remove=remove)
250         return self

```

3.9 其他布局

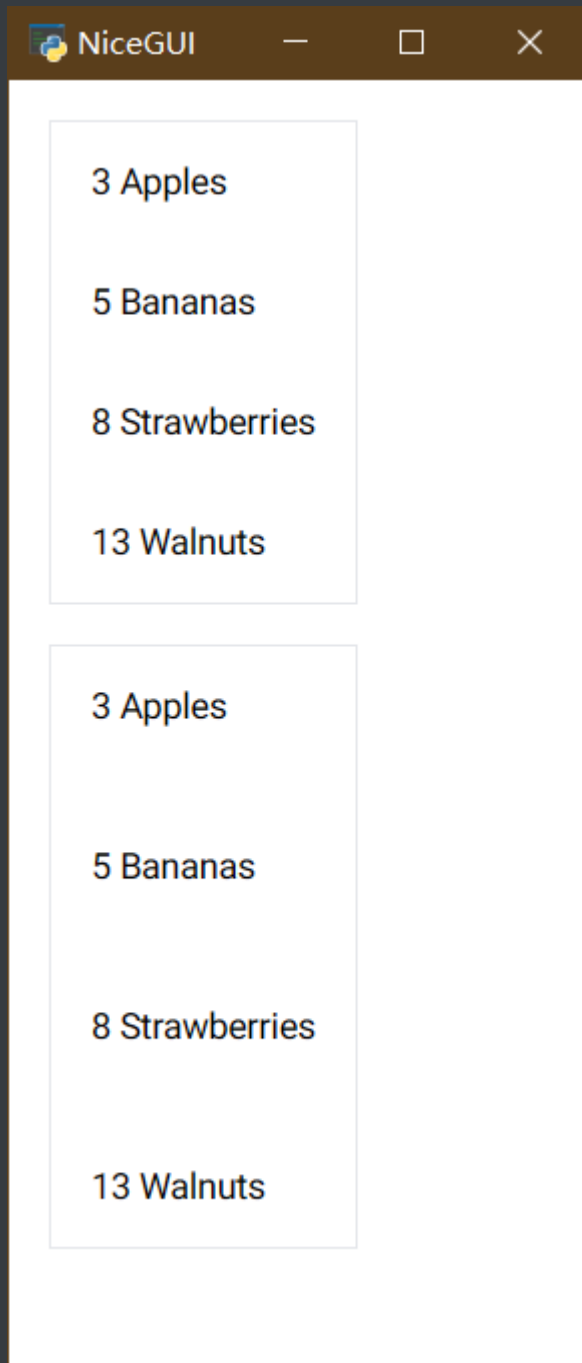
前面介绍过常用的布局，其实NiceGUI支持的布局控件有很多，下面提到的这些不常用，但有些需求比较刁钻，用这些布局刚好可以减少不必要的工作。

3.9.1 ui.list

列表布局，看上去有点像 `ui.column`，但列表布局主要是给 `ui.item` 用的，当然，要是想在一般布局的时候使用也没问题，不过需要注意一下二者的区别。

以代码为例：

```
1 from nicegui import ui
2
3 with ui.list().classes('border'):
4     ui.item('3 Apples')
5     ui.item('5 Bananas')
6     ui.item('8 Strawberries')
7     ui.item('13 Walnuts')
8
9 with ui.column().classes('border'):
10     ui.item('3 Apples')
11     ui.item('5 Bananas')
12     ui.item('8 Strawberries')
13     ui.item('13 Walnuts')
14
15 ui.run(native=True)
```



为了能清楚看到二者的区别，这里给两种布局加了个边框，方便看到边界。可以看出，`ui.list` 比 `ui.column` 的默认行距小，整体看上去更加紧凑。当然，`ui.list` 主要是给 `ui.item` 用的，默认的紧凑是有别的用途，如果在一般布局中需要紧凑一些的观感，可以根据需求调整样式，而不是使用 `ui.list` 代替 `ui.column`。

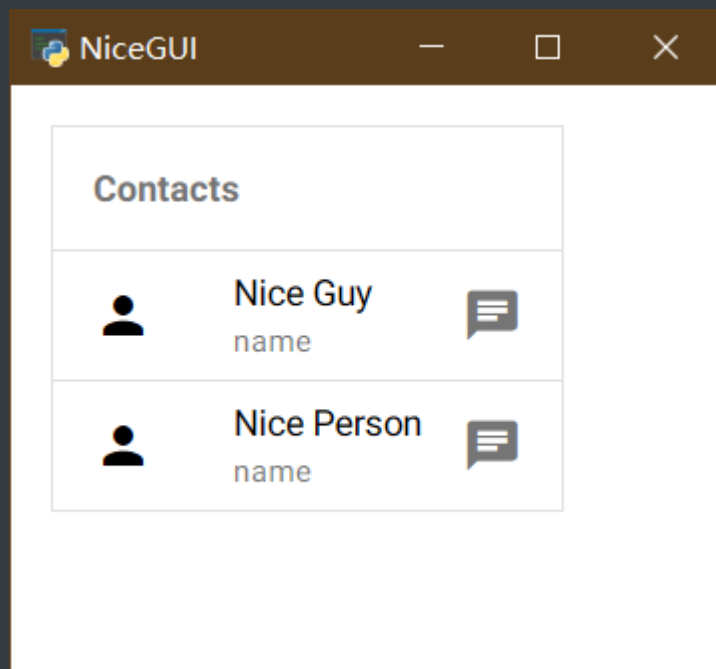
关于 `ui.list` 更恰当的用途，还是以代码为例：

```
1 from nicegui import ui
2
3 with ui.list().props('bordered separator'):
4     ui.item_label('Contacts').props('header').classes('text-bold')
```

```

5     ui.separator()
6     with ui.item(on_click=lambda: ui.notify('Selected contact 1')):
7         with ui.item_section().props('avatar'):
8             ui.icon('person')
9         with ui.item_section():
10            ui.item_label('Nice Guy')
11            ui.item_label('name').props('caption')
12        with ui.item_section().props('side'):
13            ui.icon('chat')
14    with ui.item(on_click=lambda: ui.notify('Selected contact 2')):
15        with ui.item_section().props('avatar'):
16            ui.icon('person')
17        with ui.item_section():
18            ui.item_label('Nice Person')
19            ui.item_label('name').props('caption')
20        with ui.item_section().props('side'):
21            ui.icon('chat')
22
23    ui.run(native=True)

```



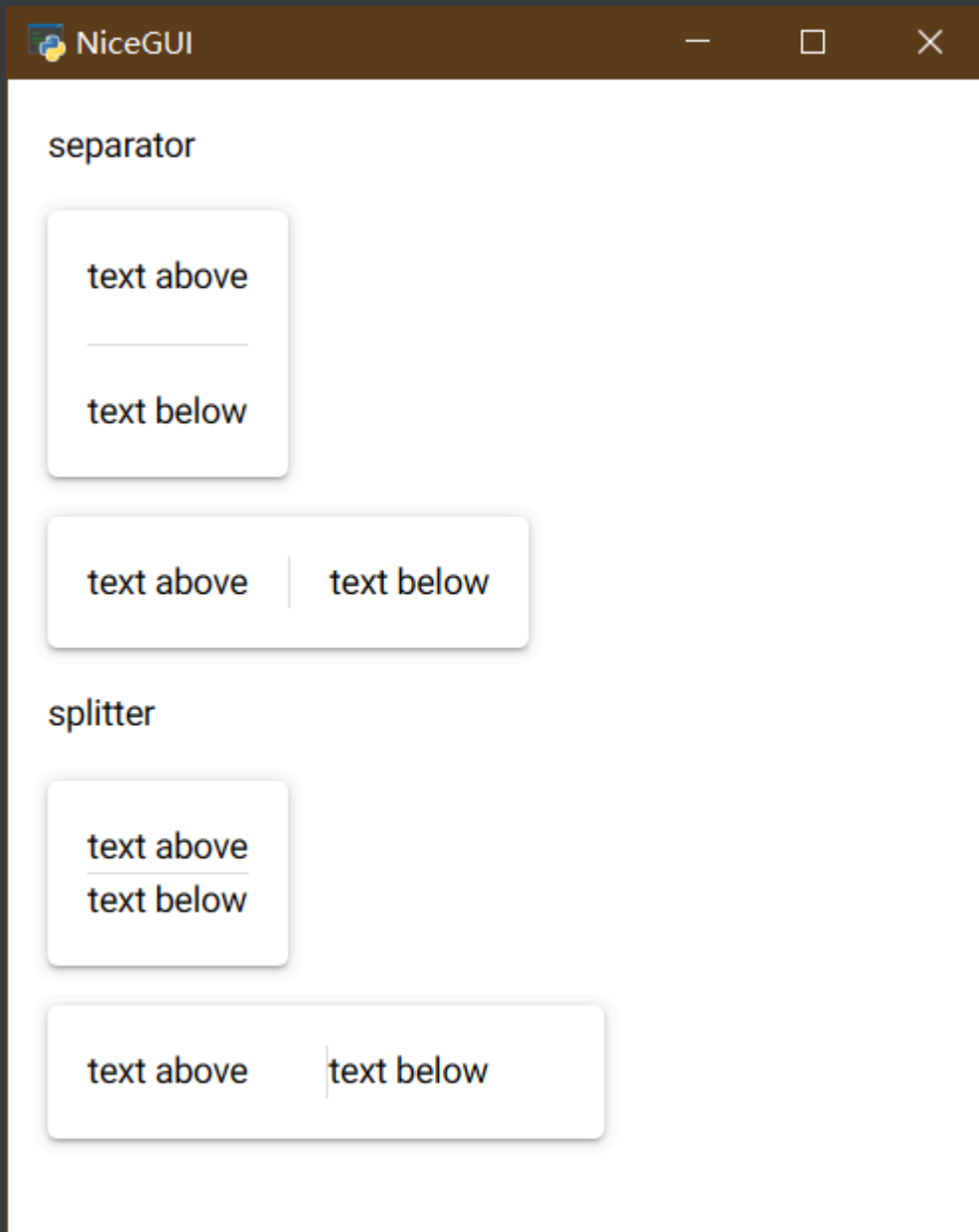
给 `ui.item` 内增加 `ui.item_section`、`ui.item_label` 等控件，可以实现类似通讯录的布局。不同于使用基本布局组合实现需要较多的样式调整，这些预定义的控件本身有很多属性样式，只需设置好对应的属性，就能让样式接近想要的效果。而且，本身的含义也比较直观，后续维护起来也方便。

3.9.2 ui.splitter

前面介绍过 `ui.separator`，可以生成一条水平或者垂直的分隔线，用来不明显地区分控件。但是，在NiceGUI中，除了 `ui.separator` 外，还有一种分隔线，那就是 `ui.splitter`，也可以生成一条水平或者垂直的分隔线。不过，在前面没介绍这个控件，是因为这个控件的用法可比 `ui.separator` 复杂，用途也多，只是简单分隔一下，还是用 `ui.separator` 比较好。倘若对于分隔有更高的要求或者其他用途，`ui.splitter` 绝对能胜任。

先看代码，分别用 `ui.separator` 和 `ui.splitter` 实现类似的效果：

```
1  from nicegui import ui
2
3  ui.label('separator')
4  with ui.card():
5      ui.label('text above')
6      ui.separator()
7      ui.label('text below')
8  with ui.card(),ui.row():
9      ui.label('text above')
10     ui.separator().props('vertical')
11     ui.label('text below')
12
13 ui.label('splitter')
14 with ui.card():
15     splitter = ui.splitter(horizontal=True)
16     with splitter.before:ui.label('text above')
17     with splitter.after:ui.label('text below')
18 with ui.card().classes('w-56'),ui.row().classes('w-full'):
19     splitter2 = ui.splitter().classes('w-full')
20     with splitter2.before:ui.label('text above')
21     with splitter2.after:ui.label('text below')
22
23 ui.run(native=True)
```



单看效果对比的话，`ui.splitter` 看上去只是比 `ui.separator` 紧凑一些，似乎没什么不同。如果仔细看代码，就会发现 `ui.splitter` 的代码量比 `ui.separator` 多不少。

`ui.splitter` 可以看做是一个容器，定义 `ui.splitter` 之后，会产生 `ui.splitter().before`、`ui.splitter().after`、`ui.splitter().separator` 三个子容器，需要分别给前两个容器填充内容，才能看到分隔效果。而且，中间的分隔线也支持添加内容和鼠标交互，可以用鼠标左右拖动。

当然，`ui.splitter` 支持的参数不少，用起来也比 `ui.separator` 复杂：

`horizontal` 参数，布尔类型，是否将 `ui.splitter` 水平显示。

`limits` 参数，浮点元组类型，表示拖动分隔线的范围（最小、最大百分比），这是一个二元元组，默认为 `(0,100)`，第一个元素表示拖动范围的最小值，，第二个元素表示拖动范围的最大值。

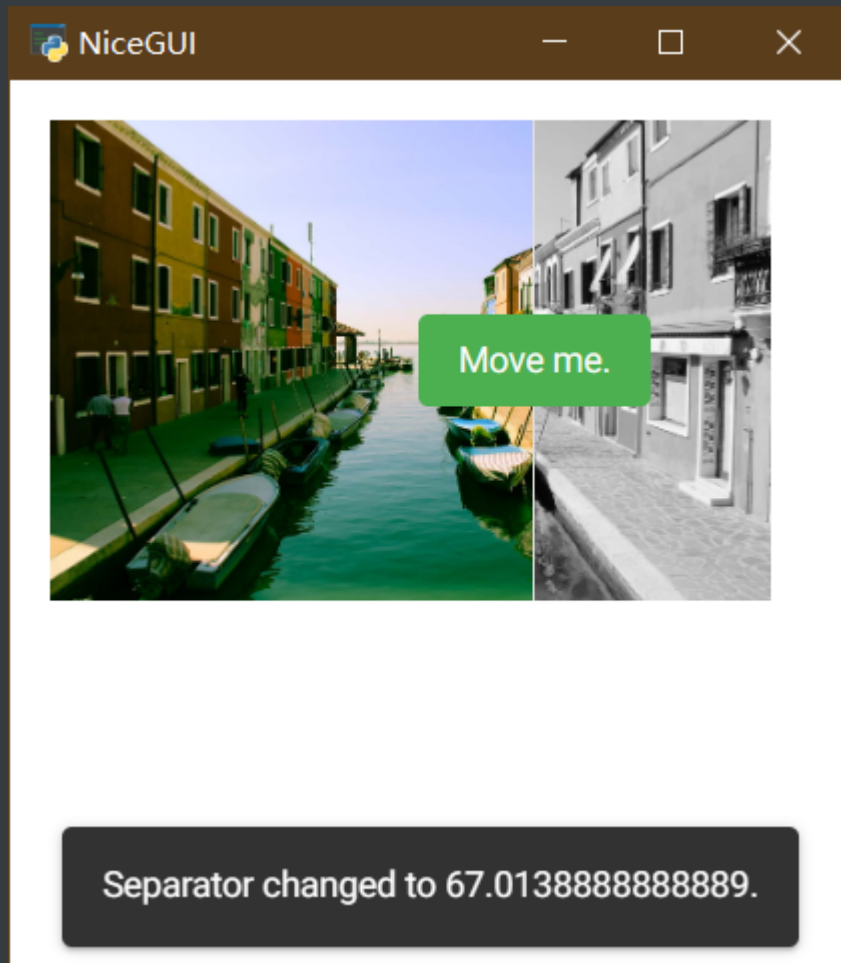
`value` 参数, 浮点类型, 表示 `ui.splitter().before` 的初始大小百分比。

`reverse` 参数, 布尔类型, 表示是否反转 `ui.splitter().before` 和 `ui.splitter().after` 先后顺序。

`on_change` 参数, 可调用类型, 表示拖动分隔线时执行的操作。

以下代码是 `ui.splitter` 的典型运用, 通过拖动分隔线, 显示彩色、黑白图像的对比效果, 同时弹出通知显示当前分隔线的位置:

```
1  from nicegui import ui
2
3  with ui.splitter(
4      horizontal=False,
5      reverse=False,
6      limits=(0, 100),
7      value=50,
8      on_change=lambda e: ui.notify(f"Separator changed to {e.sender.value}."),
9  ).classes("w-72 h-48").props(
10     "before-class=overflow-hidden after-class=overflow-hidden"
11 ) as splitter:
12     with splitter.separator:
13         ui.icon("lightbulb").classes("text-green")
14         ui.tooltip("Move me.").classes("bg-green").props(
15             'anchor="center middle" self="center middle"'
16         )
17     with splitter.before:
18         ui.image("https://cdn.quasar.dev/img/parallax1.jpg").classes(
19             "w-72 absolute-top-left"
20         )
21     with splitter.after:
22         ui.image("https://cdn.quasar.dev/img/parallax1-bw.jpg").classes(
23             "w-72 absolute-top-right"
24         )
25
26  ui.run(native=True)
```

3.9.3 ui.tabs

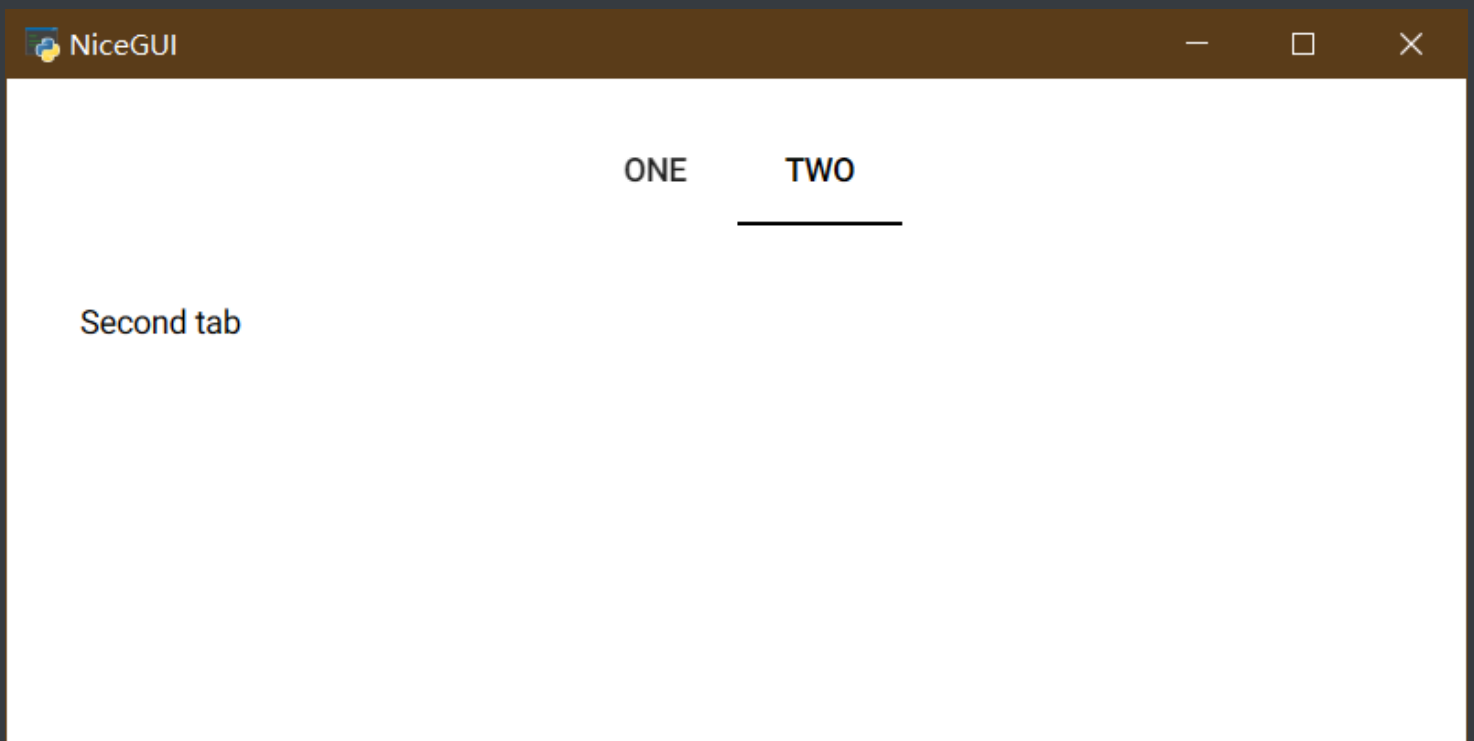
选项卡，现代浏览器的基本布局，可以通过点击上面的选项卡，切换下面的内容。

在NiceGUI中，和选项卡相关的控件有 `ui.tabs`、`ui.tab`、`ui.tab_panels` 和 `ui.tab_panel`。
`ui.tabs` 是放置选项卡的容器，`ui.tab` 是选项卡，`ui.tab_panels` 是放置选项卡关联内容的容器，`ui.tab_panel` 则是选项卡的关联内容。

以下面的代码为例，`ui.tab` 放置在 `ui.tabs` 内，`ui.tab_panel` 放置在 `ui.tab_panels` 内，想要让选项卡正确关联、交互，需要确保以下参数正确设置：

1. `ui.tab_panels` 的参数 `tabs` 需要传递已经创建的 `ui.tabs` 实例；
2. `ui.tab_panel` 的参数 `name` 需要传递已经创建的 `ui.tab` 实例，或者实例的字符串参数 `name`。

```
1 from nicegui import ui
2
3 with ui.tabs().classes("w-full") as tabs:
4     one = ui.tab(name="One")
5     two = ui.tab(name="Two")
6 with ui.tab_panels(tabs=tabs, value=two).classes("w-full"):
7     with ui.tab_panel(name=one):
8         ui.label("First tab")
9     with ui.tab_panel(name='Two'):
10         ui.label("Second tab")
11
12 ui.run(native=True)
```

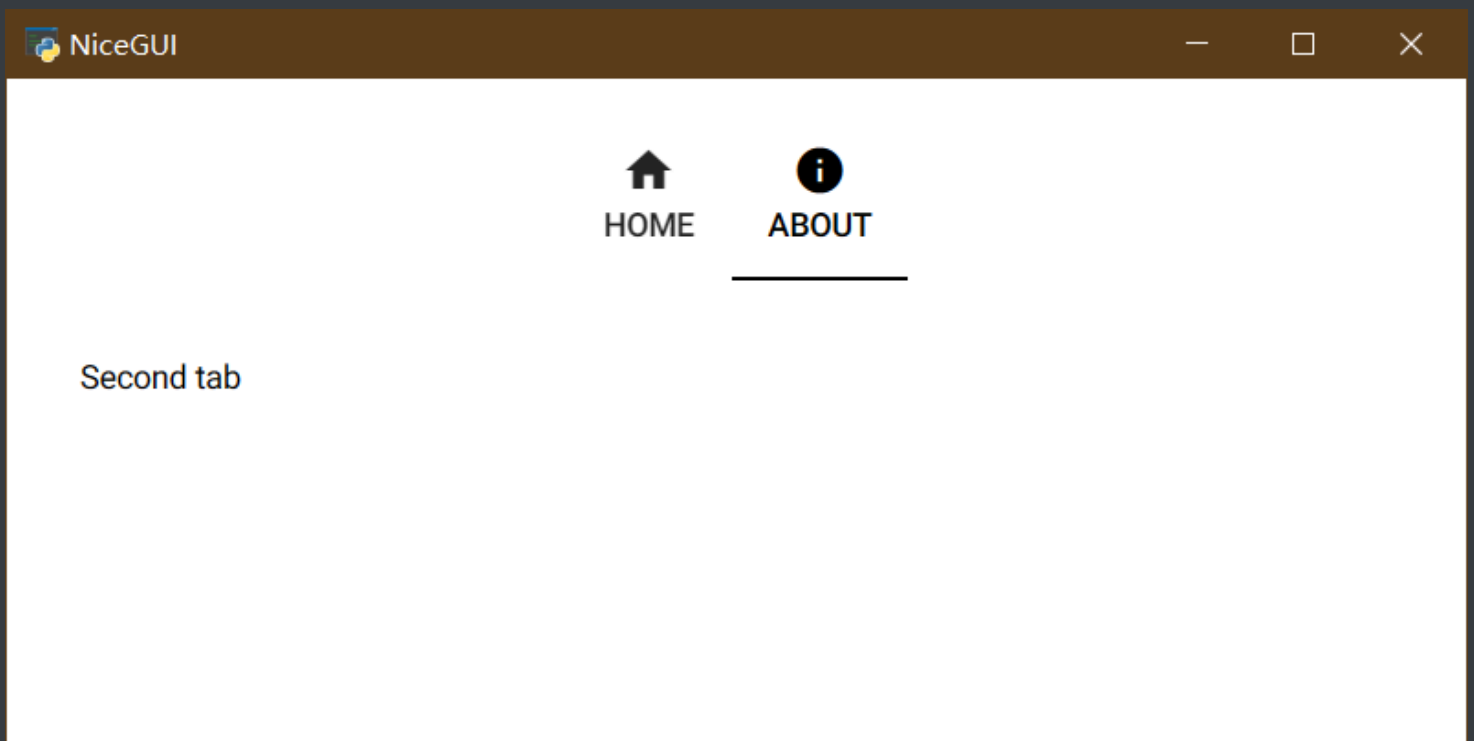


`ui.tab` 的 `name` 参数是用来区分选项卡的唯一标识符，称之为id也可以，但是，肯定有读者觉得不太方便，如果想要修改选项卡的名字，势必影响到下面 `ui.tab_panel` 的关联。其实，并不会导致这样的问题，`ui.tab` 还有一个字符串参数 `label`，如果设置了这个参数，显示在选项卡上的内容就会变成 `label` 而不是 `name`。此外，`ui.tab` 还支持像 `ui.button` 一样设置图标，只需给参数 `icon` 传入图标字符的名字即可：

```

1 from nicegui import ui
2
3 with ui.tabs().classes("w-full") as tabs:
4     one = ui.tab(name="One",label='Home',icon='home')
5     two = ui.tab(name="Two",label='About',icon='info')
6 with ui.tab_panels(tabs=tabs, value=two).classes("w-full"):
7     with ui.tab_panel(name=one):
8         ui.label("First tab")
9     with ui.tab_panel(name='Two'):
10         ui.label("Second tab")
11
12 ui.run(native=True)

```



除了通过点击交互来切换选项卡，调用 `ui.tabs`、`ui.panels` 的 `set_value` 方法也能切换选项卡：

```

1 from nicegui import ui
2
3 with ui.tabs().classes("w-full") as tabs:
4     one = ui.tab(name="One",label='Home',icon='home')
5     two = ui.tab(name="Two",label='About',icon='info')
6 with ui.tab_panels(tabs=tabs, value=two).classes("w-full") as panels:
7     with ui.tab_panel(name=one):

```

```

8         ui.label("First tab")
9         with ui.tab_panel(name='Two'):
10             ui.label("Second tab")
11
12 ui.button('GoTo 1', on_click=lambda: panels.set_value(one))
13 ui.button('GoTo 2', on_click=lambda: tabs.set_value('Two'))
14
15 ui.run(native=True)

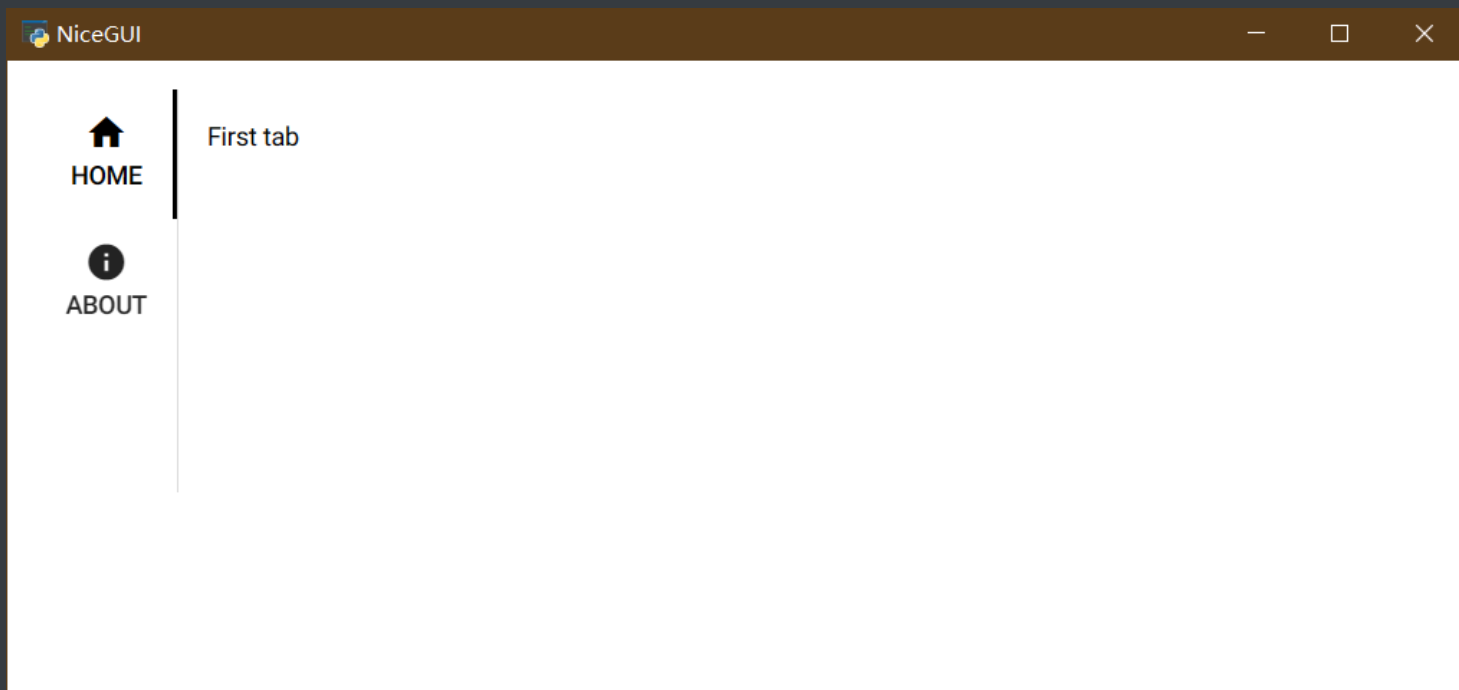
```

想要让选项卡从水平变成垂直，只需调用 `props('vertical')`，设置 `'vertical'` 即可。不过，只是设置一下，界面并不会如预想中那样改变，还需要借用前面介绍到的 `ui.splitter`：

```

1 from nicegui import ui
2
3 with ui.splitter(value=10).classes('w-full h-56') as splitter:
4     with splitter.before, ui.tabs().props('vertical').classes('w-full h-56') as tabs:
5         one = ui.tab(name="One", label='Home', icon='home')
6         two = ui.tab(name="Two", label='About', icon='info')
7     with splitter.after, ui.tab_panels(tabs=tabs,
8 value=two).props('vertical').classes('w-full h-56') as panels:
9         with ui.tab_panel(name=one):
10             ui.label("First tab")
11         with ui.tab_panel(name='Two'):
12             ui.label("Second tab")
13
14 ui.run(native=True)

```



`ui.tabs` 支持两个参数：

`value` 参数，字符串类型或者 `ui.tab` 类型或者 `ui.tab_panel` 类型，用于指定初始化选择的 `ui.tab` 或者 `ui.tab_panel`，如果是字符串类型，则值为 `ui.tab` 或者 `ui.tab_panel` 的 `name` 属性的值。但是，因为定义 `ui.tabs` 时，`ui.tab` 和 `ui.tab_panel` 通常还没定义，这个参数一般不需要知道，也没法指定。

`on_change` 参数，可调用类型，当选项卡切换时执行的操作。

`ui.tab` 支持三个参数：

`name` 参数，字符串类型，表示选项卡的名字，也可以称之为标识符，如果对应的 `ui.tab_panel` 也定义了 `name`，需要保持一致。

`label` 参数，字符串类型，表示选项卡的标签，如果此参数没有定义，默认使用 `name` 的值。

`icon` 参数，字符串类型，表示选项卡的图标。

`ui.tab_panels` 支持五个参数：

`tabs` 参数，`ui.tabs` 类型，与已经创建的 `ui.tabs` 关联，`ui.tabs` 切换选项卡时，`ui.tab_panels` 也切换为相应的选项卡。

`value` 参数，字符串类型或者 `ui.tab` 类型或者 `ui.tab_panel` 类型，用于指定初始化选择的 `ui.tab` 或者 `ui.tab_panel`，如果是字符串类型，则值为 `ui.tab` 或者 `ui.tab_panel` 的 `name` 属性的值。但是，因为定义 `ui.tab_panels` 时，`ui.tab_panel` 通常还没定义，这个参数一般不设置为 `ui.tab_panel` 类型对象，而是指定为字符串或者已经定义的 `ui.tab`。

`on_change` 参数，可调用类型，当选项卡切换时执行的操作。

`animated` 参数，布尔类型，表示是否启用切换动画，默认为 `True`。

`keep_alive` 参数，布尔类型，表示是否对容器内的控件启用VUE的 `keep-alive` 组件。启用此组件，容器内控件不会在不可见的时候销毁，而是一直保持存活，以免再次访问控件时，控件的状态因为重建而被重置。

`ui.tab_panel`支持一个字符串参数 `name`，与 `ui.tab` 的 `name` 相同，如果值一致，则会将 `ui.tab_panel` 与 `ui.tab` 关联。

3.9.4 `ui.scroll_area`

对于较多的内容放置在网页，会导致网页又臭又长，可以使用滚动区域当容器。滚动区域控件会生成滚动条，让内容只在指定的大小内显示，拖动滚动条可以显示其余内容。

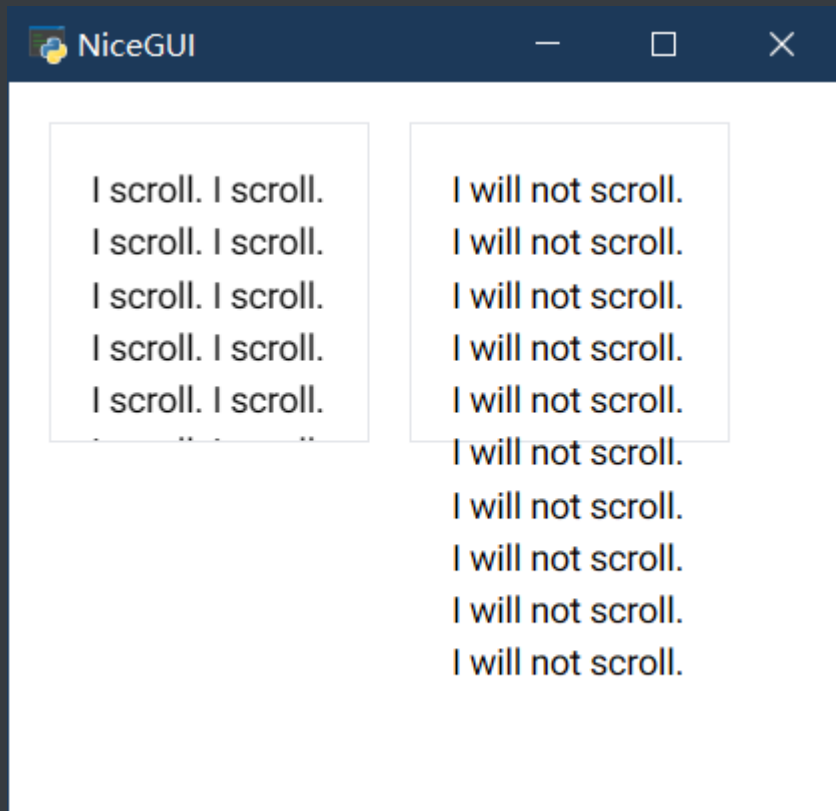
滚动区域控件支持一个可调用类型参数 `on_scroll` 作为响应滚动的执行操作，通过一个事件参数捕获滚动的响应事件，并将 `vertical_position`（当前位置的垂直位置，单位像素）、`vertical_percentage`（当前位置的垂直位置，单位百分比）、`vertical_size`（滚动内容的垂直大小，单位像素）、`vertical_container_size`（滚动区域容器的垂直大小，单位像素）、`horizontal_position`（当前位置的水平位置，单位像素）、`horizontal_percentage`（当前位置的水平位置，单位百分比）、`horizontal_size`（滚动内容的水平大小，单位像素）、`horizontal_container_size`（滚动区域容器的水平大小，单位像素）等属性传递出来。此外，该控件的 `scroll_to` 方法可以设置控件内容滚动到什么位置。

以下是简单的示例：

```

1  from nicegui import ui
2
3  with ui.row():
4      with ui.scroll_area().classes('w-32 h-32 border'):
5          ui.label('I scroll. ' * 20)
6      with ui.column().classes('p-4 w-32 h-32 border'):
7          ui.label('I will not scroll. ' * 10)
8
9  ui.run(native=True)

```

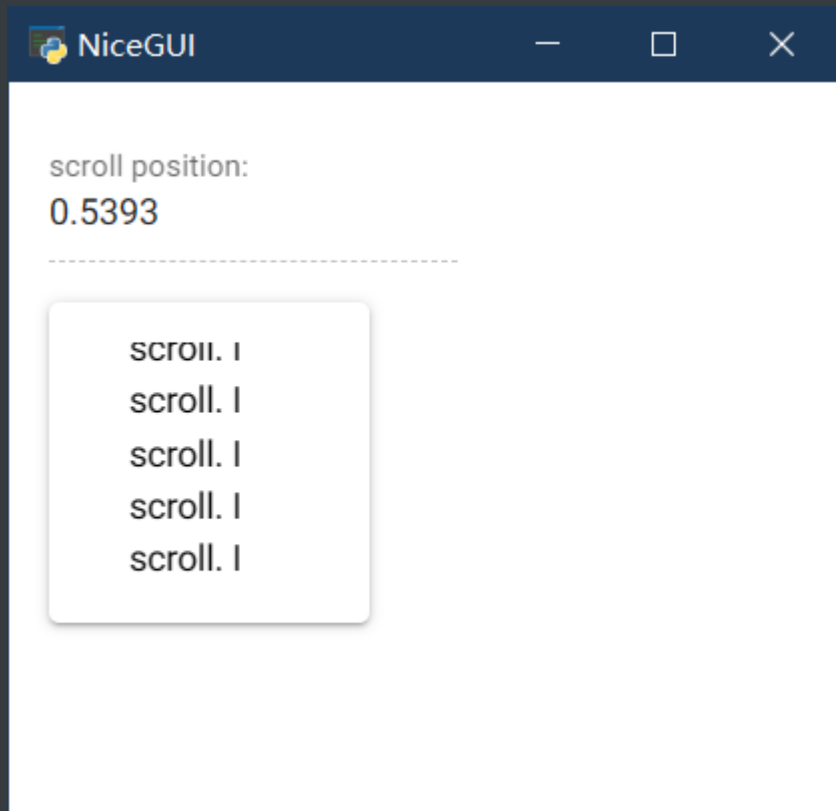


给 `on_scroll` 参数传入带一个参数的lambda表达式，可以捕获滚动的响应事件，该事件的属性均为浮点类型。以下面的代码为例，代码中捕获的该事件的 `vertical_percentage`，并将其赋给 `ui.number`：

```

1 from nicegui import ui
2
3 position = ui.number('scroll position:').props('readonly')
4 with ui.card().classes('w-32 h-32'):
5     with ui.scroll_area(on_scroll=lambda e:
6         position.set_value(e.vertical_percentage)):
7         ui.label('I scroll. ' * 20)
8
9 ui.run(native=True)

```



`scroll_to` 方法可以设置控件内容滚动到什么位置，该方法有四个参数：

`pixels` 参数，浮点类型，用像素表示目标位置，不能与 `percent` 参数同时指定。

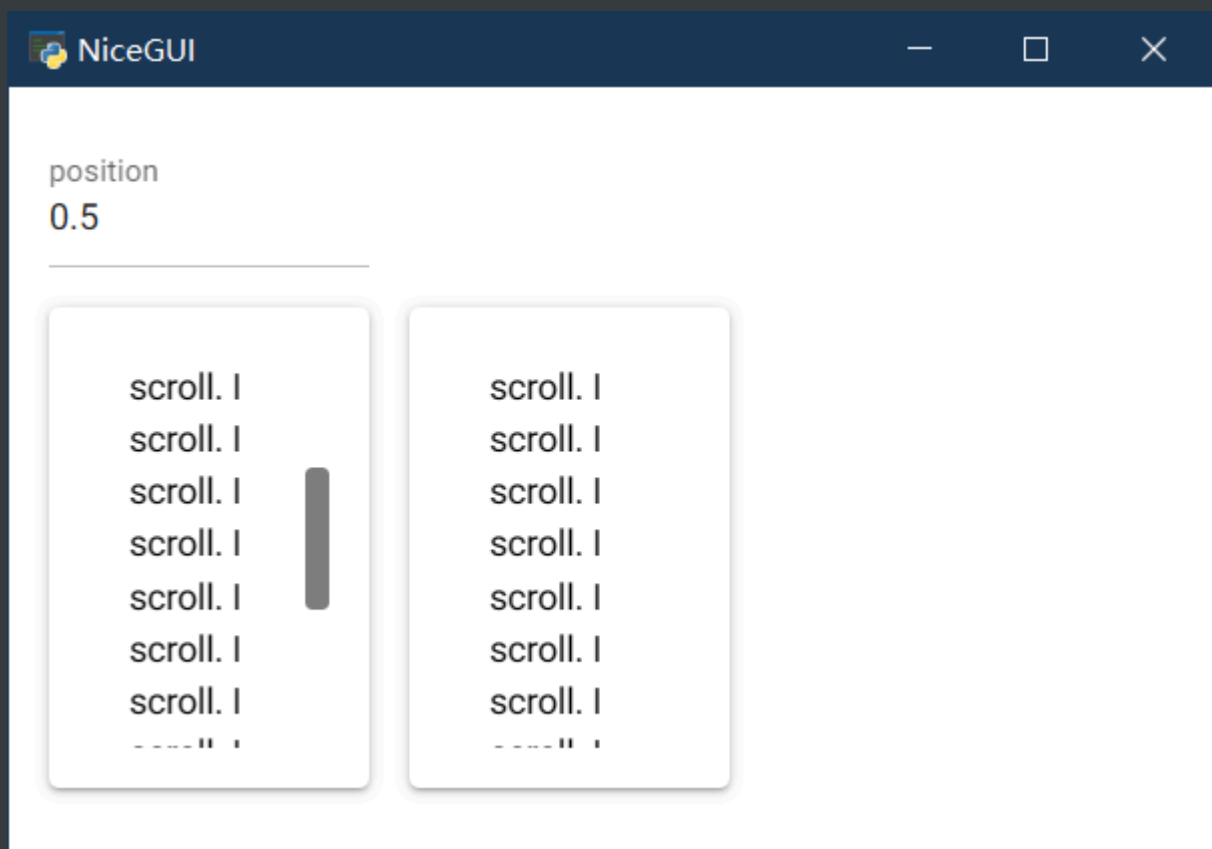
`percent` 参数，浮点类型，用百分比表示目标位置，不能与 `pixels` 参数同时指定。

`axis` 参数，字符串类型，限定为 `'vertical'` 或 `'horizontal'`，默认为 `'vertical'`，表示滚动的方向是水平还垂直。

`duration` 参数，浮点类型，表示滚动动画的持续时间，默认是 `0`，表示启用滚动动画。

以下代码在左边滚动区域的响应事件中添加执行了右边滚动区域的 `scroll_to` 方法，让左右的滚动保持一致：

```
1 from nicegui import ui
2
3 ui.number('position', value=0, min=0, max=1, step=0.1,
4          on_change=lambda e: area1.scroll_to(percent=e.value)).classes('w-32')
5
6 with ui.row():
7     with ui.card().classes('w-32 h-48'):
8         with ui.scroll_area(on_scroll=lambda e:
9                               area2.scroll_to(percent=e.vertical_percentage)) as area1:
10             ui.label('I scroll. ' * 20)
11
12     with ui.card().classes('w-32 h-48'):
13         with ui.scroll_area() as area2:
14             ui.label('I scroll. ' * 20)
15
16 ui.run(native=True)
```

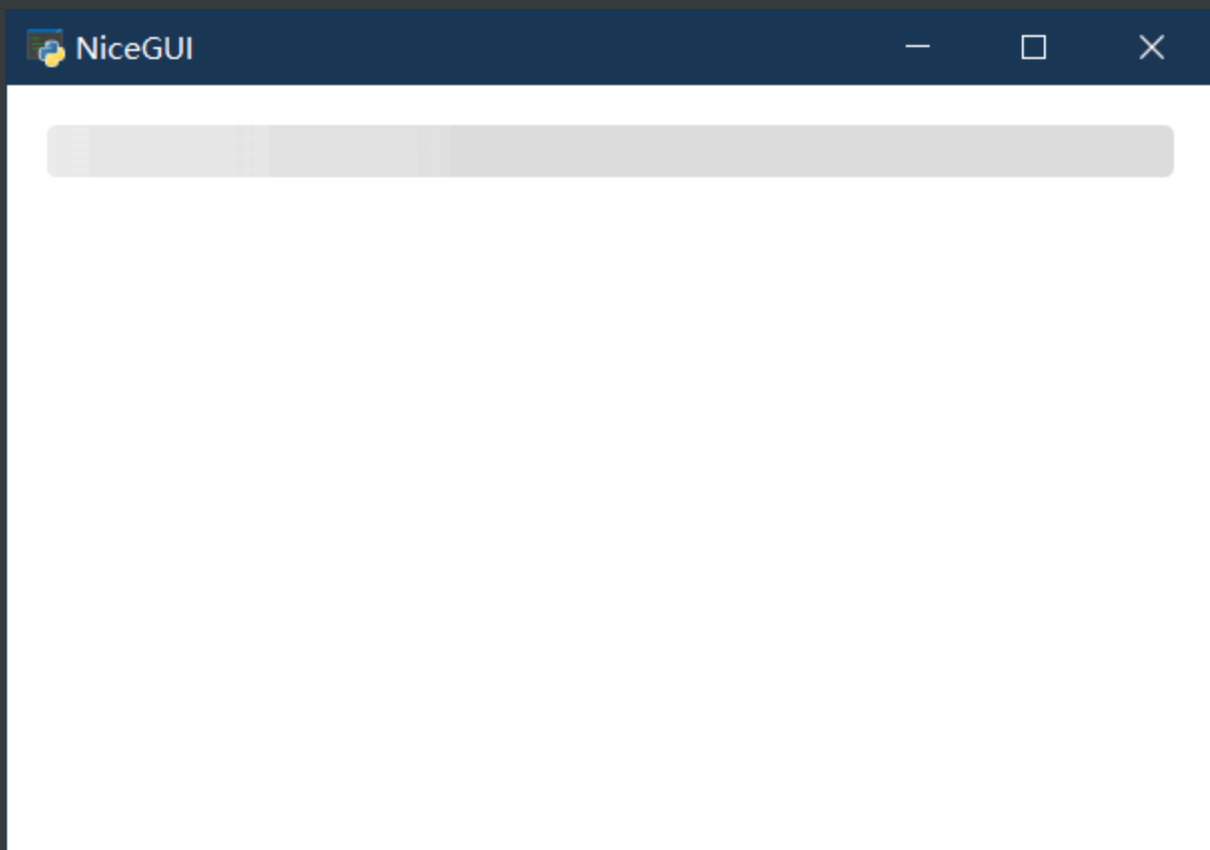


3.9.5 ui.skeleton

骨架控件可以提供一系列占位轮廓，当内容还没有加载的时候，用占位轮廓提供网页的内容结构预览。同时，骨架控件上的鼠标样式会显示为正忙的样式，表示内容正在加载。

以下为代码示例：

```
1 from nicegui import ui
2
3 ui.skeleton(type='rect', tag='div', animation='wave', animation_speed=1.5,
4             square=False, bordered=False, size=None, width=None,
5             height=None).classes('w-full')
6 ui.run(native=True)
```



`type` 参数，字符串类型，表示骨架的基本形状，支持 'text'、'rect'、'circle'、'QBtn'、'QBadge'、'QChip'、'QToolbar'、'QCheckbox'、'QRadio'、'QToggle'、'QSlider'、'QRange'、'QInput'、'QAvatar'，默认为 'rect'。

`tag` 参数，字符串类型，表示创建骨架控件用的HTML标签，默认为 'div'。

`animation` 参数, 字符串类型, 骨架控件的动画, 因为是在加载过程中占位显示, 必须要有动画, 以缓解用户等待期间的焦虑。支持 'wave'、'pulse'、'pulse-x'、'pulse-y'、'fade'、'blink'、'none', 默认为 'wave'。

`animation_speed` 参数, 浮点类型, 表示动画的速度, 即在多少秒完成一次动画循环, 默认为 1.5。

`square` 参数, 布尔类型, 表示是否移除骨架控件的圆角, 默认为 False。

`bordered` 参数, 布尔类型, 表示是否显示骨架控件的边框, 默认为 False。

`size` 参数, 字符串类型, 表示使用CSS的大小单位指定骨架控件的大小, 此时骨架控件显示为正方形或者圆形(取决于 `type` 参数), 并且会覆盖 `width` 参数和 `height` 参数的设置, 默认为 None。

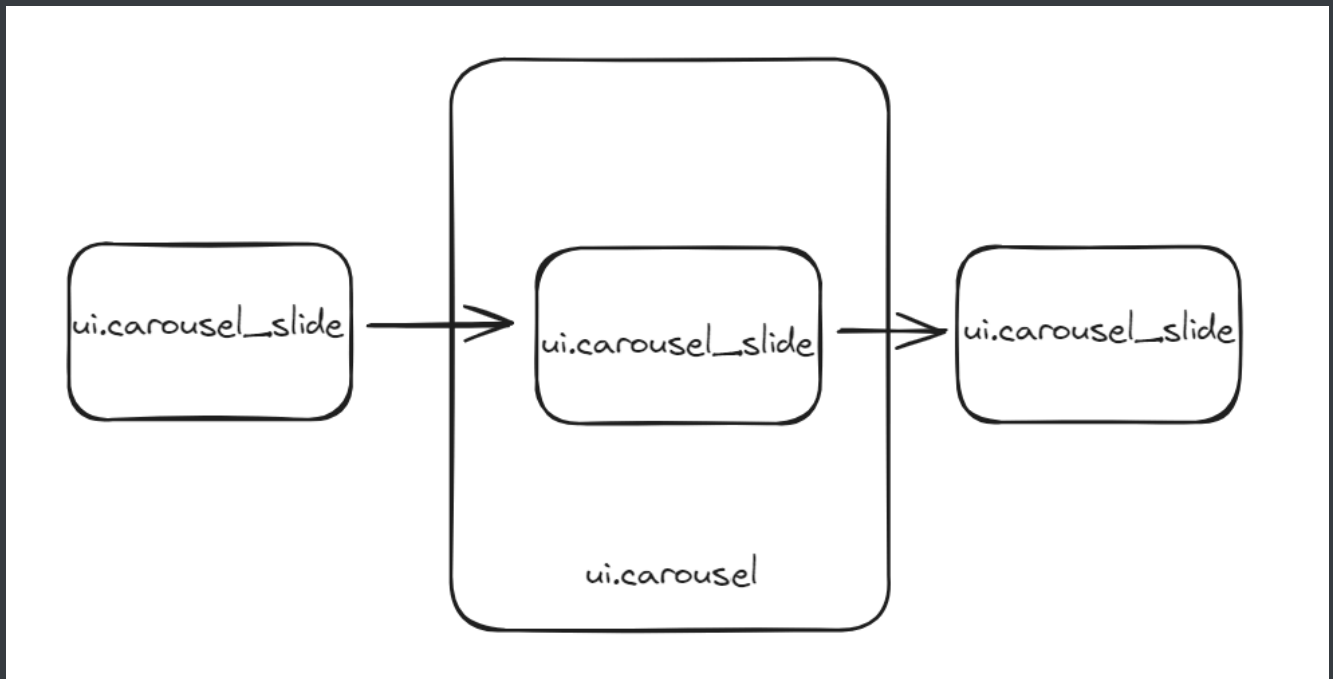
`width` 参数, 字符串类型, 表示使用CSS的大小单位指定骨架控件的宽度, 会被 `size` 参数覆盖, 默认为 None。

`height` 参数, 字符串类型, 表示使用CSS的大小单位指定骨架控件的高度, 会被 `size` 参数覆盖, 默认为 None。

3.9.6 ui.carousel

轮播图是一种常见的网页设计元素, 主要用于提供网页内容的快速展示和导航。它通过切换多个图片或内容, 吸引用户的注意力, 提高页面的视觉吸引力。轮播图支持自定义轮播图片、轮播动画效果等, 能够在可视化应用中展示多张图片轮流播放的效果。

在NiceGUI中, `ui.carousel` 就是可以实现轮播图效果的控件, 不过它的英文是carousel, 翻译过来的话是旋转木马, 听起来不太像控件, 这里就用轮播图代替。



NiceGUI的轮播图控件，本质上是一种容器，轮播图会依次展示每个子控件，放在其中的子控件就是像一页一页的幻灯片。一般来说，轮播图子控件应该是 `ui.carousel_slide`，实际上用其他控件也可以。

`ui.carousel` 有五个参数：

`value` 参数，字符串类型或者 `ui.carousel_slide` 类型，表示轮播图初始展示哪一个子控件，默认为 `None`，表示展示第一个。

`on_value_change` 参数，可调用类型，表示轮播图当前展示的子控件变化时，执行什么操作。

`animated` 参数，布尔类型，表示是否开启切换动画。

`arrows` 参数，布尔类型，表示是否显示手动切换上、下一个子控件的箭头。

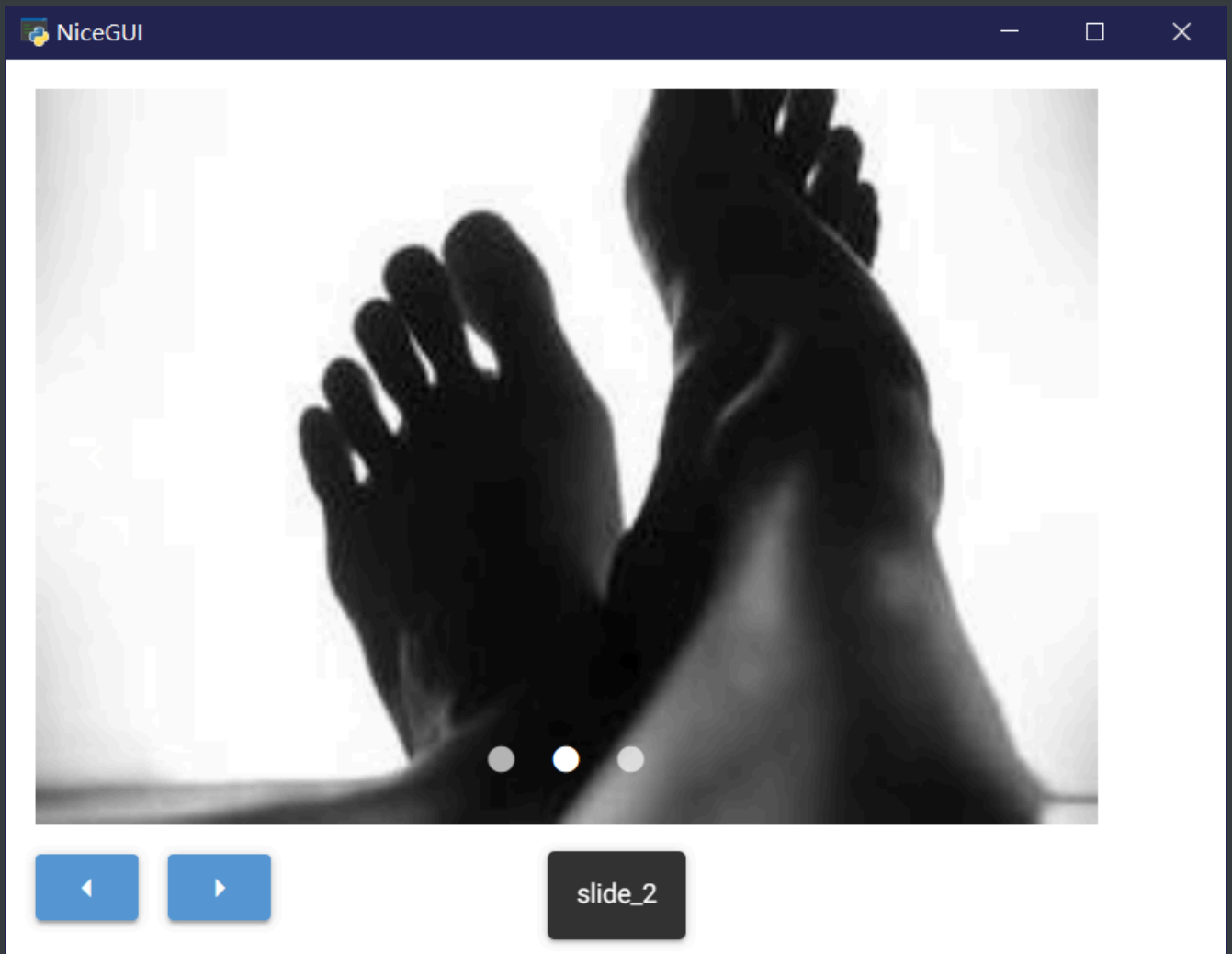
`navigation` 参数，布尔类型，表示是否显示下面直接切换哪一个子控件的圆点。

`previous` 方法，切换上一个子控件。

`next` 方法，切换下一个子控件。

`ui.carousel_slide` 只有一个字符串参数 `name`，也就是上面 `ui.carousel` 中 `value` 用到的指定当前页的值。默认没有指定的话，这个值是自动生成的——`'slide_1'` 这种名字，下划线后的数字代表当前控件在所有子控件中的排序。

```
1 from nicegui import ui
2
3 with ui.carousel(animated=True, arrows=True, navigation=True, on_value_change=lambda
  e: ui.notify(e.value))\
4     .props('autoplay=5000 infinite') as carousel:
5     with ui.carousel_slide().classes('p-0'):
6         ui.interactive_image(
7             'https://picsum.photos/id/30/270/180').classes('w-full h-full')
8     with ui.carousel_slide().classes('p-0'):
9         ui.interactive_image(
10            'https://picsum.photos/id/31/270/180').classes('w-full h-full')
11    with ui.carousel_slide().classes('p-0'):
12        ui.interactive_image(
13            'https://picsum.photos/id/32/270/180').classes('w-full h-full')
14 with ui.row():
15     ui.button(icon='arrow_left', on_click=carousel.previous)
16     ui.button(icon='arrow_right', on_click=carousel.next)
17
18 ui.run(native=True)
```



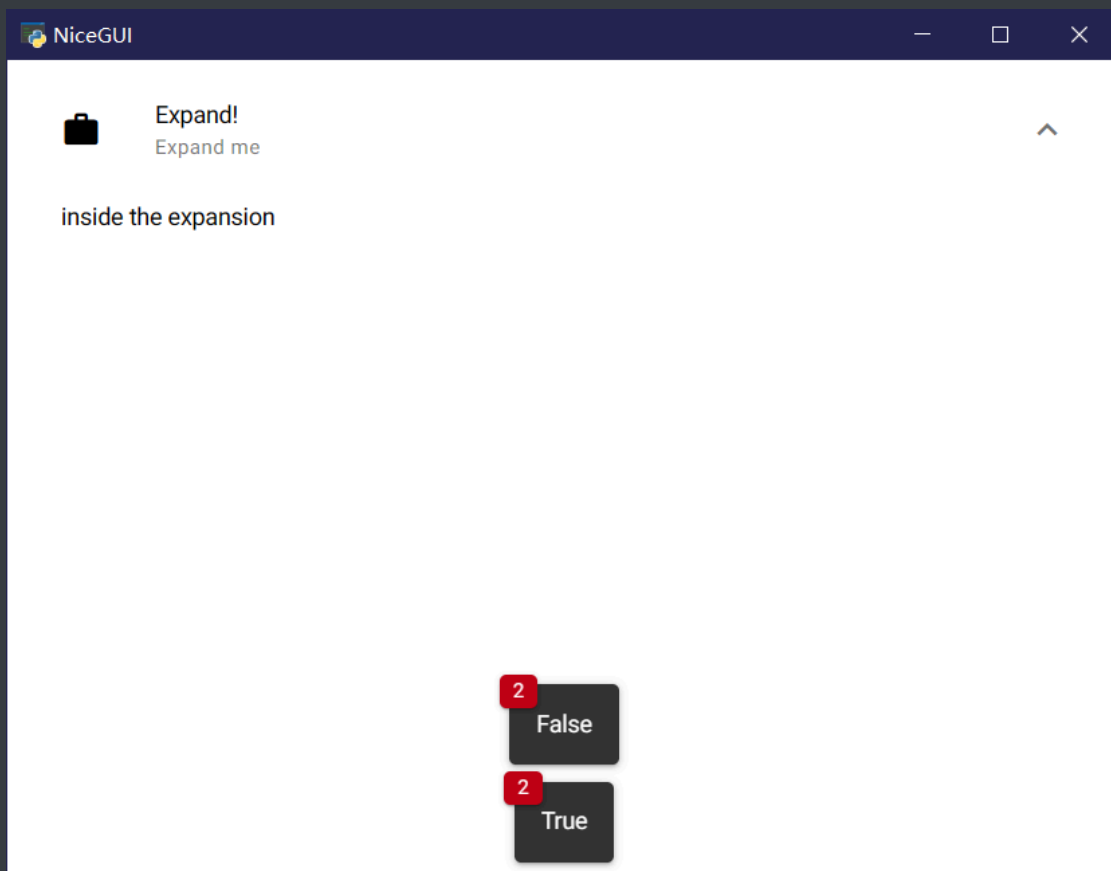
3.9.7 ui.expansion

在其他UI框架中可能叫做Accordion或者手风琴，这里也称之为手风琴控件，而不是直译其为扩大。

手风琴控件是一个用来展示多个面板的控件，这些面板同时只能展开一项（也可以一项都不展开），和选项卡控件有异曲同工之妙。手风琴控件允许在一个紧凑的空间中显示许多链接，通过单击或点击来展开和折叠，使得用户可以在一个可视化的界面中快速访问不同的信息或功能。这种控件通常包括一个或多个可折叠的面板，每个面板可以包含文本、图像或其他内容，用户可以通过点击面板的标题来展开或折叠对应的内容。手风琴控件的设计旨在提供一个直观且用户友好的界面，使用户能够轻松地在不同部分之间切换，同时保持界面的整洁和有序。

先看代码和效果图：

```
1 from nicegui import ui
2
3 with ui.expansion(
4     text="Expand!",
5     caption="Expand me",
6     icon="work",
7     group=None,
8     value=False,
9     on_value_change=lambda e:ui.notify(e.value),
10 ).classes("w-full"):
11     ui.label("inside the expansion")
12
13 ui.run(native=True)
```



`ui.expansion` 有六个参数：

`text` 参数，字符串类型，表示手风琴控件的文本内容。

`caption` 参数，字符串类型，表示手风琴控件的说明性文本内容（也可以称之为副标题或者标签文本，比文本内容小一点）。

`icon` 参数，字符串类型，表示手风琴控件的图标。

`group` 参数，字符串类型，表示手风琴控件的分组，默认为 `None`。其实，单个 `ui.expansion` 没法组成手风琴控件，需要多个配合。在没有指定此参数的情况下，每个 `ui.expansion` 都是独立打开、关闭的，只有在指定相同的此参数之后，各个 `ui.expansion` 的开闭才会关联，每组 `ui.expansion` 中只允许一个 `ui.expansion` 开启，点开其他 `ui.expansion` 会让已经打开的 `ui.expansion` 关闭。

`value` 参数，布尔类型，表示 `ui.expansion` 的开关状态，默认为 `False`。

`on_value_change` 参数，可调用类型，表示手风琴控件的值变化时执行什么操作。

`open` 方法，调用此方法会打开手风琴控件。

`close` 方法，调用此方法会关闭手风琴控件。

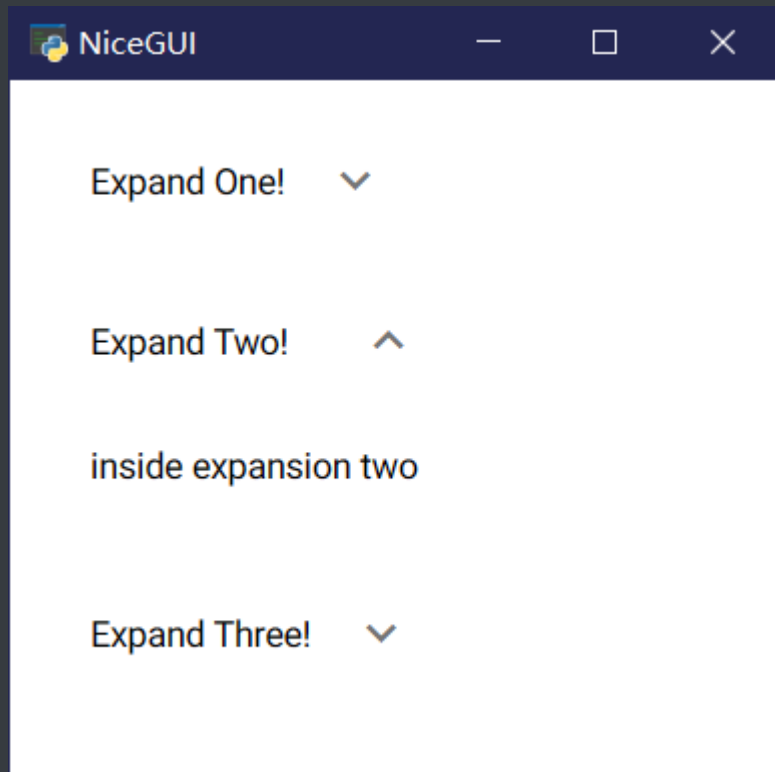
如果想用图片代替手风琴控件的图标，可以这样操作：

```
1  from nicegui import ui
2
3  with ui.expansion() as expansion:
4      with expansion.add_slot('header'):
5          with ui.row().classes('items-center'):
6              ui.image('https://nicegui.io/logo.png').classes('w-16')
7              ui.label('LOGO')
8          ui.label('What a nice GUI!')
9
10 ui.run(native=True)
```




前面提到的 `group` 参数，用法也很简单：

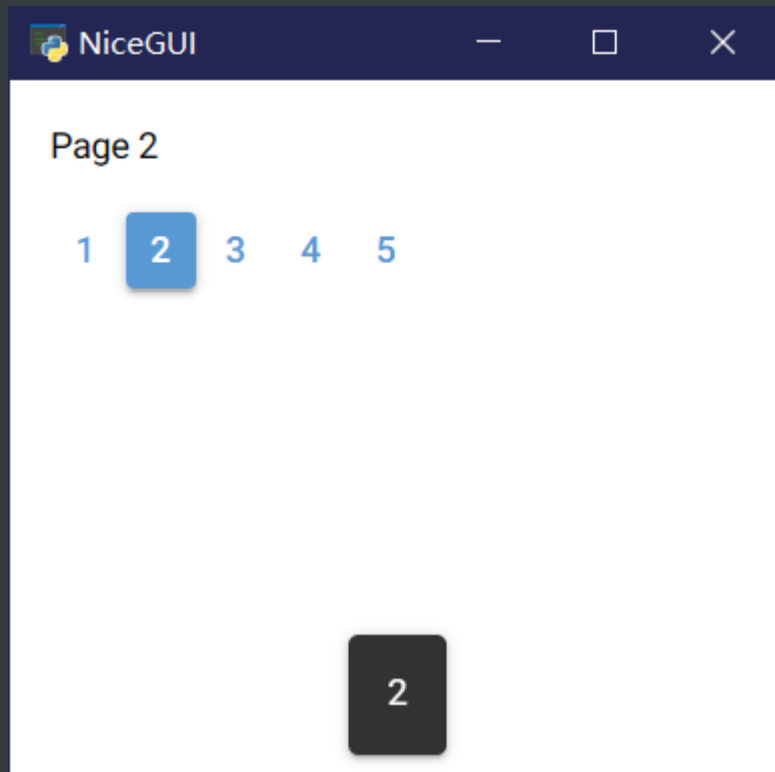
```
1 from nicegui import ui
2
3 with ui.expansion(text='Expand One!', group='group'):
4     ui.label('inside expansion one')
5 with ui.expansion(text='Expand Two!', group='group'):
6     ui.label('inside expansion two')
7 with ui.expansion(text='Expand Three!', group='group'):
8     ui.label('inside expansion three')
9
10 ui.run(native=True)
```



3.9.8 ui.pagination

常在网页中看到多页内容的底部有标着页码的分页控件，点击后一页或者对应页码可以直接跳转。在NiceGUI，实现此功能的是 `ui.pagination`。

```
1 from nicegui import ui
2
3 p = {"value": 0}
4 ui.label().bind_text_from(p, "value", lambda v: f"Page {v}")
5 ui.pagination(
6     min=1,
7     max=5,
8     direction_links=False,
9     value=1,
10    on_change=lambda e: ui.notify(e.value)
11 ).bind_value_to(p)
12
13
14 ui.run(native=True)
```



`ui.pagination` 有五个参数：

`min` 参数，整数类型，分页控件的页码最小值。

`max` 参数，整数类型，分页控件的页码最小值。

`direction_links` 参数，布尔类型，是否显示前一页、后一页的链接。

`value` 参数，整数类型，分页控件的页码初始值。如果没有指定初始值，初始值是最小值。

`on_change` 参数，可调用类型，表示分页控件的值变化时执行什么操作。

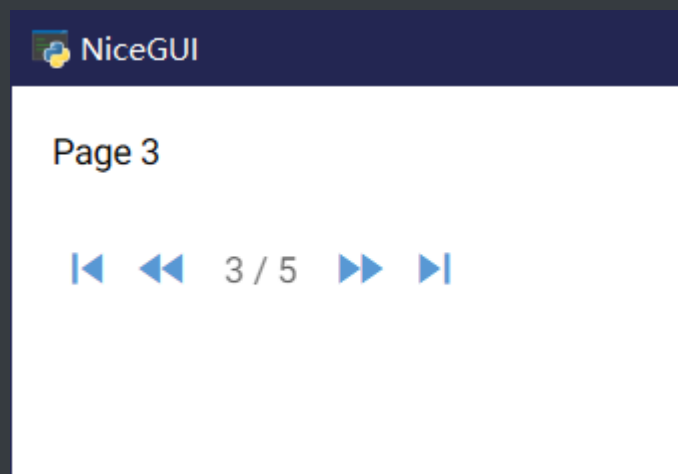
分页控件的更多样式设计可以参考[Quasar官网](#)：

```
1 from nicegui import ui
2
3 p = {"value": 0}
4 ui.label().bind_text_from(p, "value", lambda v: f"Page {v}")
5 ui.pagination(
6     min=1,
7     max=5,
8     direction_links=True,
```

```

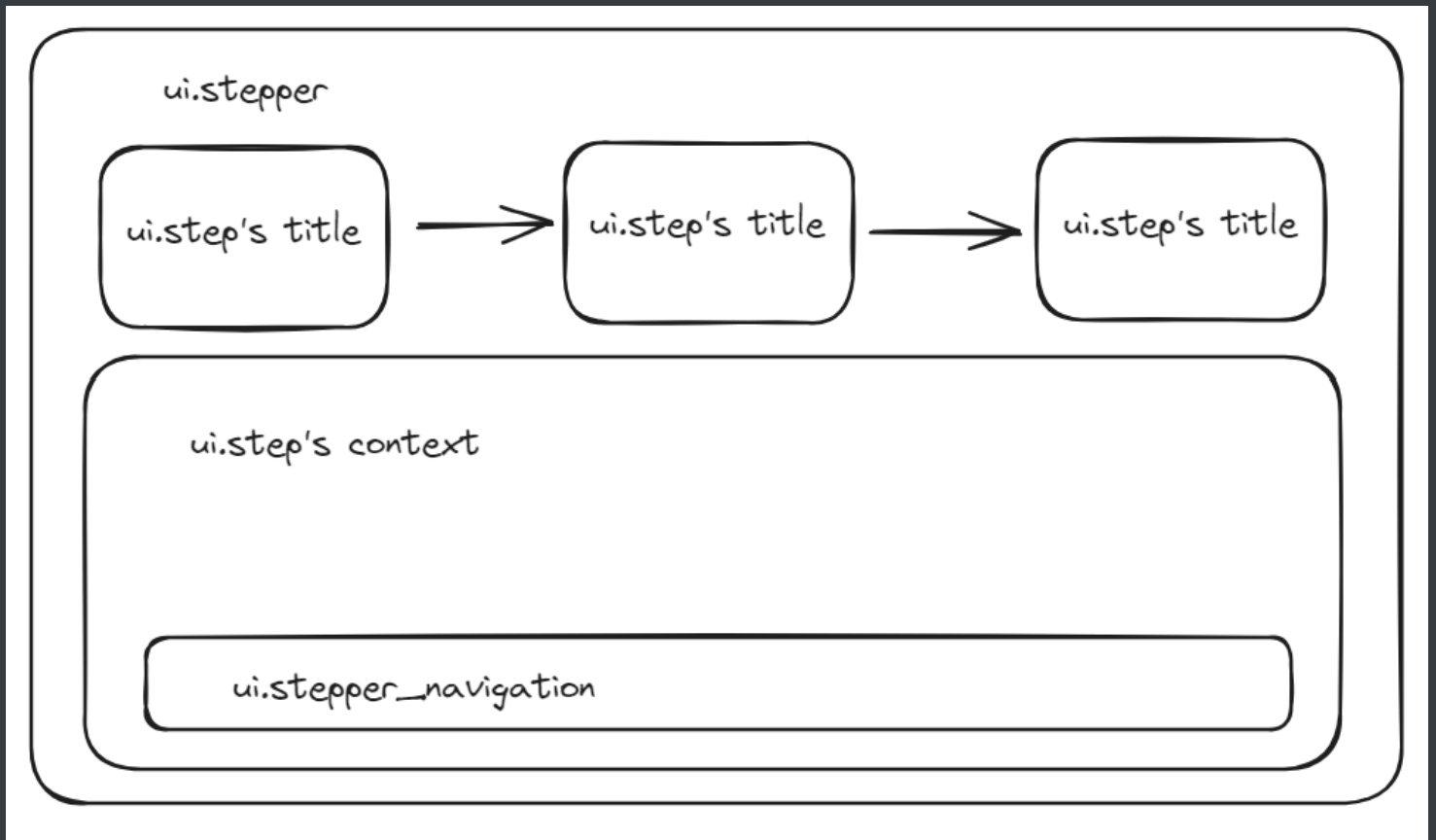
9     value=1,
10     on_change=lambda e: ui.notify(e.value)
11 ).props(''
12     boundary-links
13     input
14     icon-first="skip_previous"
15     icon-last="skip_next"
16     icon-prev="fast_rewind"
17     icon-next="fast_forward"
18     '').bind_value_to(p)
19
20 ui.run(native=True)

```



3.9.9 ui.stepper

`ui.stepper` 是一个类似向导功能的综合控件，可以给用户提供直观的操作引导。从本质上说，`ui.stepper` 是个容器，有点像前面提到的轮播图，每次都是展示当前子控件的内容。不同的是，`ui.stepper` 会在上方（或者左边）提供全部子控件标题作为步骤的预览，大致结构如下图所示：



与 `ui.stepper` 相关的控件有 `ui.step` 和 `ui.stepper_navigation`。`ui.step` 是每一步的步骤，放在 `ui.stepper` 的上下文内。`ui.step` 的上下文主要放置步骤的操作说明，这里的内容会展示在步骤预览的下方。`ui.stepper_navigation` 是导航栏，里面放置下一步、上一步之类的操作按钮，通常放置在 `ui.step` 内，也可以放置在外面。

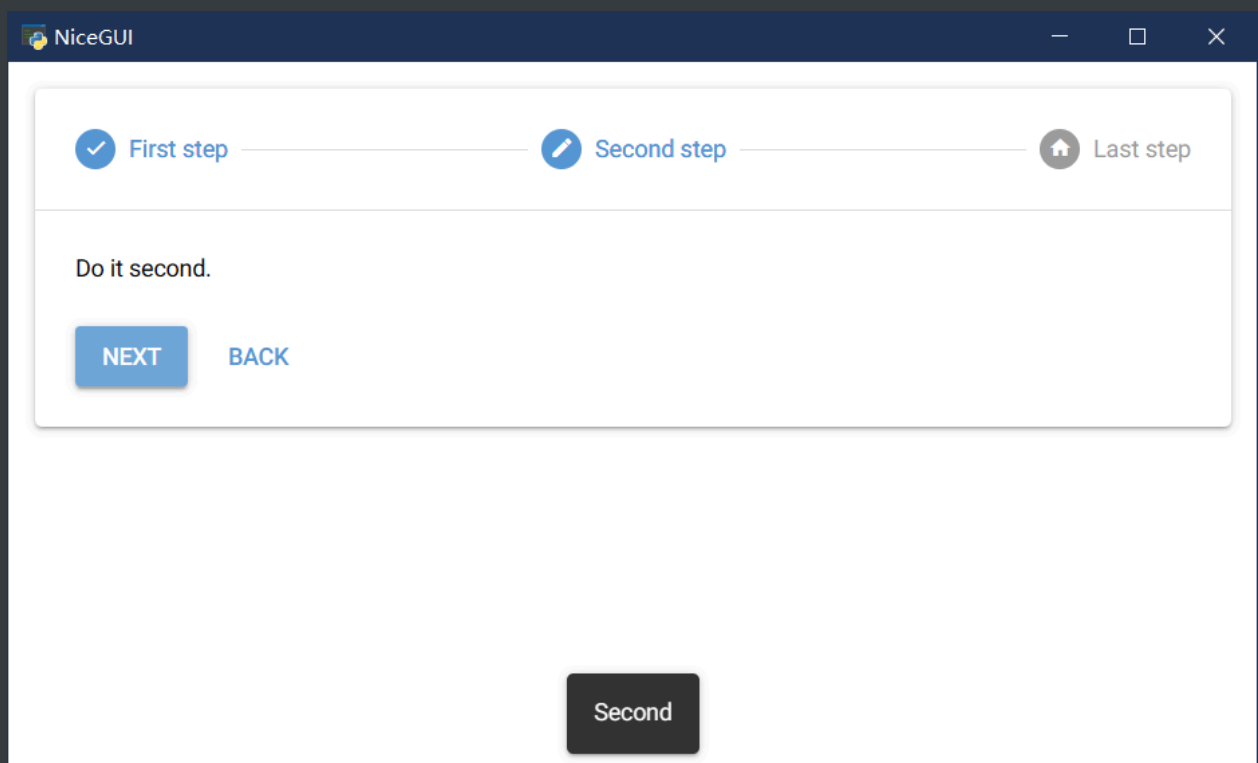
顺着上面的思路，代码如下：

```
1 from nicegui import ui
2
3 with ui.stepper(
4     value='First',
5     on_value_change=lambda e: ui.notify(e.value),
6     keep_alive=True
7 ).classes('w-full') as stepper:
8     with ui.step(name='First', title='First step', icon='home') as first:
9         ui.label('Do it fisrt.')
10         with ui.stepper_navigation(wrap=True):
11             ui.button('Next', on_click=stepper.next)
12     with ui.step(name='Second', title='Second step', icon='home') as second:
13         ui.label('Do it second.')
```

```

14     with ui.stepper_navigation(wrap=True):
15         ui.button('Next', on_click=stepper.next)
16         ui.button('Back', on_click=stepper.previous).props('flat')
17     with ui.step(name='last', title='Last step', icon='home') as last:
18         ui.label('Do it last.')
19     with ui.stepper_navigation(wrap=True):
20         ui.button('Done', on_click=lambda: ui.notify(
21             'Done!', type='positive'))
22         ui.button('Back', on_click=stepper.previous).props('flat')
23
24 ui.run(native=True)

```



`ui.stepper` 支持三个参数：

`value` 参数，字符串类型或者 `ui.step` 类型，表示初始选择的步骤是哪一个，默认为 `None`，即第一个。因为步骤的定义在这一行后面，直接使用 `ui.step` 类型会触发未定义报错，要指定非第一个当初始选择步骤，一般只能用字符串类型的名字，即 `ui.step` 的 `name` 属性的值。实际上 `ui.stepper` 点击下一步的操作，也就是将 `ui.stepper` 的 `value` 设置为指定 `ui.step` 的 `name` 的值。

`on_value_change` 参数，可调用类型，当 `ui.stepper` 的 `value` 变化时执行的操作。

`keep_alive` 参数，布尔类型，表示是否对容器内的控件启用VUE的 `keep-alive` 组件。启用此组件，容器内控件不会在不可见的时候销毁，而是一直保持存活，以免再次访问控件时，控件的状态因为重建而被重置。

`previous` 方法，切换上一步骤。

`next` 方法，切换下一步骤。

`ui.step` 支持三个参数：

`name` 参数，字符串类型，和 `ui.tab` 类似，`name` 也是 `ui.step` 的唯一标识符。同时，当 `title` 没有被定义时，这也是 `title` 的默认值。

`title` 参数，字符串类型，表示步骤的标题，当此参数没有定义、为默认的 `None` 时，此参数会取 `name` 的值。

`icon` 参数，字符串类型，表示步骤的默认图标。注意，此图标只有在当前步骤没有被选定或者完成的时候显示。

`ui.stepper_navigation` 支持一个布尔类型参数 `wrap`，表示里面的内容如果超出容器宽度的话，是否自动换行，默认为 `True`。

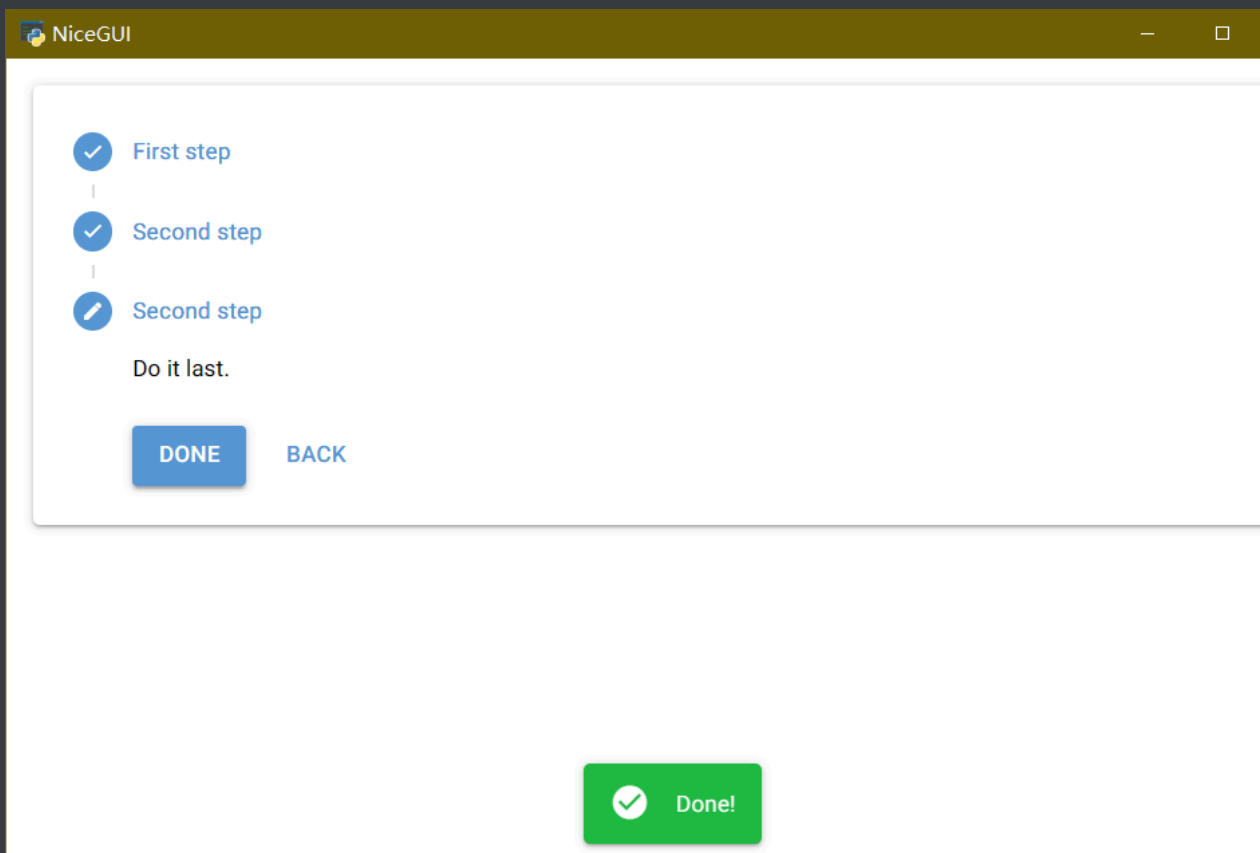
`ui.stepper` 除了横向显示步骤，还可以变成竖向显示，只需调用 `props('vertical')`，设置 `'vertical'` 即可：

```
1 from nicegui import ui
2
3 with ui.stepper(
4     value='First',
5     on_value_change=lambda e: ui.notify(e.value),
6     keep_alive=True
7 ).classes('w-full').props('vertical') as stepper:
8     with ui.step(name='First', title='First step', icon='home') as first:
9         ui.label('Do it fisrt.')
10         with ui.stepper_navigation(wrap=True):
11             ui.button('Next', on_click=stepper.next)
12     with ui.step(name='Second', title='Second step', icon='home') as second:
13         ui.label('Do it second.')
```

```

14     with ui.stepper_navigation(wrap=True):
15         ui.button('Next', on_click=stepper.next)
16         ui.button('Back', on_click=stepper.previous).props('flat')
17     with ui.step(name='last', title='Second step', icon='home') as last:
18         ui.label('Do it last.')
19     with ui.stepper_navigation(wrap=True):
20         ui.button('Done', on_click=lambda: ui.notify(
21             'Done!', type='positive'))
22         ui.button('Back', on_click=stepper.previous).props('flat')
23
24 ui.run(native=True)

```

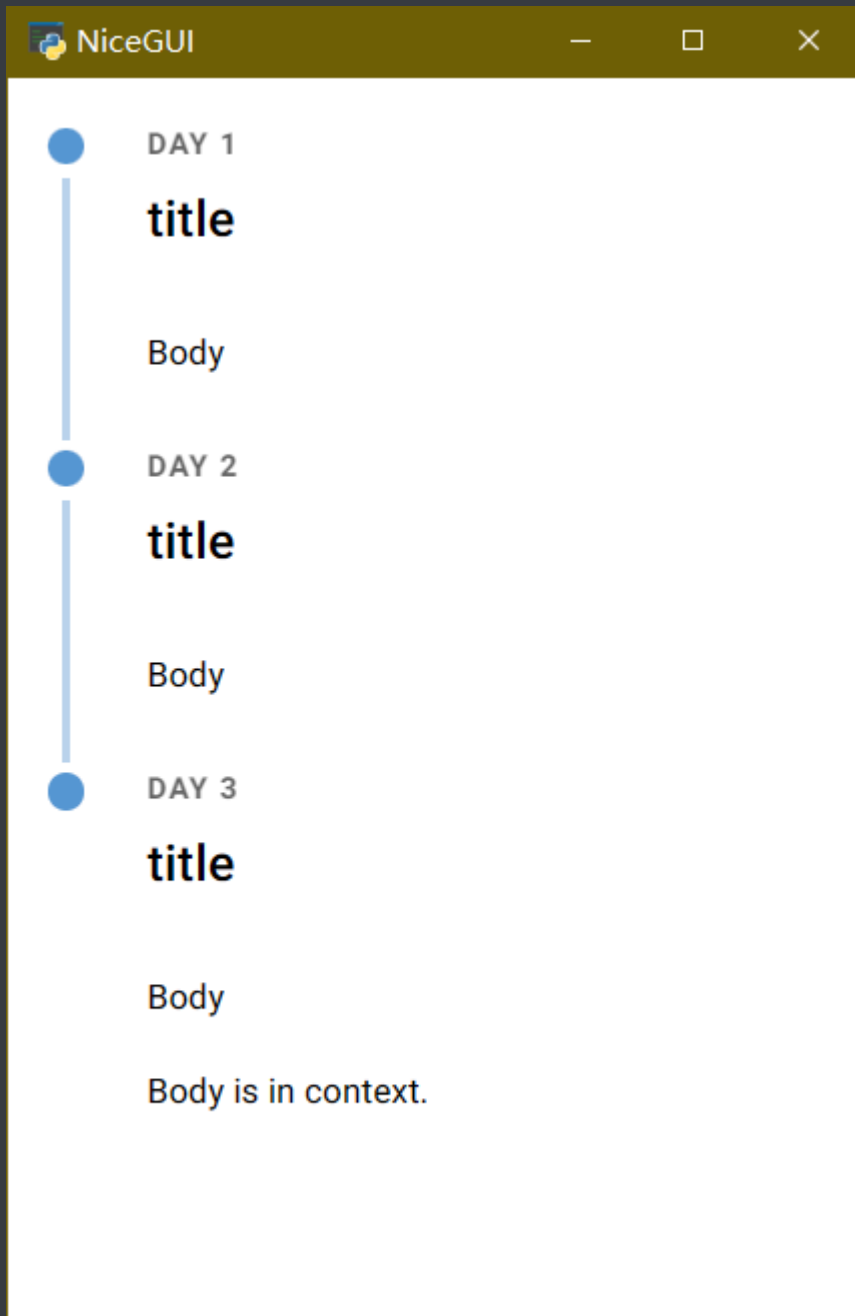


3.9.10 ui.timeline

时间线控件有点像 `ui.stepper`，不同的是，时间线控件只是展示为主的控件，并不具备交互功能。时间线控件可以像绘制思维导图一样，提供一个按照时间排序展示内容的控件。

以代码为例，`ui.timeline` 需要内嵌 `ui.timeline_entry`，才能构成完整的时间线控件：

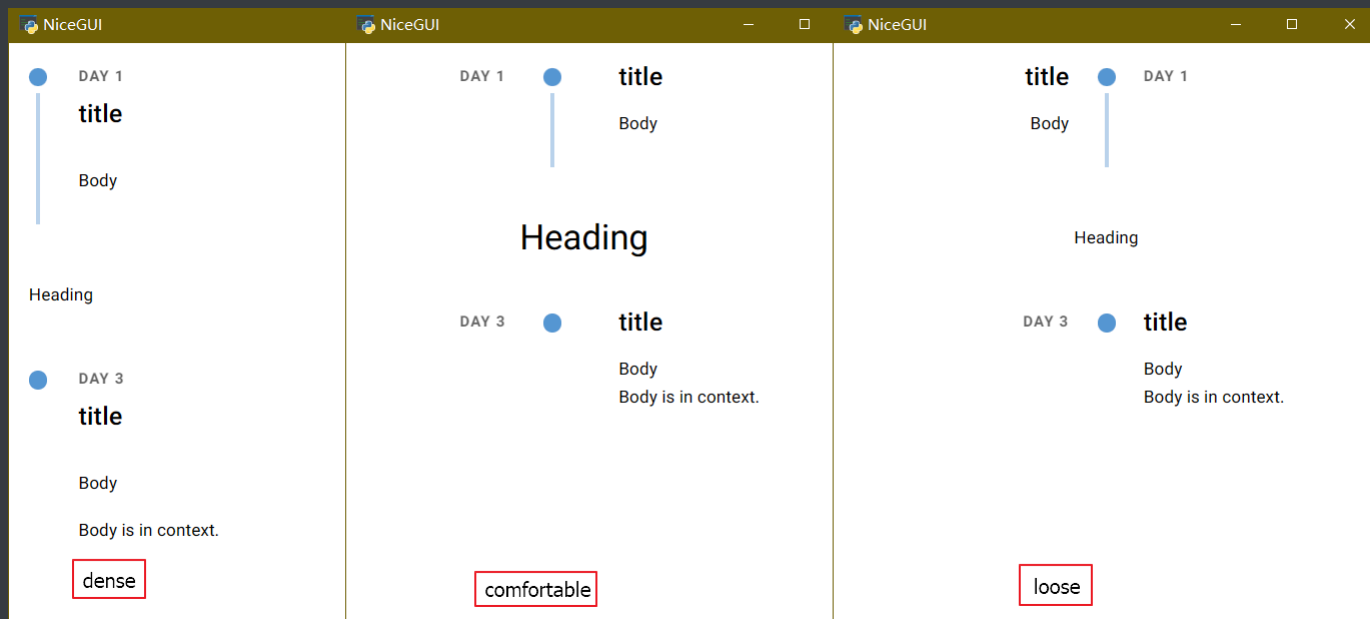

```
1 from nicegui import ui
2
3 with ui.timeline(side='right', layout='dense', color=None):
4     for i in range(1,3):
5         ui.timeline_entry(body='Body', side='left', heading=False, tag=None,
6                             icon=None, avatar=None, title='title', subtitle=f'Day {i}',
7                             color=None)
8         with ui.timeline_entry(body='Body', side='right', heading=False, tag=None,
9                                 icon=None, avatar=None, title='title', subtitle=f'Day {i+1}',
10                                 color=None):
11             ui.label('Body is in context.')
12
13 ui.run(native=True)
```



`ui.timeline` 支持三个参数:

`side` 参数, 字符串类型, 表示时间线的内容在时间线的哪边, 允许值是 `'left'` 和 `'right'`, 默认是 `'left'`, 即左边。

`layout` 参数, 字符串类型, 表示时间线的布局, 允许值是 `'dense'`、`'comfortable'`、`'loose'`, 默认是 `'dense'`。dense布局是时间点的大字标题、标题、副标题和主要内容在时间线指定的 `side` 一边。comfortable布局是时间点的大字标题、标题和主要内容在时间线指定的 `side` 一边, 副标题在另一边。loose布局是大字标题在中间, 标题和主要内容在时间点指定的 `side` 一边, 副标题在另一边。三种布局的对比如下:



`color` 参数，字符串类型，表示时间线的颜色。

`ui.timeline_entry` 支持的控件比较多，有九个：

`body` 参数，字符串类型，表示时间点的主要内容。

`side` 参数，字符串类型，表示时间线的内容在时间线的哪边，允许值是 `'left'` 和 `'right'`，默认是 `'left'`，即左边。注意，`ui.timeline_entry` 的 `side` 只有在 `ui.timeline` 的 `layout` 属性为 `'loose'` 时生效。

`heading` 参数，布尔类型，表示时间点是否为大字标题，默认为 `False`。如果为 `True`，当前时间点将只显示主要内容，并且样式与其他时间点不同，具体参考上面的布局对比图片。

`tag` 参数，字符串类型，表示当前时间点如果被设定为大字标题，使用什么HTML标签当做大字标题的外围标签，默认是 `'h3'`。

`icon` 参数，字符串类型，表示当前时间点的图标。

`avatar` 参数，字符串类型，表示当前时间点的头像，可以用图片的地址。注意，如果指定了 `icon` 参数，则只会显示图标，因为图标的优先级比头像高。

`title` 参数，字符串类型，表示当前时间点的标题。

`subtitle` 参数，字符串类型，表示当前时间点的副标题。

`color` 参数，字符串类型，表示时间点的颜色，如果当前时间点没有指定颜色，则采用时间线的颜色。

3.9.11 `ui.notification`

`ui.notification` 也是通知提示控件，相比 `ui.notify`，`ui.notification` 可以在显示的同时更新内容和一些其他属性，还可以使用 `dismiss` 方法手动移除。

而且，`ui.notification` 的参数也不藏着掖着，直接放在提示里：

`message` 参数，字符串类型，信息文本，显示在通知中的主要内容。

`position` 参数，字符串类型，通知出现的位置，有 `"top-left"`、`"top-right"`、`"bottom-left"`、`"bottom-right"`、`"top"`、`"bottom"`、`"left"`、`"right"`、`"center"` 可选，默认为 `"bottom"`。

`close_button` 参数，字符串类型或者布尔型，是否显示关闭按钮，如果是字符串类型，关闭按钮的文字就是给定的文字，默认为 `False`。

`type` 参数，字符串类型，通知的类型，有 `"positive"`、`"negative"`、`"warning"`、`"info"`、`"ongoing"`，默认为 `None`，不是其中的任何一种。

`color` 参数，字符串类型，通知的背景颜色。

`multi_line` 参数，布尔类型，是否让通知内容以多行格式显示。

`icon` 参数，字符串类型，通知的图标，默认为 `None`。

`spinner` 参数，布尔类型，是否显示转盘动，默认为 `False`。

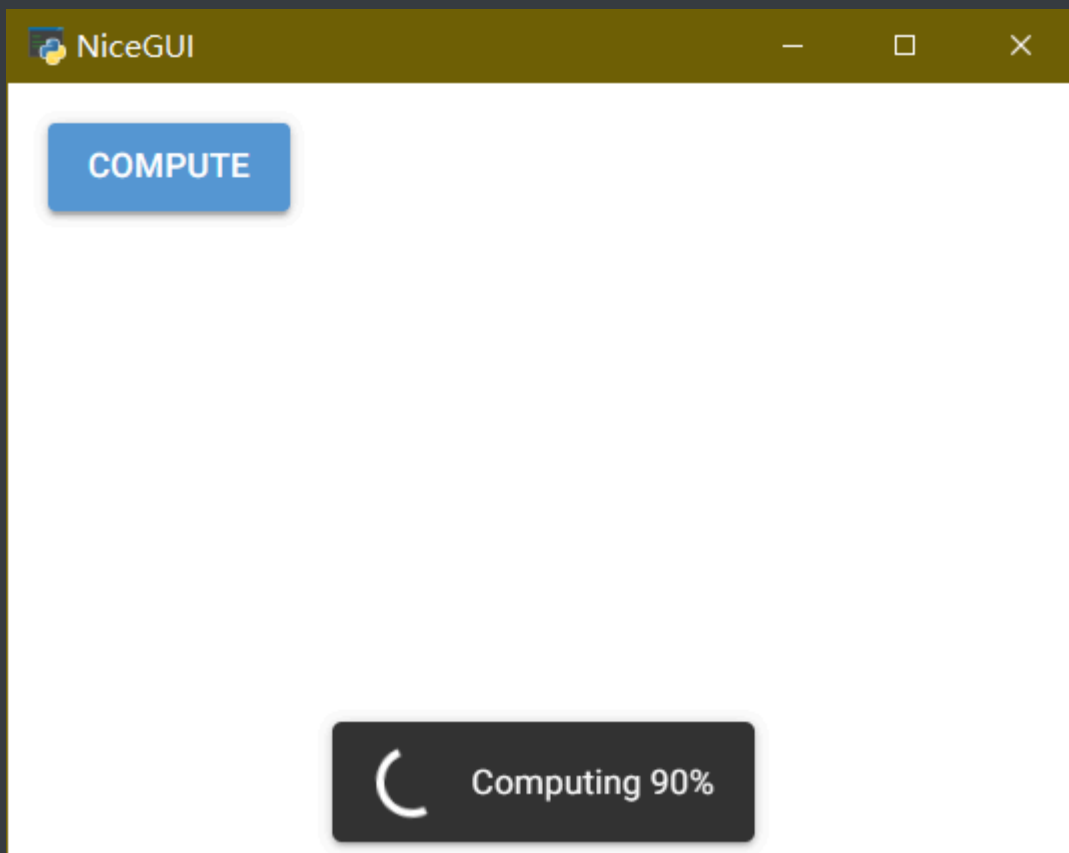
`timeout` 参数，整数型，通知自动消失的时间，单位秒，默认为 `5`，为 `0` 就是不消失。但是要确保 `close_button` 不是 `False`，或者有调用 `dismiss` 的方法，否则通知没法正常消除，影响用户体验。

`on_dismiss` 参数，可调用类型，当通知消失时执行的操作。

`options` 参数，字典类型，其他 [Quasar](#) 的通知控件API中可用的参数可以传入此字典。

由于 `ui.notification` 的特殊性，现在可以弹出一个显示百分比进度并自动消失的通知了（并不需要估计执行时间）：

```
1 import asyncio
2 from nicegui import ui
3
4 async def compute():
5     n = ui.notification(timeout=None)
6     for i in range(10):
7         n.message = f'Computing {i/10:.0%}'
8         n.spinner = True
9         await asyncio.sleep(0.2)
10    n.message = 'Done!'
11    n.spinner = False
12    await asyncio.sleep(1)
13    n.dismiss()
14
15 ui.button('Compute', on_click=compute)
16
17 ui.run(native=True)
```



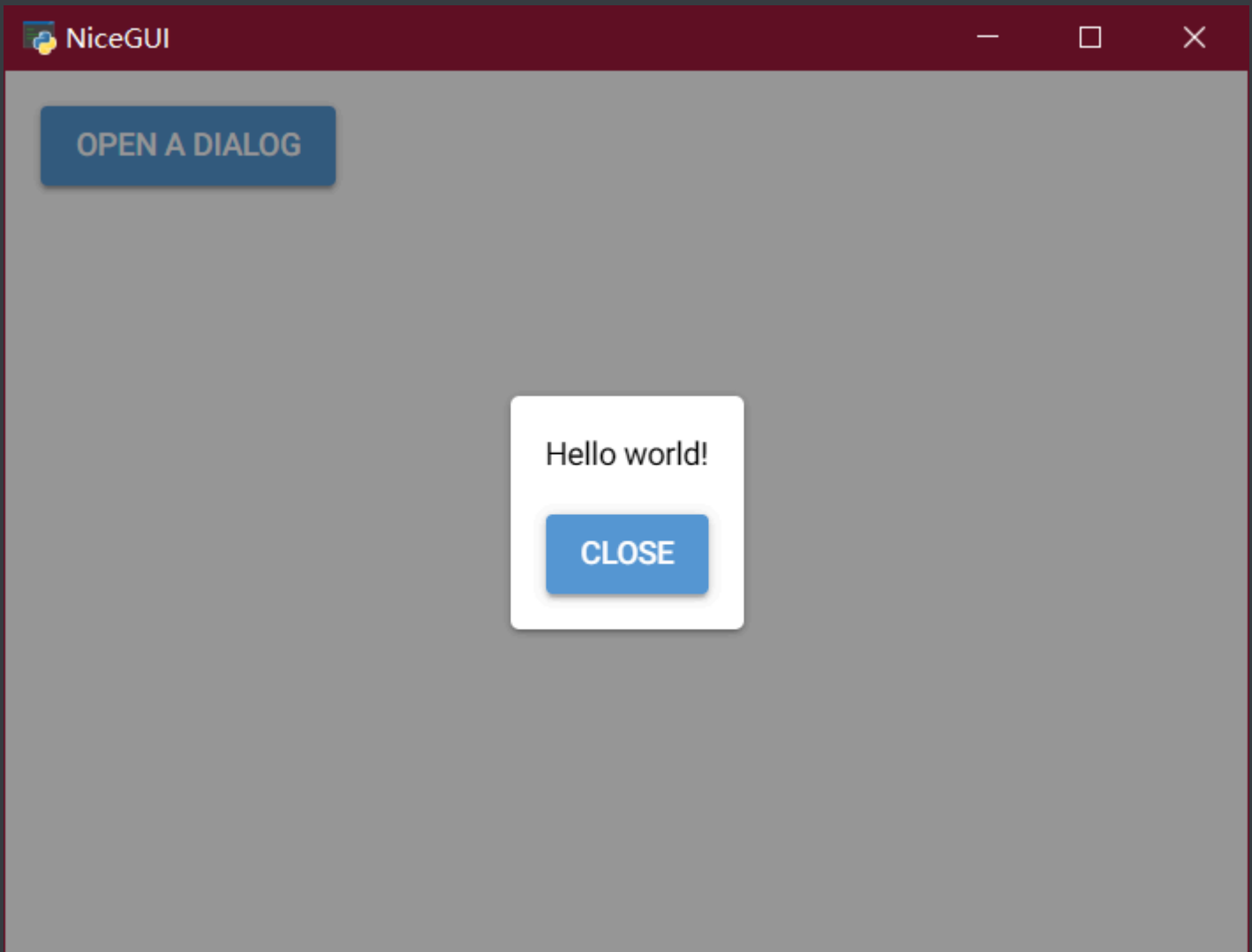
3.9.12 ui.dialog

对话框常见于桌面程序，在网页中并不多见。因为网页经常通过页面切换来跳转到特定界面，实现对话框一样的功能。但是，NiceGUI是混合框架，网页中也没法完全不用对话框。所以，还是有必要看一下对话框的用法。

`ui.dialog` 只有一个布尔类型参数 `value`，表示对话框初始的开启状态，默认为 `False`。其余的对话框设计样式可以参考[Quasar的文档](#)。

以下是示例：

```
1 from nicegui import ui
2
3 with ui.dialog() as dialog:
4     with ui.card():
5         ui.label('Hello world!')
6         ui.button('Close', on_click=dialog.close)
7
8 ui.button('Open a dialog', on_click=dialog.open)
9
10 ui.run(native=True)
```



调用对话框的 `open` 方法打开对话框，调用 `close` 方法、点击关闭按钮、点击空白处、按 `esc` 键都可以关闭对话框。如果想设置为只有点击关闭按钮才能关闭对话框，可以使用 `.props('persistent')` 添加属性 `'persistent'`。

需要注意的是，对话框实际运行时只创建一次，后续是重复使用的。关闭对话框并不会销毁对话框，只是隐藏对话框，而且下次开启时会重复使用已经创建的对话框。如果想要确保对话框内容准确，要么在打开对话框前更新对话框内容，要么每次打开前重新创建一次。更新对话框内容有两种方法，一是遍历对话框内每个控件，调用对应的更新方法；二是重新创建对话框内的内容，使用 `clear` 方法清除之后重新创建，或者使用 `refreshable` 方法包装需要更新的控件，调用 `refresh` 方法触发刷新重建。

以下面的代码为例，为了更新对话框的内容，先调用 `clear` 方法清除原有内容，然后创建新内容：

```
1 from nicegui import ui
2
3 def replace():
4     dialog.clear()
```

```

5     with dialog, ui.card().classes('w-64 h-64'):
6         ui.label('New Content')
7     dialog.open()
8
9     with ui.dialog() as dialog, ui.card():
10         ui.label('Hello world!')
11
12     ui.button('Open', on_click=dialog.open)
13     ui.button('Replace', on_click=replace)
14
15     ui.run(native=True)

```

除了在对话框里修改全局变量来传递用户选择的结果，对话框还可以通过异步等待的方式返回结果，代码如下：

```

1     from nicegui import ui
2
3     with ui.dialog() as dialog, ui.card():
4         ui.label('Are you sure?')
5         with ui.row():
6             ui.button('Yes', on_click=lambda: dialog.submit('Yes'))
7             ui.button('No', on_click=lambda: dialog.submit('No'))
8
9     async def show():
10         result = await dialog
11         ui.notify(f'You chose {result}')
12
13     ui.button('Await a dialog', on_click=show)
14
15     ui.run(native=True)

```

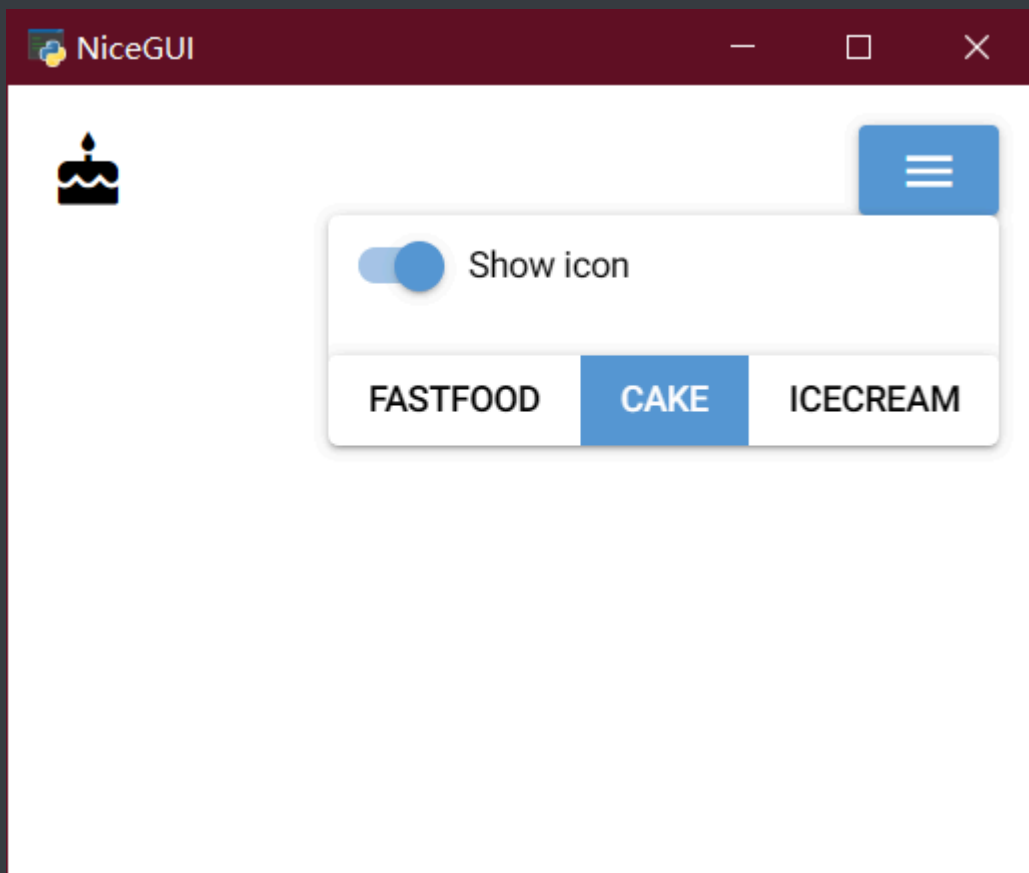
3.9.13 ui.menu 补充

`ui.menu` 中除了可以嵌入 `ui.menu_item`，还可以嵌入其他控件，有时候会有意想不到的效果：


```

1  from nicegui import ui
2
3  with ui.row().classes('w-full items-center'):
4      icon = ui.icon('', size='md').classes('mr-auto')
5      ui.space()
6      with ui.button(icon='menu')as button:
7          with ui.menu().props('auto-close'):
8              with ui.column():
9                  switch =ui.switch('Show icon')
10                 toggle = ui.toggle(['fastfood', 'cake', 'icecream'],
value='fastfood')
11             icon.bind_name_from(toggle, 'value').bind_visibility_from(switch,'value')
12
13  ui.run(native=True)

```



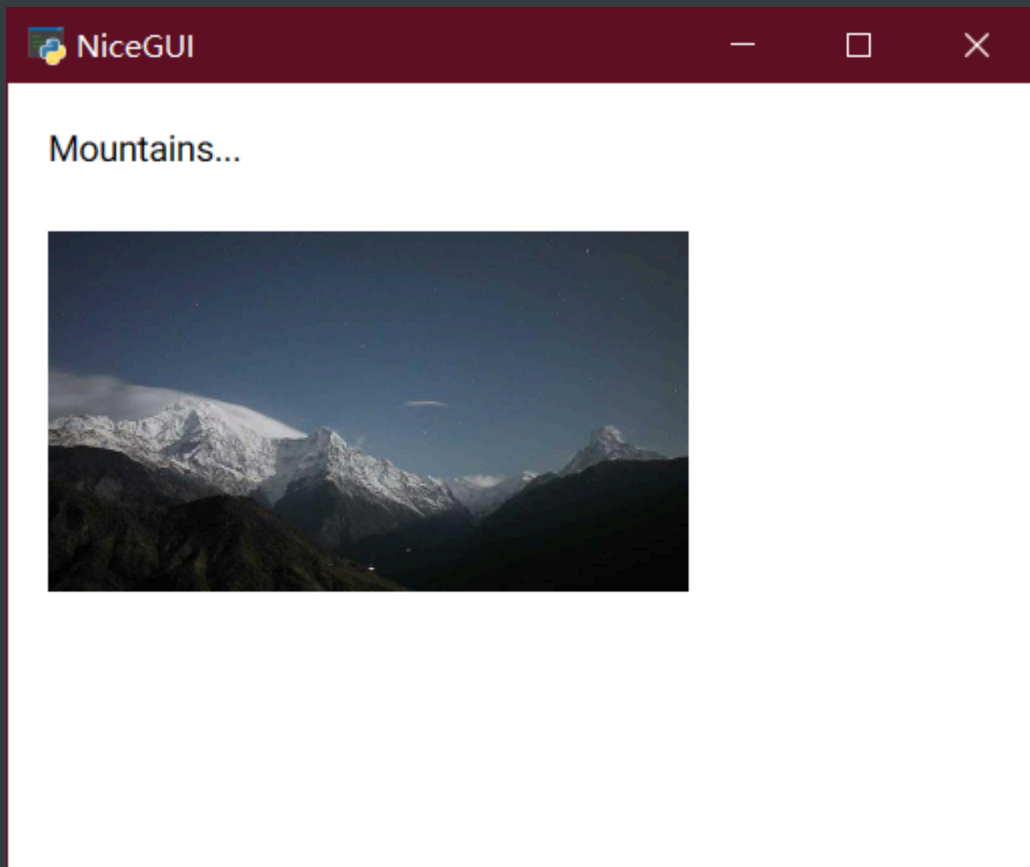
3.9.14 ui.tooltip 补充 (2025.01.21更新)

对于像 `ui.html`、`ui.markdown`、`ui.upload` 等不支持添加 `tooltip` 的元素，可以使用 `ui.element` 包装来间接实现：

```
1 from nicegui import ui
2
3 with ui.element().tooltip('...with a tooltip!'):
4     ui.html('This is <u>HTML</u>...')
5
6 ui.run(native=True)
```

tooltip 里除了显示一般的文本，还可以显示图像等其他内容。不过，不建议在 tooltip 内放置需要交互的内容，因为被添加 tooltip 的控件一旦失去焦点，tooltip 就会消失，里面的交互内容永远无法交互：

```
1 from nicegui import ui
2
3 with ui.label('Mountains...'):
4     with ui.tooltip().classes('bg-transparent'):
5         ui.image('https://picsum.photos/id/377/640/360').classes('w-64')
6
7 ui.run(native=True)
```



前面说过 `tooltip` 方法返回的是控件本身，而不是 `tooltip`。但是，这并不是说就没有办法设置 `tooltip` 方法生成的 `tooltip`。如果想要获取到控件 `tooltip` 方法设置的 `tooltip`，可以遍历控件来获取控件内部的其他控件，再判断控件是不是需要的类型：

```
1 from nicegui import ui
2
3 with ui.button(icon='thumb_up'):
4     ui.tooltip('I like this').classes('bg-green')
5
6 button = ui.button(icon='thumb_up')
7 button.tooltip('I like this')
8 for i in button:
9     if isinstance(i,ui.tooltip):
10         i.classes('bg-green')
11
12 ui.run(native=True)
```

也可以使用 `ElementFilter` 方法，简单快捷地设置控件内部的 `tooltip`：

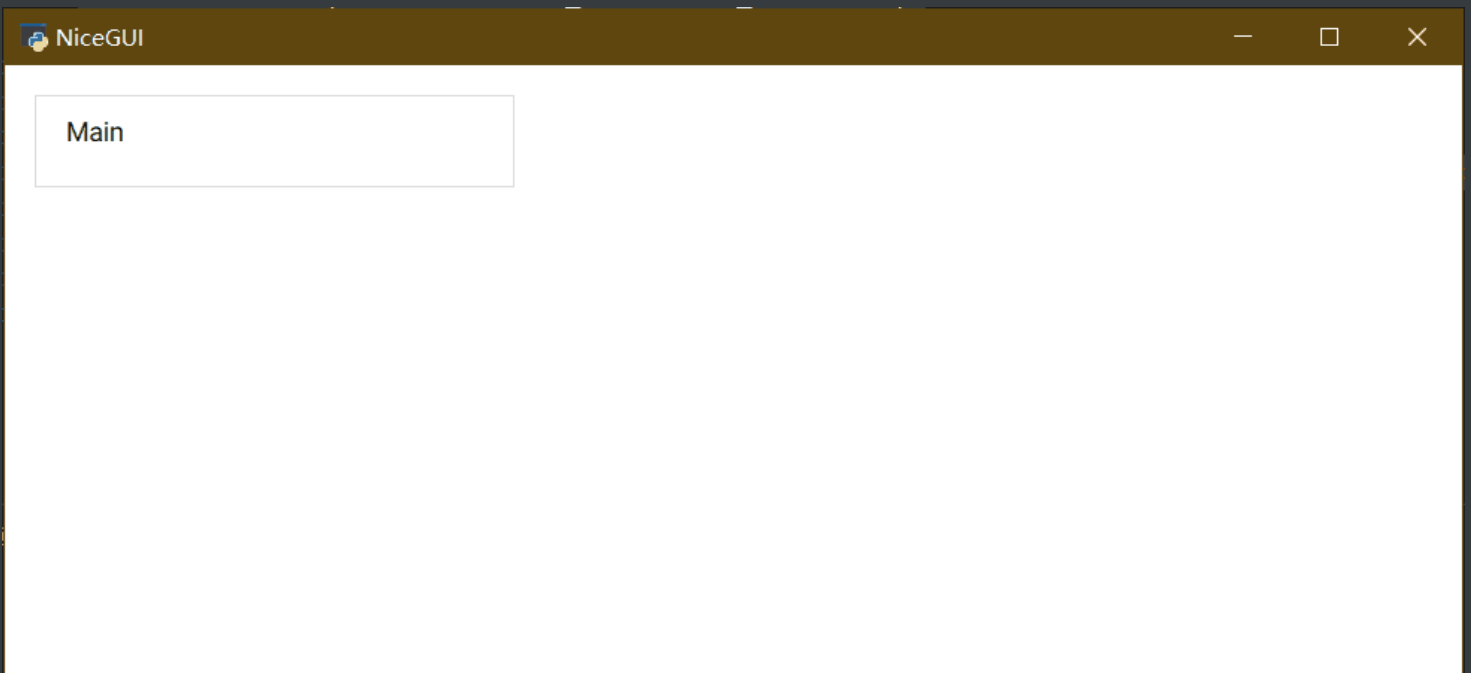
```
1 from nicegui import ui,ElementFilter
2
3 with ui.button(icon='thumb_up'):
4     ui.tooltip('I like this').classes('bg-green')
5
6 button = ui.button(icon='thumb_up')
7 button.tooltip('I like this')
8
9 with button:
10     ElementFilter(kind=ui.tooltip,local_scope=True).classes('bg-green')
11
12 ui.run(native=True)
```

3.9.15 `ui.slide_item` (2025.03.04更新)

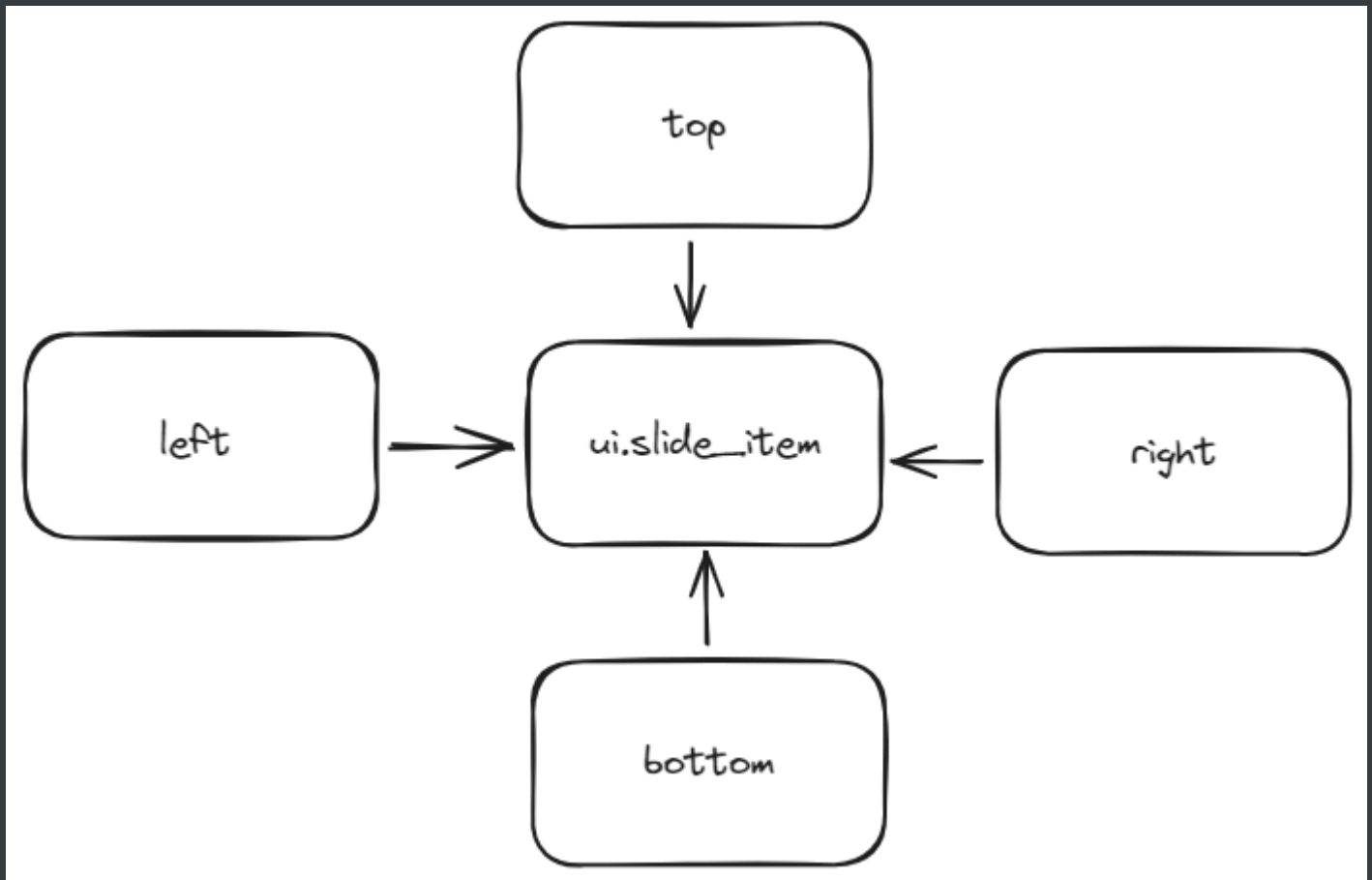
`ui.slide_item` 实现了类似滑动解锁一样操作，可以在空间有限的情况下，四向滑动，进入不同的子控件中。实现源自于Quasar的[QSlidItem](#)。

依照惯例，先看一下示例：

```
1 from nicegui import ui
2
3 with ui.list().props('bordered'), ui.slide_item(
4     text='Main',
5     on_slide=lambda e: ui.notify(f'Slide: {e.side}')
6 ).classes('w-64') as slide_item:
7     with slide_item.left(text='left'):
8         ui.button(text='reset', on_click=slide_item.reset)
9     with slide_item.right():
10         ui.button(text='reset', on_click=slide_item.reset)
11     with slide_item.top(text='top', on_slide=slide_item.reset, color='red'):
12         ...
13     with slide_item.action(side='bottom', text='bottom', on_slide=slide_item.reset,
14                             color='red'):
15         ...
16 ui.run(native=True)
```



看完了示例，接下来开始介绍 `ui.slide_item` 的交互原理。实际上，`ui.slide_item` 可以看作是类似轮播图的万向非自动版。当用户滑动时，控件显示的内容会切换为来向的子控件，如下图所示，箭头表示滑动方向，用户滑动之后，对应的子控件会根据滑动方向而进入主显示区：



同样的，每个子控件也是一个容器，除了传入参数直接显示文字之外，还可以继续添加其他控件到内部。

了解了交互原理之后，下面正式介绍一下参数和方法。

`ui.slide_item` 支持以下参数：

`text` 参数，字符串类型，表示显示在控件内的文字。在实际使用时，可以不使用此参数，直接进入 default slot，自定义控件内的内容。

`on_slide` 参数，可调用类型，当滑动切换到对应的子控件时执行的操作。该操作接收一个消息参数，消息参数的字符串类型属性 `slide` 表示滑动的方向。

相比于参数的简单，`ui.slide_item` 支持的方法就复杂了一点。

`left` 方法、`right` 方法、`top` 方法、`bottom` 方法本质上是一样的，就是对应左、右、上、下四个方向上的子控件，因此参数也是一样：

`text` 参数，同 `ui.slide_item` 的 `text` 参数。

`on_slide` 参数，同 `ui.slide_item` 的 `on_slide` 参数。

`color` 参数，字符串类型或者 `None`，表示子控件的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让按钮变成默认颜色），默认为 `'primary'`，即和主题颜色一致。

`left` 方法、`right` 方法、`top` 方法、`bottom` 方法其实是对 `action` 方法的包装，返回的是 `action` 方法的执行结果。`action` 方法除了支持这些方法的参数之外，还支持字符串类型参数 `slide`，表示子控件对应的方向，该参数仅支持 `['left','right','top','bottom']` 中的值，参数位置在 `text` 参数前。

此外，需要注意的是，切换到对应子控件之后，需要调用 `reset` 方法才能从子控件返回：

```
1  from nicegui import ui
2
3  with ui.list().props('bordered'):
4      with ui.slide_item() as slide_item:
5          ui.item('Slide me')
6          with slide_item.left(color='blue'):
7              ui.item('Left')
8          with slide_item.right(color='purple'):
9              ui.item('Right')
10
11 ui.button('Reset', on_click=slide_item.reset)
12
13 ui.run(native=True)
```

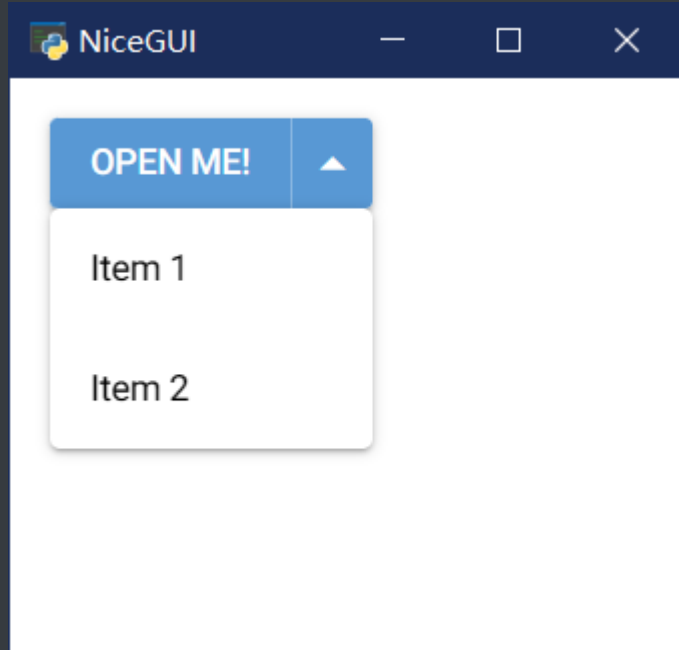
3.10 其他常用控件

3.10.1 `ui.dropdown_button`

前面在介绍菜单控件的时候，是用按钮内嵌入菜单，实现点击按钮弹出菜单的效果。其实，在 NiceGUI 中，有个控件可以实现一步到位，无需嵌入，那就是 `ui.dropdown_button` —— 下拉按钮。

先看示例：

```
1 from nicegui import ui
2
3 with ui.dropdown_button('Open me!', auto_close=True, split=True):
4     ui.item('Item 1', on_click=lambda: ui.notify('You clicked item 1'))
5     ui.item('Item 2', on_click=lambda: ui.notify('You clicked item 2'))
6
7 ui.run(native=True)
```



可以看到，下拉按钮的弹出效果很像在按钮中嵌入了菜单，但需要的代码更少，结构也更清晰。当然，样式上因为多了一个下拉的三角，没法像嵌入菜单那样灵活，读者可以根据需求自主选择。

下拉按钮支持以下参数：

`text` 参数，字符串类型，表示显示在按钮上的文字，和普通按钮一样。

`value` 参数，布尔类型，表示下拉按钮的初始状态是否为下拉内容弹出，默认为 `False`。

`on_value_change` 参数，可调用类型，表示下拉按钮的 `value` 变化（即下拉内容弹出状态变化）时执行什么操作。

`on_click` 参数，可调用类型，表示点击下拉按钮主体（不是右边的小三角）时执行什么操作，和普通按钮一样。

`color` 参数，字符串类型或者 `None`，表示按钮的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让按钮变成默认颜色），默认为 `'primary'`，即和主题颜色一致。

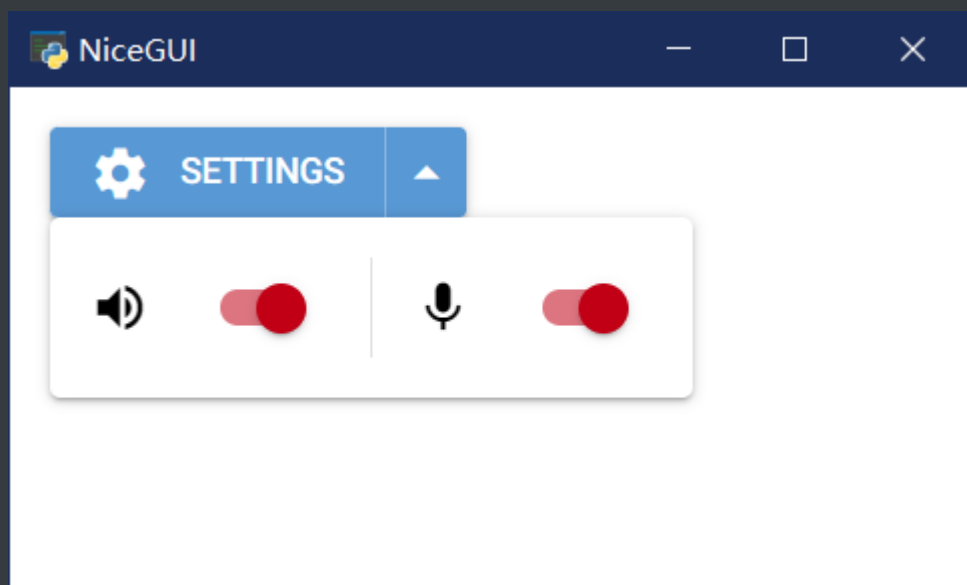
`icon` 参数，字符串类型，表示按钮额外显示的图标，支持传入字符串类型的图标名，和普通按钮一样。

`auto_close` 参数，布尔类型，表示点击下拉内容之后是否自动关闭下拉内容的弹出，就和菜单项一样，默认为 `False`。

`split` 参数，布尔类型，表示是否显示分隔符来区分按钮主体和下拉弹出响应区，默认为 `False`。

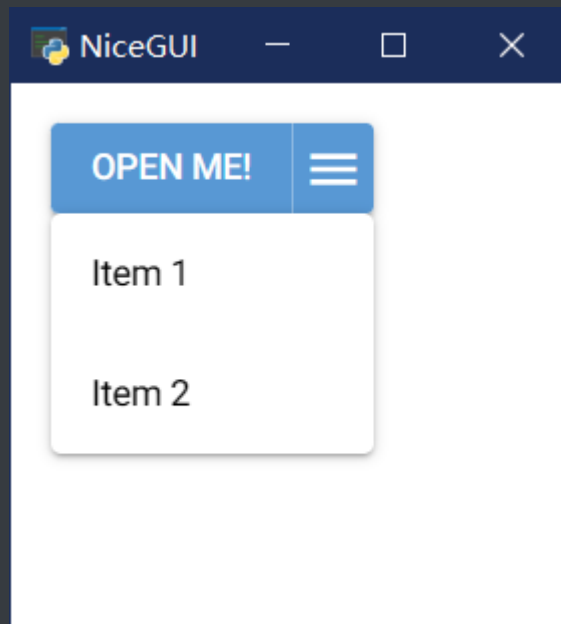
同样的，下拉按钮也可以嵌入其他控件，实现想要的效果：

```
1 from nicegui import ui
2
3 with ui.dropdown_button('Settings', icon='settings', split=True):
4     with ui.row().classes('p-4 items-center'):
5         ui.icon('volume_up', size='sm')
6         ui.switch().props('color=negative')
7         ui.separator().props('vertical')
8         ui.icon('mic', size='sm')
9         ui.switch().props('color=negative')
10
11 ui.run(native=True)
```



想要修改下拉触发区的图标，可以参考[API文档](#)，修改 'dropdown-icon' 属性（ props ） ，

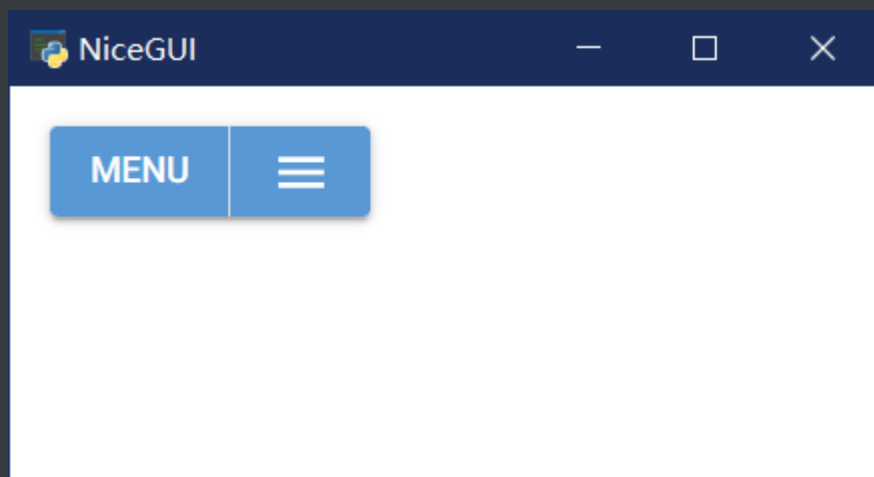
```
1 from nicegui import ui
2
3 with ui.dropdown_button("Open me!", auto_close=True, split=True).props(
4     'no-icon-animation dropdown-icon="menu"'
5 ):
6     ui.item("Item 1", on_click=lambda: ui.notify("You clicked item 1"))
7     ui.item("Item 2", on_click=lambda: ui.notify("You clicked item 2"))
8
9 ui.run(native=True)
```



3.10.2 ui.button_group

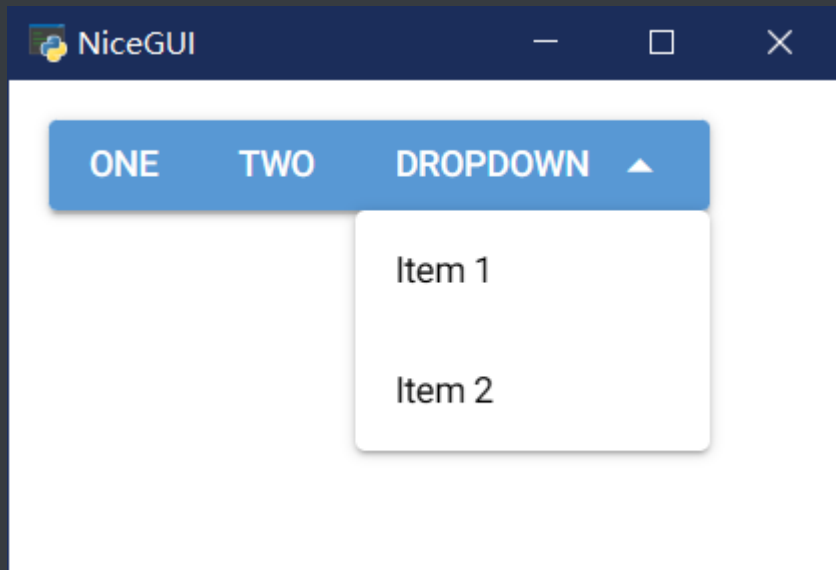
下拉按钮看起来就像两个按钮组合到一起，也可以用按钮组控件模拟实现，让右边的按钮支持更多功能：

```
1 from nicegui import ui
2
3 with ui.button_group():
4     ui.button('menu')
5     ui.separator().props('vertical')
6     ui.button(icon='menu')
7
8 ui.run(native=True)
```



下拉按钮也是按钮，一样可以和按钮组组合使用：

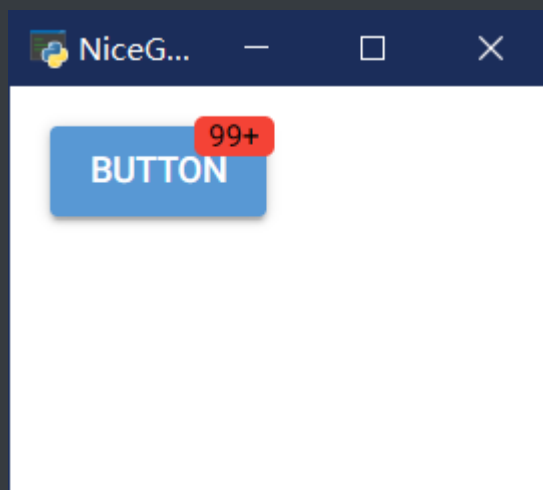
```
1 from nicegui import ui
2
3 with ui.button_group():
4     ui.button('One')
5     ui.button('Two')
6     with ui.dropdown_button('Dropdown'):
7         ui.item('Item 1', on_click=lambda: ui.notify('Item 1'))
8         ui.item('Item 2', on_click=lambda: ui.notify('Item 2'))
9
10 ui.run(native=True)
```



3.10.3 ui.badge

角标控件，可以在一个控件的上层显示简单的文字，就像手机图标上提示有多少消息未读的角标一样：

```
1 from nicegui import ui
2
3 with ui.button('Button'):
4     ui.badge(text='99+',
5             color='red',text_color='black',outline=False).props('floating')
6
7 ui.run(native=True)
```



角标控件支持以下参数：

`text` 参数，字符串类型，显示在角标内的文字。

`color` 参数，字符串类型或者 `None`，表示角标的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让角标变成默认颜色），默认为 `'primary'`，即和主题颜色一致。

`text_color` 参数，字符串类型或者 `None`，表示文字的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让文字变成默认颜色）。

`outline` 参数，布尔类型，是否启用轮廓线风格，默认为 `False` 即填充风格。

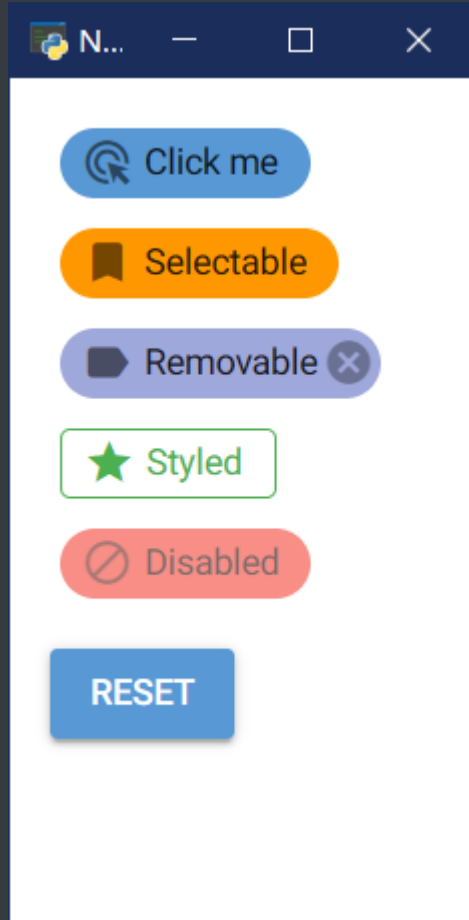
更多角标的设计属性可以参考[API文档](#)，示例代码中的 `'floating'` 就是修改了设计属性，让角标显示在右上角。其他有用的属性有 `'rounded'`、`'transparent'`、`'label'` 等。

3.10.4 ui.chip

薄片控件，看上去有点像角标，但交互性远远高于角标。可以点击、选择、移除，设计属性也比角标多。先看示例代码：

```
1 from nicegui import ui
2
3 with ui.row().classes("gap-1") as row:
4     ui.chip("Click me", icon="ads_click", on_click=lambda: ui.notify("Clicked"))
5     chip = ui.chip(
6         text="Selectable",
7         selectable=True,
8         icon="bookmark",
9         color="orange",
10        on_selection_change=lambda e: ui.notify(e.sender.selected),
11    )
12    chip2 = ui.chip(
13        text="Removable",
14        removable=True,
15        icon="label",
16        color="indigo-3",
17        on_value_change=lambda e: ui.notify(e.value),
18    )
19    ui.chip("Styled", icon="star", color="green").props("outline square")
20    ui.chip("Disabled", icon="block", color="red").set_enabled(False)
```

```
21 ui.button("RESET", on_click=lambda: (chip.set_selected(False),
    chip2.set_value(True)))
22
23 ui.run(native=True)
```



薄片控件支持以下参数：

`text` 参数，字符串类型，显示在控件内的内容。

`icon` 参数，字符串类型，控件内的图标。

`color` 参数，字符串类型，表示控件的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名），默认为 'primary'，即和主题颜色一致。

`text_color` 参数，字符串类型或者 `None`，表示文字的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让文字变成默认颜色）。

`on_click` 参数，可调用类型，当点击控件时执行什么操作。注意，设置此参数，会同时添加控件的 `'clickable'` 属性（`props`），启用控件的鼠标悬停效果，让控件响应 `'click'` 事件。也就是说，如果不设置 `on_click` 参数或者调用 `on_click` 方法来设置点击事件，而是直接使用

`ui.chip('test').on('click', handler=lambda: ui.notify("Clicked"))` 设置点击事件响应，这样操作并不能成功，只有 `ui.chip('test').props('clickable').on('click', handler=lambda:`

`ui.notify("Clicked"))` 这样同时添加 `'clickable'` 属性才行。否则，只能用

`ui.chip('test').on('mousedown', handler=lambda: ui.notify("Clicked"))` 这样直接监听鼠标事件的响应才行，但这样会导致控件没有鼠标悬停效果。

`selectable` 参数，布尔类型，控件是否可选择，默认为 `False`。

`selected` 参数，布尔类型，控件是否已经被选择，默认为 `False`。

`on_selection_change` 参数，可调用类型，当控件被选择时执行什么操作。

`removable` 参数，布尔类型，控件是否可被移除，默认为 `False`。如果设置为 `True`，控件上会多一个 `"X"` 移除按钮。

`on_value_change` 参数，可调用类型，当控件的被移除状态变化时执行什么操作。实际上，控件被移除、未被移除，就是将控件的 `value` 设置为 `False`、`True`。

更多的设计属性可以参考[官方API](#)。

以下是一个动态添加、移除控件的代码示例：

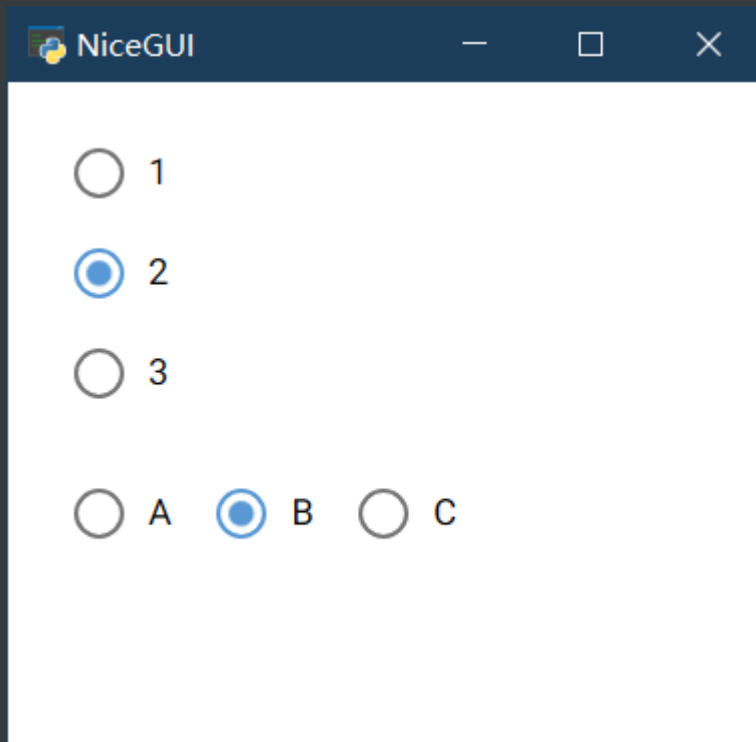
```
1 from nicegui import ui
2
3 def add_chip():
4     with chips:
5         ui.chip(label_input.value, icon='label', color='silver', removable=True)
6         label_input.value = ''
7
8 label_input = ui.input('Add label').on('keydown.enter', add_chip)
9 with label_input.add_slot('append'):
10     ui.button(icon='add', on_click=add_chip).props('round dense flat')
11
12 with ui.row().classes('gap-0') as chips:
13     ui.chip('Label 1', icon='label', color='silver', removable=True)
```

```
14
15 ui.button('Restore removed chips', icon='unarchive',
16           on_click=lambda: [chip.set_value(True) for chip in chips]) \
17           .props('flat')
18
19 ui.run(native=True)
```

3.10.5 ui.radio

单选按钮，每次只能选择其中一个值。

```
1 from nicegui import ui
2
3 radio1 = ui.radio([1, 2, 3], value=1)
4 radio2 = ui.radio({1: 'A', 2: 'B', 3: 'C'}).props('inline').bind_value(radio1,
5                                 'value')
6
7 ui.run(native=True)
```



单选按钮的参数很简单：

`options` 参数，列表类型或者字典类型，表示单选按钮的所有选项。如果是列表，每个元素既是当前选择的值，也是显示出来的文本。如果是字典，则键（key）是当前选择的值，值（value）是显示出来的文本。

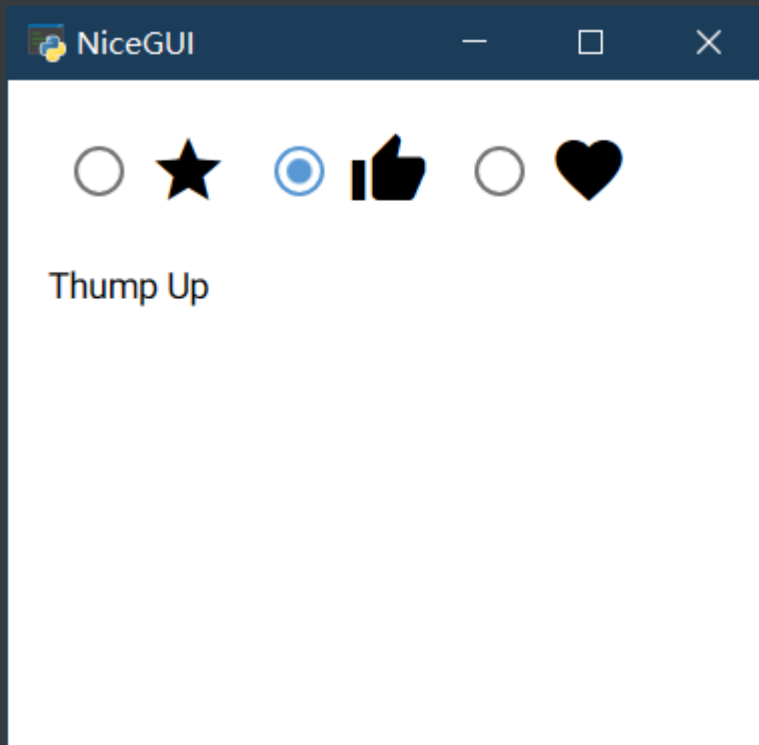
`value` 参数，表示控件初始选择的值。

`on_change` 参数，可调用类型，当值变化时执行什么操作。

更多设计属性参考[官方API](#)。

除了让选项显示文本，借用 `ui.teleport` 的功能，还能让选项显示图标：

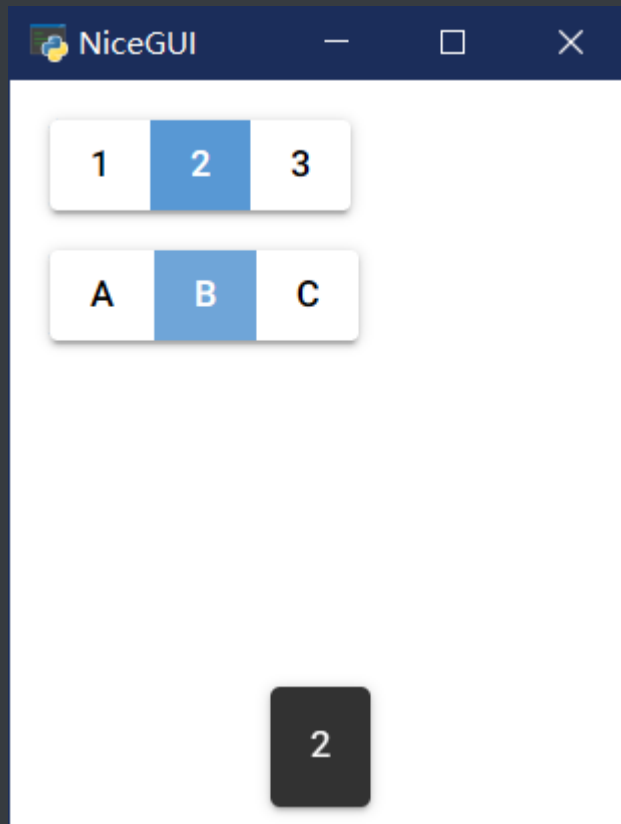
```
1 from nicegui import ui
2
3 options = ['Star', 'Thump Up', 'Heart']
4 radio = ui.radio({x: '' for x in options}, value='Star').props('inline')
5 with ui.teleport(f'#c{radio.id} > div:nth-child(1) .q-radio__label'):
6     ui.icon('star', size='md')
7 with ui.teleport(f'#c{radio.id} > div:nth-child(2) .q-radio__label'):
8     ui.icon('thumb_up', size='md')
9 with ui.teleport(f'#c{radio.id} > div:nth-child(3) .q-radio__label'):
10     ui.icon('favorite', size='md')
11 ui.label().bind_text_from(radio, 'value')
12
13 ui.run(native=True)
```

3.10.6 ui.toggle

切换控件，功能上和单选按钮一样，操作起来有点像老式磁带播放机的按钮，每次只能点选一个：

```
1 from nicegui import ui
2
3 toggle1 = ui.toggle(
4     options=[1, 2, 3], value=1, clearable=True, on_change=lambda e: ui.notify(e.value)
5 )
6 toggle2 = ui.toggle({1: "A", 2: "B", 3: "C"}).bind_value(toggle1, "value")
7
8 ui.run(native=True)
```



切换控件支持以下参数：

`options` 参数，列表类型或者字典类型，表示切换控件的所有选项。如果是列表，每个元素既是当前选择的值，也是显示出来的文本。如果是字典，则键（key）是当前选择的值，值（value）是显示出来的文本。

`value` 参数，表示控件初始选择的值。

`on_change` 参数，可调用类型，当值变化时执行什么操作。

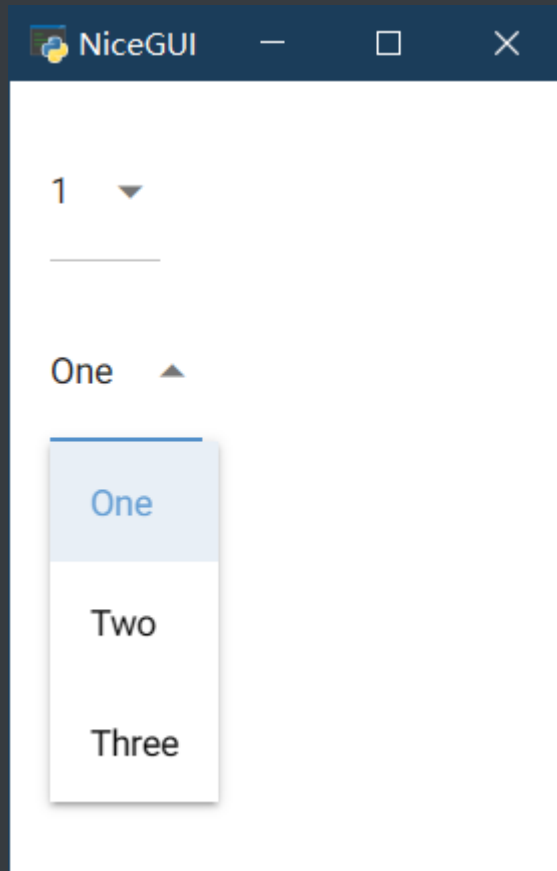
`clearable` 参数，布尔类型，表示是否可以通过点击当前选择的选项来取消选择，默认为 `False`，即默认必须选择一个，没法取消选择。

更多设计属性参考[官方API](#)。

3.10.7 `ui.select`

下拉选择框，同样是一个提供多种选择内容的控件，只不过下拉选择框是弹出可选择的内容，在选择之后，将被选择的内容显示在框内。NiceGUI的下拉选择框支持的参数和设计属性比较多，也让这个看起来简单的选择控件有无数可能。在详细学习之前，先看简单的示例：

```
1 from nicegui import ui
2
3 select1 = ui.select([1, 2, 3], value=1)
4 select2 = ui.select({1: 'One', 2: 'Two', 3: 'Three'}).bind_value(select1, 'value')
5
6 ui.run(native=True)
```



下拉选择框支持以下参数：

options 参数，列表类型或者字典类型，表示控件的所有选项。如果是列表，每个元素既是当前选择的值，也是显示出来的文本。如果是字典，则键（key）是当前选择的值，值（value）是显示出来的文本。

label 参数，字符串类型，直译的话是标签，表示显示在选择框上方的文本，但不是选择的文本，如果当前选择的内容是空的，点击选择之前会显示在选择框内，点击之后会移动到选择框上方。

value 参数，表示控件初始选择的值。

on_change 参数，可调用类型，当值变化时执行什么操作。

`with_input` 参数，布尔类型，表示是否显示一个输入框在选择框内，用输入框内的内容筛选选项，默认为 `False`，即不显示输入框。

`new_value_mode` 参数，字符串类型或 `None`，表示在选择框内输入值而不是选择当前定义好的选项，直接回车的话，执行什么样的操作。这个参数只支持 `'add'`、`'add-unique'`、`'toggle'`。其中，`'add'` 表示只能添加没有的值，即根据输入的值筛选当前已有的选项，如果有筛选结果，选择第一个，没有筛选结果则添加这个值。`'add-unique'` 表示添加当前值到选项中，哪怕值是相同的，也能添加为新的选项。`'toggle'` 表示没有就添加，和 `'add-unique'` 一样的添加；如果有就删除，注意，删除时候需要取消下拉弹出选项的焦点，确保选择框为输入状态，但没有弹出的选项，回车即可删除与当前输入框的内容相同的新增选项。默认为 `None`，表示不会添加新的选项。

`multiple` 参数，布尔类型，表示是否支持多选。

`clearable` 参数，布尔类型，表示是否添加一个清除当前选择的按钮。

`validation` 参数，可调用类型、字典类型或者 `None`，表示验证选择的内容是否有效。如果传入可调用类型参数，该参数返回错误信息表示内容无效，返回 `None` 表示内容有效。如果传入字典类型参数，则字典的键（key）表示错误信息，字典的值（value）为可调用类型参数，字典的值（value）返回 `True` 表示内容有效，返回 `False` 则表示内容无效并输出错误信息。默认值为 `None`，表示不验证选择的内容。

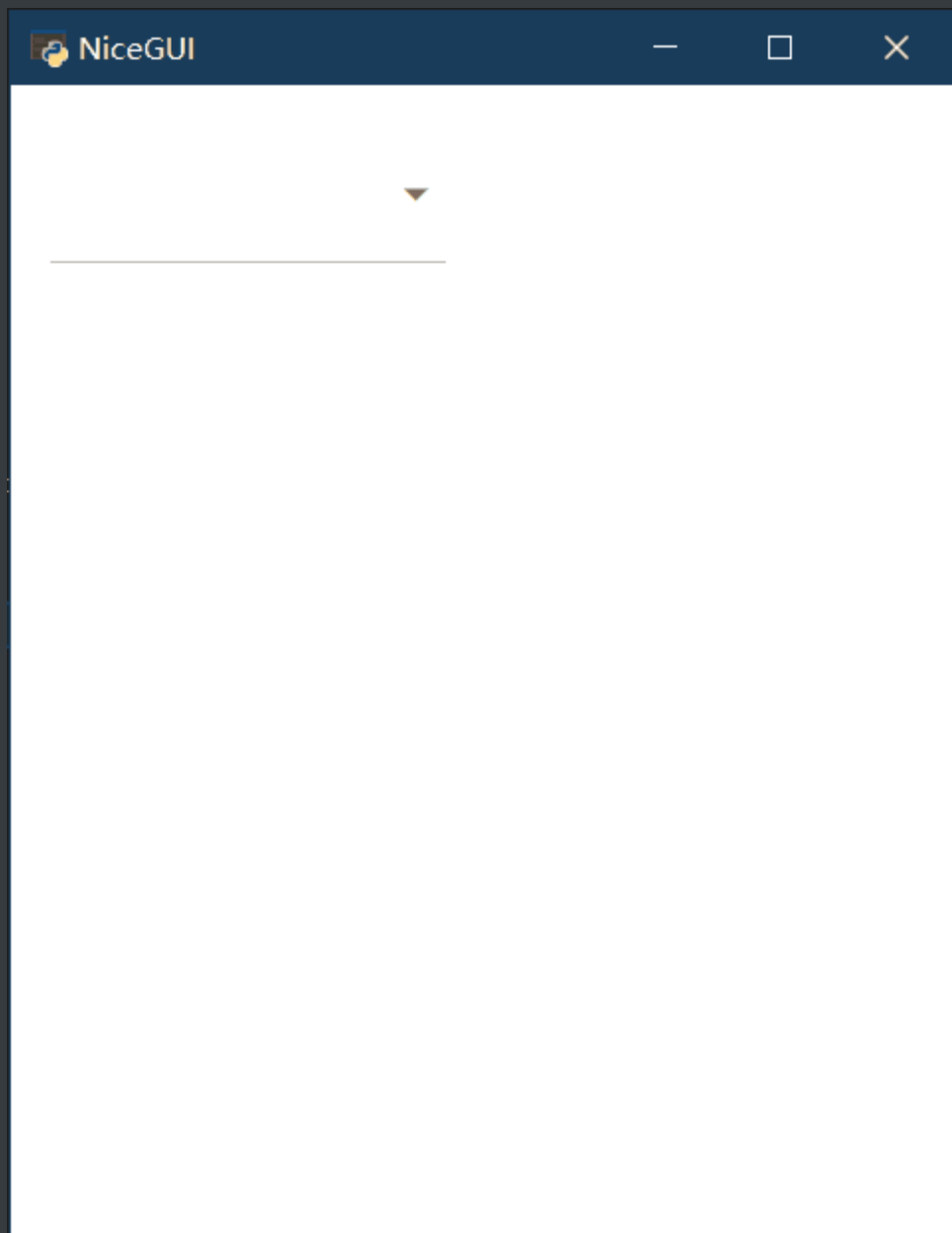
`key_generator` 参数，生成器类型、迭代器类型或者可调用类型，当 `options` 为字典类型且 `new_value_mode` 不为 `None` 时，此参数用于生成字典的键（key）。当此参数为生成器类型和迭代器类型时，每次新添加选项的键（key）就是依次遍历此参数获得，该选项的显示文本就是输入的内容。当此参数不能继续遍历时，新的选项则没法添加。如果此参数为可调用类型，每次新添加选项的键（key）就是将输入内容当做参数、可调用参数返回的执行结果，该选项的显示文本就是输入的内容。注意，当 `new_value_mode` 为 `'add'` 时，此参数必须正确设置，否则会报错。

更多设计属性参考[官方API](#)。

使用添加输入框参数，可以同时启用搜索功能：

```
1 from nicegui import ui
2
3 continents = [
4     'Asia',
5     'Africa',
6     'Antarctica',
```

```
7     'Europe',
8     'Oceania',
9     'North America',
10    'South America',
11 ]
12 ui.select(options=continents, with_input=True,
13           on_change=lambda e: ui.notify(e.value)).classes('w-40')
14
15 ui.run(native=True)
```

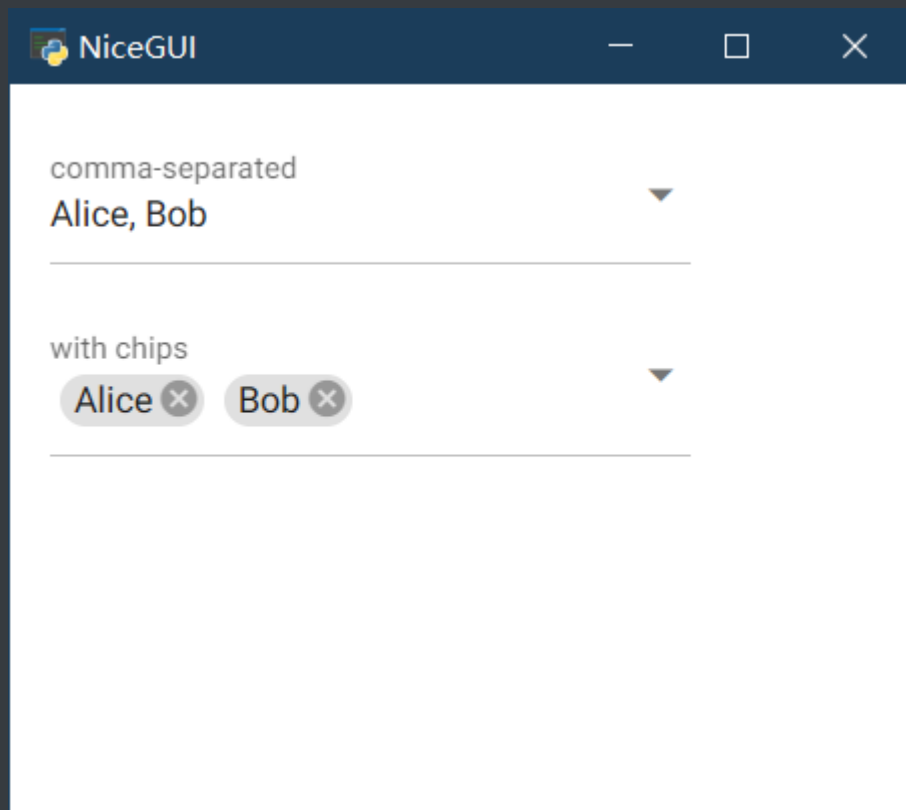


多选功能:

```

1  from nicegui import ui
2
3  names = ['Alice', 'Bob', 'Carol']
4  ui.select(names, multiple=True, value=names[:2], label='comma-separated') \
5      .classes('w-64')
6  ui.select(names, multiple=True, value=names[:2], label='with chips') \
7      .classes('w-64').props('use-chips')
8
9  ui.run(native=True)

```



调用 `set_options` 方法更新选项：

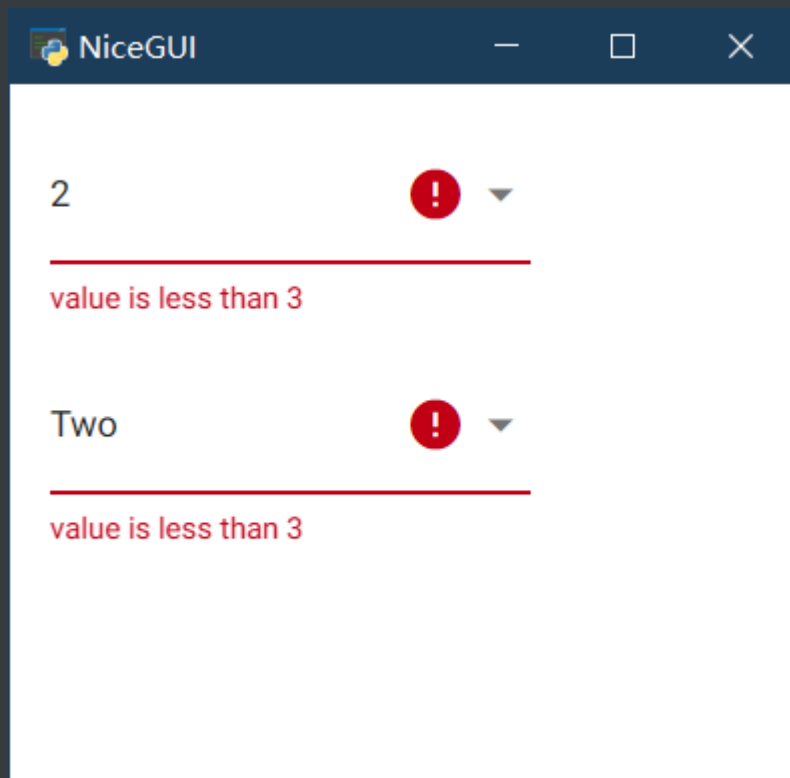
```

1  from nicegui import ui
2
3  select = ui.select([1, 2, 3], value=1)
4  with ui.row():
5      ui.button('4, 5, 6', on_click=lambda: select.set_options([4, 5, 6], value=4))
6      ui.button('1, 2, 3', on_click=lambda: select.set_options([1, 2, 3], value=1))
7
8  ui.run(native=True)

```

两种验证格式：

```
1 from nicegui import ui
2
3 ui.select(
4     options=[1, 2, 3],
5     value=1,
6     validation={
7         "value is less than 3": lambda v: v >= 3
8     }
9 ).classes("w-48")
10 ui.select(
11     options={1: "One", 2: "Two", 3: "Three"},
12     value=1,
13     validation=lambda v: "value is less than 3" if v < 3 else None,
14 ).classes("w-48")
15
16 ui.run(native=True)
```



使用生成器和迭代器作为 key_generator :

```

1  from nicegui import ui
2
3  select1 = ui.select(
4      options={1: "One", 2: "Two", 3: "Three"},
5      value=1,
6      new_value_mode='add',
7      key_generator=(i for i in ['',4,5,6])
8  ).classes("w-48")
9
10 select2 = ui.select(
11     options={1: "One", 2: "Two", 3: "Three"},
12     value=1,
13     new_value_mode='add',
14     key_generator=iter([4,5,6])
15 ).classes("w-48")
16
17 ui.run(native=True)

```

注意，如果传入的 `key_generator` 是生成器，默认控件会执行一次 `next` 方法，然后才开始执行 `send` 方法，会导致生成器第一个生成（`yield`）的值被抛弃，这样做的目的是每一步都能接收到 `send` 方法的参数。所以，如果是简易的生成器，不接收 `send` 方法的参数，或者不太熟悉生成器语法的话，可以在头部插入任意值用于执行 `next` 方法，也可以使用 `iter` 方法转换为迭代器来使用。

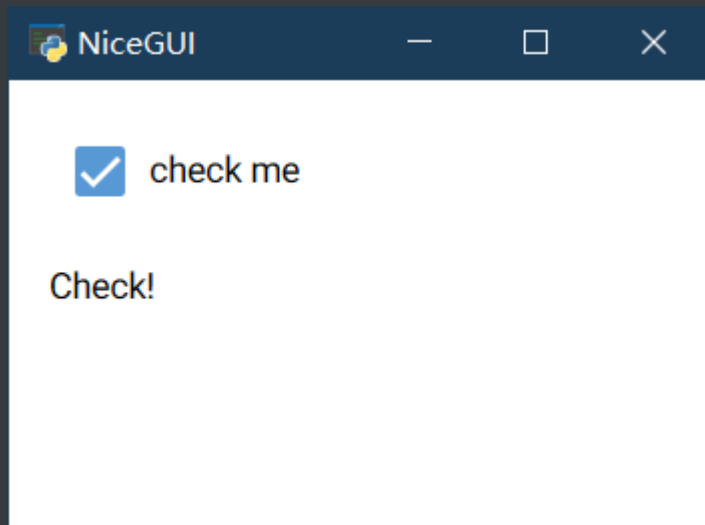
3.10.8 ui.checkbox

复选框控件，用于显示一个支持勾选的复选框。

```

1  from nicegui import ui
2
3  checkbox = ui.checkbox('check me')
4  ui.label('Check!').bind_visibility_from(checkbox, 'value')
5
6  ui.run(native=True)

```

复选框控件的参数很简单：

`text` 参数，字符串类型，显示在控件右边的文字内容。

`value` 参数，布尔类型，控件初始的勾选状态，默认为 `False`，即不勾选。实际上，勾选复选框也就是将控件的 `value` 属性设置为 `True`，调用控件的 `set_value` 方法可以改变勾选状态。

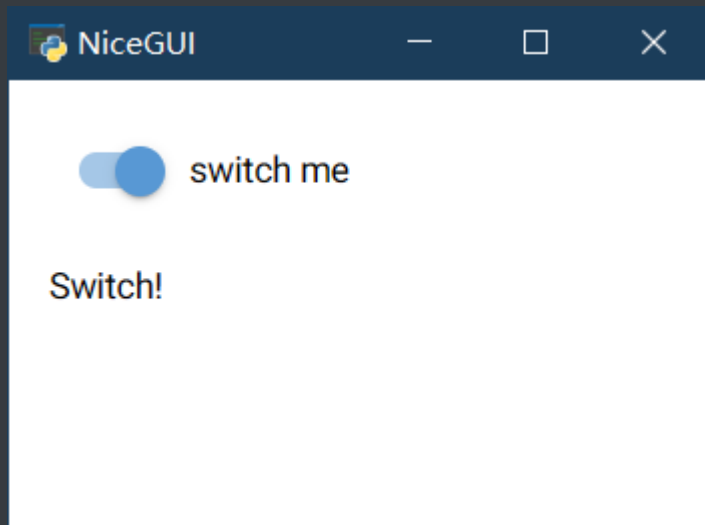
`on_change` 参数，可调用类型，当控件的勾选状态变化时执行什么操作。

更多设计属性参考[官方API](#)。

3.10.9 `ui.switch`

开关控件，显示一个像开关一样可以切换开启、关闭状态的控件。

```
1 from nicegui import ui
2
3 switch = ui.switch('switch me')
4 ui.label('Switch!').bind_visibility_from(switch, 'value')
5
6 ui.run(native=True)
```



开关控件的参数很简单：

`text` 参数，字符串类型，显示在控件右边的文字内容。

`value` 参数，布尔类型，控件初始的开关状态，默认为 `False`，即关闭。实际上，切换为打开状态也就是将控件的 `value` 属性设置为 `True`，调用控件的 `set_value` 方法可以改变开关状态。

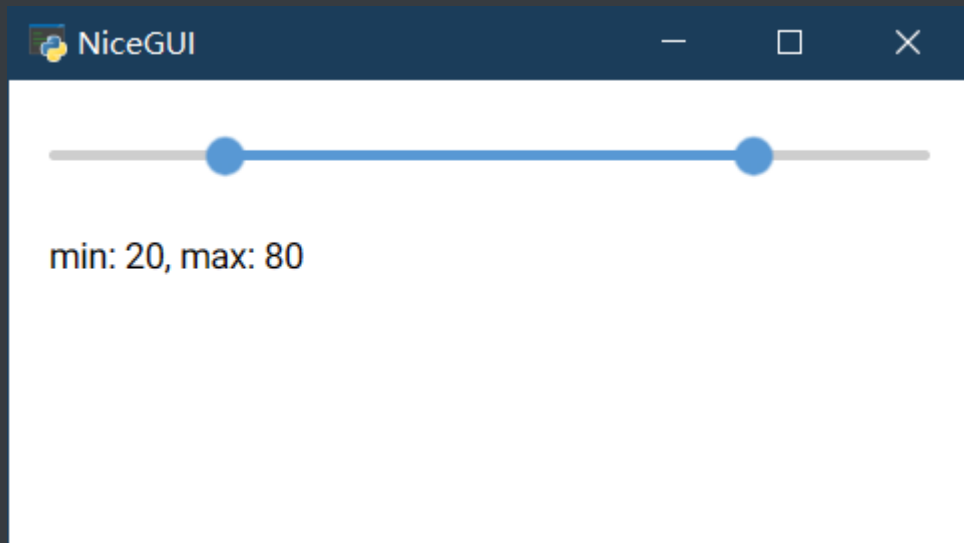
`on_change` 参数，可调用类型，当控件的开关状态变化时执行什么操作。

更多设计属性参考[官方API](#)。

3.10.10 `ui.range`

范围条控件有点像之前提到的 `ui.slider`，参数都是一样的，只是 `value` 参数略有差异。

```
1 from nicegui import ui
2
3 min_max_range = ui.range(min=0, max=100, value={'min': 20, 'max': 80})
4 ui.label().bind_text_from(min_max_range, 'value',
5                             backward=lambda v: f'min: {v["min"]}, max: {v["max"]}')
6
7 ui.run(native=True)
```



范围条控件支持以下参数：

`min` 参数，浮点类型，范围条最小值。

`max` 参数，浮点类型，范围条最大值。

`step` 参数，浮点类型，范围条的步长。

`value` 参数，字典类型，范围条当前值。注意，因为范围条有两个滑块，不像 `ui.slider` 只有一个，因此，`ui.slider` 值只是一个浮点数，范围条的值是一个字典，`{'min': 20, 'max': 80}`，键（key）是字符串，必须包含 `'min'` 和 `'max'`，对应的值（value）就是左右滑块的当前位置。注意，虽然交互过程中，左边的滑块可以越过右边的滑块位置，不会影响最大值、最小值的输出，但在指定初始值的时候，最小值必须小于等于最大值，否则会显示异常。

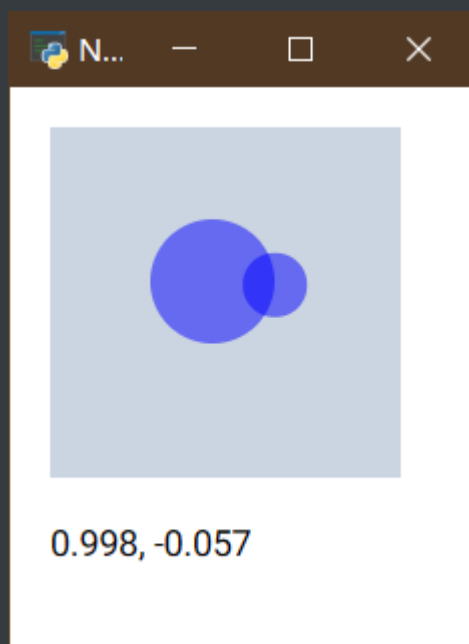
`on_change` 参数，可调用类型，拖动任一滑块触发的操作。

更多设计属性参考[官方API](#)。

3.10.11 `ui.joystick`

虚拟摇杆控件，生成一个可以交互的虚拟摇杆。

```
1 from nicegui import ui
2
3 ui.joystick(
4     color='blue', size=50,
5     on_move=lambda e: coordinates.set_text(f'{e.x:.3f}, {e.y:.3f}'),
6     on_end=lambda _: coordinates.set_text('0, 0'),
7 ).classes('bg-slate-300')
8 coordinates = ui.label('0, 0')
9
10 ui.run(native=True)
```



虚拟摇杆是使用第三方JavaScript库实现的，NiceGUI提供的显式参数不多，想要了解更多用法的话，只能去官方文档了解。

虚拟摇杆支持以下参数：

`on_start` 参数，可调用类型，当用户开始触摸控件时执行的操作。

`on_move` 参数，可调用类型，当用户开始移动摇杆时执行的操作。

`on_end` 参数，可调用类型，当用户停止触摸控件（此时虚拟摇杆消失）时执行的操作。

`throttle` 参数，浮点类型，检测用户移动事件的间隔，默认为 `0.05`，单位秒。

options 参数，关键字参数，通过关键字参数的形式传递给此参数，第三方库支持的其他选项，可以参考下表或者[官方文档](#)：

```
1 var options = {
2     zone: Element,           // active zone
3     color: String,
4     size: Integer,
5     threshold: Float,       // before triggering a directional event
6     fadeTime: Integer,      // transition time
7     multitouch: Boolean,
8     maxNumberOfNipples: Number, // when multitouch, what is too many?
9     dataOnly: Boolean,      // no dom element whatsoever
10    position: Object,        // preset position for 'static' mode
11    mode: String,            // 'dynamic', 'static' or 'semi'
12    restJoystick: Boolean|Object, // Re-center joystick on rest state
13    restOpacity: Number,     // opacity when not 'dynamic' and rested
14    lockX: Boolean,          // only move on the X axis
15    lockY: Boolean,          // only move on the Y axis
16    catchDistance: Number,   // distance to recycle previous joystick in
17                               // 'semi' mode
18    shape: String,           // 'circle' or 'square'
19    dynamicPage: Boolean,    // Enable if the page has dynamically visible
    elements
20    follow: Boolean,         // Makes the joystick follow the thumbstick
21 };
```

常用的选项也就以下几个：

color：字符串，虚拟摇杆的颜色。

size：整数，虚拟摇杆的大小。

mode：字符串，虚拟摇杆的显示模式。 'dynamic' 即动态显示，按下的话，不显示虚拟摇杆。 'static' 是静态显示，无论是否按下，虚拟摇杆都一直显示。 'semi' 是半动态，不按下之前，不显示，一旦按下，就会在按下位置始终显示。

shape：字符串，虚拟摇杆的形状， 'circle' 圆形或者 'square' 方形。

```
1 from nicegui import ui
2
3 ui.joystick(
4     color='blue',
5     size=50,
6     mode='dynamic',
7     shape='square'
8 ).classes('bg-slate-300')
9
10 ui.run(native=True)
```

3.10.12 ui.textarea

文本区域控件其实就是基于 `ui.input` 实现的，大部分参数和 `ui.input` 的一样。能用的参数主要有这几个：

`label` 参数，字符串类型，直译的话是标签，表示显示在文本区域上方的文本，但不是输入的文本，如果输入的内容是空的，点击输入的之前会显示在文本区域内，点击之后会移动到文本区域上方。

`value` 参数，字符串类型，表示文本区域内的内容，也就是文本区域的值。

`on_change` 参数，可调用类型，表示文本区域的值变化时执行的函数。

`placeholder` 参数，字符串类型，表示文本区域的占位符，即文本区域没有输入之前，显示什么内容。与标签不太一样的是，占位符是文本区域获得焦点时候显示的，标签是文本区域没有获得焦点时候显示的。

`validation` 参数，可调用类型、字典类型或者 `None`，表示验证输入的内容是否有效。如果传入可调用类型参数，该参数返回错误信息表示内容无效，返回 `None` 表示内容有效。如果传入字典类型参数，则字典的键（key）表示错误信息，字典的值（value）返回 `True` 表示内容有效，返回 `False` 则表示内容无效并输出错误信息。默认值为 `None`，表示不验证输入的内容。

3.10.13 ui.number

数字输入框同样也是基于 `ui.input` 实现的，只不过数字输入框额外支持的几个特有的参数：

`label` 参数，字符串类型，直译的话是标签，表示显示在数字输入框上方的文本，但不是输入的文本，如果输入的内容是空的，点击输入的之前会显示在数字输入框内，点击之后会移动到数字输入框上方。

`value` 参数，浮点类型，表示数字输入框内的内容，也就是数字输入框的值。

`on_change` 参数，可调用类型，表示数字输入框的值变化时执行的函数。

`placeholder` 参数，字符串类型，表示数字输入框的占位符，即数字输入框没有内容的话，显示什么内容。

`validation` 参数，可调用类型、字典类型或者 `None`，表示验证输入的内容是否有效。如果传入可调用类型参数，该参数返回错误信息表示内容无效，返回 `None` 表示内容有效。如果传入字典类型参数，则字典的键（key）表示错误信息，字典的值（value）返回 `True` 表示内容有效，返回 `False` 则表示内容无效并输出错误信息。默认值为 `None`，表示不验证输入的内容。

`min` 参数，浮点类型，数字输入框允许的最小值。

`max` 参数，浮点类型，数字输入框允许的最大值。

`precision` 参数，整数类型，数字输入框内数值的精度。如果是正数，表示保留到小数点之后的位数；如果是负数，表示保留到小数点前的位数，默认不限制精度。

`step` 参数，浮点型，每次点击增减按钮增加、减少的数值。

`prefix` 参数，字符串类型，表示显示在数字前的前缀。

`suffix` 参数，字符串类型，表示显示在数字后的后缀。

`format` 参数，字符串类型，表示数字的显示格式，该字符串采用类似 `"%.2f"` 的旧式%格式字符串。注意，不要尝试其他输出非纯浮点数的格式，会导致显示异常，包括但不限于前导填充、正负号等，数字输入框不支持添加正号的浮点数。

如果改变了数字输入框的数字或者精度，导致显示的内容不符合要求，可以手动调用 `sanitize` 方法：

```
1 from nicegui import ui
2
3 n = ui.number(value=3.14159265359, precision=5)
4 n.sanitize()
5
6 ui.run(native=True)
```

3.10.14 ui.color_picker

颜色选择器，可以用于选择指定颜色。为了实现弹出效果，颜色选择器是基于Quasar官方的color_picker 和 menu 实现的。

```
1 from nicegui import ui
2
3 with ui.button(icon='colorize') as button:
4     ui.color_picker(on_pick=lambda e: button.classes(f'!bg-[{e.color}]'))
5
6 ui.run(native=True)
```




颜色选择器的参数很简单：

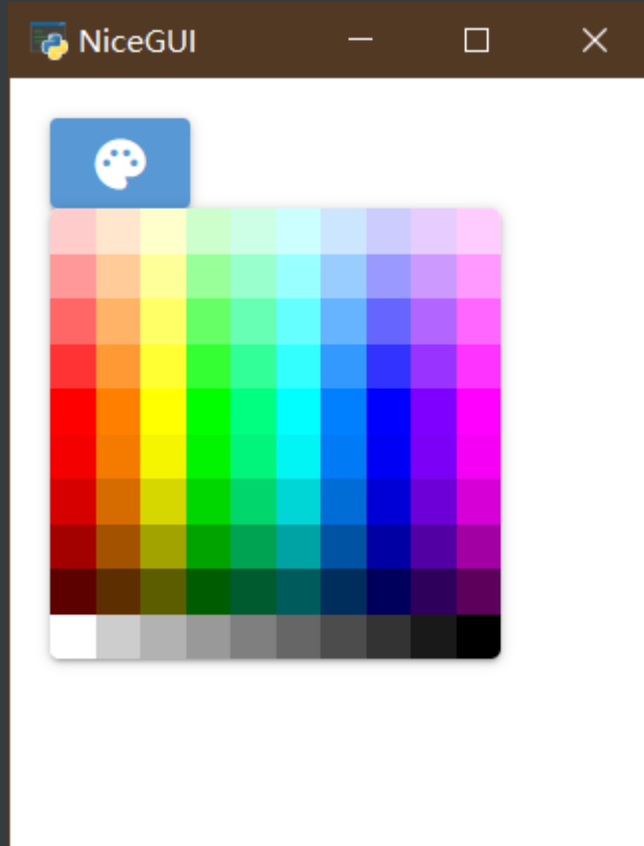
`on_pick` 参数，可调用类型，表示颜色选择器完成选择之后执行的操作。

`value` 参数，布尔类型，表示默认颜色选择器的弹出状态，默认为 `False`，即不弹出。

更多设计属性参考[官方API](#)。

默认的颜色面板提供多种颜色显示模式，想要降低颜色选择复杂度，可以参考API：

```
1 from nicegui import ui
2
3 with ui.button(icon='palette'):
4     picker = ui.color_picker(on_pick=lambda e: ui.notify(f'You chose {e.color}'))
5     picker.q_color.props('default-view=palette no-header no-footer')
6
7 ui.run(native=True)
```



3.10.15 ui.color_input

颜色输入框则是颜色选择器与输入框的结合体，从参数上看，也有输入框的影子：

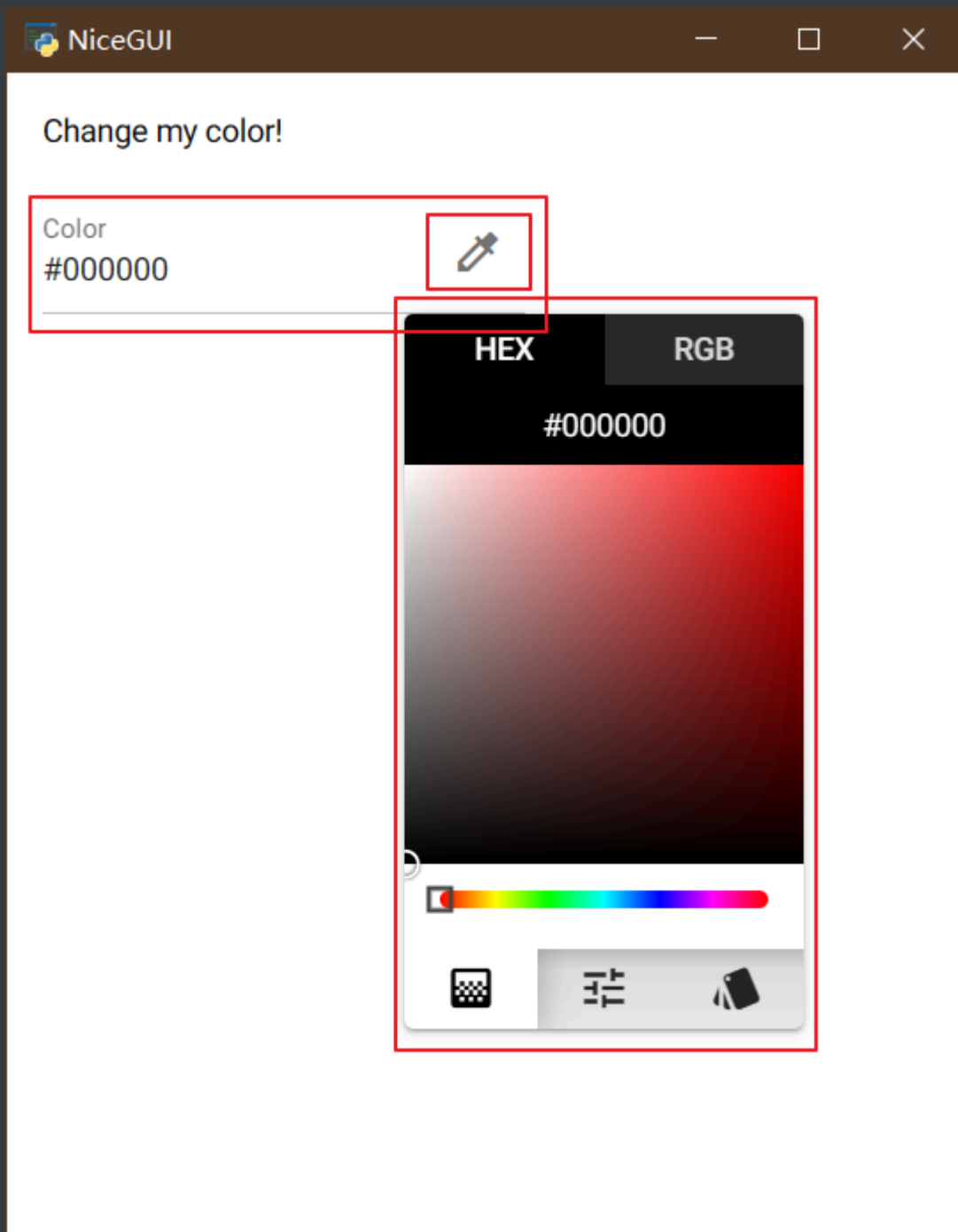
`label` 参数，字符串类型，表示显示在输入框上方的文本。

`placeholder` 参数，字符串类型，表示输入框的占位符，仅在没有颜色值的时候显示。

`value` 参数，字符串类型，表示输入框内的内容，也就是当前选择的颜色的值。

`on_change` 参数，可调用类型，表示输入框的值变化时执行的函数。

`preview` 参数，布尔类型，表示是否让取色器按钮的背景颜色与当前选择的颜色一致，默认为 `False`，即不改变按钮的背景颜色。



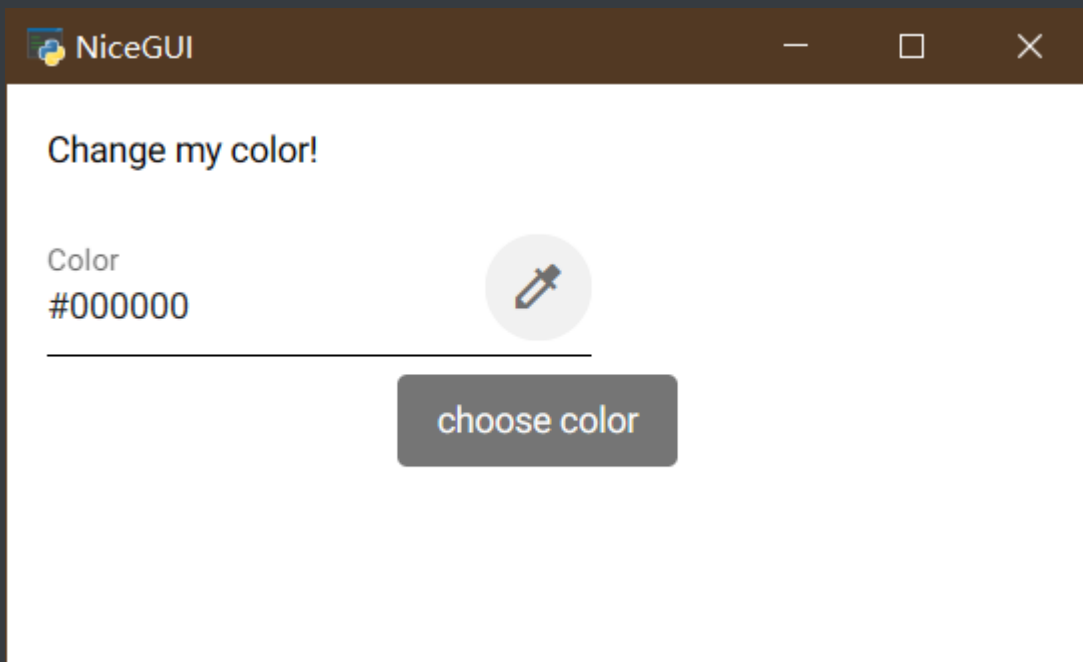
从控件的外观上看，控件主要由三部分组成：主体的输入框、右侧取色器图标代表的按钮、点击按钮之后弹出的颜色选择器面板。与之对应的，便是控件本身、控件的 `button` 成员、控件的 `picker` 成员，如果想要修改对应部分的外观、属性，可以访问对应的对象进行修改。

比如，想要给按钮增加一个工具提示，代码可以这样写：

```

1 from nicegui import ui
2
3 label = ui.label('Change my color!')
4 ci = ui.color_input(label='Color', value='#000000',
5                     on_change=lambda e: label.style(f'color:{e.value}'),
6                     preview=False)
7
8 ci.button.tooltip('choose color')
9
10 ui.run(native=True)

```



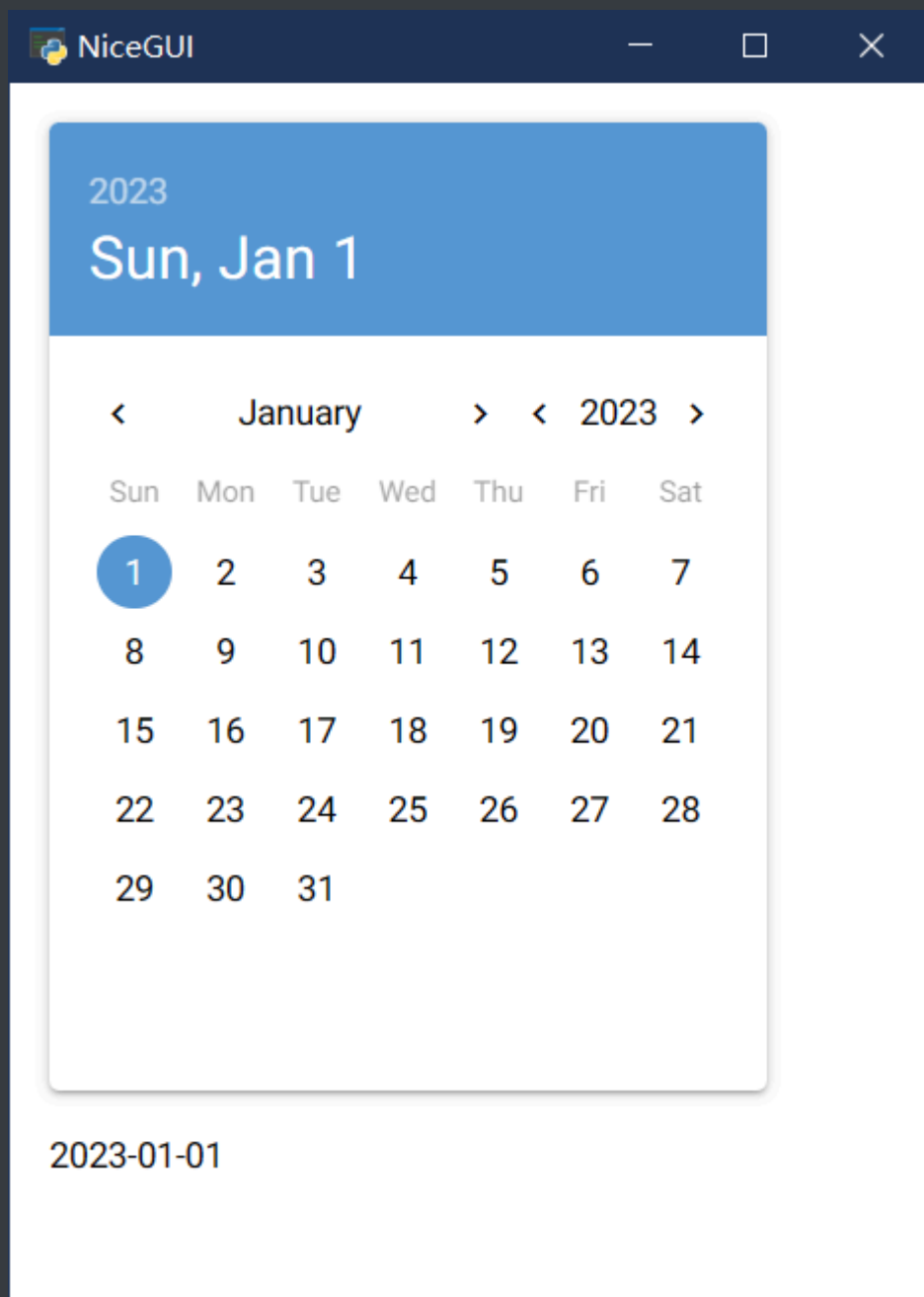
3.10.16 ui.date

日期控件支持显示、选择日期，用法简单直观，没有太复杂的用法，只是其中的 `mask` 参数需要多看看文档：

```

1 from nicegui import ui
2
3 ui.date(value='2023-01-01', mask='YYYY-MM-DD',
4        on_change=lambda e: result.set_text(e.value))
5 result = ui.label()
6
7 ui.run(native=True)

```



控件支持以下参数：

`value` 参数，字符串类型，控件的初始值。

`on_change` 参数，可调用类型，当控件的值发生变化时执行什么操作。

`mask` 参数，字符串类型，控件值的格式代码，默认为 'YYYY-MM-DD' 。

`mask` 支持的格式代码参考[官网文档](#)或者下表：

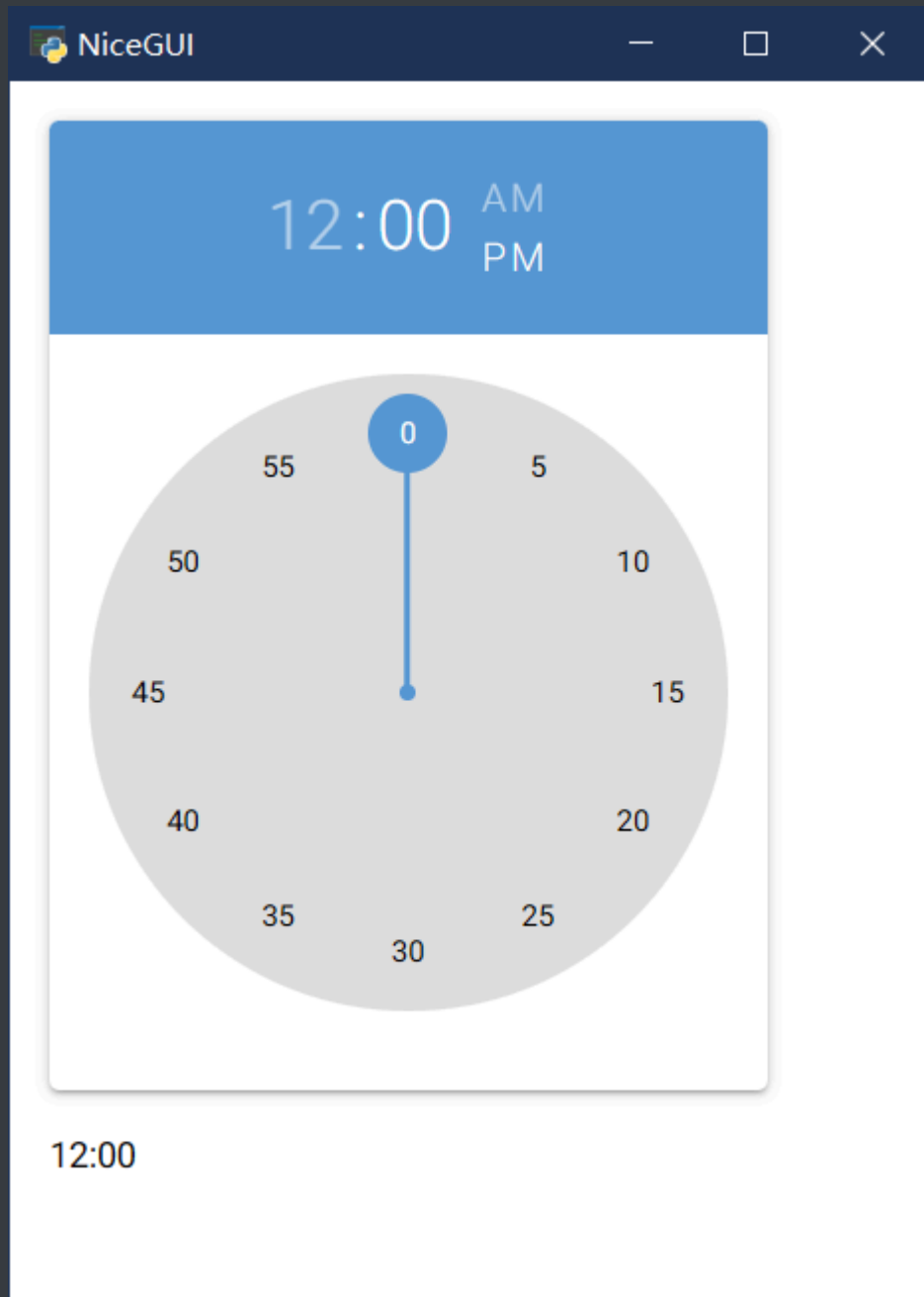
时间单位	格式代码及效果
年	YY : 70, 71.....29, 30 YYYY : 1970, 1971.....2029, 2030
月	M : 1, 2.....11, 12 Mo : 1st, 2nd.....11th, 12th MM : 01, 02.....11, 12 MMM : Jan, Feb.....Nov, Dec MMMM : January, February.....November, December
季度	Q : 1, 2, 3, 4 Qo : 1st, 2nd, 3rd, 4th
本月第几天	D : 1, 2.....30, 31 Do : 1st, 2nd.....30th, 31st DD : 01, 02.....30, 31
本年第几天	DDD : 1, 2.....364, 365 DDDo : 1st, 2nd.....364th, 365th DDDD : 001, 002.....364, 365
本周第几天	d : 0, 1.....5, 6 do : 0th, 1st.....5th, 6th dd : Su, Mo.....Fr, Sa ddd : Sun, Mon.....Fri, Sat dddd : Sunday, Monday.....Friday, Saturday
本周第几天 (ISO标准)	E : 1, 2.....6, 7
本年第几周	w : 1, 2.....52, 53 wo : 1st, 2nd.....52nd, 53rd ww : 01, 02.....52, 53

更多设计属性参考[官方API](#)。

3.10.17 ui.time

时间控件支持显示、选择时间，用法简单直观，没有太复杂的用法，只是其中的 `mask` 参数需要多看看文档：

```
1 from nicegui import ui
2
3 ui.time(value='12:00', mask='HH:mm',
4         on_change=lambda e: result.set_text(e.value))
5 result = ui.label()
6
7 ui.run(native=True)
```



控件支持以下参数：

`value` 参数，字符串类型，控件的初始值。

`on_change` 参数，可调用类型，当控件的值发生变化时执行什么操作。

`mask` 参数，字符串类型，控件值的格式代码，默认为 `'HH:mm'` 。

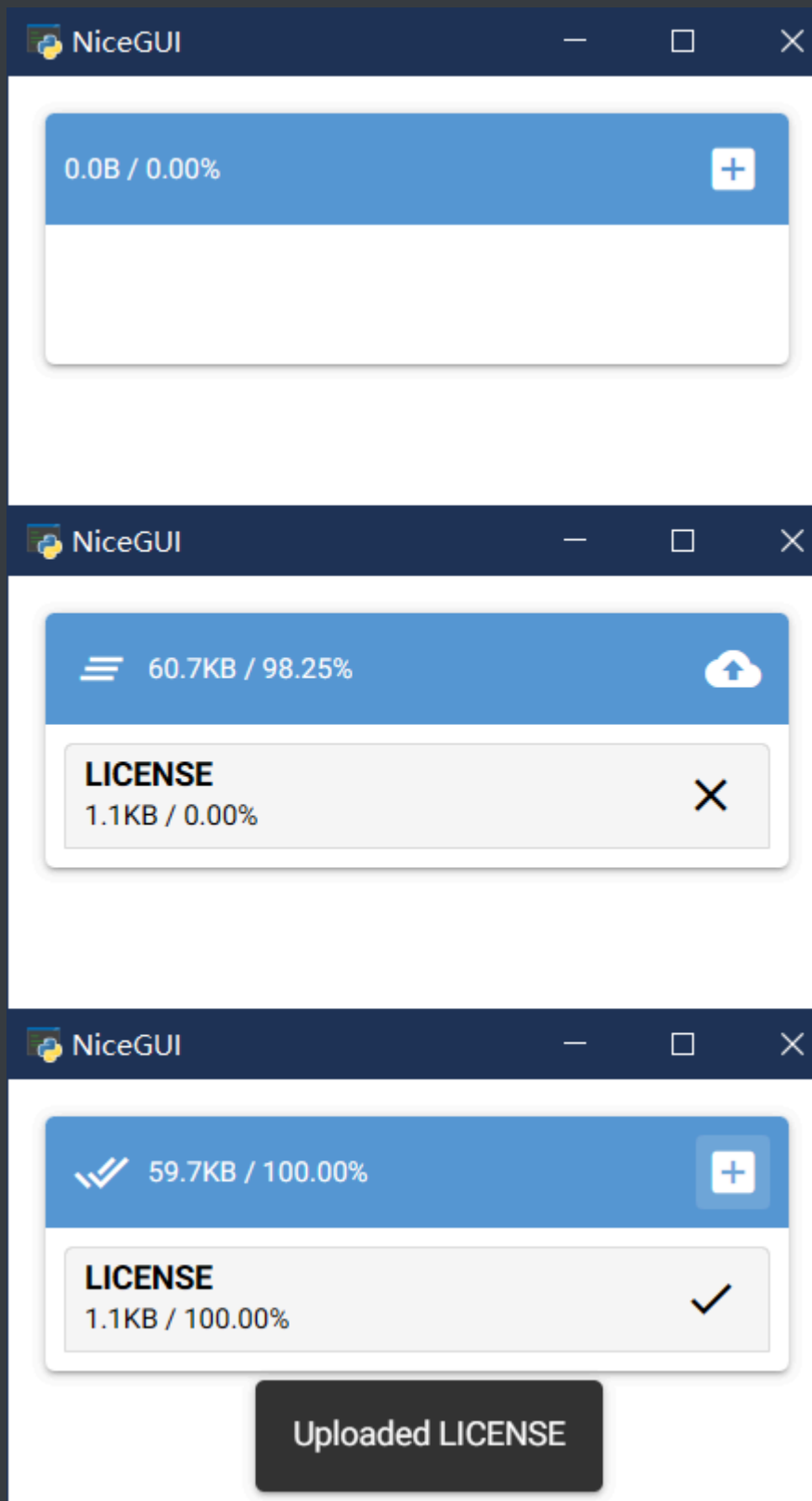
`mask` 支持的格式代码参考[官网文档](#)或者下表：

时间单位	格式代码及效果
时	H : 0, 1.....22, 23 HH : 00, 01.....22, 23 h : 0, 1.....11, 12 hh : 01, 02.....11, 12
分	m : 0, 1.....58, 59 mm : 00, 01.....58, 59
秒	s : 0, 1.....58, 59 ss : 00, 01.....58, 59
小数秒（秒的小数部分）	S : 0, 1.....8, 9 SS : 00, 01.....98, 99 SSS : 000, 001.....998, 999
时区	Z : -07:00, -06:00.....+06:00, +07:00 ZZ : -0700, -0600.....+0600, +0700
上下午	A : AM, PM a : am, pm aa : a.m, p.m
Unix时间戳	X : 1360013296 x （毫秒）：1360013296123

更多设计属性参考[官方API](#)。

3.10.18 ui.upload

之前提到过 `ui.download` 文件下载，有下载就有上传，相比于下载操作，上传操作就复杂了一些。



点击加号按钮，弹出文件选择窗口，选择文件后点确认，然后点击上传按钮，完成上传。

上传控件支持以下参数：

`multiple` 参数，布尔类型，表示是否支持上传多个文件，默认为 `False`。

`max_file_size` 参数, 整数类型, 上传文件的大小限制, 单位字节, 默认为 0 , 即不限制。

`max_total_size` 参数, 整数类型, 上传文件的总大小限制, 单位字节, 默认为 0 , 即不限制。

`max_files` 参数, 整数类型, 上传文件的数量限制, 默认为 0 , 即不限制。

`on_upload` 参数, 可调用类型, 完成一个文件上传之后执行的操作。

`on_multi_upload` 参数, 可调用类型, 完成所有文件上传之后执行的操作。

`on_rejected` 参数, 可调用类型, 当文件的上传被拒绝 (超出大小限制或者不符合要求的扩展名等) 之后执行的操作。

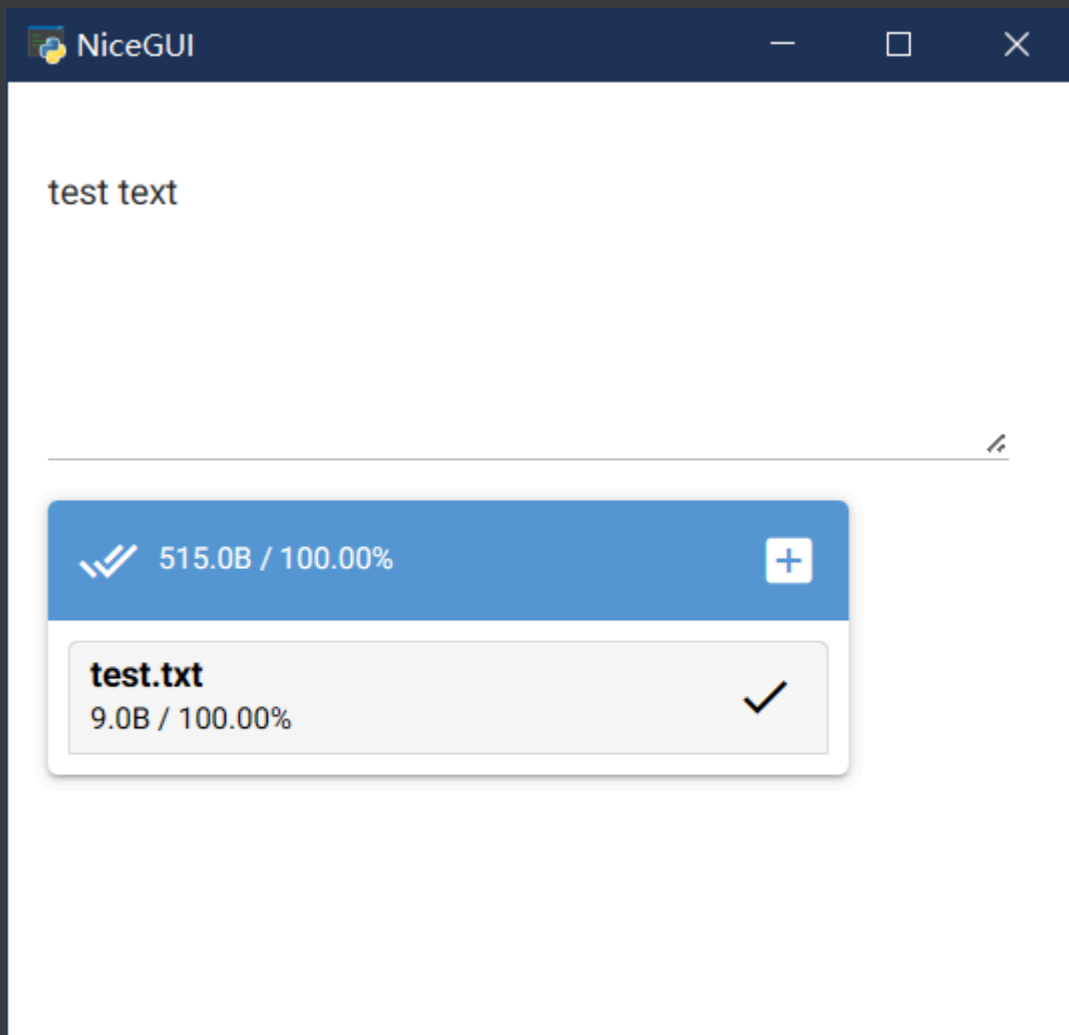
`label` 参数, 字符串类型, 显示在控件上部的说明性文字。

`auto_upload` 参数, 布尔类型, 表示是否开启自动上传, 即完成选择之后就上传, 默认为 `False` 。

更多设计属性参考[官方API](#)。

如果想要读取文件内容, 可以读取事件参数的 `content` 属性:

```
1 from nicegui import events, ui
2
3 content = ui.textarea().classes('w-96')
4
5 def handle_upload(e: events.UploadEventArguments):
6     text = e.content.read().decode('utf-8')
7     content.set_value(text)
8
9 ui.upload(on_upload=handle_upload).props('accept=.txt').classes('max-w-full')
10
11 ui.run(native=True)
```



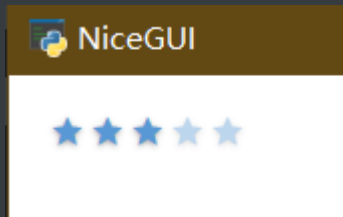
因为底层 `starlette` 库默认文件大小参数的设置，上传大文件可能会导致一些潜在的问题。为了确保更平滑地上传大文件，建议调整 `starlette` 的 `MultiPartParser` 类中的参数 `max_file_size`，将默认的 `1024 * 1024`（1 MB）调大。下面的代码就将该参数调大到5MB，来让更大的文件切片保存到服务器内存中。加大此参数并不是解除大文件的限制，而是让缓存到内存的文件块更大，以便快速处理，不然，文件会直接存入磁盘，可能会产生卡顿现象。另外，此参数也不能无限制加大，此参数过大会导致占用太多的内存，反而会导致内存不足的问题。

```
1 from nicegui import ui
2 from starlette.formparsers import MultiPartParser
3
4 MultiPartParser.max_file_size = 1024 * 1024 * 5 # 5 MB
5
6 ui.upload(on_upload=lambda e: ui.notify(f'Uploaded {e.name}')).classes('max-w-full')
7
8 ui.run(native=True)
```

3.10.19 ui.rating (2025.03.04更新)

`ui.rating` 是一个打分的交互组件，实现源于Quasar的[QRating](#)。先看示例：

```
1 from nicegui import ui
2
3 ui.rating(value=4)
4
5 ui.run(native=True)
```



可以点击对应的星级，来确定最终的评分。

`ui.rating` 支持以下参数：

`value` 参数，浮点类型，表示初始的分值。

`max` 参数，整数类型，表示最多的分值，同时也是显示在组件中评分图标个数，默认为 5。

`icon` 参数，字符串类型，表示没有选定的评分图标，默认为 'star'。

`icon_selected` 参数，字符串类型，表示选定之后的评分图标，默认同 `icon` 参数。

`icon_half` 参数，字符串类型，表示半分的评分图标，默认同 `icon` 参数。

`color` 参数，字符串类型或者 `None`，表示评分图标的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让按钮变成默认颜色），默认为 'primary'，即和主题颜色一致。

`size` 参数，表示评分图标的大小，字符串类型，采用CSS语法的大小表示方式。

`on_change` 参数，可调用类型，表示评分改变时执行的操作，可以使用lambda表达式，也可以使用函数名。

3.11 多媒体控件的使用技巧

3.11.1 ui.interactive_image 的交互技巧

interactive，顾名思义，就是交互性。既然 ui.interactive_image 是交互性图像，只是当成普通图像的加强版使用，而不学习它的交互能力实在说不过去。

通过捕获鼠标事件参数，可以获取到鼠标点击的事件种类 type 、鼠标位置 image_x\image_y：

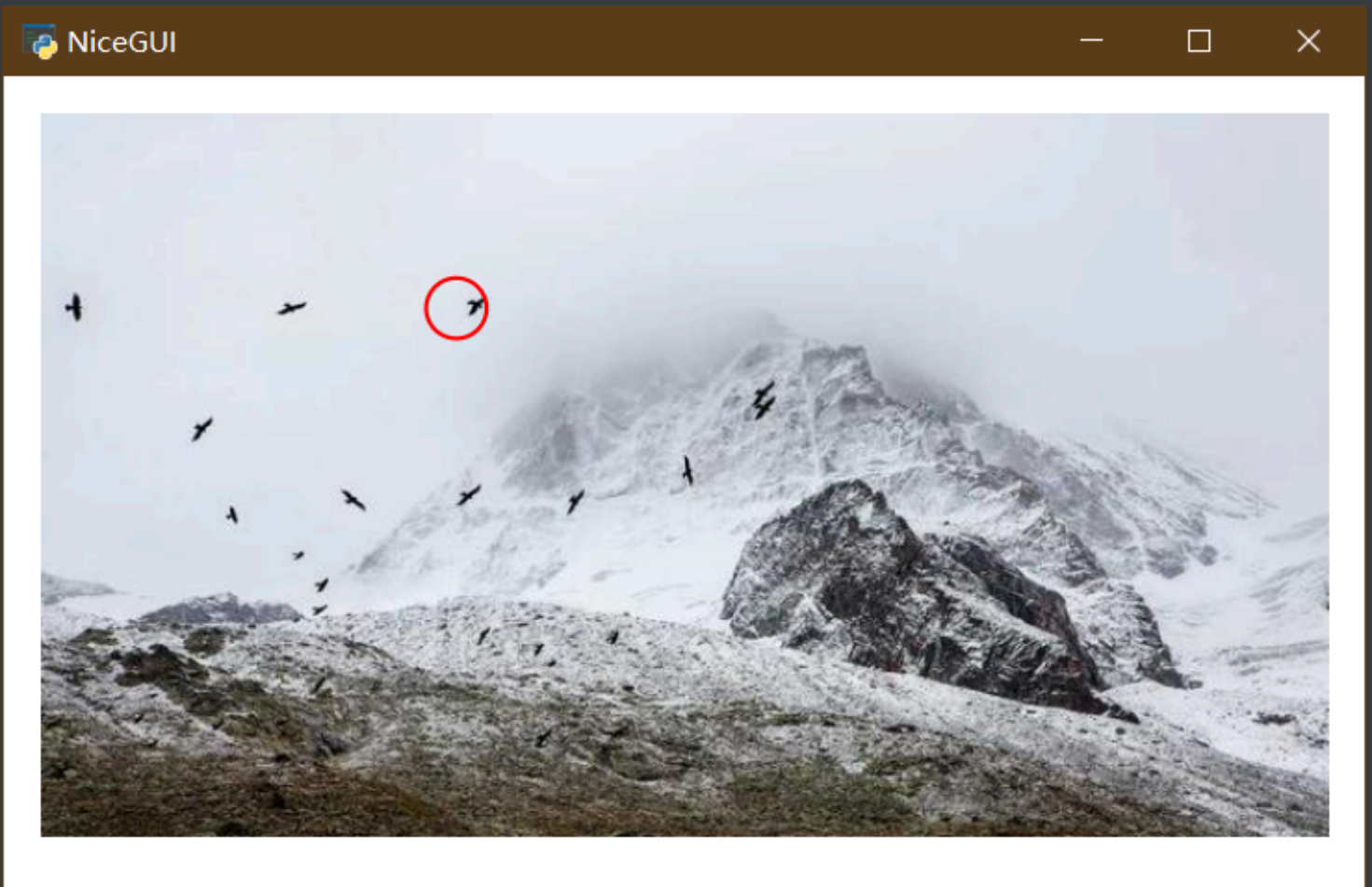
```
1 from nicegui import ui
2
3 src = 'https://picsum.photos/id/565/640/360'
4 ui.interactive_image(src, on_mouse=lambda e: ui.notify(
5     f'{e.type} at ({e.image_x:.1f}, {e.image_y:.1f})'), cross=True)
6
7 ui.run(native=True)
```

NiceGUI



既然可以通过点击获取鼠标位置，在鼠标位置画点东西自然也不是问题，还是上面的代码，可以在 `on_mouse` 参数的执行函数里，添加一些绘画的内容。前面介绍过，`content` 参数表示覆盖在图片之上的 SVG 内容，函数里要操作的，也是 `content` 属性。其中，使用 `e.sender.content` 来获取 `content` 属性；因为这个属性实际上是描述 SVG 内容的字符串，因此每次绘制的内容会自动替换上一次绘制的内容，如果需要保存之前绘制的内容，赋值操作 `=` 就要换成增加并赋值操作 `+=`：

```
1 from nicegui import ui, events
2
3 def draw(e: events.MouseEventArguments):
4     e.sender.content = f'<circle cx="{e.image_x}" cy="{
5         e.image_y}" r="15" fill="none" stroke="Red" stroke-width="2" />'
6
7 src = 'https://picsum.photos/id/565/640/360'
8 ui.interactive_image(src, on_mouse=draw, cross=True)
9
10 ui.run(native=True)
```



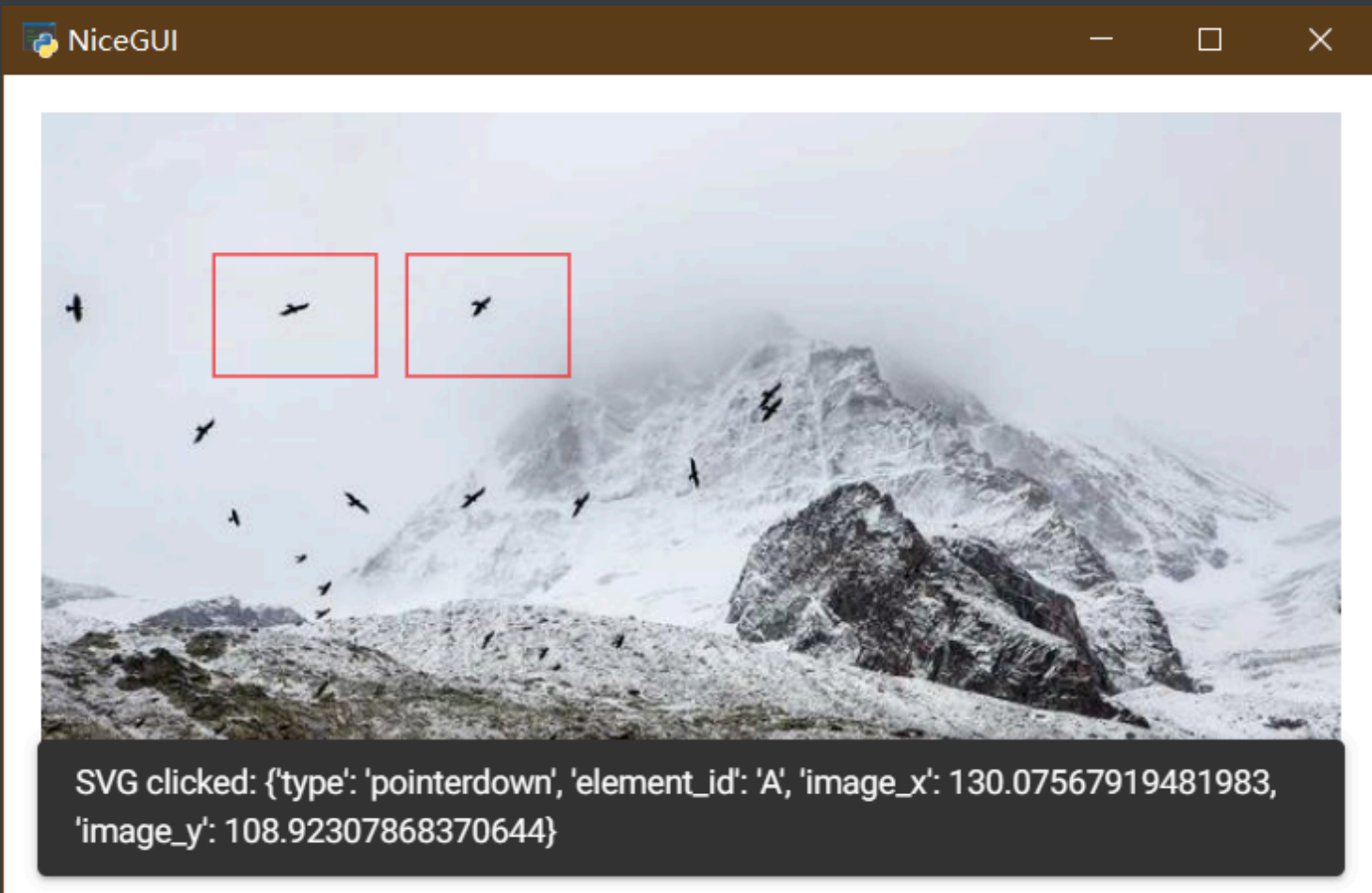
对于绘制在 `content` 上的SVG图形，其实也有交互事件，`ui.interactive_image` 也能捕捉。只不过，需要设定SVG的 `pointer-events` 为 `"all"`，来确保SVG图形接收所有事件响应，具体含义可以参考[手册](#)。

目前 `ui.interactive_image` 可以捕捉以下SVG事件：

- `pointermove`
- `pointerdown`
- `pointerup`
- `pointerover`
- `pointerout`
- `pointerenter`
- `pointerleave`
- `pointercancel`

想要在 `ui.interactive_image` 中响应SVG的事件，只需订阅`"svg:"`开头、后接SVG事件名的混合事件名即可响。

```
1 from nicegui import ui
2
3 src = 'https://picsum.photos/id/565/640/360'
4 content = '''
5     <rect id="A" x="85" y="70" width="80" height="60" fill="none" stroke="red"
6     pointer-events="all" cursor="pointer" />
7     <rect id="B" x="180" y="70" width="80" height="60" fill="none" stroke="red"
8     pointer-events="all" cursor="pointer" />
9     '''
10 ui.interactive_image(src, content=content, cross=True).on('svg:pointerdown', lambda
    e: ui.notify(f'SVG clicked: {e.args}'))
11
12 ui.run(native=True)
```



3.12 ui.add_*和app.add_*的技巧

3.12.1 app.add_static_file 和 app.add_static_files

前面提到过 `ui.image`、`ui.video`、`ui.audio` 等提供视听效果。不过，前面的例子中只用了网络图片地址，并没有使用本地地址，肯定有读者在尝试使用本地地址之后发现了一个奇怪的现象，以为 NiceGUI 有 bug。

以下面的代码为例，`os.path.dirname(os.path.abspath(__file__))` 可以获取代码文件的当前目录，在代码文件的同目录下放一个图片文件 `LOGO.png`，下面的代码就能显示这个图片。看起来没问题。但是，一旦复制这个图片的地址，将后面的文件名换成其他同目录下的文件名，就会报 404 错误（文件未找到）。

```
1 from nicegui import ui
2 import os
3
4 ui.image(f'{os.path.dirname(os.path.abspath(__file__))}/LOGO.png')
5
6 ui.run(native=True)
```

其实这不是NiceGUI的bug，而是默认的安全和缓存机制，只有代码中使用的静态文件才会生成地址映射，其他没有使用的文件即使存在，直接输入地址访问也会报不存在。以上面的代码为例，图片文件的地址是

`http://127.0.0.1:8000/_nicegui/auto/static/cf276f9ca066376dc8588fbf61afe905/LOGO.png`，中间的 `cf276f9ca066376dc8588fbf61afe905` 是图片的hash码，而不是真实存在的目录。之所以会变成这样，是因为NiceGUI会对小的静态文件进行缓存，提高访问速度。因为网页中经常存在大量图片、JavaScript代码、CSS代码等文件，使用缓存可以提高访问速度，不必每次刷新都要从服务器获取。而且，采用缓存机制，还能避免黑客恶意猜测服务器的文件目录，进而获取到影响安全的文件。

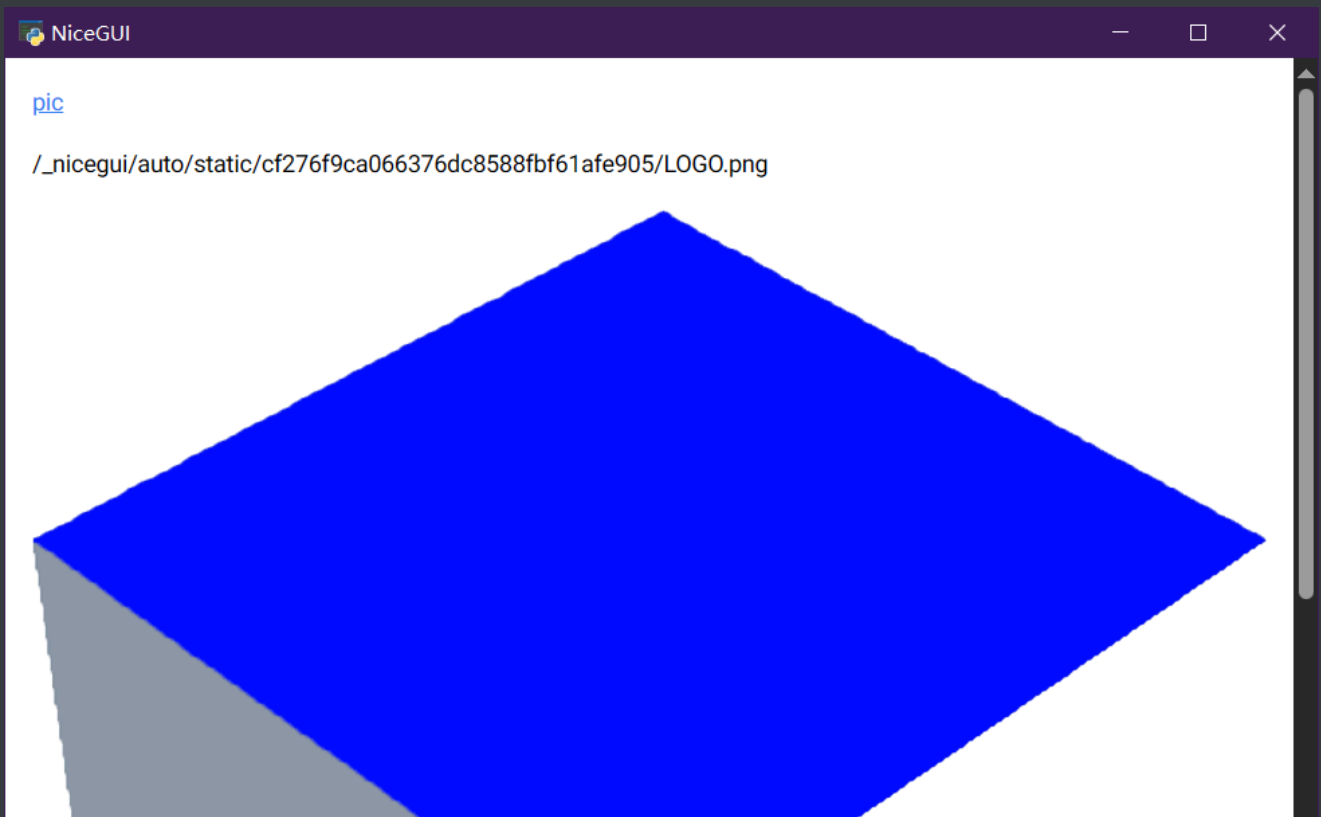
这个时候，理解这一切的读者已经恍然大悟，随之而来的是另一个问题——如果想创建一个图片的链接但图片随时修改怎么办？总不能每次都用 `ui.image` 生成一次，然后复制地址过去吧？就算使用代码实现自动化，看上去也不够优雅。

这时，就需要正式介绍一下本节要说的功能—— `app.add_static_file`。`app.add_static_file` 可以返回本地文件的服务器地址，也可以将本地文件映射为固定的服务器地址。

以代码为例：

```
1 from nicegui import ui, app
2 import os
3
4 src = app.add_static_file(
5     local_file=f'{os.path.dirname(os.path.abspath(__file__))}/LOGO.png',
6     url_path=None,
7     single_use=False
8 )
9
10 ui.link('pic', src)
11 ui.label(src)
12 ui.image(src)
```

```
13
14 ui.run(native=True)
```



可以看到，给 `app_add_static_file` 的 `local_file` 传入本地文件地址，返回的正是服务器地址，和 `ui.image` 的图片地址一致，这下，`ui.link` 也能使用图片，而不必担心图片变化还要手动复制地址。

`app_add_static_file` 有三个参数：

`local_file` 参数，字符串类型或者 `Path` 类型，表示本地文件地址。

`url_path` 参数，字符串类型，表示服务器地址，默认为 `None`，即自动生成服务器地址，也可以传入参数，例如 `'/logo.png'`，就是固定的服务器地址。

`single_use` 参数，布尔类型，表示文件是否在下载一次后移除服务器地址，默认为 `False`。

假如要添加的图片比较多，但都在一个文件夹内，是不是还要一个一个添加？不用，`app_add_static_files` 可以将本地文件夹映射为服务器地址。

`app_add_static_files` 有三个参数：

`local_directory` 参数，字符串类型或者 `Path` 类型，表示本地文件夹地址。

`url_path` 参数，字符串类型，表示服务器目录地址，必须传入 '/' 开头的字符串，例如 `'/pic'`，同时不能为 `'/'`，否则会报错。

`follow_symlink` 参数，布尔类型，表示是否追踪符号链接，即目录下如果存在符号链接的话，会将符号链接代表的实际路径连接到当前路径下，让服务器地址访问符号链接就和本地访问符号链接一样。这个参数默认为 `False`，即不处理符号链接，服务器地址没法访问符号链接。注意，此参数为 `True` 并且在 Windows 平台下的话，代码中使用的 `os.path.abspath(__file__)` 会导致获取到文件路径中的磁盘符号为小写，将导致底层代码出错进而上报 404 错误。此时应该将 `os.path.abspath(__file__)` 换成 `os.path.realpath(__file__)`。如果后续遇到 Windows 平台下开启 `app.add_static_files` 的追踪符号链接后，报 404 错误，可以按照这个思路检查一下传入的 `local_directory` 参数中，磁盘符号是不是小写。

3.12.2 `app.add_media_file` 和 `app.add_media_files`

前面的 `app.add_static_file`、`app.add_static_files` 用于添加小的静态文件，本节要介绍的 `app.add_media_file`、`app.add_media_files` 则用于添加媒体文件。看名字的话，和前两者相似，一个是添加单个文件，一个是添加文件夹，那 NiceGUI 为何要设计重复的功能？

重复当然是不可能重复的，既然是用于媒体文件，肯定与静态文件不同。媒体文件通常是音视频等需要流式传输的文件，不会一下子全部加载，而是一点一点加载，这一点与静态文件不同。毕竟媒体文件通常比较大，一下子全部缓存，一不小心就会让缓存空间爆满。之所以采用流式传输，是因为媒体文件需要支持播放时跳转到指定时间点，如果是采用静态文件那种缓存全部再加载的机制，跳转到指定时间点的功能会失效，只有流式传输才支持跳转到指定时间点。

`app.add_media_file`、`app.add_media_files` 得到的服务器地址就是采用流式传输，而不是缓存机制。

以下面的代码为例，可以看一下区别，因为此代码需要本地视频文件，这里就不提供直接运行的代码了，视频文件地址由读者自己修改：

```
1 from nicegui import ui,app
2
3 video = r'{视频文件的本地路径}'
4 app.add_static_file(local_file=video,url_path='/video1')
5 app.add_media_file(local_file=video,url_path='/video2')
6
7 ui.video('/video1')
8 ui.video('/video2')
9
10 ui.run(native=True)
```

通常情况下，`app.add_static_file` 得到的视频文件地址，在播放器中没法拖动进度条，`app.add_media_file` 得到的视频文件地址，在播放器中和正常播放一样，可以自由拖动时间轴。但是，大多数服务器、浏览器、播放器有容错优化，实际上两种方法得到的视频地址都可以正常播放，只有某些要求严格的接口、播放程序才会有区别。一般建议读者使用 `app.add_media_file` 添加媒体文件，以免特定情况下出现不兼容的问题。

`app.add_media_file` 的参数和 `app.add_static_file` 的一样，这里不再赘述。

`app.add_media_files` 的参数则比 `app.add_static_files` 少了 `follow_symlink` 参数。

3.12.3 `ui.add_head_html` 和 `ui.add_body_html`

`ui.add_head_html` 可以添加HTML代码到页面 `head` 标签内，`ui.add_body_html` 可以添加HTML代码到页面 `body` 标签内。对于页面加载来说，`head` 标签内的内容一般不显示，而且因为是从上到下加载，`head` 标签内的内容会先被加载，这里通常放着需要第一时间执行的前置脚本和样式设置。`body` 标签内放着页面显示内容的主体，使用 `ui.add_body_html` 会在NiceGUI其他控件加载前嵌入HTML代码，因此 `ui.add_body_html` 通常是为了实现在NiceGUI其他内容显示之前放置内容，包括但不限于显示的内容、执行前置脚本和样式设置。

这两个方法都有两个参数字符串参数 `code` 和布尔参数 `shared`。前者表示要嵌入的HTML代码，后者表示是否在所有页面（`ui.page`）执行嵌入操作，对于使用 `ui.page` 装饰的页面，后者可以让嵌入操作的代码只写一次，就能应用于所有页面上。

前面说过，`ui.html` 也可以添加HTML代码，那和 `ui.add_head_html`、`ui.add_body_html` 有什么区别？

`ui.html` 会返回一个元素，可以使用一般元素的方法，`ui.add_head_html` 和 `ui.add_body_html` 是直接将HTML代码嵌入页面，不会返回任何对象，没法调用一般元素的方法。不过，它们支持嵌入JavaScript代码，而 `ui.html` 只能是纯HTML。

```
1 from nicegui import ui
2
3 ui.add_head_html('<script>alert("yes")</script>')
4 ui.add_body_html('<script>alert("yes")</script>')
5 #ui.html('<script>alert("yes")</script>')
6
7 ui.run(native=True)
```

3.12.4 `ui.add_css`、`ui.add_scss`、`ui.add_sass`

这三个功能都可以添加CSS代码，只是对应的CSS代码语法规则不同。

`ui.add_scss` 和 `ui.add_sass` 依赖的 `libsass` 默认不安装，如果想要使用此控件，可以在项目根目录下使用以下命令安装：

```
1 pdm add nicegui[sass]
2 #或者下面这条
3 pdm add libsass
```

如果运行环境是全局环境或者想用pip安装，可以执行下面的命令安装：

```
1 pip install nicegui[sass]
2 #或者下面这条
3 pip install libsass
```

SASS是一款强化 CSS 的辅助工具，它在 CSS 语法的基础上增加了变量 (variables)、嵌套 (nested rules)、混合 (mixins)、导入 (inline imports) 等高级功能，这些拓展令 CSS 更加强大与优雅。简单一点理解的话，SASS是CSS扩展版本。SASS有两种语法风格：以 `.scss` 为后缀的，是语法风格和CSS一致的版本，即采用大括号表示所属，用分号表示一句内容的结束；以 `.sass` 为后缀的，是语法风格变成用缩进代替大括号、用换行代替分号的版本。

因此 `ui.add_css`、`ui.add_scss`、`ui.add_sass` 分别代表可以添加标准CSS代码、scss风格代码、sass风格代码。因为scss语法风格和CSS一致，基本兼容CSS，所以，可以用 `ui.add_scss` 添加CSS代码，反之不行。

`ui.add_css` :

```
1 from nicegui import ui
2
3 ui.add_css('''
4     .red {
5         color: red;
6     }
7 ''')
8 ui.label('This is red with CSS.').classes('red')
9
10 ui.run()
```

`ui.add_scss` :

```
1 from nicegui import ui
2
3 ui.add_scss('''
4     .green {
5         background-color: lightgreen;
6         .blue {
7             color: blue;
8         }
9     }
10 ''')
11 with ui.element().classes('green'):
12     ui.label('This is blue on green with SCSS.').classes('blue')
13
14 ui.run()
```

`ui.add_sass` :


```

1  from nicegui import ui
2
3  ui.add_sass('''
4      .yellow
5          background-color: yellow
6      .purple
7          color: purple
8  ''')
9  with ui.element().classes('yellow'):
10     ui.label('This is purple on yellow with SASS.').classes('purple')
11
12  ui.run()

```

3.13 ui.keyboard 的事件处理技巧

`ui.keyborad` 可以在页面添加一个按键事件的响应。

`ui.keyborad` 有以下四个参数：

`on_key` 参数，可调用类型，表示发生按键事件之后要执行的函数。

`active` 参数，布尔类型，表示是否激活该功能，默认为 `True`。

`repeating` 参数，布尔类型，表示当按键持续按下的时候是否重复发送按键事件，默认为 `True`。

`ignore` 参数，字符串列表类型，表示当哪些元素激活时忽略按键事件的响应，默认为 `['input', 'select', 'button', 'textarea']`。

对于 `on_key` 参数，可以传递一个 `KeyEventArguments` 响应对象作为可调用对象的参数，该对象有以下属性：

- `sender`：事件的发送者，即 `ui.keyboard` 元素。
- `client`：客户端对象。
- `action`：`KeyboardAction` 对象，表示按键当前的动作，该对象有以下属性：
 - `keydown`：按键是否按下。
 - `keyup`：按键是否松开。

- `repeat` : 按键是否重复。
- `key` : `KeyboardKey` 对象, 表示当前按键, 该对象有以下属性:
 - `name` : 按键名。比如: `"a"`、`"Enter"`、`"ArrowLeft"` 等。可以参考[这里](#)提供的按键名清单。
 - `code` : 按键的代号。比如: `"KeyA"`、`"Enter"`、`"ArrowLeft"` 等。
 - `location` : 按键的位置。0 表示标准键盘区, 1 表示左边的按键 (指的是 `Ctrl`、`Alt`、`Shift` 这种左右都有的按键), 2 表示右边的按键, 3 表示数字键盘区。
- `modifiers` : `KeyboardModifiers` 对象, 表示当前修饰键 (`Ctrl`、`Alt`、`Shift`、`Win` 这种可以与字母、数字、功能键等组合使用的按键), 该对象有以下属性:
 - `alt` : `Alt` 键 (Mac下的 `Opt` 键) 是否被按下。
 - `ctrl` : `Ctrl` 键是否被按下。
 - `meta` : `Meta` 键 (Win的 `Win` 键或者Mac下的 `Cmd` 键) 是否被按下。
 - `shift` : `Shift` 键是否被按下。

为了方便, `KeyboardKey` 对象还有以下属性:

- `is_cursorkey` : 方向键是否被按下 (数字键盘区的方向键不算)。
- `number` : 按下了主键盘区上方的哪个数字键。0 - 9 表示对应的数字键, `None` 没有按下上方的数字键。
- `backspace`、`tab`、`enter`、`shift`、`control`、`alt`、`pause`、`caps_lock`、`escape`、`space`、`page_up`、`page_down`、`end`、`home`、`arrow_left`、`arrow_up`、`arrow_right`、`arrow_down`、`print_screen`、`insert`、`delete`、`meta`、`f1`、`f2`、`f3`、`f4`、`f5`、`f6`、`f7`、`f8`、`f9`、`f10`、`f11`、`f12` : 表示对应的按键是否被按下。

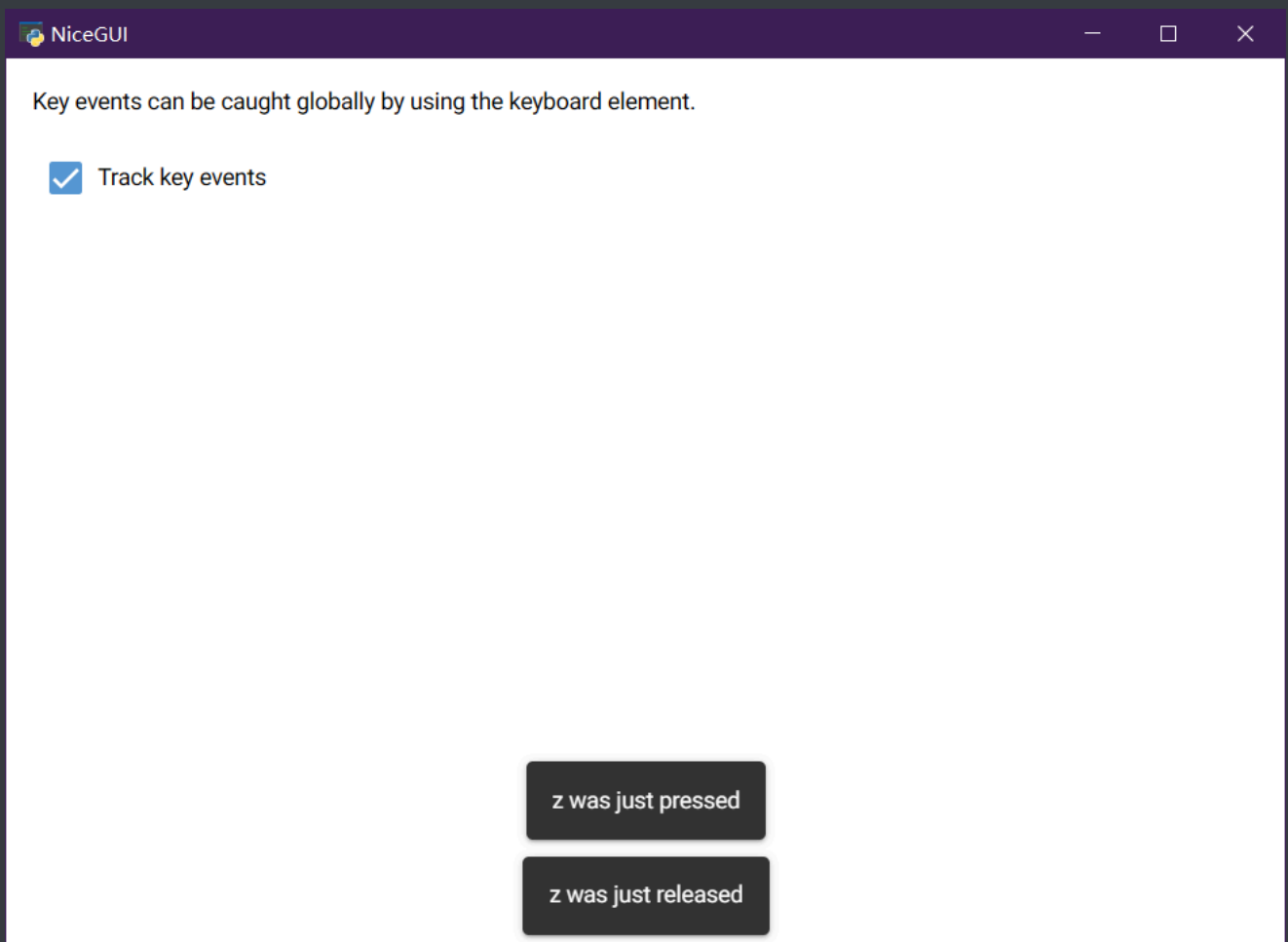
下面代码中, 通过勾选、取消复选框来启用、禁用按键事件捕捉。在按键的处理函数里, 通过判断 `Ctrl` 键是否被按下来决定是否通知另一个按钮的按下、弹起状态:

```
1 from nicegui import ui
2 from nicegui.events import KeyEventArguments
3
4 def handle_key(e: KeyEventArguments):
5     if e.modifiers.ctrl and not e.key.control:
6         if e.action.keyup:
7             ui.notify(f"{e.key} was just released")
```

```

8         elif e.action.keydown:
9             ui.notify(f"{e.key} was just pressed")
10
11 keyboard = ui.keyboard(
12     on_key=handle_key,
13     active=True,
14     repeating=True,
15     ignore=["input", "select", "button", "textarea"],
16 )
17 ui.label("Key events can be caught globally by using the keyboard element.")
18 ui.checkbox("Track key events").bind_value_to(keyboard, "active")
19
20 ui.run(native=True)

```



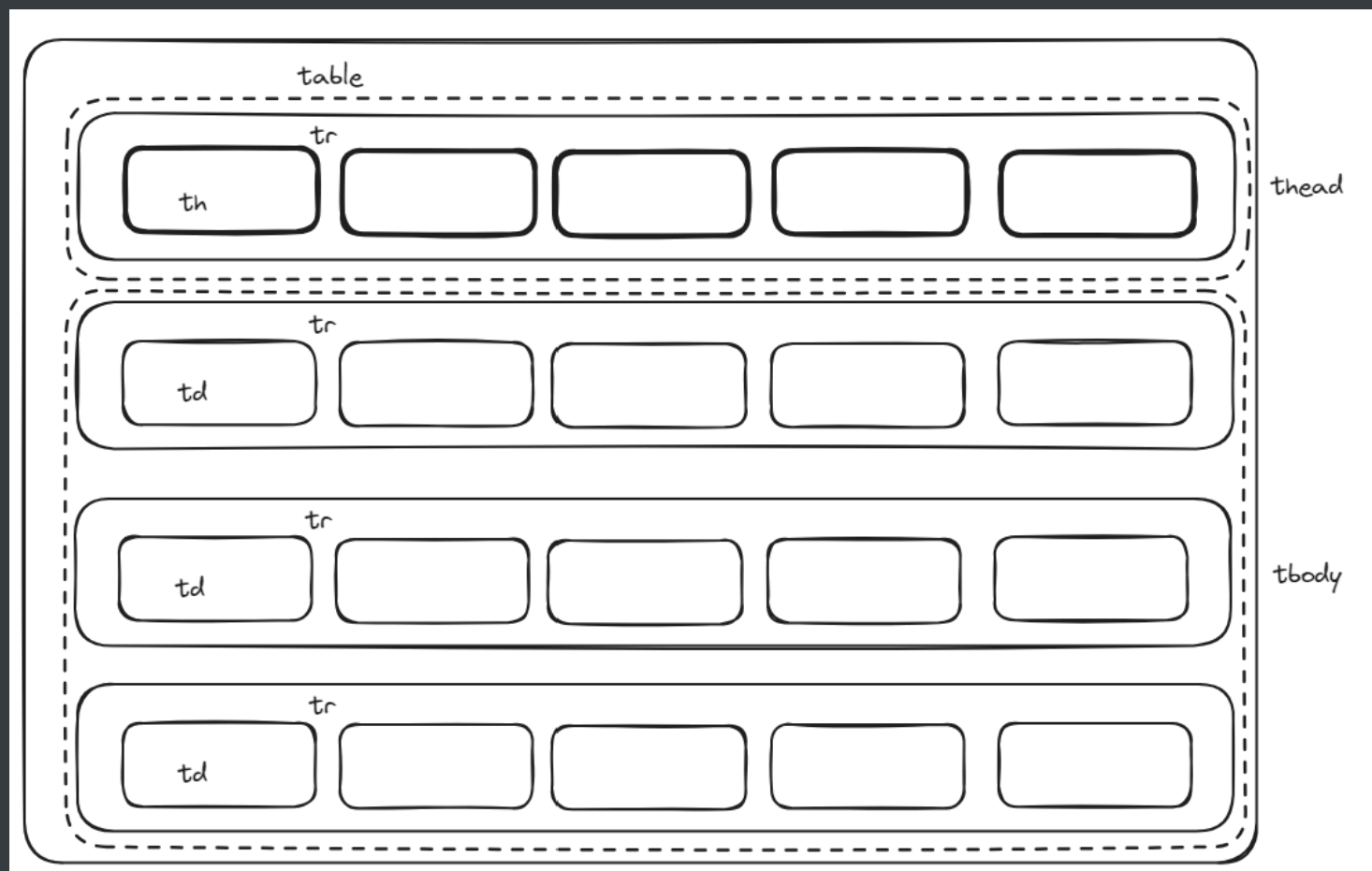
3.14 其他数据展示控件

因为大部分数据展示控件使用的是第三方扩展库，主要用法、API全在第三方网站上，这里只重点讲一下非第三方扩展库的控件。使用扩展库的控件，只简单过一遍基本用法，更多用法则以问题补充的形式添加，就不在教程中细讲。

3.14.1 ui.table

在学习NiceGUI的表格之前，各位读者需要先了解一下HTML的表格结构，等日后熟悉表格控件之后，想要对表格进行深入美化时，很有帮助。

在HTML中，定义表格常用这几种标签：`table`、`tbody`、`thead`、`tr`、`td`、`th`。`table` 表示整个表格，`thead` 表示表头，`tbody` 表示表格内容主体，`tr` 表示一整行，`td` 表示每个单元格，`th` 表示表头中的单元格。具体结构如下图所示：



在HTML中，想要定义一个表格，需要自己写一堆标签。哪怕有插件，对于每个单元格内的数据，操作起来也没那么简单。

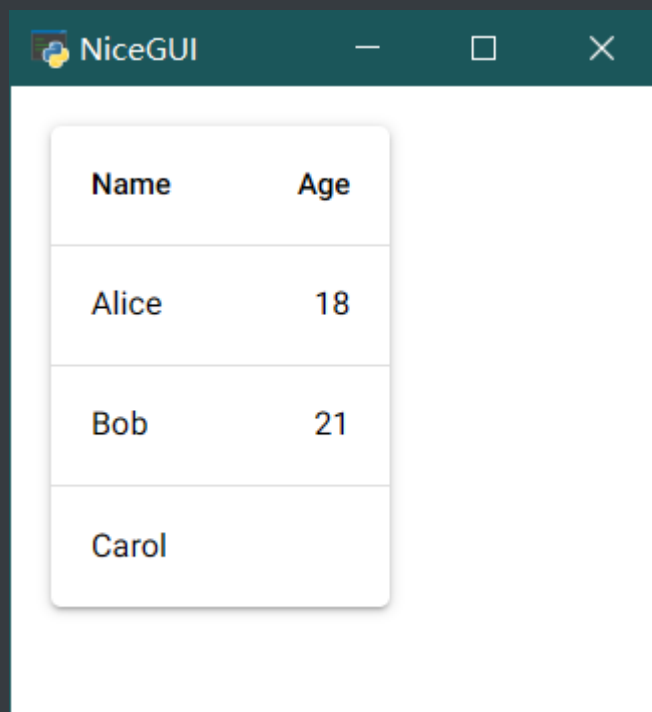
不过，NiceGUI的表格控件简化了绘制的过程，让开发者更加专注于数据的处理，确实是快捷展示数据的一种途径。先看一段简单的代码示例：

```
1 from nicegui import ui
2
3 columns = [
```

```

4      {'name': 'firstname', 'label': 'Name', 'field': 'firstname', 'required': True,
      'align': 'left'},
5      {'name': 'age', 'label': 'Age', 'field': 'age', 'sortable': True},
6  ]
7  rows = [
8      {'firstname': 'Alice', 'age': 18},
9      {'firstname': 'Bob', 'age': 21},
10     {'firstname': 'Carol'},
11  ]
12  table = ui.table(columns=columns, rows=rows, row_key='firstname')
13
14  ui.run(native=True)

```



在正式介绍表格控件的参数用法前，先来学习一下表头定义。

`columns` 这个字典通过关键字给Quasar的表头定义传入一系列参数，进而一想到表格控件的展示。在Quasar中，表头定义支持以下参数：

`name`：表格每一列的独特的id，用字符串表示。这个参数并不是显示出来的表头，只是表示这一列的变量标识符，就和在Python中定义一个变量一样。这个参数后续会用在 `'body-cell-[name]'` slot（插槽）中的 `name` 和 `pagination` 中的 `sortBy` 等一系列API中特指 `name` 的地方。

`label`：表示每一列显示的表头内容。如果此参数没有定义，会取 `rows` 中每一行的key当表头（这个参考后续介绍的表格最简用法）。

`field`：表示该列对应每一行的哪个key的数据。看示例代码可知，每一行的数据也是用字典表示，而字典的key对应的就是表头定义里的 `field`。这样，显示在这一列里的，就是每一行的这个key的数据。

`required`：表示该列的数据是否为必须的。这个值设置为 `True` 的列，不受 '`visible-columns`' 的影响，会强制显示出来。

设置 '`visible-columns`' 属性（`props`）为字符串列表之后，只有列的 `name` 在字符串列表中，列才会显示出来。比如 `table.props['visible-columns'] = ["age","firstname"]`、`table.props.update(visibleColumns = ["age","firstname"])`、`table.props('' :visible-columns="['firstname','age']" '')`、`table.props(f'' :visible-columns="{['firstname','ages']}" '')`。注意，在第二个示例代码中，字典 `update` 方法用的关键字，就是将对应属性名的蛇形命名法（使用下划线或连字符分隔每个字段）转化为骆驼命名法（即去掉蛇形命名法的下划线或者连字符之后，除了第一个字段，每个字段的首字母都大写）。另外，该属性只接受JavaScript的字符串列表，直接修改 `props` 字典的话，NiceGUI会自动转换。如果是使用 `props` 方法传入，不管是使用f-string传入英文引号包起来的Python字符串列表，还是直接传英文引号包起来的字符串列表，结果都是普通字符串，并不会转换为JavaScript的字符串列表（`props` 方法不支持解析Python列表，所以要用英文引号包起来）。因此，在第三、四个示例代码中，需要给被设置的属性添加英文冒号（`':'`）前缀，启用计算表达式功能，让客户端将该属性的值当成JavaScript表达式执行一次，这样就能转换为JavaScript的字符串列表。

`align`：对齐，表示这一列的对齐方向。

`sortable`：是否可排序，即点击表头可以启用该列数据的排序。

`sort`：排序的计算方法，使用JavaScript语法的函数定义。根据 `(a, b, rowA, rowB) => parseInt(a, 10) - parseInt(b, 10)` 返回值是否小于0来判断前者是否小于后者。此方法需要JavaScript基础和对相关API的了解，读者可以按需使用。

`rawSort`：排序的计算方法，使用JavaScript语法的函数定义。根据 `(a, b, rowA, rowB) => parseInt(a, 10) - parseInt(b, 10)` 返回值是否小于0来判断前者是否小于后者。此方法需要JavaScript基础和对相关API的了解，读者可以按需使用。

`format`：使用JavaScript语法的函数定义该列的数据显示为什么格式。比如：

```
1 (val, row) => `${val}%`  
2 val => val ? "\u2611":"\u2610"
```

注意，以上三个需要使用JavaScript函数表达式的参数，在Python中传参时，需要在前面加冒号，如：':format':'val => val ? "\u2611":"\u2610"'。这样才能启用计算表达式功能，不然不会生效。

`sortOrder`：点击表头时，先使用从小到大排序（递增）还是先使用从大到小排序（递减）。只支持 'ad' 或 'da'，并且此参数会覆盖 'column-sort-order' 属性。

下面两个参数只对单元格内容生效：

`style`：内容样式，采用CSS语法。

`classes`：内容样式类名。

下面两个参数只对表头生效：

`headerStyle`：表头样式，采用CSS语法。

`headerClasses`：表头样式类名。

下面正式介绍table的参数：

`rows` 参数，字典类型，表示表格的内容。字典的键（key）需要对应表头定义的 `field` 参数。

`columns` 参数，字典类型，表示表头定义，具体参数含义参考上面的内容或者官网文档。如果没有此参数，表头会自动取 `rows` 字典的键（key）。

`column_defaults` 参数，字典类型，表示默认的表头定义。对于每个表头都需要设置的定义，为了减少重复操作的工作量，可以使用此参数传递。

`row_key` 参数，字符串类型，表示确定每行数据唯一性的键（key）值，默认是 'id'，可以指定。注意，请务必保证指定唯一性键（key）值后，每行数据用该键（key）查询不会重复，否则会导致表格数据异常。

`title` 参数，字符串类型，表格的标题。

`selection` 参数，字符串类型，使用鼠标单击会不会选择对应的行，支持 "single"（单选）、"multiple"（多选），默认为 `None`（不选择）。如果启用了单选或者多选，表格控件的 `selected` 属性会返回当前选择的行。

`pagination` 参数，字典类型或整数类型，如果是整数类型，表示分页时每页几行（0 表示无数行，也就是分页常见的单页显示）；如果是字典类型，表示分页的定义。默认为 `None`，即不分页。关于分页的定义，支持以下参数：

- `rowsPerPage`：每页多少行。
- `sortBy`：根据哪个表头定义的 `name` 排序。
- `descending`：是否递减排序，默认为 `False`。
- `page`：启用分页时，当前显示哪一页，默认为第一页。

`on_select` 参数，可调用类型，当选择的行变化时执行什么操作。

`on_pagination_change` 参数，可调用类型，当分页（每页多少行、当前页、排序等相关属性）变化时执行什么操作。

注意，如果启用了单选或者多选，表格控件的 `selected` 属性会返回当前选择的行。

别看表格控件涉及到的参数很多，想要快速创建也很简单，只需记住一个参数，那就是 `rows`。以下是最简化的表格用法：

```
1 from nicegui import ui
2
3 ui.table(rows=[
4     {'make': 'Toyota', 'model': 'Celica', 'price': 35000},
5     {'make': 'Ford', 'model': 'Mondeo', 'price': 32000},
6     {'make': 'Porsche', 'model': 'Boxster', 'price': 72000},
7 ])
8
9 ui.run(native=True)
```

如果想要表格列的动态隐藏，可以给对应列的表头和单元格设置样式 `'hidden'`，代码如下：

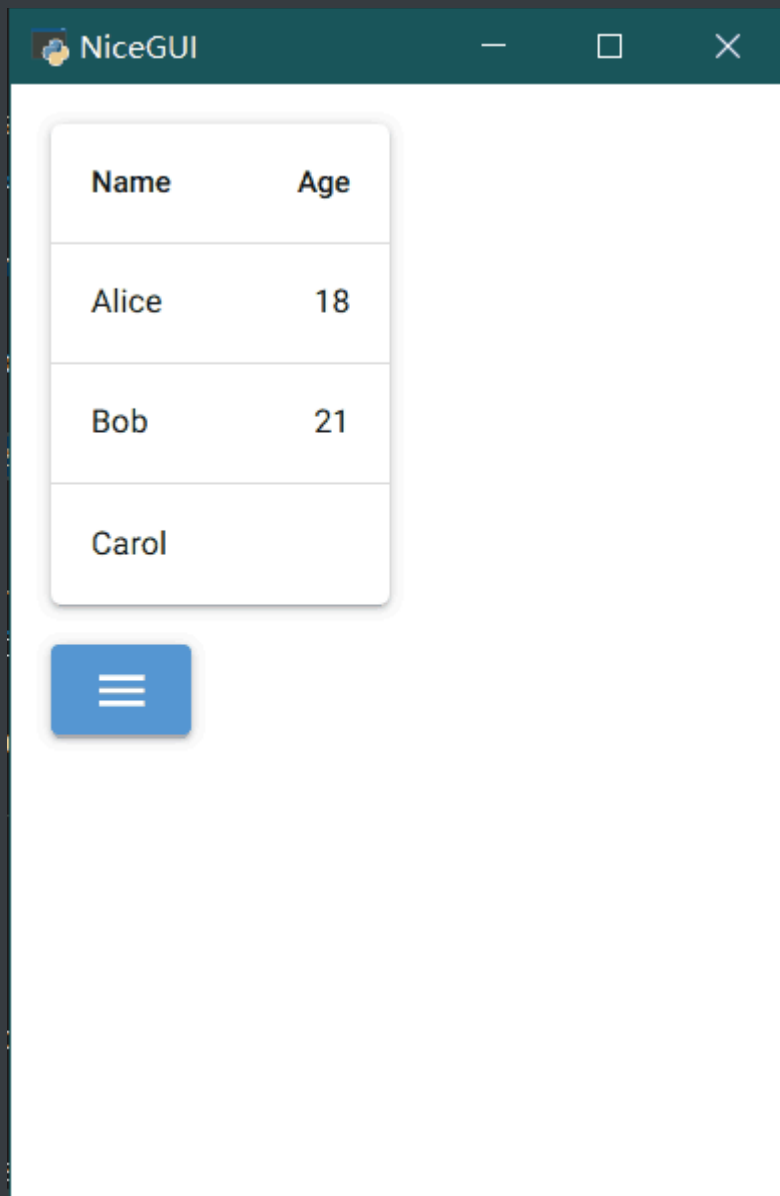
```
1 from nicegui import ui
```



```

2  from typing import Dict
3
4  columns = [
5      {'name': 'name', 'label': 'Name', 'field': 'name',
6        'required': True, 'align': 'left'},
7      {'name': 'age', 'label': 'Age', 'field': 'age', 'sortable': True},
8  ]
9  rows = [
10     {'name': 'Alice', 'age': 18},
11     {'name': 'Bob', 'age': 21},
12     {'name': 'Carol'},
13 ]
14 table = ui.table(columns=columns, rows=rows, row_key='name')
15
16
17 def toggle(column: Dict, visible: bool) -> None:
18     column['classes'] = '' if visible else 'hidden'
19     column['headerClasses'] = '' if visible else 'hidden'
20     table.update()
21
22 with ui.button(icon='menu'):
23     with ui.menu(), ui.column().classes('gap-0 p-2'):
24         for column in columns:
25             ui.switch(
26                 text=column['label'],
27                 value=True,
28                 on_change=lambda e, c=column: toggle(
29                     column=c,
30                     visible=e.value
31                 )
32             )
33
34 ui.run(native=True)

```



想要给表格添加数据的话，可以使用 `add_row` 方法添加一行，或者用 `add_rows` 方法添加多行：

```

1 from datetime import datetime
2 from nicegui import ui
3
4 def add():
5     table.add_row({'date': datetime.now().strftime('%c')})
6     table.run_method('scrollTo', len(table.rows)-1)
7
8 columns = [{'name': 'date', 'label': 'Date', 'field': 'date'}]
9 table = ui.table(columns=columns, rows=[]).classes('h-52').props('virtual-scroll')
10 ui.button('Add row', on_click=add)
11
12 ui.run(native=True)

```

想要在表格中使用来自 pandas 的 DataFrame 数组，可以使用 from_pandas 方法：

```

1 import pandas as pd
2 from nicegui import ui
3
4 df = pd.DataFrame(data={'col1': [1, 2], 'col2': [3, 4]})
5 ui.table.from_pandas(df).classes('max-h-40')
6
7 ui.run(native=True)

```

自定义该列的数据排序方法和数据显示格式，可以使用前面提到的表头定义中的 'sort' 和 'format'：

```

1 from nicegui import ui
2
3 columns = [
4     {
5         'name': 'name',
6         'label': 'Name',
7         'field': 'name',
8         'sortable': True,
9         ':sort': '(a, b, rowA, rowB) => b.length - a.length',
10    },
11    {

```



```

22  ui.table(
23      columns=columns,
24      rows=rows,
25      on_pagination_change=lambda e: ui.notify(e.value),
26      pagination={"rowsPerPage": 4, "sortBy": "age", "page": 2},
27  )
28
29  ui.run(native=True)

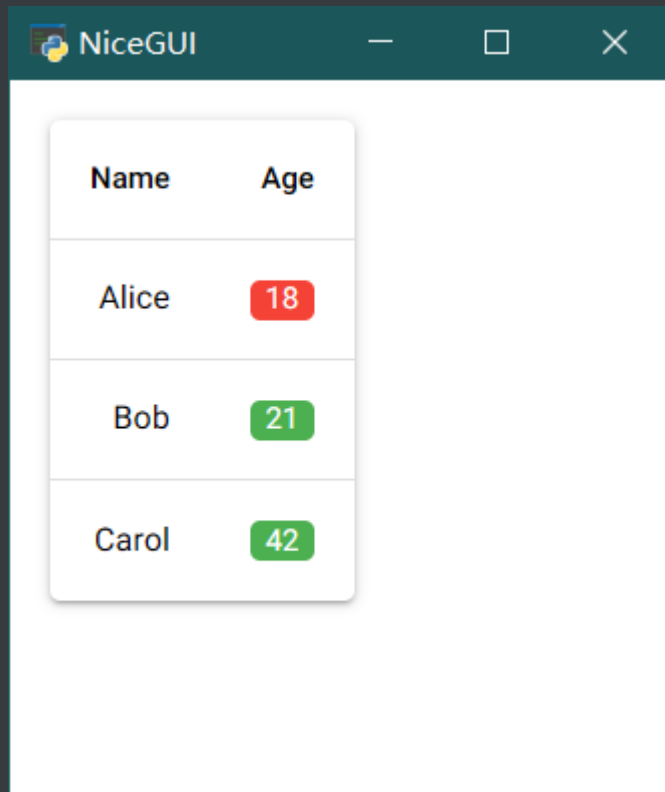
```

一种根据内容确定显示格式的方法，使用的是 'body-cell-{name}' slot而不是 'format' 参数，可扩展性更强，但要求更属性Quasar框架和前端知识，仅作了解，有余力的可以自行学习。这段代码的作用是根据 age 是否大于21，来让单元格内的 q-badge 的显示显示为绿色或者红色：

```

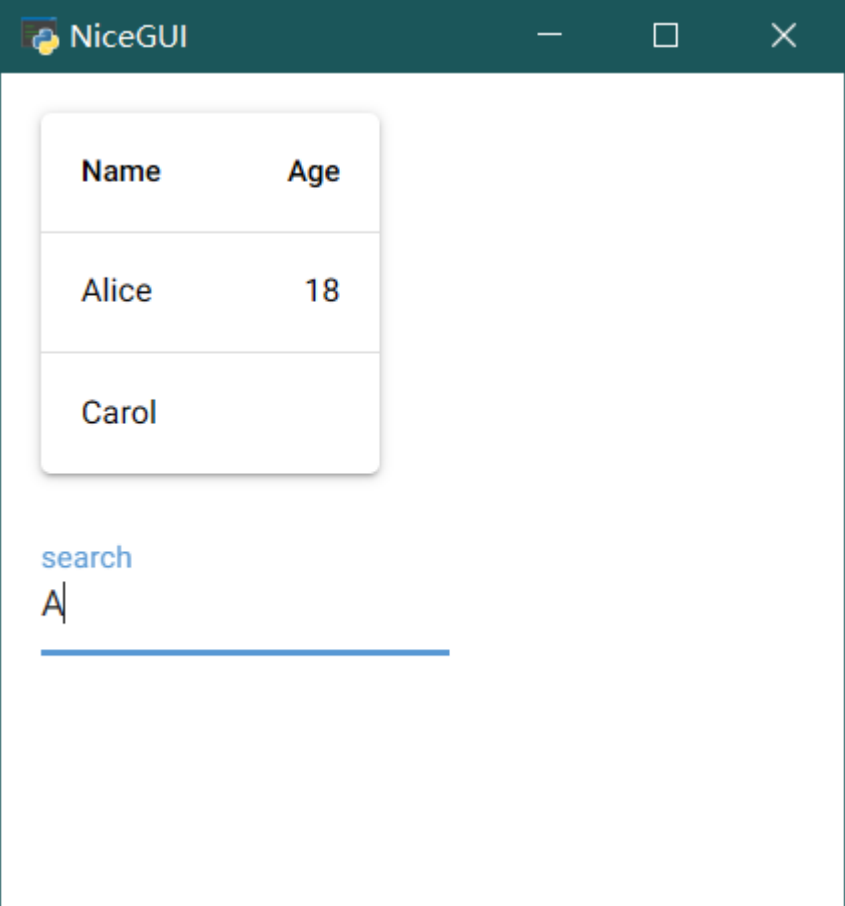
1  from nicegui import ui
2
3  columns = [
4      {'name': 'name', 'label': 'Name', 'field': 'name'},
5      {'name': 'age', 'label': 'Age', 'field': 'age'},
6  ]
7  rows = [
8      {'name': 'Alice', 'age': 18},
9      {'name': 'Bob', 'age': 21},
10     {'name': 'Carol', 'age': 42},
11 ]
12 table = ui.table(columns=columns, rows=rows, row_key='name')
13 table.add_slot('body-cell-age', '''
14     <q-td key="age" :props="props">
15         <q-badge :color="props.value < 21 ? 'red' : 'green'">
16             {% raw %}{{ props.value }}{% endraw %}
17         </q-badge>
18     </q-td>
19 ''')
20
21 ui.run(native=True)

```



如果表格的数据比较多，想要通过输入关键字来搜索、筛选数据，可以将输入框的值绑定到表格的 `filter` 属性：

```
1 from nicegui import ui
2
3 columns = [
4     {'name': 'name', 'label': 'Name', 'field': 'name', 'required': True, 'align':
5       'left'},
6     {'name': 'age', 'label': 'Age', 'field': 'age', 'sortable': True},
7 ]
8 rows = [
9     {'name': 'Alice', 'age': 18},
10    {'name': 'Bob', 'age': 21},
11    {'name': 'Carol'},
12 ]
13 table = ui.table(columns=columns, rows=rows, row_key='name')
14 ui.input(label='search').bind_value_to(table, 'filter')
15 ui.run(native=True)
```

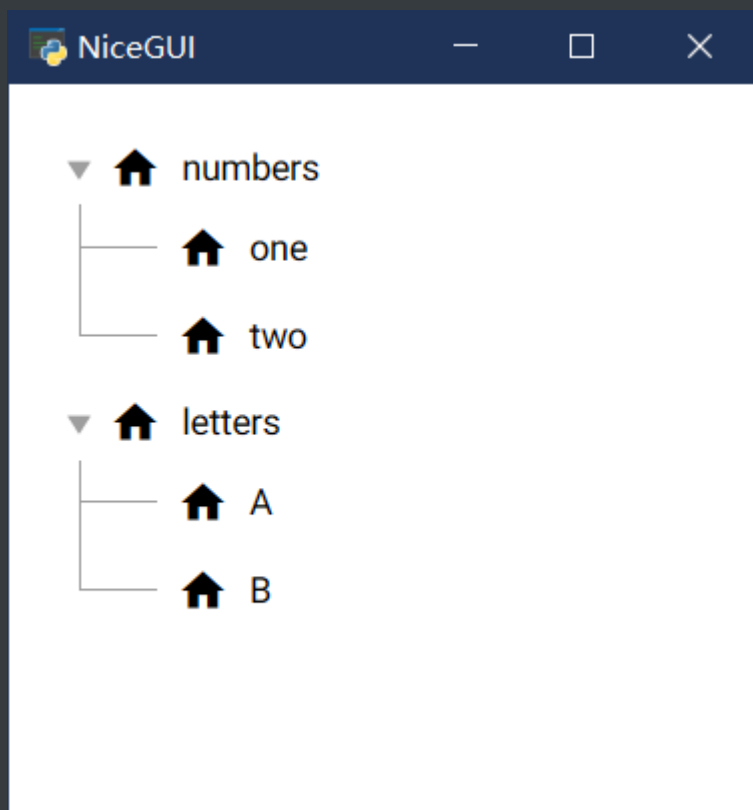


3.14.2 ui.tree

树形图控件，可以用树形图的方式展示给定的数据。

先看示例代码：

```
15         {
16             'id': '2',
17             'icon': 'home',
18             'label': 'two'
19         }
20     ]
21 },
22 {
23     'id': 'letters',
24     'label': 'letters',
25     'icon': 'home',
26     'children': [
27         {
28             'id': 'a',
29             'icon': 'home',
30             'label': 'A'
31         },
32         {
33             'id': 'b',
34             'icon': 'home',
35             'label': 'B'
36         }
37     ]
38 },
39 ],
40 node_key='id',
41 label_key='label',
42 children_key='children',
43 on_select=lambda e: ui.notify(f'{e.value} selected.'),
44 on_expand=lambda e: ui.notify(f'{e.value} expanded.'),
45 on_tick=lambda e: ui.notify(f'{e.value} ticked.'),
46 tick_strategy=None
47 )
48 tree.expand()
49 ui.run(native=True)
```

树形图控件支持以下参数：

`nodes` 参数，字典列表，树的节点层次数据源，通过构建特定结构的数据，可以让树形图按照指定结构显示。列表为根节点，每个列表元素表示一个子节点。在字典中，添加 `'children'` 键 (key) 并设置类似结构的列表为值 (value)，即可为该节点扩展出子节点。在数据源中，不同的键有不同的含义：

`'id'`：表示节点的编号，具有唯一性，不可重复。如果需要获取已经选择、勾选、展开的节点数据，这个键对应的值会用于标识节点，出现在返回的列表中。

`'label'`：表示节点显示的文字，如果没有对应的值，该节点不会显示文字。

`'children'`：表示节点的子节点，如果有对应的值，该节点会变成可以展开的样式。该键的对应值是字典列表，需要与传给 `nodes` 参数结构保持一致，才能确保子节点显示正常。

`'icon'`：表示节点的图标。

`node_key` 参数，字符串类型，表示 `nodes` 参数中用于确定节点唯一性的键，默认值是 `"id"`，可以根据实际情况修改为其他值。

`label_key` 参数，字符串类型，表示 `nodes` 参数中用于显示为节点文本的键，默认值是 `"label"`，可以根据实际情况修改为其他值。

`children_key` 参数，字符串类型，表示 `nodes` 参数中用于确定子节点的键，默认值是 "children"，可以根据实际情况修改为其他值。

`on_select` 参数，可调用类型，当节点的选择状态（点击节点即选择、取消选择该节点）时执行的操作。

`on_expand` 参数，可调用类型，当节点的展开状态变化（点击有子节点的节点前的三角形可以展开、收回该节点）时执行的操作。

`on_tick` 参数，可调用类型，当节点的展开状态变化（点击有节点前的复选框可以勾选、取消勾选该节点）时执行的操作。注意，只有下面的 `tick_strategy` 参数不为 `None` 时，才会显示复选框。

`tick_strategy` 参数，字符串类型，表示节点的复选框采用什么勾选策略，支持 "leaf"、"leaf-filtered"、"strict"，默认为 `None`，即没有勾选策略，不显示复选框。具体勾选策略的含义可以参考下表或者[官方文档](#)：

策略名	含义
"leaf"	如果勾选了叶子节点，会影响父节点的勾选状态（变成全选或者部分勾选）；如果勾选的是分支节点，会影响父节点的勾选状态（变成全选或者部分勾选）和子节点的勾选状态（可以被勾选子节点会被全选）。
"leaf-filtered"	和 leaf 的含义一致，只不过该策略只会影响被过滤之后依然可见的节点。也就是说，对父节点、子节点如果在过滤之后不可见，勾选状态不会改变。
"strict"	只影响被勾选的节点，不影响父节点或子节点的勾选状态。

更多设计属性参考[官方API](#)。

两个助手函数，可以帮助读者更好使用树形图控件，一个是确认给定的键（key）是不是树形图数据源的有效键（key），另一个是根据给定的键（key）获取当前节点、父节点、子节点：

```
1  #验证key列表是不是有效的key
2  #不指定node_keys则获取全部节点的key
3  #可以通过指定parent_only来只获取父节点（有子节点的）
4  def find_all_node_keys(tree, node_keys=None, parent_only=False):
5      NODES = tree._props['nodes']
6      CHILDREN_KEY = tree._props['children-key']
7      NODE_KEY = tree._props['node-key']
```

```

8     def iterate_nodes(nodes=NODES):
9         for node in nodes:
10             if parent_only:
11                 if (CHILDREN_KEY in node.keys()):
12                     yield node
13                     yield from iterate_nodes(node.get(CHILDREN_KEY, []))
14                 else:
15                     yield
16             else:
17                 yield node
18                 yield from iterate_nodes(node.get(CHILDREN_KEY, []))
19     finded_nodes = {node[NODE_KEY] for node in iterate_nodes(NODES) if node}
20     return finded_nodes if node_keys is None else {finded_node for finded_node in
    set(node_keys) if finded_node in finded_nodes}
21 #获取指定key的相关节点
22 def get_related_nodes(tree, target_key=None):
23     NODES = tree._props['nodes']
24     CHILDREN_KEY = tree._props['children-key']
25     NODE_KEY = tree._props['node-key']
26     result = {'current':None, 'parent':None, 'children':None}
27     def iterate_node(nodes=NODES, parent = None):
28         for node in nodes:
29             if target_key == node[NODE_KEY]:
30                 result['parent'] = parent
31                 result['current'] = node
32                 if (CHILDREN_KEY in node.keys()):
33                     result['children'] = node.get(CHILDREN_KEY, [])
34             else:
35                 if (CHILDREN_KEY in node.keys()):
36                     iterate_node(nodes=node.get(CHILDREN_KEY, []), parent=node)
37         return
38     iterate_node(NODES)
39     return result

```

如果想要选择、展开、勾选的方法和获取对应状态的节点，可以参照下表提供的方法：

状态	设置为该状态的方法	取消该转态的方法	获取该状态下的节点
选择	<code>.select(key)</code>	<code>.deselect()</code>	<code>.props['selected']</code>
展开	<code>.expand([key1,...keyn])</code> 不传入参数的话就是对所有节点生效	<code>.collapse([key1,...keyn])</code> 不传入参数的话就是对所有节点生效	<code>.props['expanded']</code>
勾选	<code>.tick([key1,...keyn])</code> 不传入参数的话就是对所有节点生效	<code>.untick([key1,...keyn])</code> 不传入参数的话就是对所有节点生效	<code>.props['ticked']</code>

```

1  from nicegui import ui
2
3  tree = ui.tree(
4      nodes=[
5          {
6              'id': 'numbers',
7              'label': 'numbers',
8              'icon': 'home',
9              'children': [
10                 {
11                     'id': '1',
12                     'icon': 'home',
13                     'label': 'one'
14                 },
15                 {
16                     'id': '2',
17                     'icon': 'home',
18                     'label': 'two'
19                 }
20             ]
21         },
22         {
23             'id': 'letters',
24             'label': 'letters',
25             'icon': 'home',

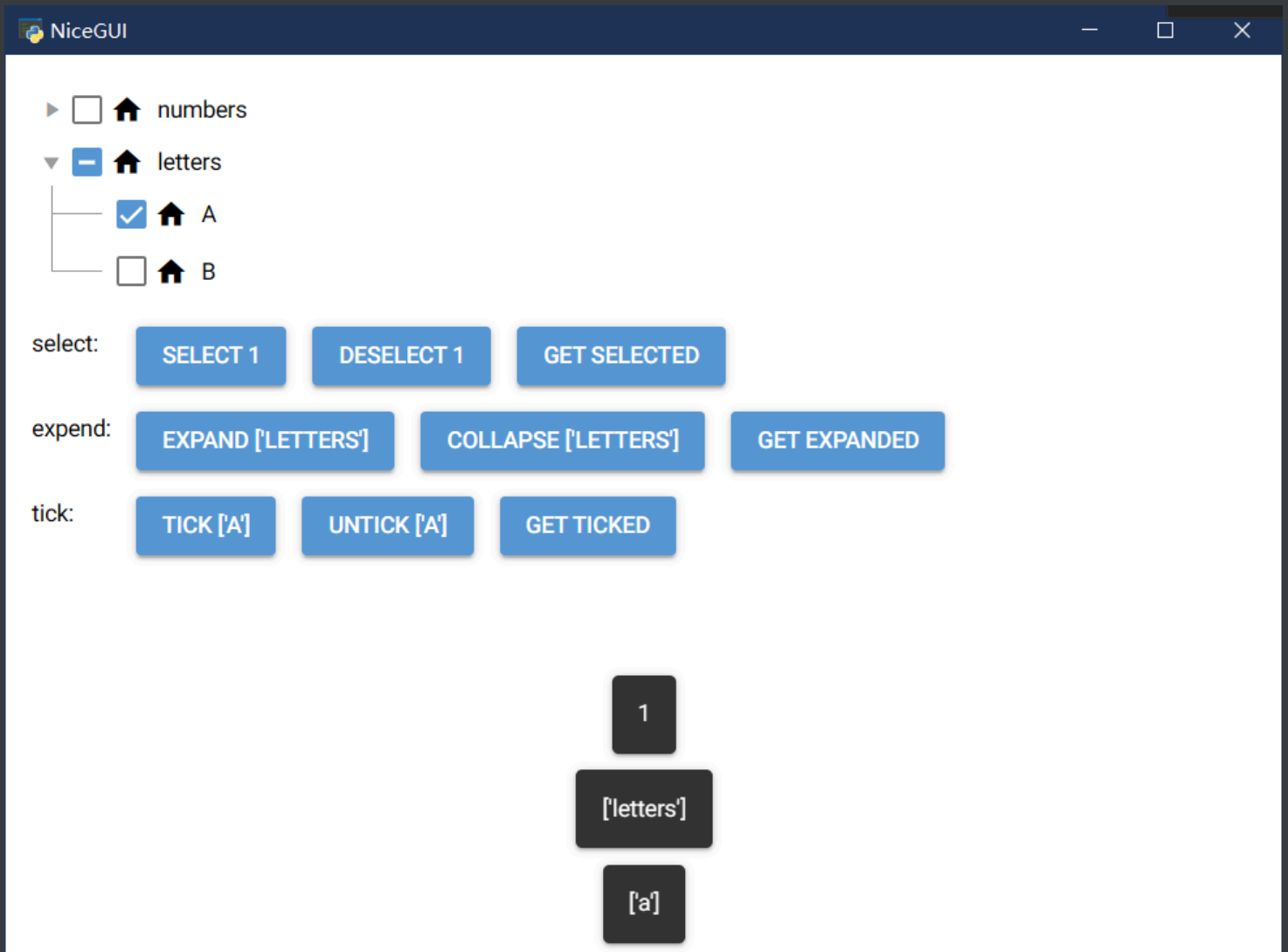
```

```
26         'children': [  
27             {  
28                 'id': 'a',  
29                 'icon': 'home',  
30                 'label': 'A'  
31             },  
32             {  
33                 'id': 'b',  
34                 'icon': 'home',  
35                 'label': 'B'  
36             }  
37         ]  
38     },  
39 ],  
40     node_key='id',  
41     label_key='label',  
42     children_key='children',  
43     tick_strategy='leaf'  
44 )  
45  
46 select='1'  
47 expand=['letters']  
48 tick=['a']  
49 with ui.column():  
50     with ui.row():  
51         ui.label('select:').classes('w-12')  
52         ui.button(f'select {select}',on_click=lambda :tree.select(select))  
53         ui.button(f'deselect {select}',on_click=lambda :tree.deselect())  
54         ui.button(f'get selected',on_click=lambda :ui.notify(tree.props['selected']))  
55     with ui.row():  
56         ui.label('expand:').classes('w-12')  
57         ui.button(f'expand {expand}',on_click=lambda :tree.expand(expand))  
58         ui.button(f'collapse {expand}',on_click=lambda :tree.collapse(expand))  
59         ui.button(f'get expanded',on_click=lambda :ui.notify(tree.props['expanded']))  
60     with ui.row():  
61         ui.label('tick:').classes('w-12')  
62         ui.button(f'tick {tick}',on_click=lambda :tree.tick(tick))  
63         ui.button(f'untick {tick}',on_click=lambda :tree.untick(tick))
```

```

64         ui.button(f'get {ticked}', on_click=lambda :ui.notify(tree.props['ticked']))
65
66     ui.run(native=True)

```

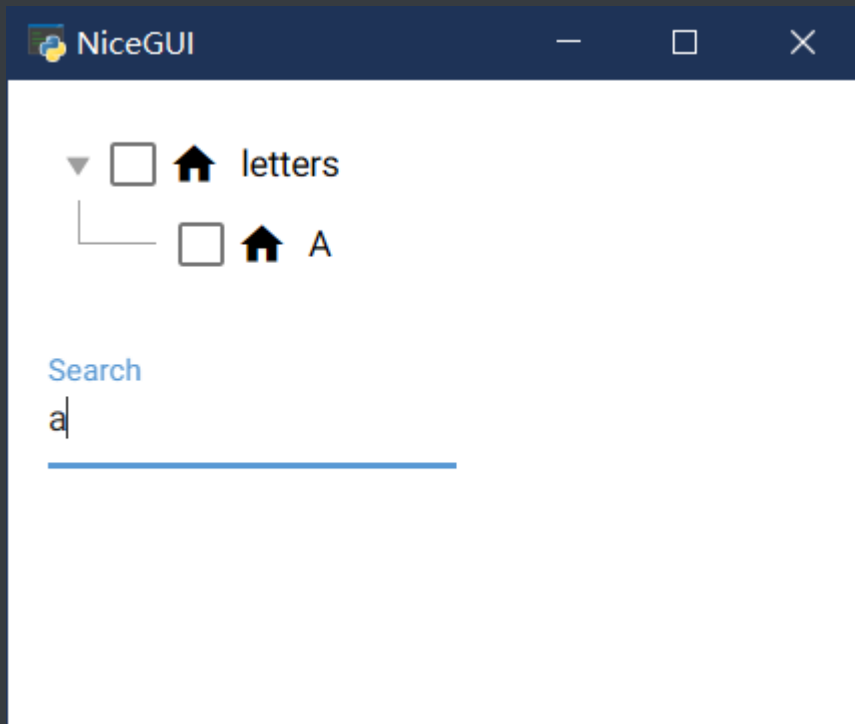


和 `ui.table` 类似，树形图也支持筛选：

```

1  from nicegui import ui
2
3  tree = ui.tree(
4      nodes=[
5          {
6              'id': 'numbers',
7              'label': 'numbers',
8              'icon': 'home',
9              'children': [
10                 {

```

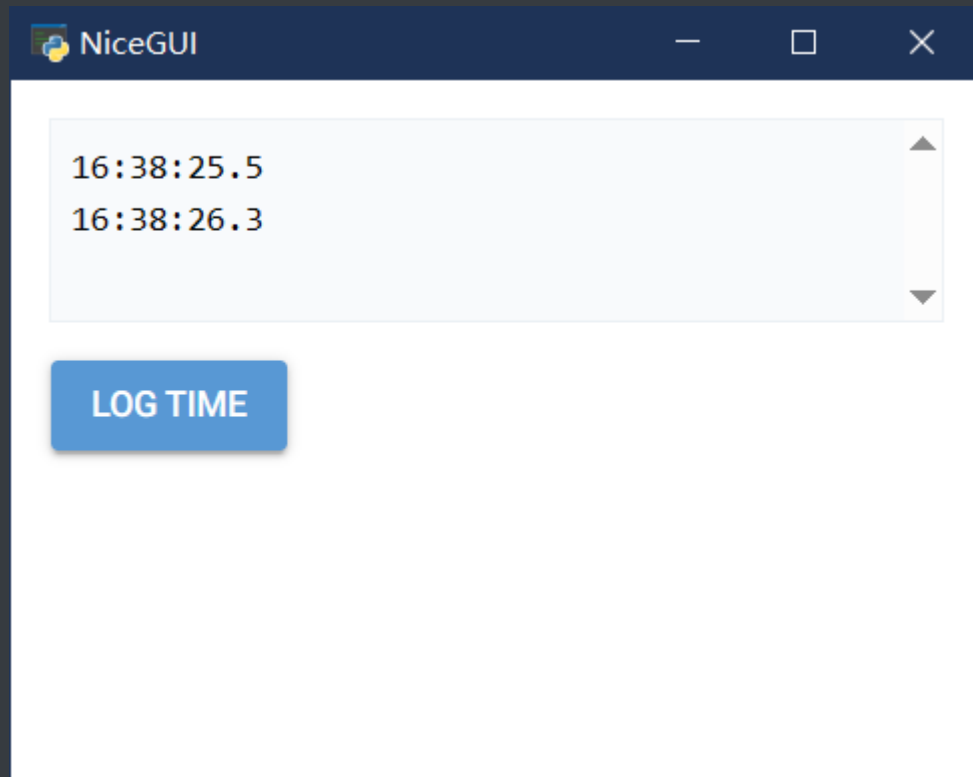
3.14.3 ui.log

日志控件，显示新推送的日志消息，并在保留历史日志消息，不需要完整传输历史消息到客户端。

控件只有一个整数参数 `max_lines`，代表日志支持的最大行数。

使用 `push` 方法将日志消息推送给控件。

```
1 from datetime import datetime
2 from nicegui import ui
3
4 log = ui.log(max_lines=10).classes('w-full h-20')
5 ui.button('Log time', on_click=lambda: log.push(datetime.now().strftime('%X.%f')
6           [:-5]))
7
8 ui.run(native=True)
```

以下代码展示了如何给日志记录模块增加推送到日志控件的操作：

```
1 import logging
2 from datetime import datetime
3 from nicegui import ui
4
5 logger = logging.getLogger()
6
7 class LogElementHandler(logging.Handler):
8     """A logging handler that emits messages to a log element."""
9
10     def __init__(self, element: ui.log, level: int = logging.NOTSET) -> None:
11         self.element = element
12         super().__init__(level)
13
14     def emit(self, record: logging.LogRecord) -> None:
15         try:
16             msg = self.format(record)
17             self.element.push(msg)
18         except Exception:
19             self.handleError(record)
20
```

```

21 log = ui.log(max_lines=10).classes('w-full')
22 handler = LogElementHandler(log)
23 logger.addHandler(handler)
24 ui.context.client.on_disconnect(lambda: logger.removeHandler(handler))
25 ui.button('Log time', on_click=lambda:
    logger.warning(datetime.now().strftime('%X.%f')[:-5]))
26
27 ui.run(native=True)

```

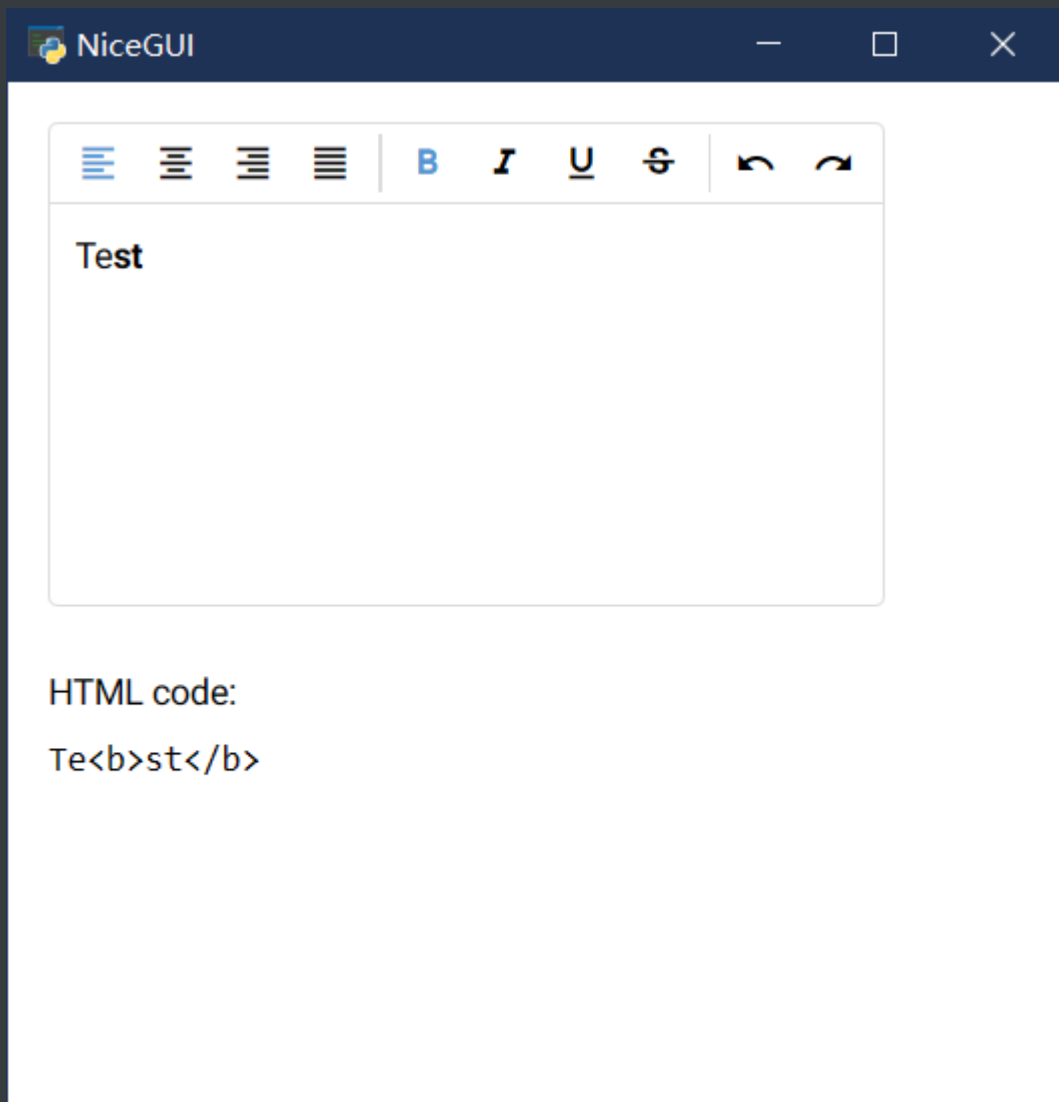
3.14.4 ui.editor

编辑器控件，可以提供一個簡易的文本編輯器。

```

1 from nicegui import ui
2
3 editor = ui.editor(placeholder='Type something here')
4 ui.markdown().bind_content_from(editor, 'value',
5                                 backward=lambda v: f'HTML code:\n```\n{v}\n```')
6
7 ui.run(native=True)

```



`placeholder` 参数，字符串类型，占位符，和输入框的一样。

`value` 参数，字符串类型，编辑器的初始文本。

`on_change` 参数，可调用类型，编辑器内文本变化时执行的操作。

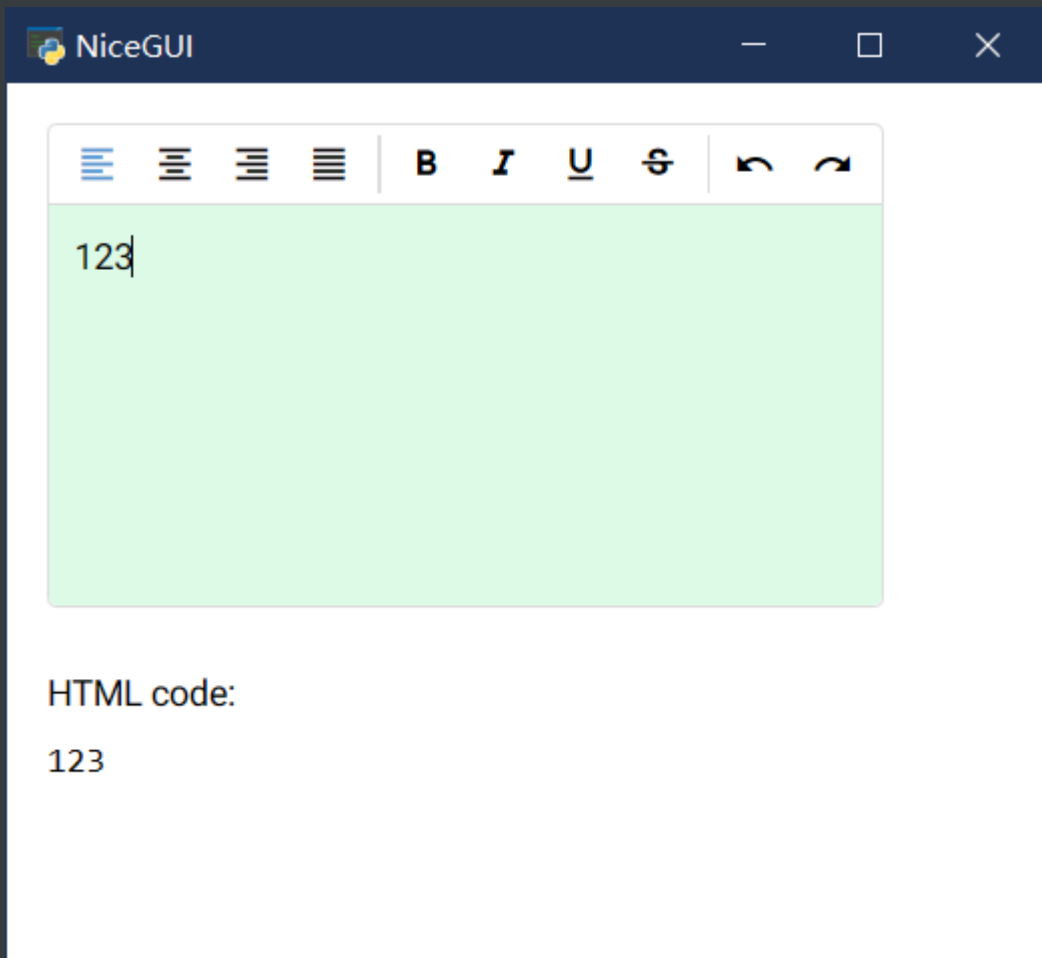
更多设计属性参考[官方API](#)。

修改 `content-class` 属性可以自定义内容区域的样式：

```

1 from nicegui import ui
2
3 editor = ui.editor(placeholder='Type something here').props('content-class="bg-green-100"')
4 ui.markdown().bind_content_from(editor, 'value',
5                                 backward=lambda v: f'HTML code:\n```\n{v}\n```')
6
7 ui.run(native=True)

```



修改 `toolbar` 属性可以实现自定义编辑器的快捷工具栏:

```

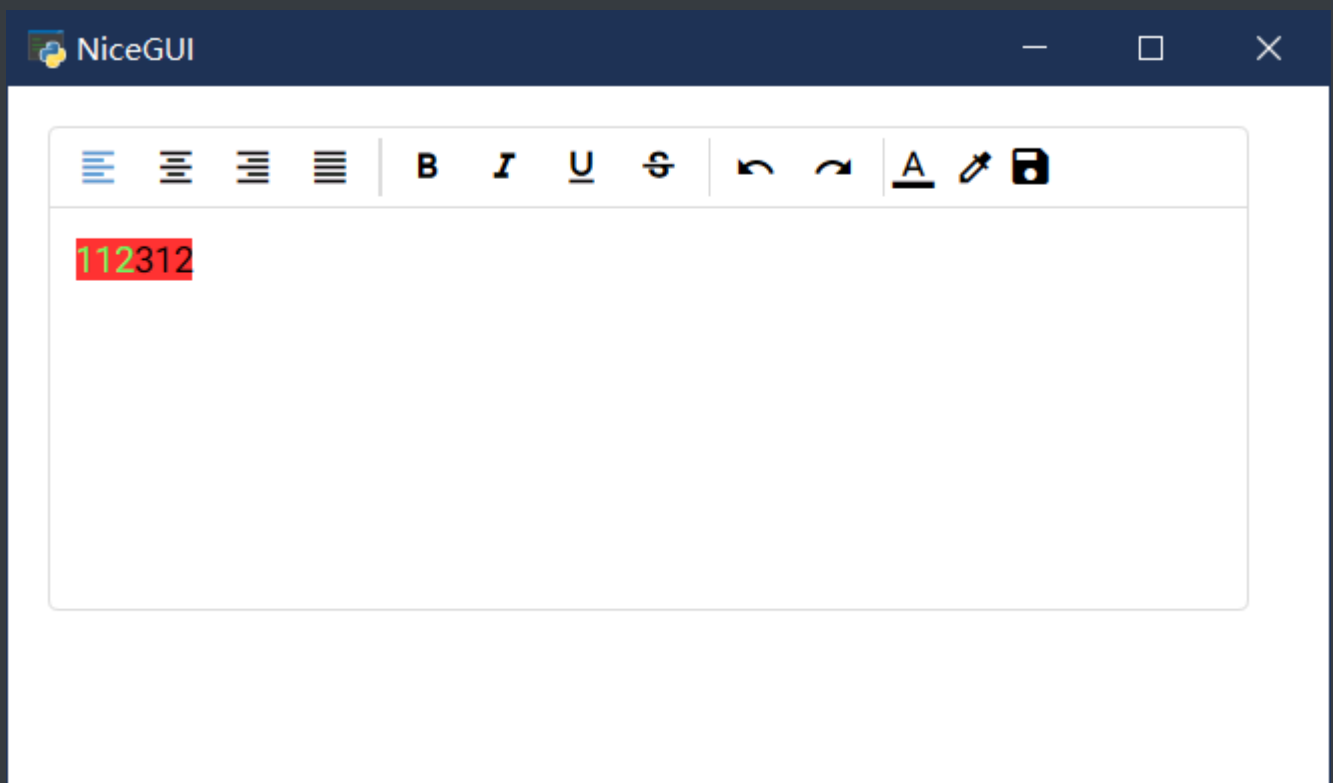
1 from nicegui import ui
2
3 with ui.editor().props('':toolbar="[['left','center','right','justify'],
4   ['bold','italic','underline','strike'], ['undo','redo'],
5   ['color','bgcolor','save']]").classes('w-[480px]') as editor:
6     with editor.add_slot("save"):

```

```

5         with ui.button(icon="save", on_click=lambda: ui.notify(f'{len(editor.value)}
chars saved!')).props("flat dense size=0.8em").classes("text-black p-0"):
6             ui.tooltip('Save text').props('delay=1200 transition-duration=300')
7         with editor.add_slot("color"):
8             with ui.icon('format_color_text', size='1.2em').classes("p-0") as icon:
9                 ui.tooltip('Change color').props(
10                     'delay=1200 transition-duration=300')
11                 picker = ui.color_picker(
12                     on_pick=lambda e: editor.run_method('runCmd', 'foreColor',
f'{e.color}'))
13                 ).props('no-focus no-refocus auto-close')
14                 picker.q_color.props('no-header no-footer default-view=palette')
15         with editor.add_slot("bgcolor"):
16             with ui.icon('colorize', size='1.2em').classes("text-black py-0 ps-2"):
17                 ui.tooltip('Change bgcolor').props(
18                     'delay=1200 transition-duration=300')
19                 picker = ui.color_picker(on_pick=lambda e: editor.run_method(
20                     'runCmd', 'backColor', f'{e.color}')).props('no-focus no-refocus
auto-close')
21                 picker.q_color.props('no-header no-footer default-view=palette')
22
23 ui.run(native=True)

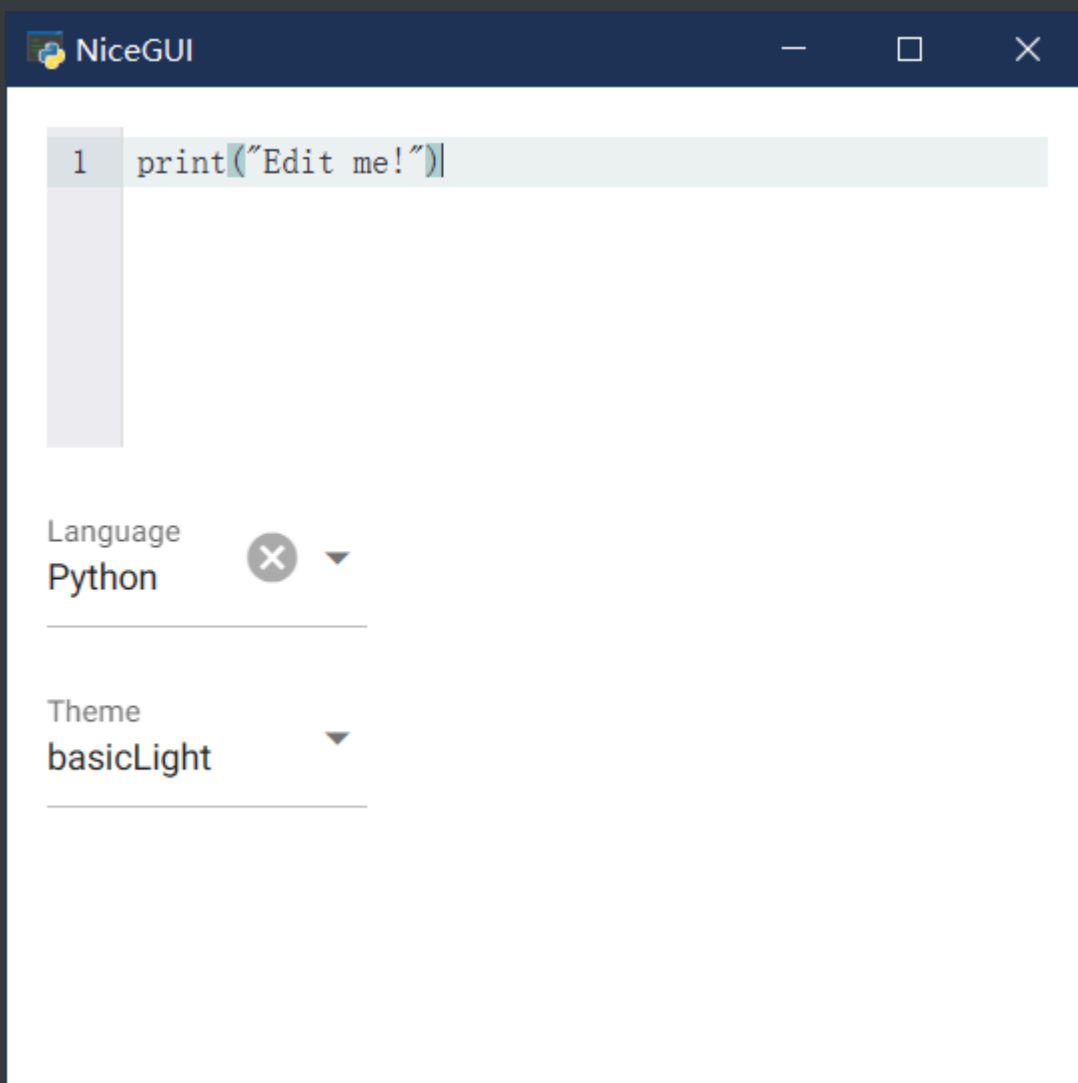
```



3.14.5 ui.codemirror

`ui.codemirror` 代码编辑器，功能实现源自[CodeMirror](#)，支持超过140种语言的语法高亮（支持列表[参考这里](#)）、超过30种界面主题（支持列表[参考这里](#)）、行号显示、代码折叠、有限的自动补全等等。

```
1 from nicegui import ui
2
3 editor = ui.codemirror('print("Edit me!")', language='Python').classes('h-32')
4 ui.select(editor.supported_languages, label='Language', clearable=True) \
5     .classes('w-32').bind_value(editor, 'language')
6 ui.select(editor.supported_themes, label='Theme') \
7     .classes('w-32').bind_value(editor, 'theme')
8
9 ui.run(native=True)
```



控件支持以下参数：

`value` 参数，字符串类型，编辑器的初始值。

`on_change` 参数，可调用类型，编辑器的值发生变化时执行的操作。

`language` 参数，字符串类型，编辑器的语法高亮方案，默认为 `None`。注意，此参数大小写敏感，如果不确定语法的准确名称，可以查询该语言字符串是否在 `ui.codemirror.supported_languages` 内。

`theme` 参数，字符串类型，编辑器的主题，默认为 `"basicLight"`。如果不确定主题的准确名称，可以查询 `ui.codemirror.supported_themes`。

`indent` 参数，字符串类型，代码缩进使用的字符，默认为4个英文空格。

`line_wrapping` 参数，布尔类型，表示是否自动换行，默认为 `False`。

`highlight_whitespace` 参数，布尔类型，表示是否高亮显示出空白字符，默认为 `False`。

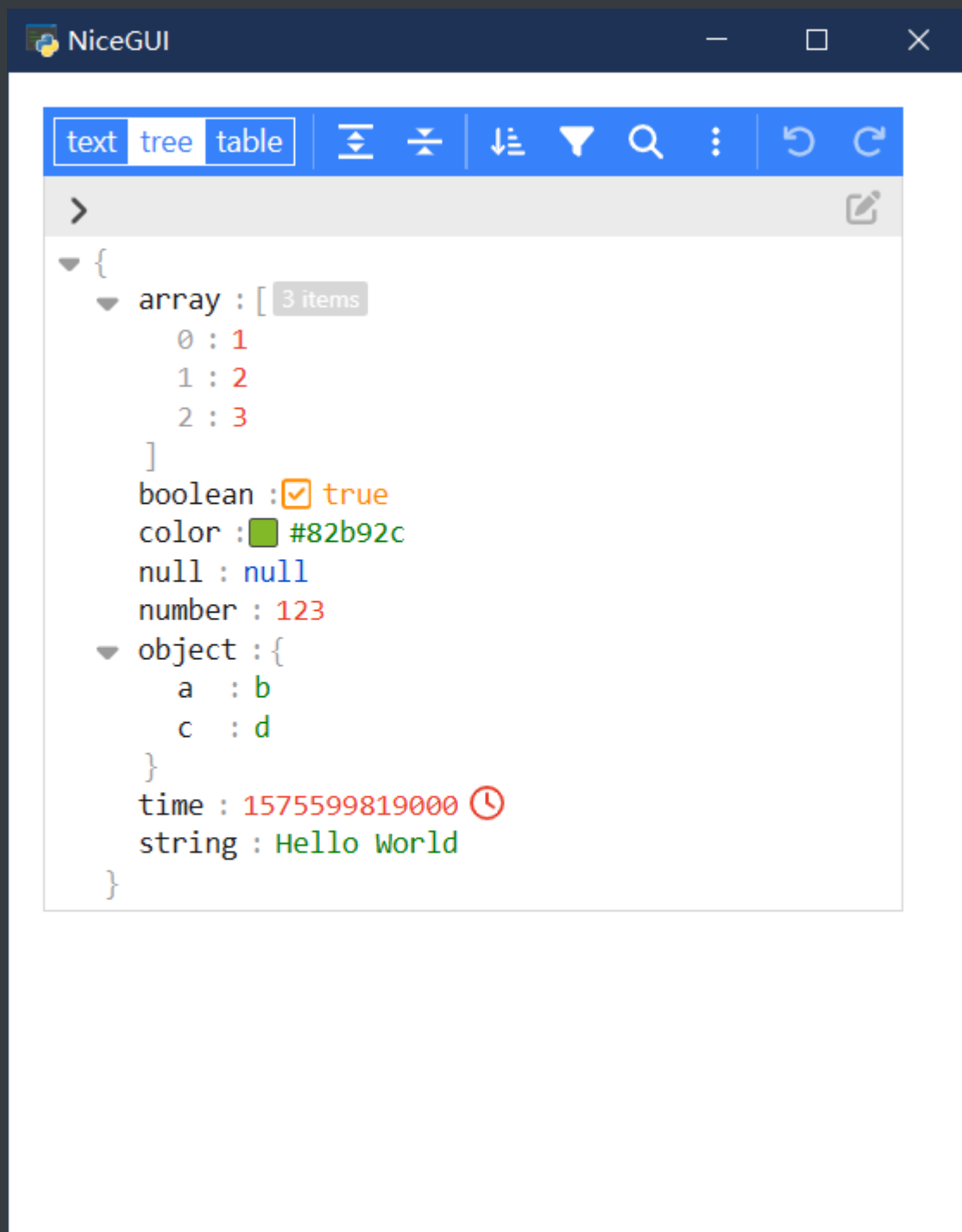
3.14.6 ui.json_editor

JSON编辑器，功能实现源自[svelte-jsoneditor](#)。

JSON（JavaScript Object Notation，JavaScript对象表示法）是基于ECMAScript的一个子集设计的，是一种开放标准的文件格式和数据交换格式，它易于人阅读和编写，同时也易于机器解析和生成。JSON独立于语言设计，很多编程语言都支持JSON格式的数据交换。JSON是一种常用的数据格式，在电子数据交换中有多种用途，包括与服务器之间的Web应用程序的数据交换。其简洁和清晰的层次结构有效地提升了网络传输效率，使其成为理想的数据交换语言。其文件通常使用扩展名 `.json`。

控件示例如下：

```
1 from nicegui import ui
2
3 json = {
4     'array': [1, 2, 3],
5     'boolean': True,
6     'color': '#82b92c',
7     None: None,
8     'number': 123,
9     'object': {
10         'a': 'b',
```

控件支持以下参数：

`properties` 参数，字典类型，传入控件的复合属性字典，其键（key）是JSON编辑器支持的属性（参考[官方API](#)）。以代码为例介绍几个常用属性：

`'content'`：表示内容主体（[官方文档](#)），其值同样是字典。字典包含 `'json'` 和 `'text'` 两个键，分别代表内容的JSON格式和TEXT格式。

`mode`：字符串类型，表示编辑器默认的编辑模式（[官方文档](#)），支持 `'tree'`、`'text'`、`'table'`，默认为 `'tree'`。

`mainMenuBar`：布尔类型，表示是否显示编辑器的主菜单栏（[官方文档](#)），默认为 `True`。

`navigationBar`：布尔类型，表示是否显示编辑器的导航栏（[官方文档](#)），默认为 `True`。

`statusBar`：布尔类型，表示是否显示编辑器文本模式的状态栏（[官方文档](#)），默认为 `True`。

`readOnly`：布尔类型，表示是否启用编辑器内容的只读模式（[官方文档](#)），默认为 `True`。

更多属性和用途参考[官方API](#)。

`on_select` 参数，可调用类型，当控件的内容被选择时执行的操作。

`on_change` 参数，可调用类型，当控件的内容被改变时执行的操作。

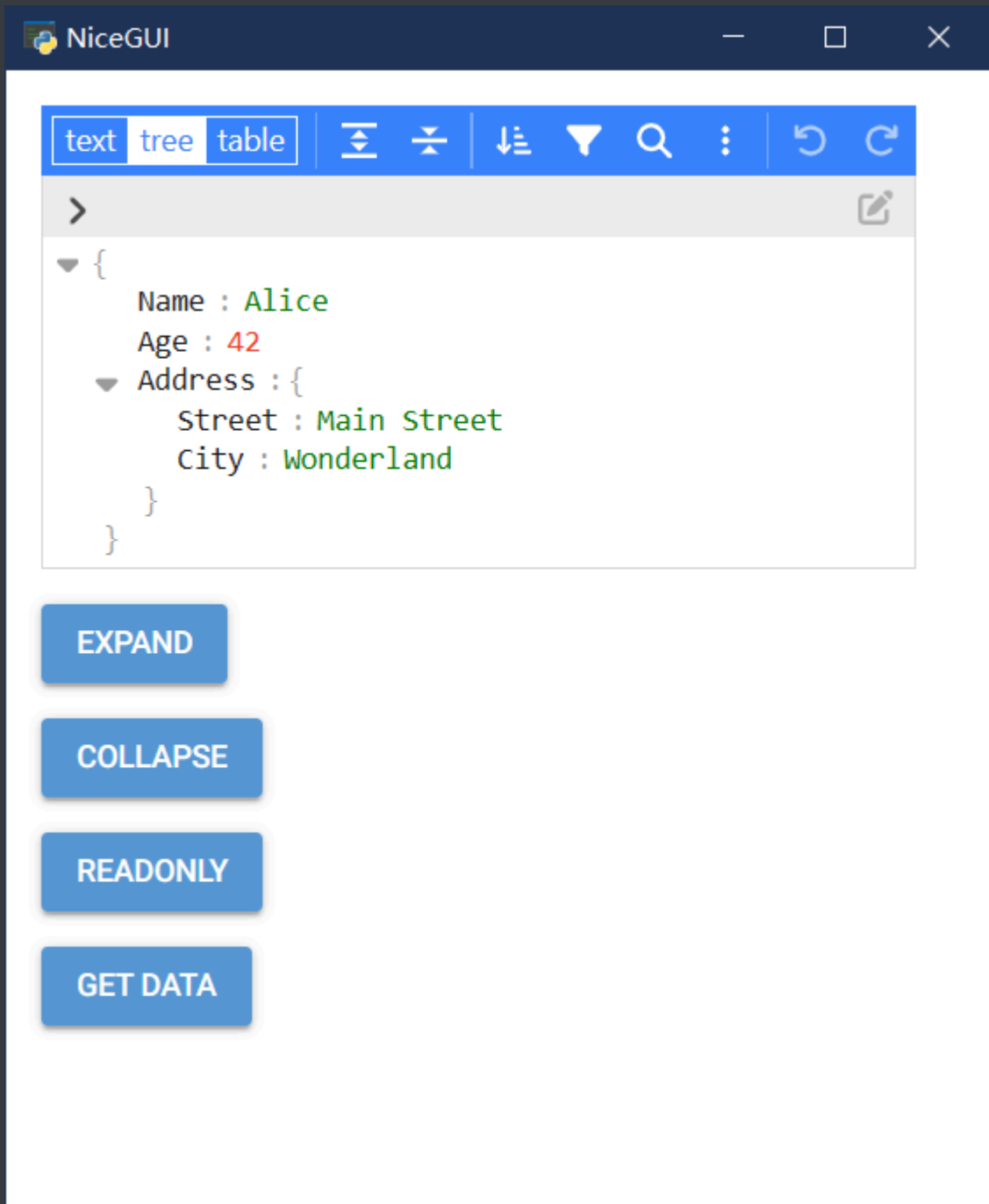
注意，除了初始化指定 `properties` 参数外，也可以修改 `properties` 属性来进而修改控件，但是，在修改属性之后，需要调用 `update` 方法才能触发界面更新，否则会导致界面显示与实际数据不一致。

控件对象的 `run_editor_method` 方法可以运行JSON编辑器支持的方法（[参考官方API](#)）。示例代码展示了如何展开、收起节点，如何获取数据。

方法名 `"expand"` 前的 `":"` 表示 `"path => true"` 是一个JavaScript表达式，该表达式会先在会在客户端计算之后，再传递给方法。

注意，从客户端获取数据（包括其他一切与客户端相关的操作），只能在 `ui.page` 页面内执行，不能在 `auto-index` 页面内执行。

```
1  from nicegui import ui
2
3  @ui.page('/')
4  def page():
5      json = {
6          'Name': 'Alice',
7          'Age': 42,
8          'Address': {
9              'Street': 'Main Street',
10             'City': 'Wonderland',
11         },
12     }
```

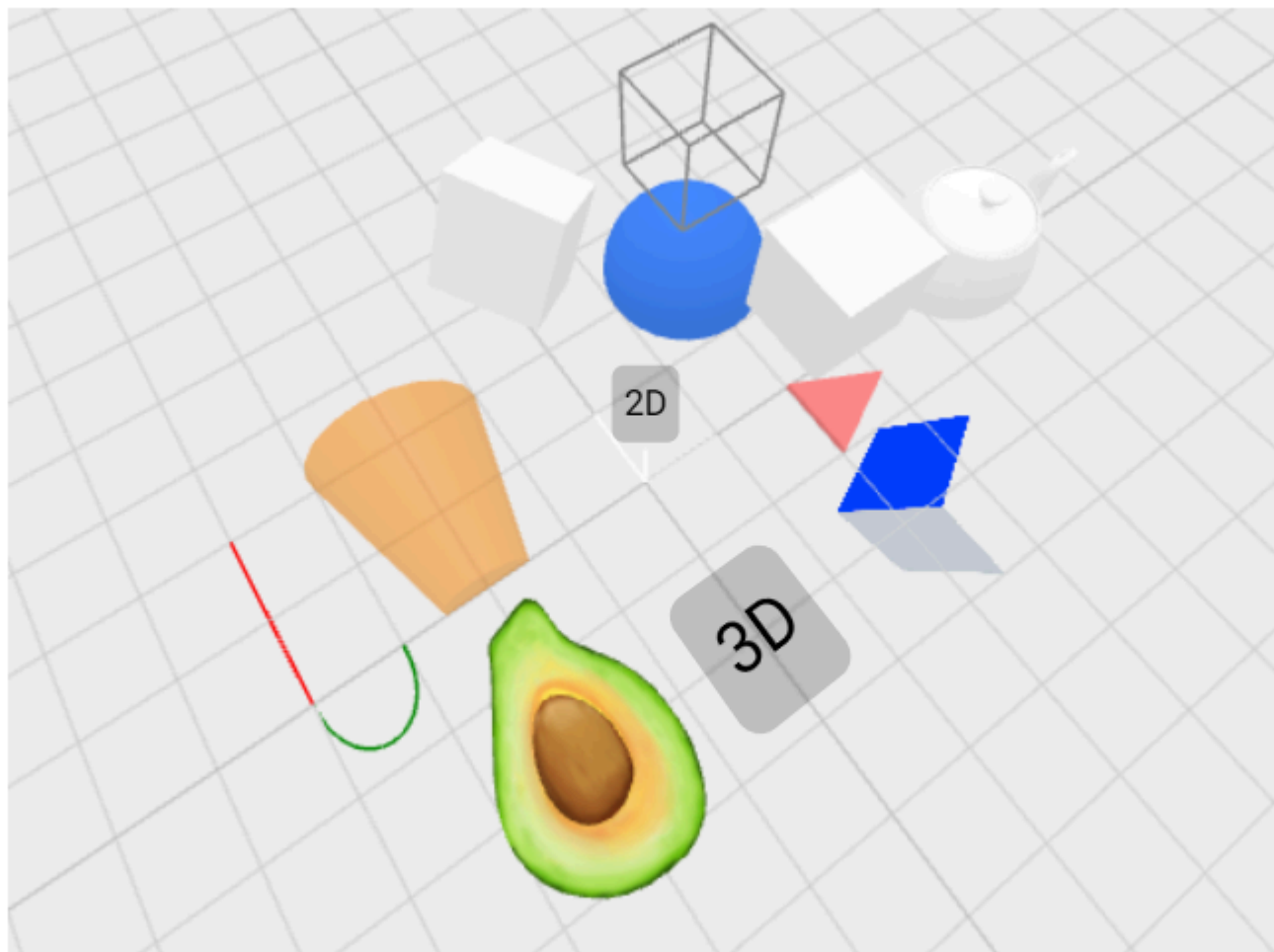
3.14.7 ui.scene

场景控件可用于显示3D内容，功能实现源于[threejs](#)。

示例代码中需要的图片文件和模型文件请自行搜索下载，放到代码文件的同目录下，并根据实际的文件名修改代码：

```
1 from nicegui import ui,app
2 import os
3
```

```
4 app.add_static_file(local_file=f'{os.path.dirname(os.path.abspath(__file__))}/LOGO.png',url_path='/logo')
5 app.add_static_file(local_file=f'{os.path.dirname(os.path.abspath(__file__))}/teapot.stl',url_path='/stl')
6 app.add_static_file(local_file=f'{os.path.dirname(os.path.abspath(__file__))}/Avocado.glb',url_path='/glp')
7
8 with ui.scene().classes('w-full h-96') as scene:
9     scene.axes_helper()
10     scene.sphere().material('#4488ff').move(2, 2)
11     scene.cylinder(1, 0.5, 2, 20).material('#ff8800', opacity=0.5).move(-2, 1)
12     scene.extrusion([[0, 0], [0, 1], [1, 0.5]], 0.1).material('#ff8888').move(2, -1)
13
14     with scene.group().move(z=2):
15         scene.box().move(x=2)
16         scene.box().move(y=2).rotate(0.25, 0.5, 0.75)
17         scene.box(wireframe=True).material('#888888').move(x=2, y=2)
18
19     scene.line([-4, 0, 0], [-4, 2, 0]).material('#ff0000')
20     scene.curve([-4, 0, 0], [-4, -1, 0], [-3, -1, 0], [-3, 0, 0]).material('#008800')
21
22     logo = '/logo'
23     scene.texture(logo, [[[0.5, 2, 0], [2.5, 2, 0]],
24                         [[0.5, 0, 0], [2.5, 0, 0]]]).move(1, -3)
25
26     teapot = '/stl'
27     scene.stl(teapot).scale(1).move(-9, -9)
28
29     avocado = '/glp'
30     scene.glTF(avocado).scale(40).move(-2, -3, 0.5)
31
32     scene.text('2D', 'background: rgba(0, 0, 0, 0.2); border-radius: 5px; padding: 5px').move(z=2)
33     scene.text3d('3D', 'background: rgba(0, 0, 0, 0.2); border-radius: 5px; padding: 5px').move(y=-2).scale(.05)
34
35 ui.run(native=True)
```



控件支持以下参数：

`width` 参数，整数类型，控件的宽度，默认为 400，单位是像素。在没有缩放控件的情况下，此参数指定了控件的固定宽度。如果使用CSS样式缩放控件宽度，此参数指定的宽度优先级最低。

`height` 参数，整数类型，控件的高度，默认为 300，单位是像素。在没有缩放控件的情况下，此参数指定了控件的固定高度。如果使用CSS样式缩放控件高度，此参数指定的高度优先级最低。

`grid` 参数，布尔类型或者整数元组类型，表示是否显示辅助网格或者网格的大小。如果为 `False`，则不显示辅助网格（显示在地平面的网格，用于确定物体的位置和大小）。如果为 `True`，则显示辅助网格。也可以指定一个整数元组来指定尺寸。元组的第一元素表示网格的大小（`size`），网格为正方形，只需指定边长即可。第二元素表示正方形的边划分为几等分（`divisions`），这样就可以得到‘等分数的平方’个正方形。网格大小的默认值是 `(100,100)`。

`camera` 参数, 指定场景使用的相机, 该参数需要传入 `ui.scene.perspective_camera` (透视相机) 或 `ui.scene.orthographic_camera` (正交相机) 的实例, 默认为透视相机。两种相机的参数不同:

`ui.scene.perspective_camera` (官方文档) :

`fov` 参数, 浮点类型, 表示视野角 (以度为单位) , 默认为 75 。

`near` 参数, 浮点类型, 表示近裁剪平面距离, 默认值为 0.1 。

`far` 参数, 浮点类型, 表示远裁剪平面距离, 默认值为 1000 。

`ui.scene.orthographic_camera` (官方文档) :

`size` 参数, 浮点类型, 表示视野角, 默认为10。此参数与视野角的关系是这样的, 官方文档中定义正交相机需要的左、右、顶、底四个视锥平面参数, NiceGUI基于此参数, 将 $(-size / 2) * \text{控件的宽高比}$ 、 $(size / 2) * \text{控件的宽高比}$ 、 $size / 2$ 、 $-size / 2$ 传递给对应参数。

`near` 参数, 浮点类型, 表示近裁剪平面距离, 默认值为 0.1 。

`far` 参数, 浮点类型, 表示远裁剪平面距离, 默认值为 1000 。

`on_click` 参数, 可调用类型, 点击3D对象时执行的操作。注意, 如果想要响应对应的点击事件, 需要在 `click_events` 参数中添加对应事件的订阅。

`click_events` 参数, 字符串列表类型, 控件订阅的JavaScript事件, 默认为 `['click', 'dblclick']` 。

`on_drag_start` 参数, 可调用类型, 开始拖动3D对象时执行的操作。

`on_drag_end` 参数, 可调用类型, 停止拖动3D对象时执行的操作。

`drag_constraints` 参数, 字符串类型, 表示用于限制被拖动的3D对象位置的JavaScript表达式, 比如: `'x = 0, z = y / 2'` 。

`background_color` 参数, 字符串类型, 场景的背景颜色, 默认为 `"#eee"` 。

如果想要想要点击事件，可以传递一个函数（或者lambda表达式这样的可调用对象）给场景控件的 `on_click` 参数。可以传递一个 `SceneClickEventArguments` 响应对象作为可调用对象的参数，该对象有以下属性：

- `click_type`：此次响应的点击的类型，`"click"`（点击）或者`"dblclick"`（双击）。
- `button`：鼠标的那个按键被按下，通常是 0 - 4 的整数数字（数字含义可以参考文档），0 表示鼠标主要按键（通常是左键），1 表示鼠标辅助按键（通常是中键），2 表示鼠标次要按键（通常是右键），3 表示鼠标第四个按键（如果有的话，一般是浏览器前进功能按键），4 表示鼠标第五个按键（如果有的话，一般是浏览器后退功能按键）。注意，如果只是订阅 `'click'` 或者 `'dblclick'` 事件，没法获取到鼠标主要按键之外的其他按键响应，只有订阅 `'mousedown'` 或者 `'mouseup'` 事件才能获取到其他鼠标按键的响应。
- `alt`：Alt 键（Mac下的 Opt 键）是否被按下。
- `ctrl`：Ctrl 键是否被按下。
- `meta`：Meta 键（Win的 Win 键或者Mac下的 Cmd 键）是否被按下。
- `shift`：Shift 键是否被按下。
- `hits`：表示被点击的结果。因为控件是二维的，空间是三维的，点击操作会覆盖到不止一个对象，所以结果返回的是一个包含多个 `SceneClickEventHit` 类型的被点击对象的列表，根据到相机的距离由近到远排序。可以取索引值为 0 的对象表示第一个被点击的对象。注意，天空不是可以点击的对象。

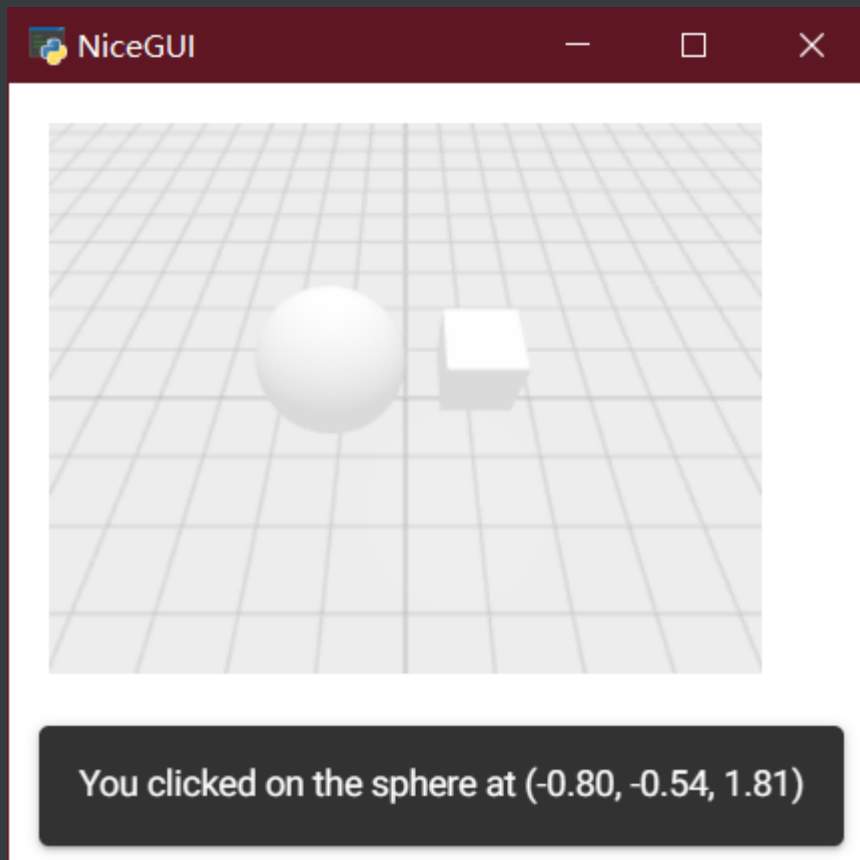
对于 `SceneClickEventHit` 对象，该对象有以下属性：

- `object_id`：被点击对象的id。如果该对象是NiceGUI创建且没有指定id（给对象的 `id` 属性赋值即可指定id），id则是自动生成的随机id。
- `object_name`：被点击对象的name。如果该对象是NiceGUI创建且没有指定name（给对象的 `name` 属性赋值或使用 `with_name` 方法即可指定name），name则为 `None`。
- `x`、`y`、`z`：鼠标点击位置的x、y、z坐标。


```

1 from nicegui import ui, events
2
3 def handle_click(e: events.SceneClickEventArguments):
4     hit = e.hits[0] if e.hits else None
5     name = (hit.object_name or hit.object_id) if hit else 'sky'
6     ui.notify(f'You clicked on the {name} at ({hit.x:.2f}, {hit.y:.2f}, {hit.z:.2f})'
7             if hit else f'You clicked on the {name}')
8
9 with ui.scene(width=285, height=220, on_click=handle_click) as scene:
10     scene.sphere().move(x=-1, z=1).with_name('sphere')
11     scene.box().move(x=1, z=1).with_name('box')
12
13 ui.run(native=True)

```



如果想要在物体上呼出右键菜单，需要订阅或者添加订阅 'contextmenu' 事件。注意，因为右键菜单创建之后属于场景控件，同一个控件只能有一个右键菜单，且菜单不支持动态刷新，如果想要实现右键菜单根据被点击对象显示不同内容，建议清除之前的菜单内容重新添加内容。代码如下：

```

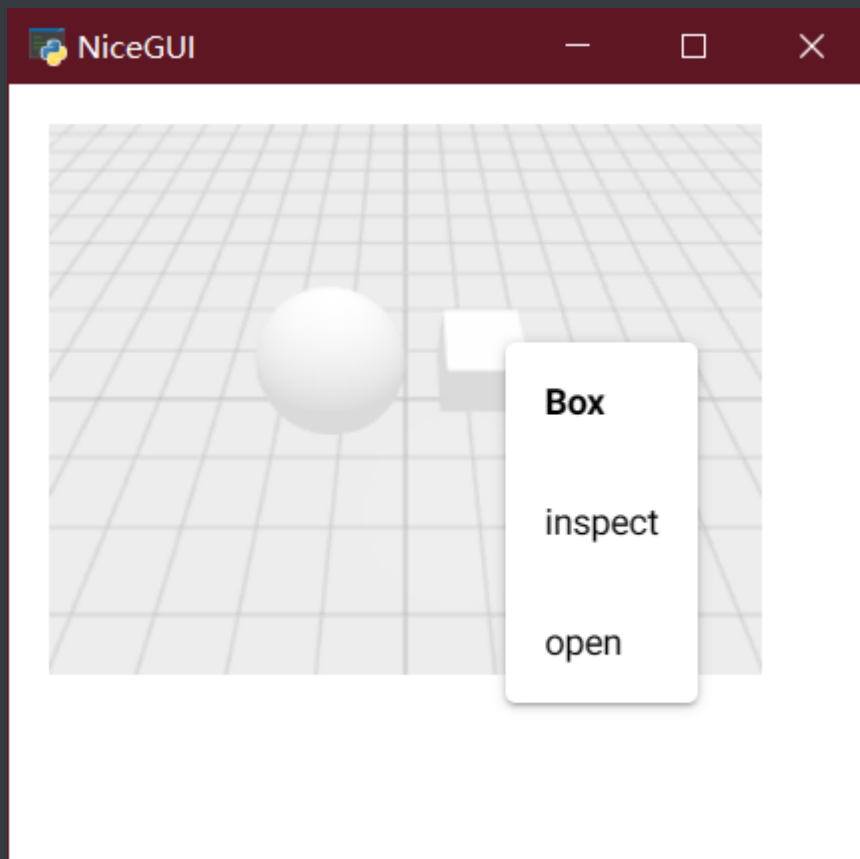
1 from nicegui import events, ui
2

```

```

3 def handle_click(e: events.SceneClickEventArguments) -> None:
4     context_menu.clear()
5     name = next((hit.object_name for hit in e.hits if hit.object_name), None)
6     with context_menu:
7         ui.item(name.capitalize()).classes('font-bold')
8         ui.menu_item('inspect')
9         ui.menu_item('open')
10
11 with ui.element():
12     context_menu = ui.context_menu()
13     with ui.scene(width=285, height=220, on_click=handle_click,
14                  click_events=['contextmenu']) as scene:
15         scene.sphere().move(x=-1, z=1).with_name('sphere')
16         scene.box().move(x=1, z=1).with_name('box')
17
18 ui.run(native=True)

```

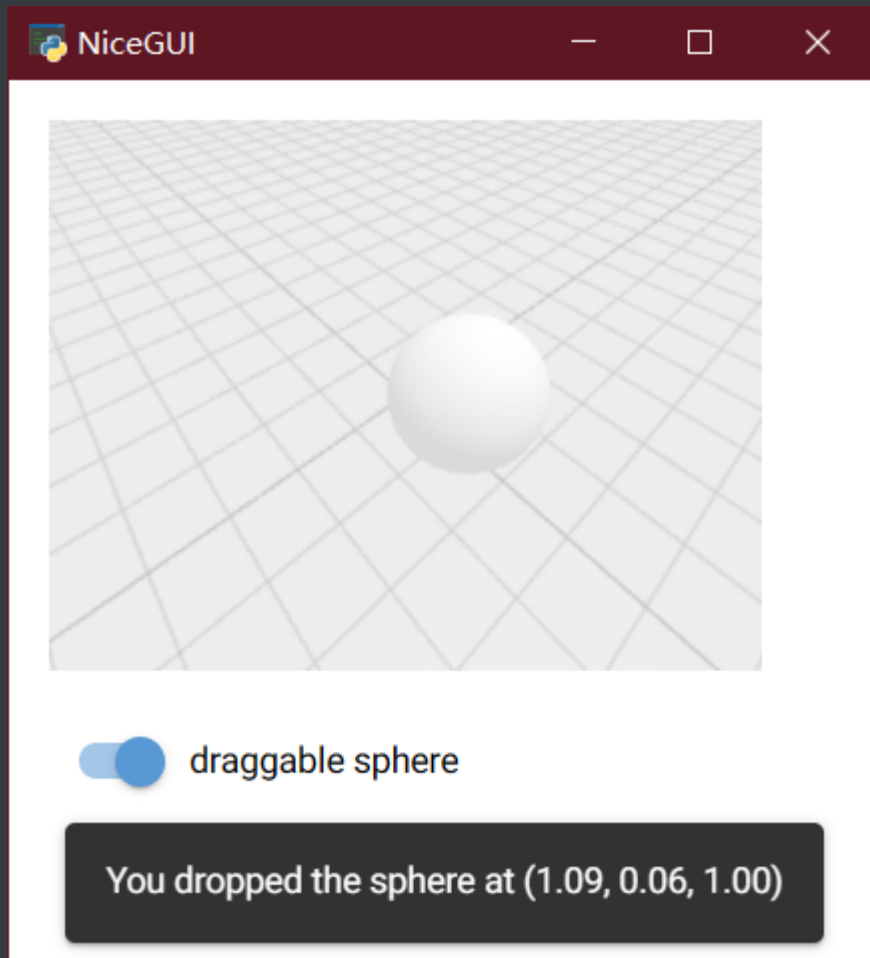


使用3D物体的 `draggable` 方法可以让对象变成可拖动对象。可以传递一个函数（或者lambda表达式这样的可调用对象）给场景控件的 `on_drag_start` 参数和 `on_drag_end` 参数，来响应可拖动对象的拖动事件。同时，还可以传递一个 `SceneDragEventArguments` 响应对象作为可调用对象的参数，该对象有以下属性：

- `type`：此次响应的拖动事件的类型，`"dragstart"`（拖动开始）或者 `"dragend"`（拖动结束）。
- `object_id`：被拖动对象的id。如果该对象是NiceGUI创建且没有指定id（给对象的 `id` 属性赋值即可指定id），`id`则是自动生成的随机id。
- `object_name`：被拖动对象的name。如果该对象是NiceGUI创建且没有指定name（给对象的 `name` 属性赋值或使用 `with_name` 方法即可指定name），`name`则为 `None`。
- `x`、`y`、`z`：被拖动对象的当前位置的x、y、z坐标。

此外，还可以设置场景控件的 `drag_constraints` 参数，来限制可拖动对象的运动规则。代码如下：

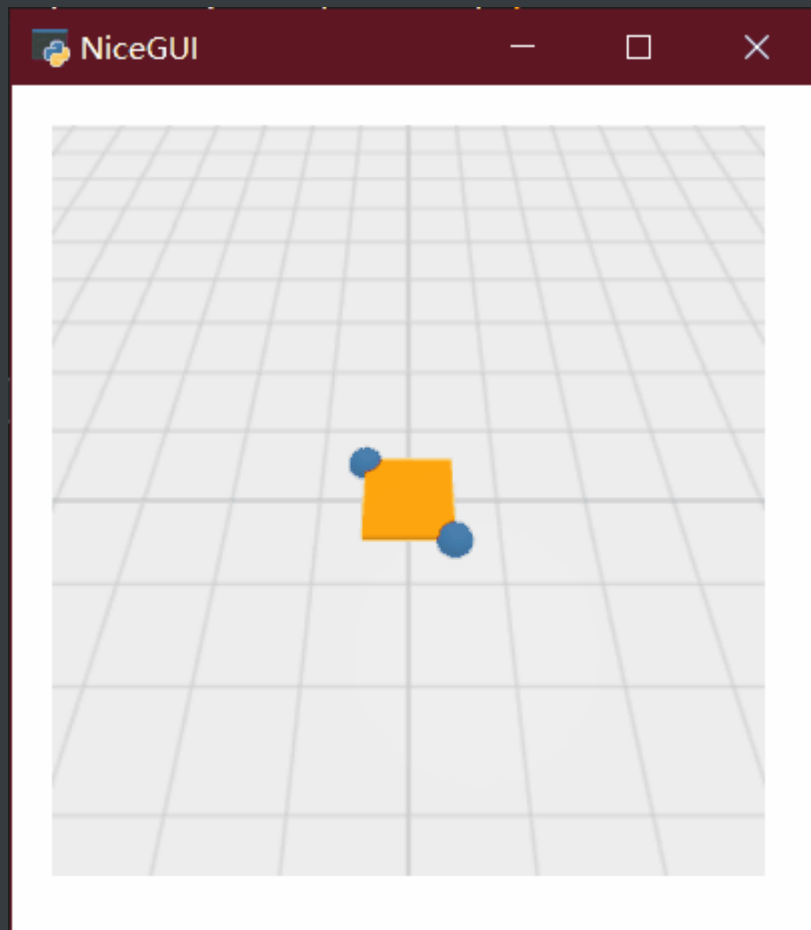
```
1 from nicegui import events, ui
2
3 def handle_drag(e: events.SceneDragEventArguments):
4     ui.notify(f'You dropped the sphere at ({e.x:.2f}, {e.y:.2f}, {e.z:.2f})')
5
6 with ui.scene(width=285, height=220,
7             drag_constraints='z = 1', on_drag_end=handle_drag) as scene:
8     sphere = scene.sphere().move(z=1).draggable()
9
10 ui.switch('draggable sphere',
11         value=sphere.draggable_,
12         on_change=lambda e: sphere.draggable(e.value))
13
14 ui.run(native=True)
```



一般来说，为了避免性能问题，拖动物体只会在拖动结束时返回物体的位置等属性。但是，可以使用 `on` 方法来显式订阅 "drag" 事件，以便实时获取物体位置。比如，在下面的代码，长方体的对角位置可以根据两个小球的位置实时刷新：

```
1 from nicegui import events, ui
2
3 with ui.scene(width=285, drag_constraints='z=0') as scene:
4     box = scene.box(1, 1, 0.2).move(0, 0).material('Orange')
5     sphere1 = scene.sphere(0.2).move(0.5, -0.5).material('SteelBlue').draggable()
6     sphere2 = scene.sphere(0.2).move(-0.5, 0.5).material('SteelBlue').draggable()
7
8 def handle_drag(e: events.GenericEventArguments) -> None:
9     x1 = sphere1.x if e.args['object_id'] == sphere2.id else e.args['x']
10    y1 = sphere1.y if e.args['object_id'] == sphere2.id else e.args['y']
11    x2 = sphere2.x if e.args['object_id'] == sphere1.id else e.args['x']
12    y2 = sphere2.y if e.args['object_id'] == sphere1.id else e.args['y']
13    box.move((x1 + x2) / 2, (y1 + y2) / 2).scale(x2 - x1, y2 - y1, 1)
14 scene.on('drag', handle_drag)
```

```
15
16 ui.run(native=True)
```



可以使用 `point_cloud` 对象来渲染点云（具体参见[官方文档](#)，但NiceGUI的构造方法与官方有差异）。

`point_cloud` 对象支持三个参数和一个方法：

`points` 参数，浮点类型列表的列表，每个浮点类型列表表示点的坐标，比如，`[[1,2,4]]`。

`colors` 参数，浮点类型列表的列表，每个浮点类型列表表示点的RGB颜色（每个元素取值0-1），比如，`[[0.5,0.5,0]]`。

`point_size` 参数，浮点类型，表示点的尺寸，默认为 `1.0`。

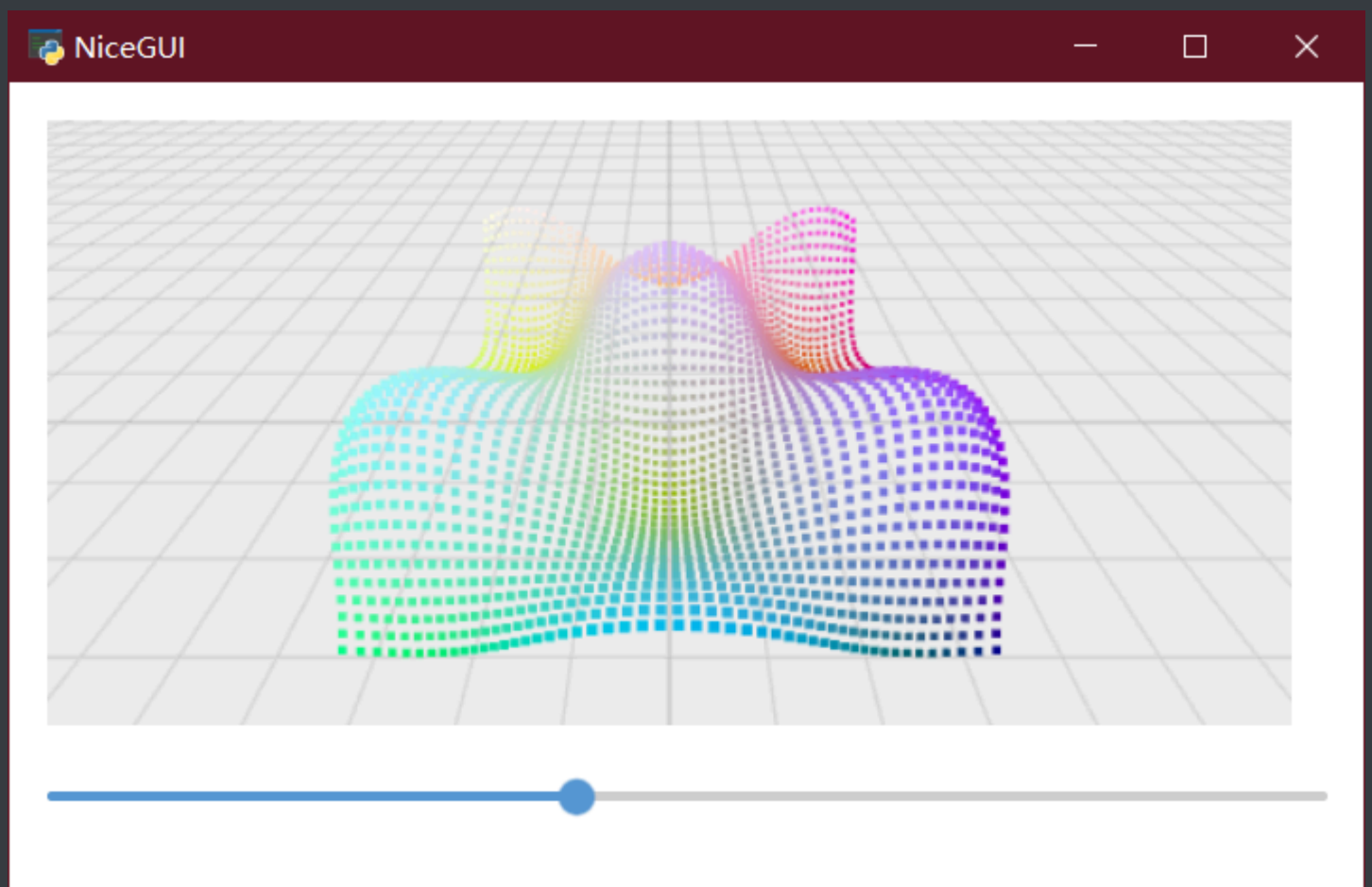
`set_points` 方法可以设置、更新点云的点数据，参数是 `points` 和 `colors`，含义同上。

示例如下：

```

1 import numpy as np
2 from nicegui import ui
3
4 def generate_data(frequency: float = 1.0):
5     x, y = np.meshgrid(np.linspace(-3, 3), np.linspace(-3, 3))
6     z = np.sin(x * frequency) * np.cos(y * frequency) + 1
7     points = np.dstack([x, y, z]).reshape(-1, 3)
8     colors = points / [6, 6, 2] + [0.5, 0.5, 0]
9     return points, colors
10
11 with ui.scene().classes('w-full h-64') as scene:
12     points, colors = generate_data()
13     point_cloud = scene.point_cloud(points, colors, point_size=0.1)
14
15 ui.slider(min=0.1, max=3, step=0.1, value=1) \
16     .on_value_change(lambda e: point_cloud.set_points(*generate_data(e.value)))
17
18 ui.run(native=True)

```



场景控件因为是基于3D渲染插件，加载过程会比一般的控件慢一些。而有的方法需要等待场景完全加载才能执行。所以，在下面的代码中，使用了 `await scene.initialized()` 来等待场景完成加载，然后再执行移动相机的操作：

```
1 from nicegui import ui
2
3 @ui.page('/')
4 async def page():
5     with ui.scene(width=285, height=220) as scene:
6         scene.sphere()
7         await scene.initialized()
8         scene.move_camera(x=1, y=-1, z=1.5, duration=2)
9
10 ui.run(native=True)
```

场景视图控件 `ui.scene_view` 可以显示一个指定场景的额外视图。注意，场景视图不同于场景，视图只是显示场景的内容，不能修改场景（无法与场景中的物体、视角交互，比如，拖动和旋转），但可以修改视图的相机来改变视图。此外，目前视图中不能显示2D文字和3D文字。

该控件支持以下参数：

`scene` 参数，`ui.scene` 类型，想要在视图内显示哪个场景的内容。

`width` 参数，整数类型，视图的宽度。

`height` 参数，整数类型，视图的高度。

`camera` 参数，视图使用的相机，该参数需要传入 `ui.scene.perspective_camera`（透视相机）或 `ui.scene.orthographic_camera`（正交相机）的实例，默认为透视相机。至于两种相机的区别和参数用法，可以参考前面场景的介绍。

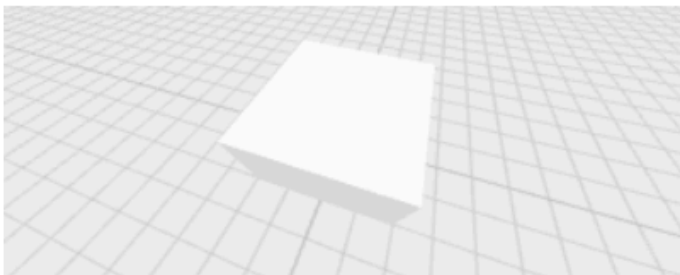
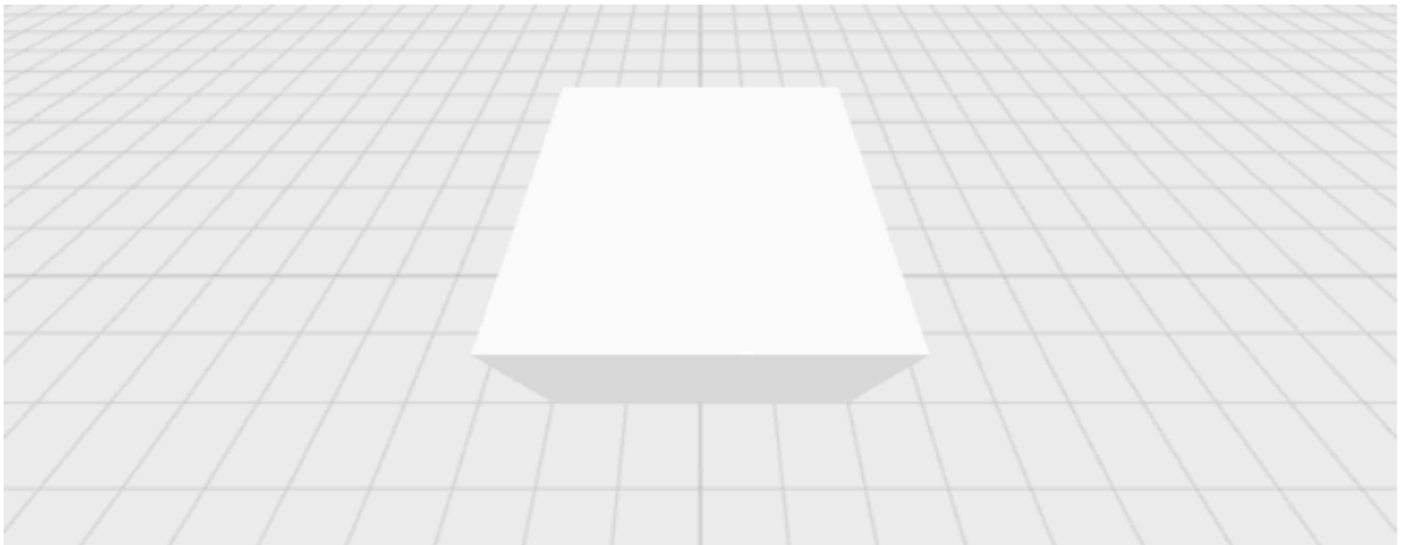
`on_click` 参数，和场景的 `on_click` 参数一致，点击视图中的物体时执行什么操作。

示例如下：

```
1 from nicegui import ui
2
3 with ui.grid(columns=2).classes('w-full'):
4     with ui.scene().classes('h-64 col-span-2') as scene:
5         scene.box().scale(4,4,4)
6
7     with ui.scene_view(scene).classes('h-32') as scene_view1:
8         scene_view1.move_camera(x=1, y=-3, z=5)
9
10    with ui.scene_view(scene).classes('h-32') as scene_view2:
11        scene_view2.move_camera(x=0, y=4, z=3)
12
13 ui.run(native=True)
```

NiceGUI

— □ ×



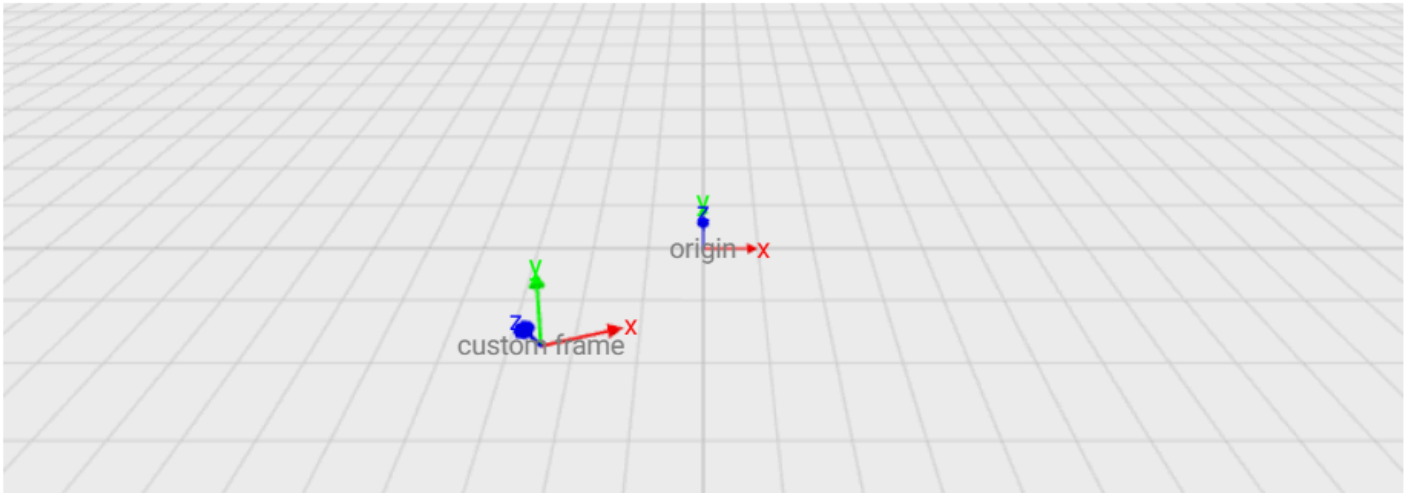
使用 `get_camera` 方法可以获取当前状态下的相机参数字典，可以从字典中读取 `position`（位置）、`rotation`（旋转角）、`fov`（视野角）等信息。完整的字典可以参考下面截取的两种相机的实际数据：

```
1  {
2      'position': {
3          'x': 0,
4          'y': -2.9999999999999987,
5          'z': 4.999999999999998
6      },
7      'up': {
8          'x': 0,
9          'y': 0,
10         'z': 1
11     },
12     'rotation': {
13         'isEuler': True,
14         '_x': 0.5404195002705839,
15         '_y': 0,
16         '_z': 0,
17         '_order': 'XYZ'
18     },
19     'quaternion': [0.26693358189581146, 0, 0, 0.9637149282107609],
20     'type': 'PerspectiveCamera',
21     'fov': 75,
22     'aspect': 2.5625,
23     'near': 0.1,
24     'far': 1000
25 }
26 {
27     'position': {
28         'x': -0.9652986025245365,
29         'y': -5.405483811638195,
30         'z': 1.9618724143227206},
31     'up': {
32         'x': 0,
33         'y': 0,
34         'z': 1
```


对于想要自定义组合3D对象的读者，可以参考下面的代码。示例代码创建了一个自定义类，将模拟三维坐标轴，可以看做是默认 `axes_helper` 对象（[官网文档](#)）的替代方案。

代码如下：

```
1 import math
2 from nicegui import ui
3
4 class CoordinateSystem(ui.scene.group):
5
6     def __init__(self, name: str, *, length: float = 1.0) -> None:
7         super().__init__()
8
9         with self:
10             for label, color, rx, ry, rz in [
11                 ('x', '#ff0000', 0, 0, -math.pi / 2),
12                 ('y', '#00ff00', 0, 0, 0),
13                 ('z', '#0000ff', math.pi / 2, 0, 0),
14             ]:
15                 with ui.scene.group().rotate(rx, ry, rz):
16                     ui.scene.cylinder(0.02 * length, 0.02 * length, 0.8 * length) \
17                         .move(y=0.4 * length).material(color)
18                     ui.scene.cylinder(0, 0.1 * length, 0.2 * length) \
19                         .move(y=0.9 * length).material(color)
20                     ui.scene.text(label, style=f'color: {color}') \
21                         .move(y=1.1 * length)
22                 ui.scene.text(name, style='color: #808080')
23
24 with ui.scene().classes('w-full h-64'):
25     CoordinateSystem('origin')
26     CoordinateSystem('custom frame').move(-2, -2, 1).rotate(0.1, 0.2, 0.3)
27
28 ui.run(native=True)
```



3.14.8 ui.aggrid

AG网格控件可以显示表格数据，NiceGUI的AG网格控件的功能实现源自于AG Grid（社区版，暂时不支持企业版）。

不同于 `ui.table`，AG网格支持的功能更多，可以使用 `run_grid_method` 方法和 `run_row_method` 方法，来与AG网格实例交互，详细用法可以参考[官方文档](#)。

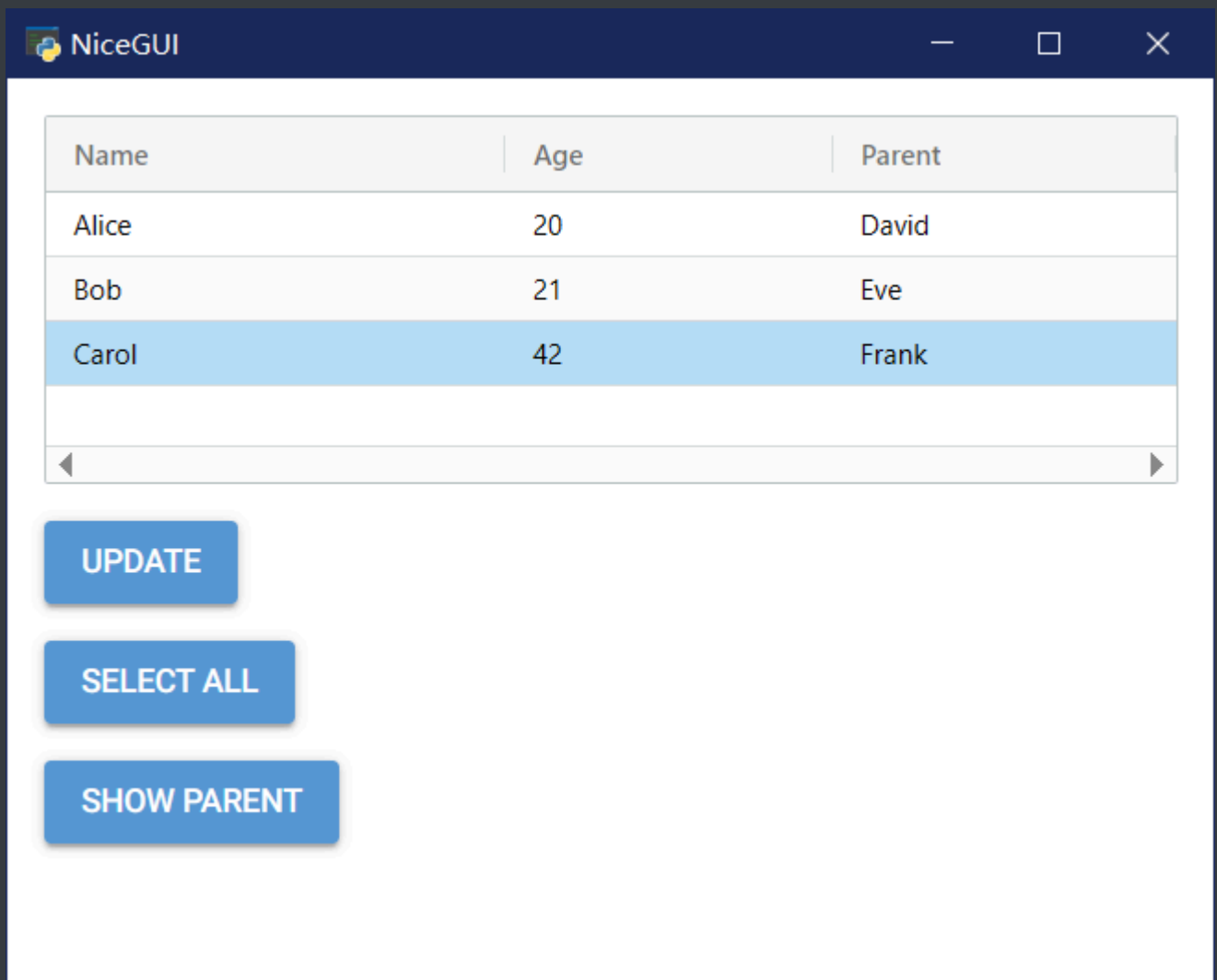
先看示例：

```
1 from nicegui import ui
2
3 grid = ui.aggrid({
4     'defaultColDef': {'flex': 1},
5     'columnDefs': [
6         {'headerName': 'Name', 'field': 'name'},
7         {'headerName': 'Age', 'field': 'age'},
8         {'headerName': 'Parent', 'field': 'parent', 'hide': True},
9     ],
10    'rowData': [
11        {'name': 'Alice', 'age': 18, 'parent': 'David'},
12        {'name': 'Bob', 'age': 21, 'parent': 'Eve'},
13        {'name': 'Carol', 'age': 42, 'parent': 'Frank'},
14    ],
15 })
```

```

15     'rowSelection': 'multiple',
16 }).classes('max-h-40')
17
18 def update():
19     grid.options['rowData'][0]['age'] += 1
20     grid.update()
21
22 ui.button('Update', on_click=update)
23 ui.button('Select all', on_click=lambda: grid.run_grid_method('selectAll'))
24 ui.button('Show parent', on_click=lambda: grid.run_grid_method('setColumnsVisible',
    ['parent'], True))
25
26 ui.run(native=True)

```



控件支持以下参数：

`options` 参数，字典类型，表示AG网格的选项定义，可以定义网格选项（具体定义内容参考官网文档），网格选项中包含列选项（对应的字典键是 `'columnDefs'`，具体定义内容参考官网文档）和具体的行数据（对应的字典键是 `'rowData'`）。以示例代码为例，`'defaultColDef'` 表示默认的列定义，适合定义每行都要设置的定义；`'rowSelection'` 表示行的选择方式，可以是单选（`'single'`）或者多选（`'multiple'`）。

`html_columns` 参数，整数列表，表示哪些列的数据当作HTML格式渲染，默认为空列表，即所有列的数据不当作HTML格式渲染。

`theme` 参数，字符串类型，表示AG网格的显示主题（支持的主题名参考官网文档），默认为 `'balham'`。

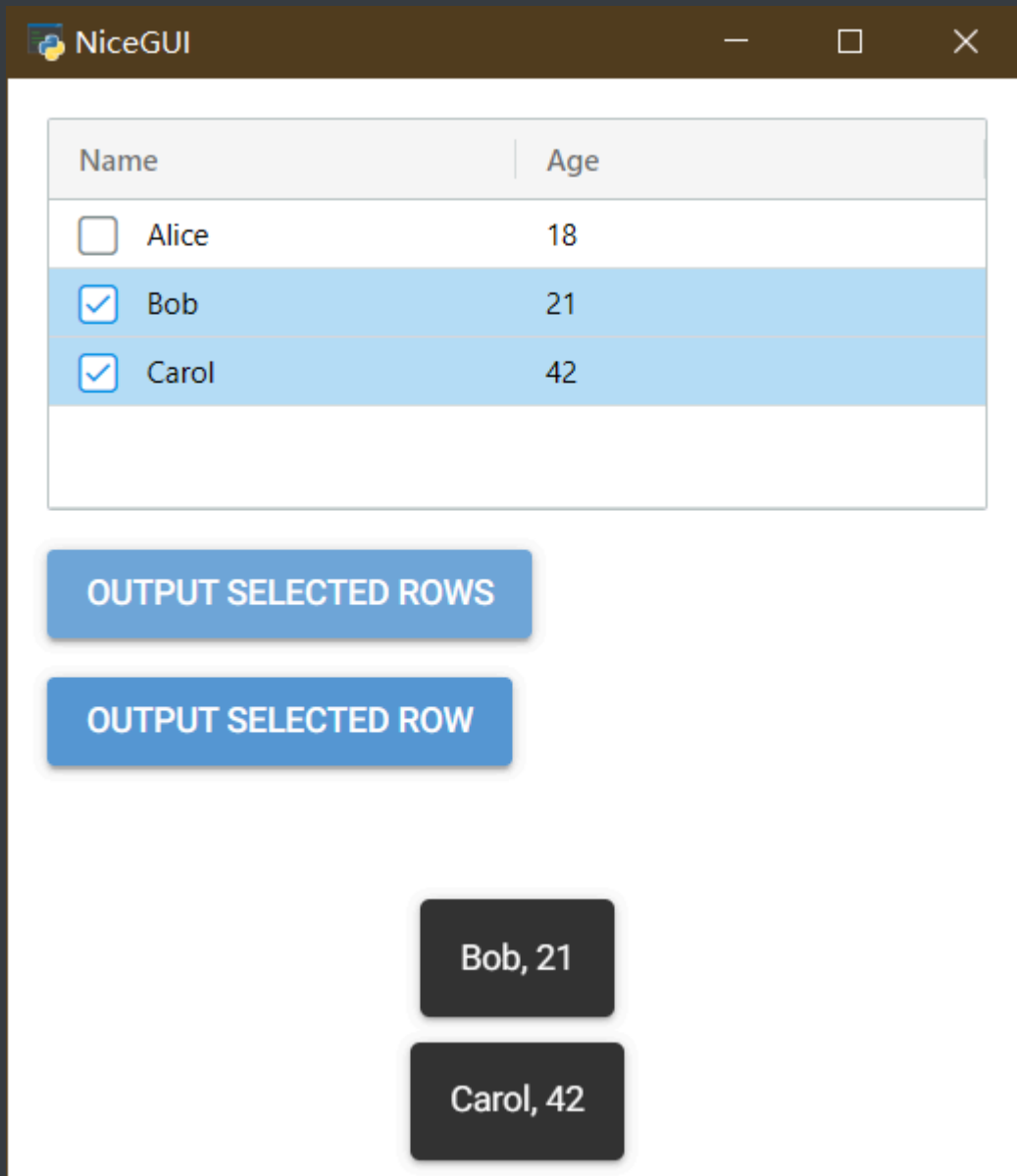
`auto_size_columns` 参数，布尔类型，表示是否根据表格内容自动调节列宽，默认为 `True`。

可以设置列选项的 `'checkboxSelection'` 为 `True` 添加单选或者多选的复选框。选择行之后，可以使用 `get_selected_rows` 方法，以字典的形式获取被选择的行。如果是单选，则可以用 `get_selected_row` 方法直接获取该行数据或者 `None`（如果没有数据被选择时）。实际上，`get_selected_row` 方法就是直接返回 `get_selected_rows` 方法结果的第一个（索引值为 0）元素，并在判断为空时返回 `None`。

代码如下：

```
1 from nicegui import ui
2
3 @ui.page('/')
4 def index():
5     grid = ui.aggrid({
6         'columnDefs': [
7             {'headerName': 'Name', 'field': 'name', 'checkboxSelection': True},
8             {'headerName': 'Age', 'field': 'age'},
9         ],
10        'rowData': [
11            {'name': 'Alice', 'age': 18},
12            {'name': 'Bob', 'age': 21},
13            {'name': 'Carol', 'age': 42},
14        ],
15        'rowSelection': 'multiple',
16    }).classes('max-h-40')
17
```

```
18     async def output_selected_rows():
19         rows = await grid.get_selected_rows()
20         if rows:
21             for row in rows:
22                 ui.notify(f"{row['name']}, {row['age']}")
23         else:
24             ui.notify('No rows selected.')
25
26     async def output_selected_row():
27         row = await grid.get_selected_row()
28         if row:
29             ui.notify(f"{row['name']}, {row['age']}")
30         else:
31             ui.notify('No row selected!')
32
33     ui.button('Output selected rows', on_click=output_selected_rows)
34     ui.button('Output selected row', on_click=output_selected_row)
35
36 ui.run(native=True)
```



在表头添加迷你过滤器（[官网文档](#)），可以用来筛选对应列下的数据。在下面的代码中，除了添加迷你过滤器（也就是漏斗图标），还启用了浮动显示过滤器（也就是第一行的输入框），这两个参数相关的过滤器选项，可以参考[官网文档](#)。过滤的类型可以是 "agTextColumnFilter"（文本过滤器）、"agNumberColumnFilter"（数字过滤器）、"agDateColumnFilter"（日期过滤器），除了这三种过滤器，还有两种是企业版功能，这里就不介绍了，相关类型可以参考[官网文档](#)。

示例如下：

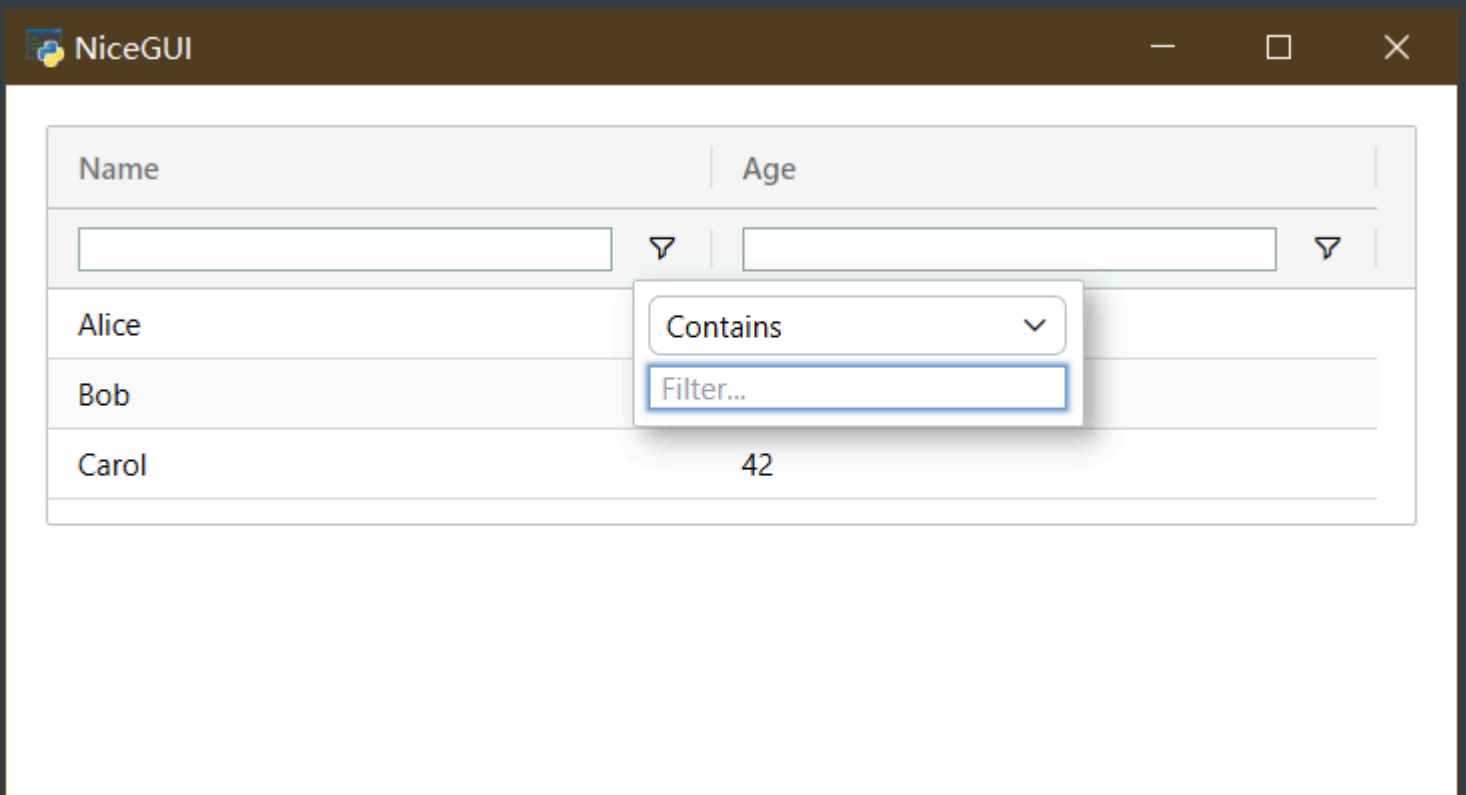
```
1 from nicegui import ui
2
3 ui.aggrid({
4     'columnDefs': [
5         {'headerName': 'Name', 'field': 'name', 'filter': 'agTextColumnFilter',
6          'floatingFilter': True},
```



```

6      {'headerName': 'Age', 'field': 'age', 'filter': 'agNumberColumnFilter',
      'floatingFilter': True},
7  ],
8  'rowData': [
9      {'name': 'Alice', 'age': 18},
10     {'name': 'Bob', 'age': 21},
11     {'name': 'Carol', 'age': 42},
12  ],
13 }).classes('max-h-40')
14
15 ui.run(native=True)

```



列定义中的 'cellClassRules' 可以实现根据网格的数据，设置网格的CSS类。其值是一个字典，字典的键（key）是要应用的CSS类名，字典的值（value）是用作判断条件的JavaScript表达式，更多用法可以参考[宜网文档](#)。注意，因为字典的值是JavaScript表达式，需要在字典的键的字符串内添加英文冒号（':'），启用计算表达式功能。

示例代码如下：

```

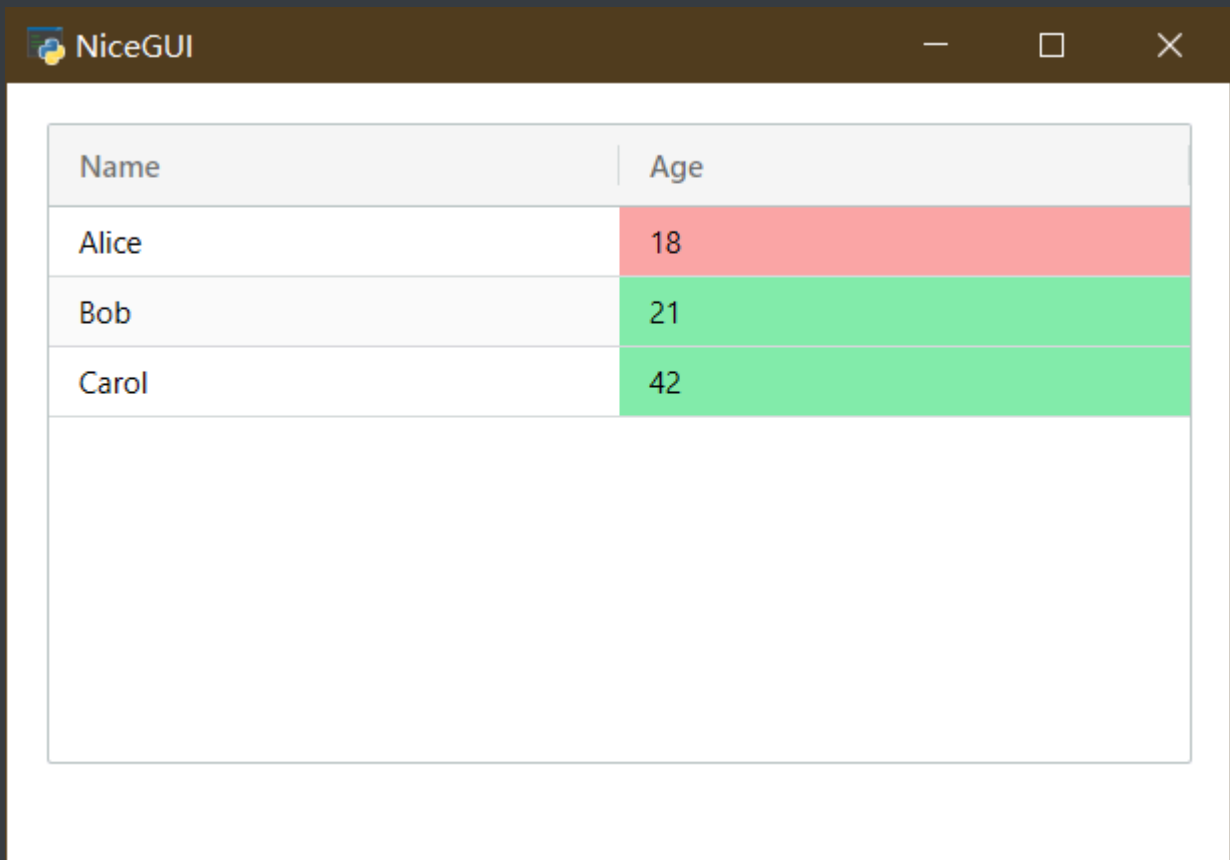
1 ui.aggrid({
2     'columnDefs': [
3         {'headerName': 'Name', 'field': 'name'},

```

```

4      {'headerName': 'Age', 'field': 'age', 'cellClassRules': {
5          ':bg-red-300': 'params => params.value < 21',
6          ':bg-green-300': 'params => params.value >= 21',
7      }},
8  ],
9  'rowData': [
10     {'name': 'Alice', 'age': 18},
11     {'name': 'Bob', 'age': 21},
12     {'name': 'Carol', 'age': 42},
13 ],
14 })
15
16 ui.run(native=True)

```



Name	Age
Alice	18
Bob	21
Carol	42

当然，如果不想写得这么复杂，可以使用映射的变量代替完整的JavaScript语法，具体参考[宜网文档](#)：

- `x`：映射了 `value`。
- `ctx`：映射了 `context`。
- `node`：映射了 `node`。

- `data`：映射了 `data`。
- `colDef`：映射了 `colDef`。
- `rowIndex`：映射了 `rowIndex`。
- `api`：映射了 `grid api`。

这样的话，只写简单的逻辑表达式就可以了，简化写法如下：

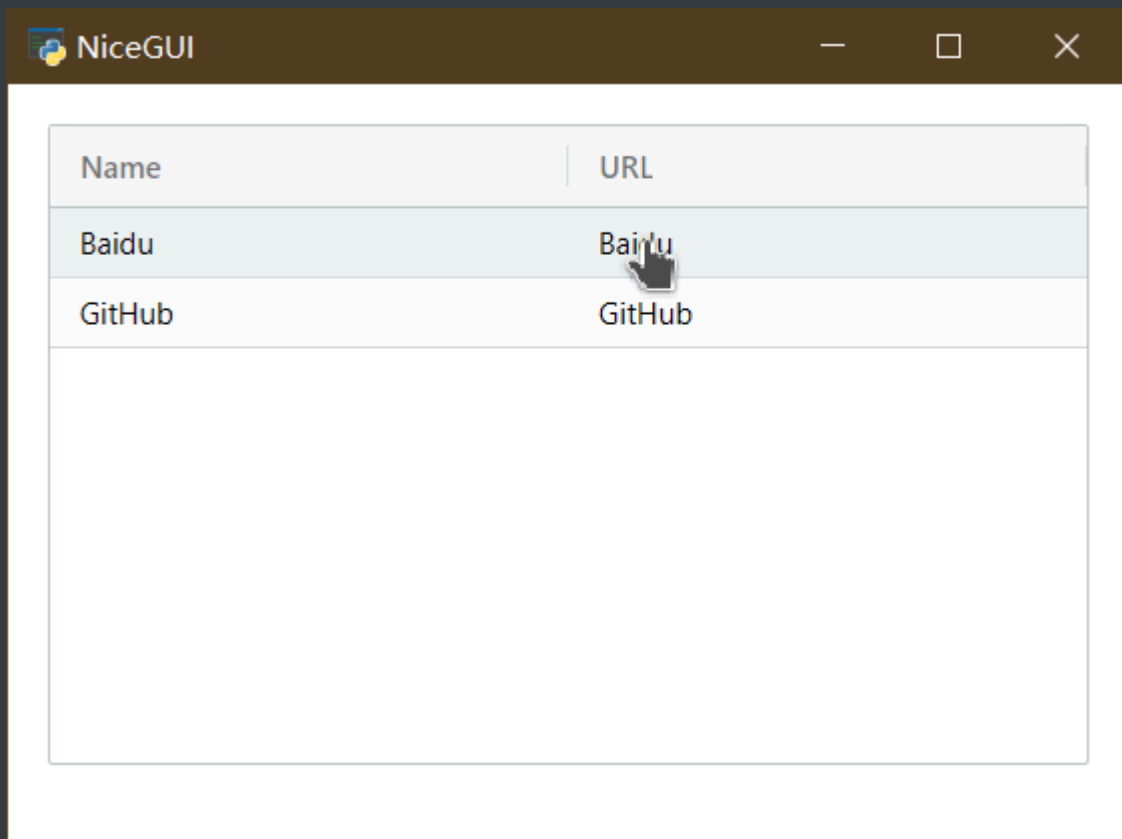
```
1 from nicegui import ui
2
3 ui.aggrid({
4     'columnDefs': [
5         {'headerName': 'Name', 'field': 'name'},
6         {'headerName': 'Age', 'field': 'age', 'cellClassRules': {
7             'bg-red-300': 'x < 21',
8             'bg-green-300': 'x >= 21',
9         }},
10    ],
11    'rowData': [
12        {'name': 'Alice', 'age': 18},
13        {'name': 'Bob', 'age': 21},
14        {'name': 'Carol', 'age': 42},
15    ],
16 })
17
18 ui.run(native=True)
```

AG网格作为一个表格控件，同样支持从 `pandas` 中导入数据，使用 `from_pandas` 方法即可：

```
1 import pandas as pd
2 from nicegui import ui
3
4 df = pd.DataFrame(data={'col1': [1, 2], 'col2': [3, 4]})
5 ui.aggrid.from_pandas(df).classes('max-h-40')
6
7 ui.run(native=True)
```

设定AG网格控件的 `html_columns` 参数，可以指定哪几列的数据当作HTML格式：

```
1 from nicegui import ui
2
3 ui.aggrid({
4     'columnDefs': [
5         {'headerName': 'Name', 'field': 'name'},
6         {'headerName': 'URL', 'field': 'url'},
7     ],
8     'rowData': [
9         {'name': 'Baidu', 'url': '<a href="https://www.baidu.com/">Baidu</a>'},
10        {'name': 'GitHub', 'url': '<a href="https://github.com/">GitHub</a>'},
11    ],
12 }, html_columns=[1])
13
14 ui.run(native=True)
```

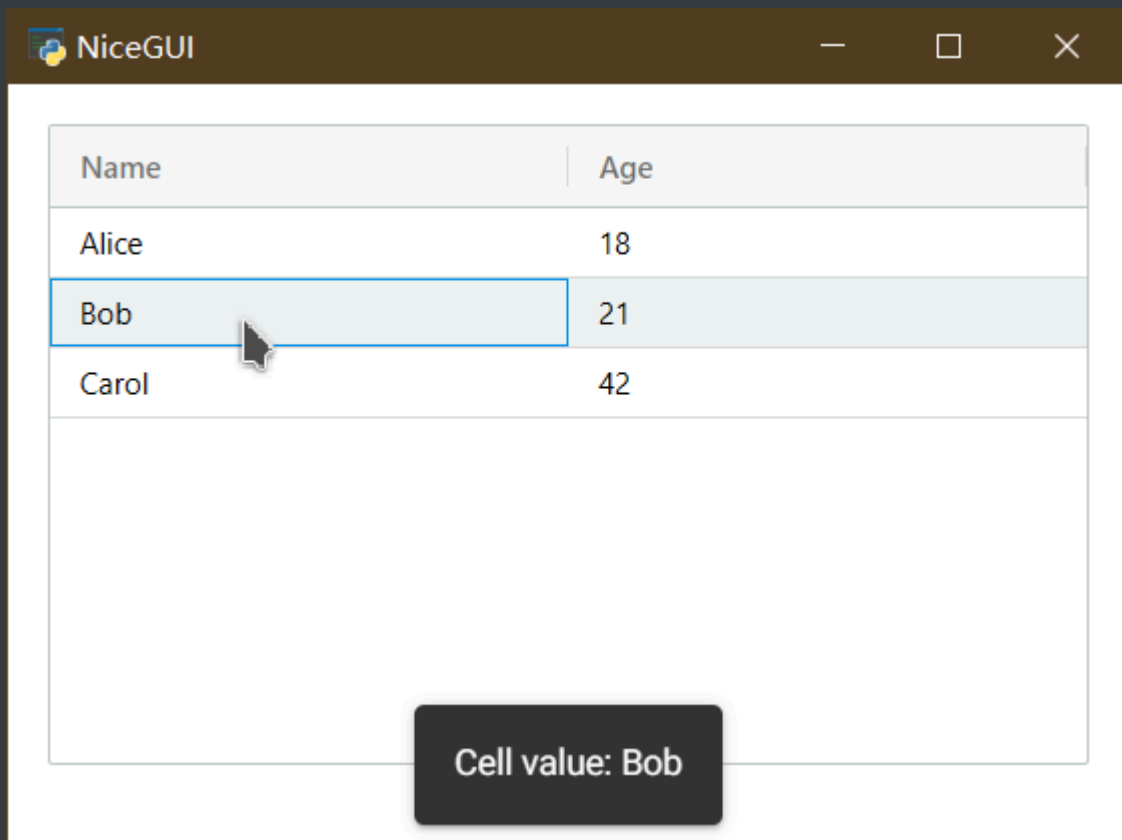


AG网格控件的所有事件（网格事件、列事件、行事件）都可以使用 `on` 方法订阅，使用Python代码编写事件的响应操作：

```

1  from nicegui import ui
2
3  ui.aggrid({
4      'columnDefs': [
5          {'headerName': 'Name', 'field': 'name'},
6          {'headerName': 'Age', 'field': 'age'},
7      ],
8      'rowData': [
9          {'name': 'Alice', 'age': 18},
10         {'name': 'Bob', 'age': 21},
11         {'name': 'Carol', 'age': 42},
12     ],
13 }).on('cellClicked', lambda event: ui.notify(f'Cell value: {event.args["value"]}'))
14
15 ui.run(native=True)

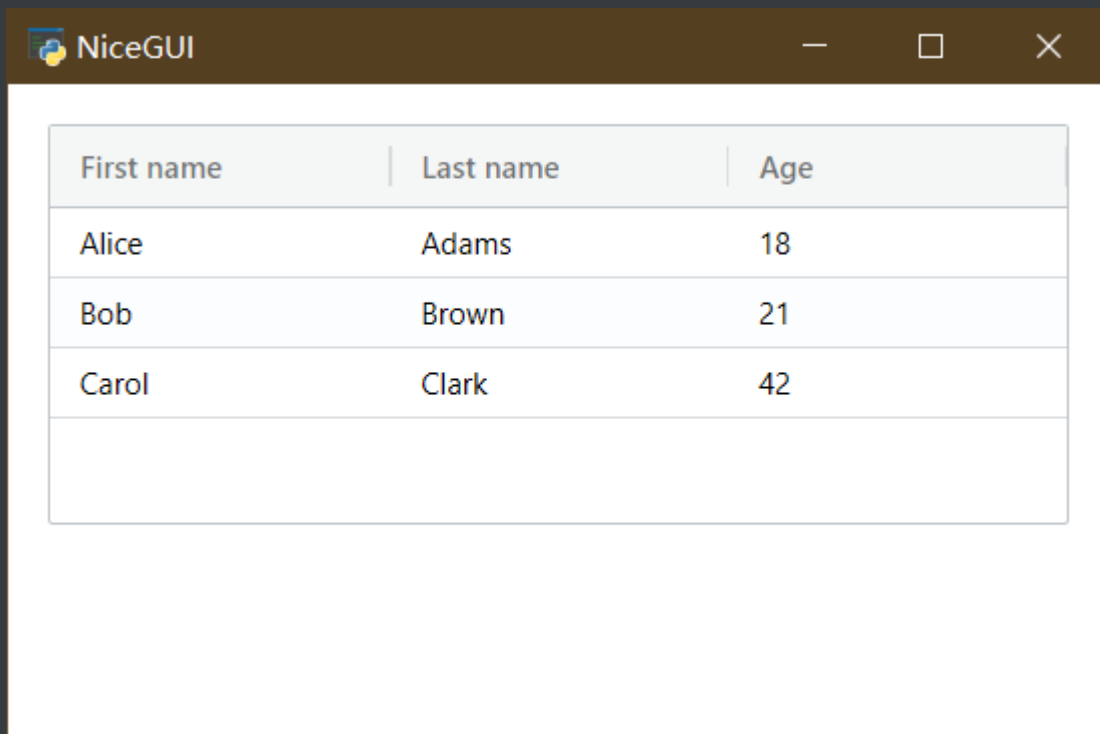
```



如果 'rowData' 的数据中，键对应的值是字典，则字典被称为子域。那么，列选项的 'field' 就可以通过英文句号选择 'rowData' 的子域数据（参考[官网文档](#)）。因此，'rowData' 的键（key）不能有英文句号。

示例代码如下：

```
1 from nicegui import ui
2
3 ui.aggrid({
4     'columnDefs': [
5         {'headerName': 'First name', 'field': 'name.first'},
6         {'headerName': 'Last name', 'field': 'name.last'},
7         {'headerName': 'Age', 'field': 'age'}
8     ],
9     'rowData': [
10         {'name': {'first': 'Alice', 'last': 'Adams'}, 'age': 18},
11         {'name': {'first': 'Bob', 'last': 'Brown'}, 'age': 21},
12         {'name': {'first': 'Carol', 'last': 'Clark'}, 'age': 42},
13     ],
14 }).classes('max-h-40')
15
16 ui.run(native=True)
```



First name	Last name	Age
Alice	Adams	18
Bob	Brown	21
Carol	Clark	42

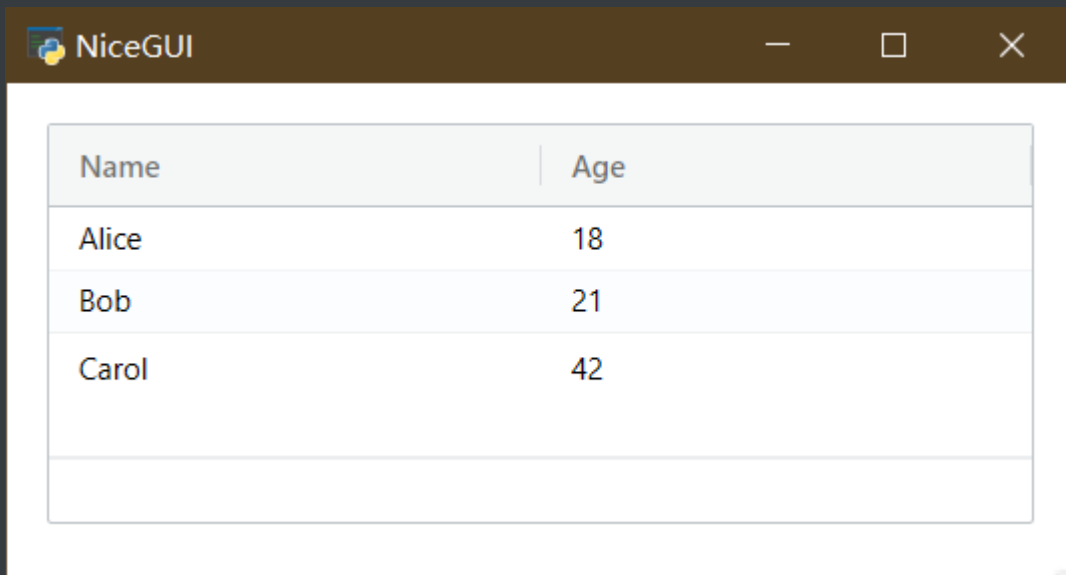
行高除了设置网格选项中的 'rowHeight' 这种统一值，还可以设置 'getRowHeight' 这种支持表达式的动态值，更多用法参考[官网文档](#)。注意，因为表达式是JavaScript语法，在NiceGUI中，需要添加英文冒号（ ':' ）当前缀，启用计算表达式功能。

示例代码如下：

```

1  from nicegui import ui
2
3  ui.aggrid({
4      'columnDefs': [{'field': 'name'}, {'field': 'age'}],
5      'rowData': [
6          {'name': 'Alice', 'age': '18'},
7          {'name': 'Bob', 'age': '21'},
8          {'name': 'Carol', 'age': '42'},
9      ],
10     ':getRowHeight': 'params => params.data.age > 35 ? 50 : 25',
11 }).classes('max-h-40')
12
13 ui.run(native=True)

```

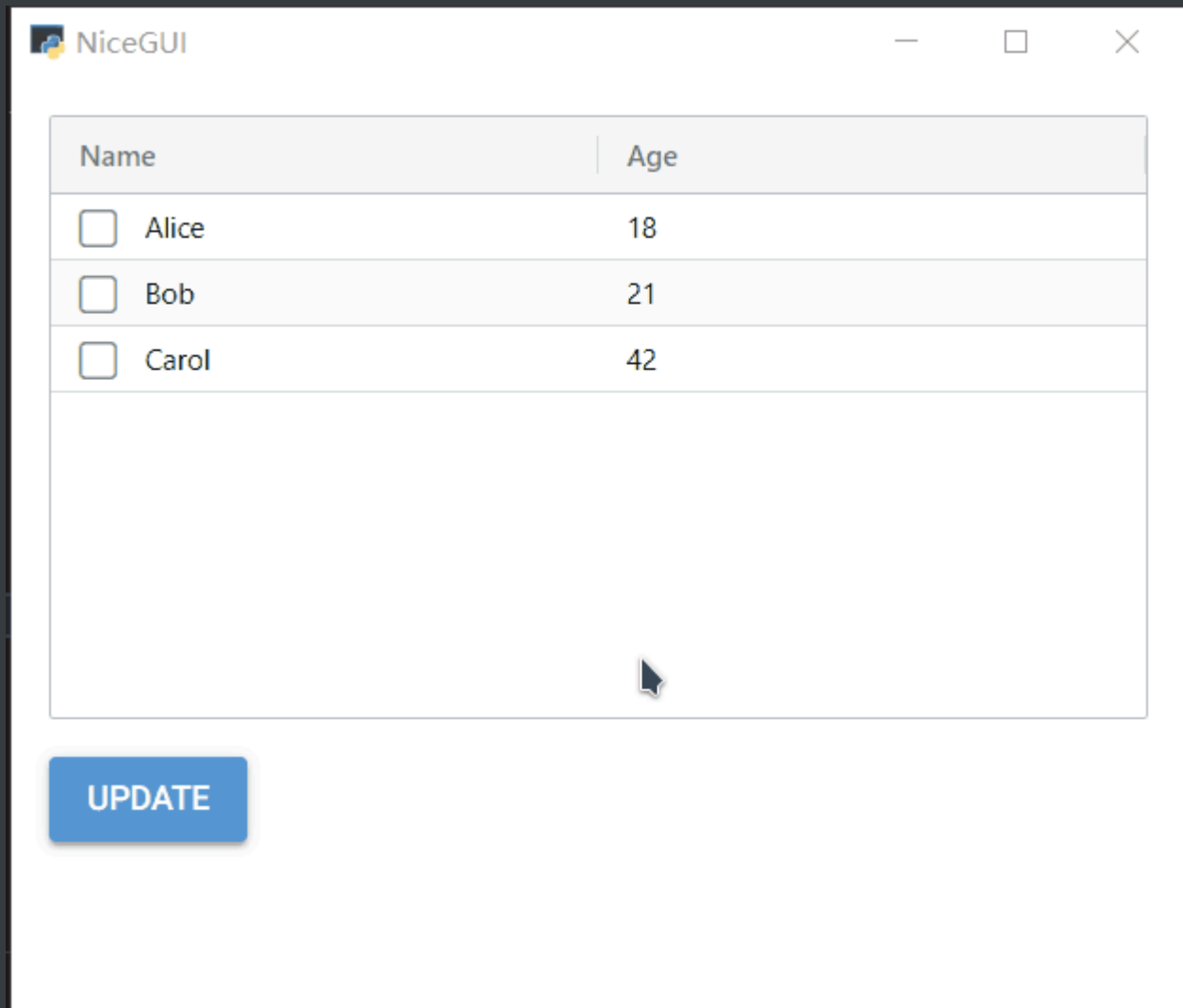


Name	Age	
Alice	18	
Bob	21	
Carol	42	

运行 `run_row_method` 方法可以调用AG Grid提供的接口方法（[官网文档](#)）来影响单行数据，这个方法参数分别是行ID、接口方法名、传给接口方法的参数。行ID可以是字符串类型的行索引值，也可以是网格选项 `'getRowId'` 提供的获取唯一ID的方法。比如，在下面的示例中，`':getRowId': '(params) => params.data.name'` 就是定义了使用 `name` 列的数据当作每行数据的唯一ID。

需要注意的是，在点击'UPDATE'按钮、显示的数据更新的同时，行的选择状态是保留的。但是，如果是使用 `update` 方法（NiceGUI提供的，在其他更新数据的情况下，为了确保显示正常，可能需要调用此方法）来更新AG网格控件，选择状态则不会保留。

示例代码如下：



除了传入方法名、方法参数给'`run_*_method`'方法来调用接口方法，还可以给方法名传入定义了JavaScript箭头函数（[参考文档](#)）的字符串，可以实现在一定程度上自由运行特定方法。

以下面的代码为例，'`g => g.getDisplayedRowIndex(0).data`'就是定义了一个JavaScript箭头函数，这种函数的用法和Python的lambda表达式一样，本质上是一个函数，只是没有函数名，可以当做函数用，字符串内的'`=>`'前是参数，后面是该方法的返回值。这个箭头函数的返回值是获取 `getDisplayedRowIndex`（[官网文档](#)）执行结果的 `data` 属性。`getDisplayedRowIndex(0)` 的执行结果是从显示的行中取第一个（索引值为 0），其 `data` 属性就是这一行的数据，可以看作是 `{'name': 'Alice'}` 这样的字典，这也就是该行代码执行的结果。

需要注意的是，因为这种方法和客户端数据相关，只能在 `ui.page` 内使用，不能用在 `auto-index` 页面。

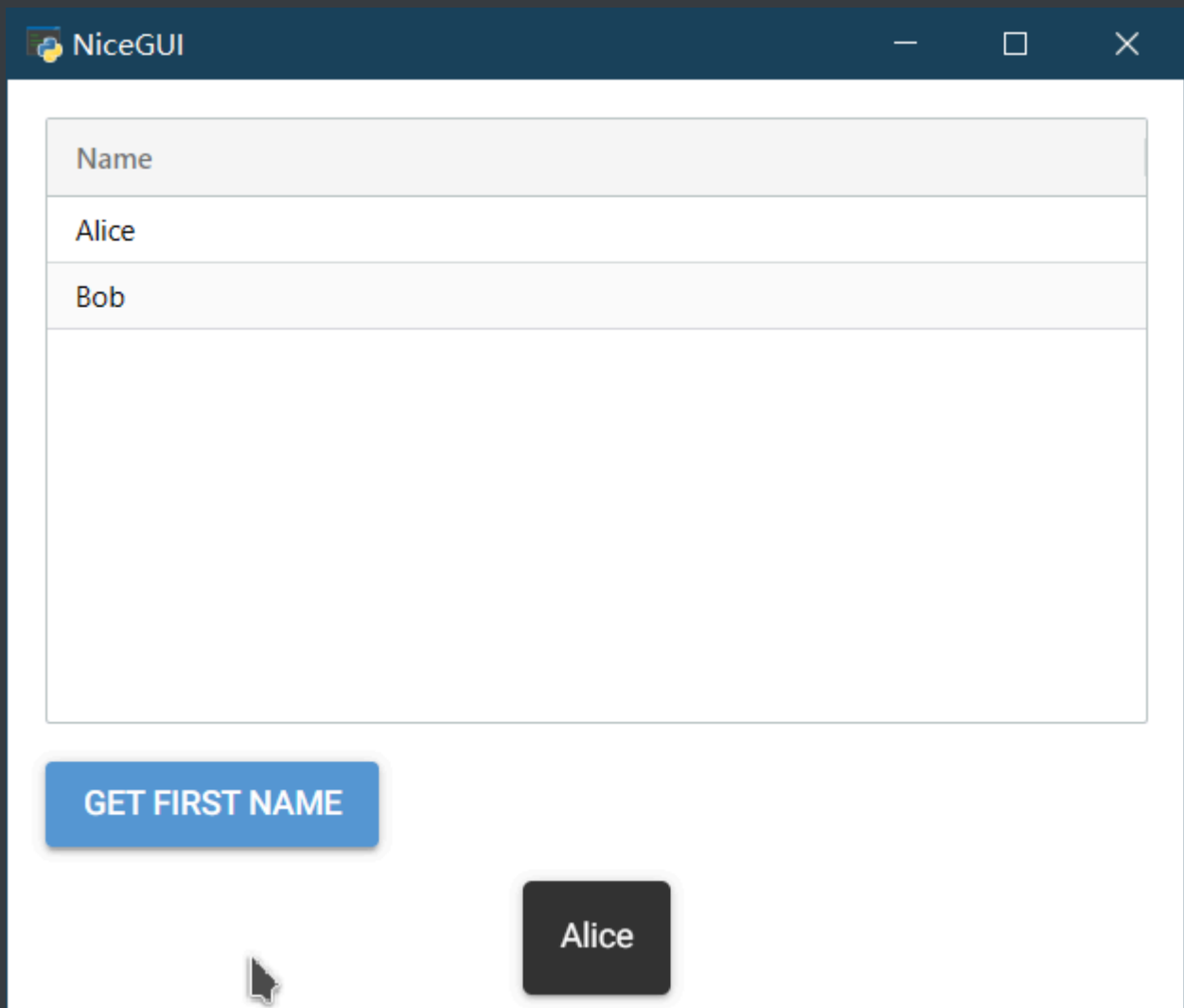
示例代码如下：

```
1 from nicegui import ui
```

```

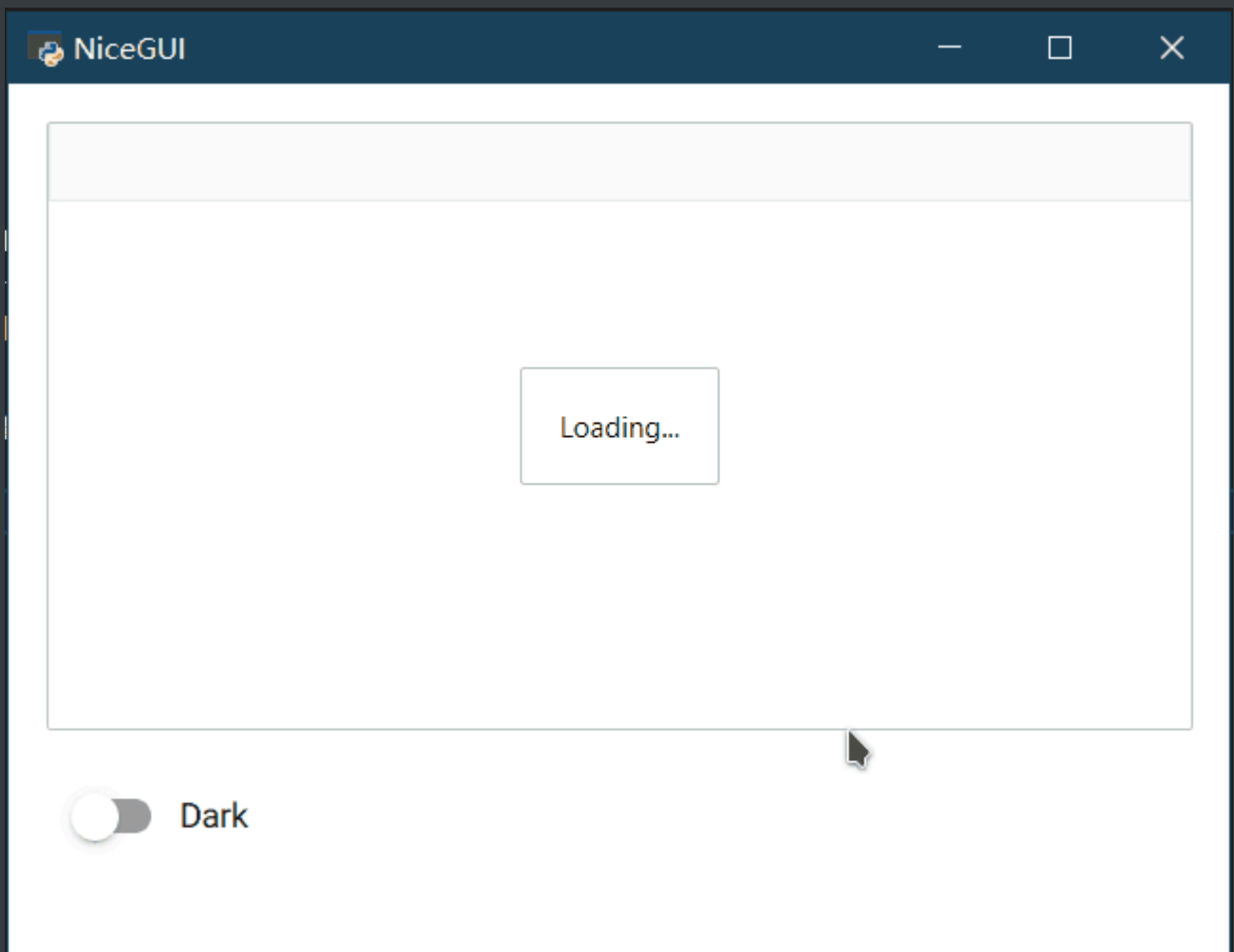
2
3 @ui.page('/')
4 def page():
5     grid = ui.aggrid({
6         'columnDefs': [{ 'field': 'name' }],
7         'rowData': [{ 'name': 'Alice' }, { 'name': 'Bob' }],
8     })
9
10    async def get_first_name() -> None:
11        row = await grid.run_grid_method('g => g.getDisplayedRowAtIndex(0).data')
12        ui.notify(row['name'])
13
14    ui.button('Get First Name', on_click=get_first_name)
15
16 ui.run(native=True)

```



AG网格有暗黑模式下的主题，但因为是第三方库，所以并不支持Quasar体系的自动切换暗黑模式。但可以使用下面的代码实现手动切换：

```
1 from nicegui import events, ui
2
3 grid = ui.aggrid({})
4 dark = ui.dark_mode()
5
6 def handle_theme_change(e: events.ValueChangeEventArguments):
7     grid.classes(add='ag-theme-balham-dark' if dark.value else 'ag-theme-balham',
8                 remove='ag-theme-balham ag-theme-balham-dark')
9
10 ui.switch('Dark', on_change=handle_theme_change).bind_value_to(dark)
11
12 ui.run(native=True)
```



3.14.9 ui.highchart

`ui.highchart` 图表控件可以绘制Highchart图表，功能实现源于[Highcharts](#)。需要注意的是，该插件商用需要购买商业许可，如有商用需求，请到官网购买许可。同样因为许可限制，NiceGUI将此控件的相关代码放在单独的仓库中维护，如果想要使用此控件，可以在项目根目录下使用以下命令安装：

```
1 pdm add nicegui[highcharts]
2 #或者下面这条
3 pdm add nicegui-highcharts
```

如果运行环境是全局环境或者想用pip安装，可以执行下面的命令安装：

```
1 pip install nicegui[highcharts]
2 #或者下面这条，但需要确保已经安装了NiceGUI
3 pip install nicegui-highcharts
```

本控件的主要用法需要读者了解Highcharts，如果没有相关基础，可以先去看看Highcharts官网的[入门文档](#)。

更新 `options` 属性，可以将数据更新推送到图表上。但是，需要注意的是，更新数据之后，需要调用 `update` 方法。

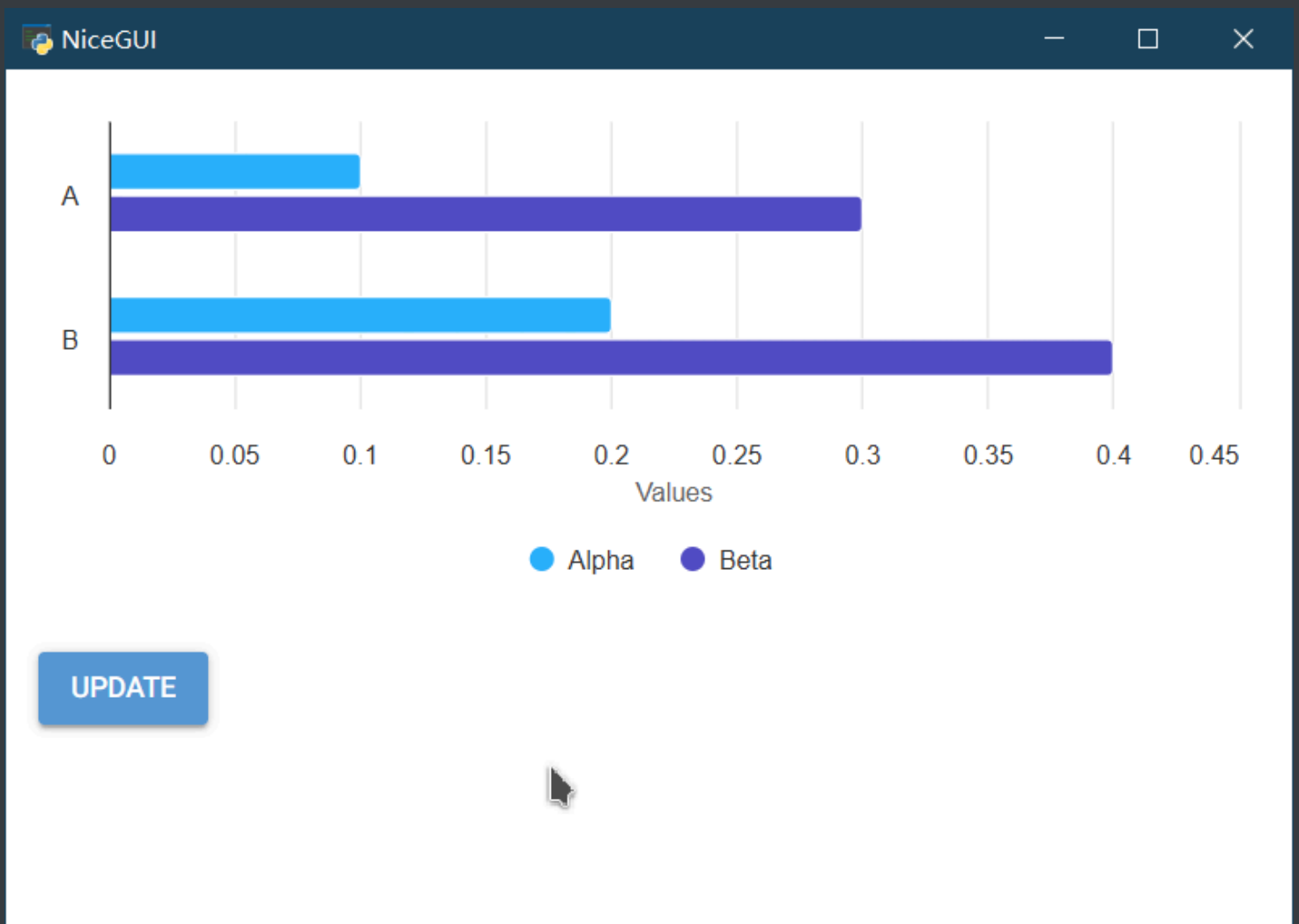
先看代码示例：

```
1 from nicegui import ui
2 from random import random
3
4 chart = ui.highchart({
5     'title': False,
6     'chart': {'type': 'bar'},
7     'xAxis': {'categories': ['A', 'B']},
8     'series': [
9         {'name': 'Alpha', 'data': [0.1, 0.2]},
10        {'name': 'Beta', 'data': [0.3, 0.4]},
11    ],
12    'credits': {'enabled': False}
13 }).classes('w-full h-64')
```

```

14
15
16 def update():
17     chart.options['series'][0]['data'][0] = random()
18     chart.update()
19
20
21 ui.button('Update', on_click=update)
22
23 ui.run(native=True)

```



控件支持以下参数：

`options` 参数，字典类型，给Highcharts传入的图表选项定义，具体参考官网文档。就示例而言，用到的主要是 `'title'`（标题，布尔类型的 `False` 为不显示，可以使用子字典定义标题样式和内容，具体见文档）、`'chart'`（图表样式，这里使用子字典定义图表类型使用柱状图 `'bar'`，如果需要设置其他类型，可以参考[绘图选项](#)，而其他图表样式具体见[文档](#)）、`'xAxis'`（设定x轴，具体见文

档)、'series' (具体数据, 具体见文档)、'credits' (版权信息, 具体见文档)。

type 参数, 字符串类型, 表示要绘制的图表大类, 默认是 "chart" (常规图表), 即官网文档中的 Highcharts.chart, 如果想绘制其他图表。比如 Highcharts.stockChart, 则需要设置此参数为 "stockChart" (股票图)。此参数支持 "chart"、"stockChart"、"mapChart"、"ganttChart" 等, 具体每一种图表大类对应的方法、数据格式均不同, 下面仅讲解常规图表。

extras 参数, 字符串列表, 表示需要导入的额外依赖 (Highcharts将不常用的功能及其依赖做成了插件, 按需导入), 比如 "annotations"、"arc-diagram"、"solid-gauge" 等。

on_point_click 参数, 可调用类型, 表示图表上的点被点击时执行的操作。

on_point_drag_start 参数, 可调用类型, 表示图表上的点开始拖动时执行的操作。

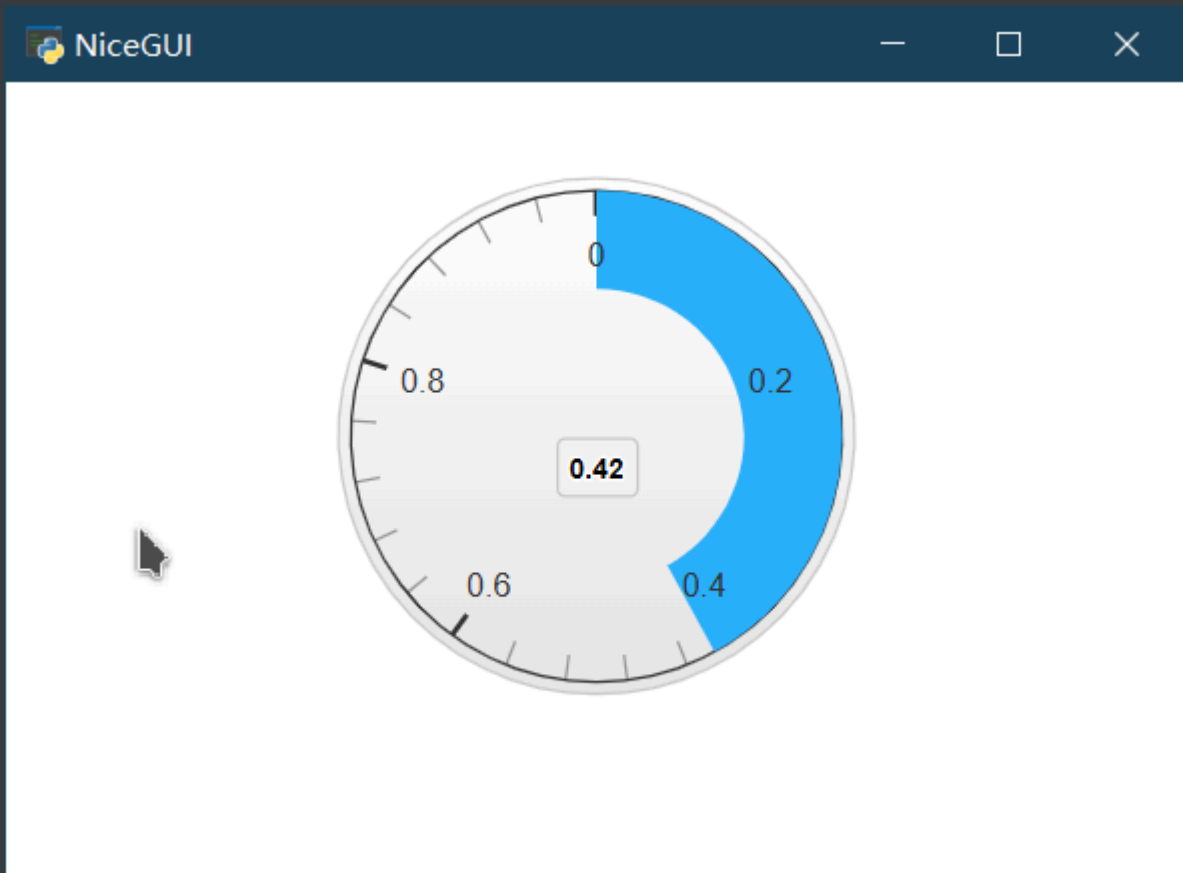
on_point_drag 参数, 可调用类型, 表示图表上的点处于拖动状态时执行的操作。

on_point_drop 参数, 可调用类型, 表示图表上的点放下时执行的操作。

有的图表类型不是常用的类型, 需要导入额外依赖, 比如下面代码中的 'solidgauge', 参考[官网文档](#), 可知其需要 `modules/solid-gauge` (没有 `requires` 的话说明不需要额外导入), 所以需要给 extras 参数传入需要导入的依赖 ['solid-gauge']:

```
1  from nicegui import ui
2
3  ui.highchart({
4      'title': False,
5      'chart': {'type': 'solidgauge'},
6      'yAxis': {
7          'min': 0,
8          'max': 1,
9      },
10     'series': [
11         {'data': [0.42]},
12     ],
13     'credits': {'enabled': False}
14 }, extras=['solid-gauge']).classes('w-full h-64')
15
```

```
16 ui.run(native=True)
```



如果设置图表允许拖动点，则可以注册以下事件来响应拖动操作：

- `on_point_click`：点击点。
- `on_point_drag_start`：开始拖动点。
- `on_point_drag`：点被拖动时。
- `on_point_drop`：点被放下时。

```
1 from nicegui import ui
2 from nicegui_highcharts import events
3
4 def handle_event(e: events.ChartEventArguments):
5     print(f'{e.event_type}: x is {e.point_x if hasattr(e, 'point_x') else None}, y is {e.point_y if hasattr(e, 'point_y') else None}.')
6 ui.highchart(
7     {
8         'title': False,
9         'plotOptions': {
```

```

10         'series': {
11             'stickyTracking': False,
12             'dragDrop': {'draggableY': True, 'dragPrecisionY': 1},
13         },
14     },
15     'series': [
16         {'name': 'A', 'data': [[20, 10], [30, 20], [40, 30]]},
17         {'name': 'B', 'data': [[50, 40], [60, 50], [70, 60]]},
18     ],
19     'credits': {'enabled': False}
20 },
21 extras=['draggable-points'],
22 on_point_click=handle_event,
23 on_point_drag_start=handle_event,
24 on_point_drag=handle_event,
25 on_point_drop=handle_event
26 ).classes('w-full h-64')
27
28 ui.run(native=True)

```

得到的结果输出（部分）如下：

```

1 point_drag_start: x is None, y is None.
2 point_drag: x is 20, y is 22.
3 point_drag: x is 20, y is 23.
4 point_drag: x is 20, y is 30.
5 point_drop: x is 20, y is 30.
6 point_drag_start: x is None, y is None.
7 point_click: x is 20, y is 30.
8 point_drag_start: x is None, y is None.
9 point_click: x is 30, y is 43.

```

可以看到，拖动一个点总是包含拖动开始、拖动时、放下三个动作；如果是点击点，则会触发开始拖动和点击这两个动作。

3.14.10 ui.echart

`ui.echart` 可以绘制echart图表，功能实现源自ECharts。

相比于HighCharts，ECharts的许可协议更友好，也不需要购买商用许可，而且官网文档有中文版本，因此更受中国用户喜欢。

本控件的主要用法需要读者了解ECharts，如果没有相关基础，可以先去看看ECharts官网的[入门文档](#)。

更新 `options` 属性，可以将数据更新推送到图表上。但是，需要注意的是，更新数据之后，需要调用 `update` 方法。

先看代码示例：

```
1  from nicegui import ui
2  from random import random
3
4  echart = ui.echart({
5      'xAxis': {'type': 'value'},
6      'yAxis': {'type': 'category', 'data': ['A', 'B'], 'inverse': True},
7      'legend': {'textStyle': {'color': 'gray'}},
8      'series': [
9          {'type': 'bar', 'name': 'Alpha', 'data': [0.1, 0.2]},
10         {'type': 'bar', 'name': 'Beta', 'data': [0.3, 0.4]},
11     ],
12 })
13
14
15 def update():
16     echart.options['series'][0]['data'][0] = random()
17     echart.update()
18
19 ui.button('Update', on_click=update)
20
21 ui.run(native=True)
```



控件支持以下参数：

`options` 参数，字典类型，给ECharts传入的图表选项定义，具体参考[官网文档](#)。

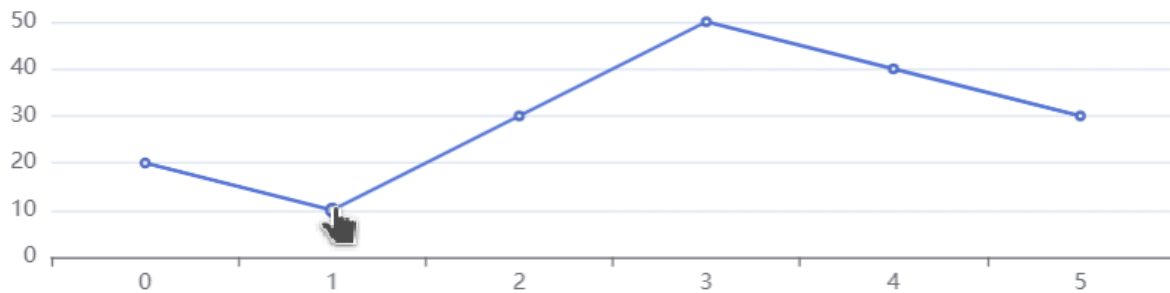
`on_click_point` 参数，可调用类型，当点击数据点时执行的操作。

`enable_3d` 参数，布尔类型，是否强制导入 `echarts-gl` 库，这样做会启用3D显示效果。

点击事件的响应操作，可以参考以下代码：

```
1 from nicegui import ui
2
3 ui.echart({
4     'xAxis': {'type': 'category'},
5     'yAxis': {'type': 'value'},
6     'series': [{'type': 'line', 'data': [20, 10, 30, 50, 40, 30]}],
7 }, on_point_click=ui.notify)
8
9 ui.run(native=True)
```

NiceGUI



```
EChartPointClickEventArguments(sender=<nicegui.elements.echart.EChart
object at 0x0000016892278680>, client=<nicegui.client.Client object at
0x00000168B7703A10>, component_type='series', series_type='line',
series_index=0, series_name='series\x000', name='1', data_index=1, data=10,
data_type=None, value=10)
```

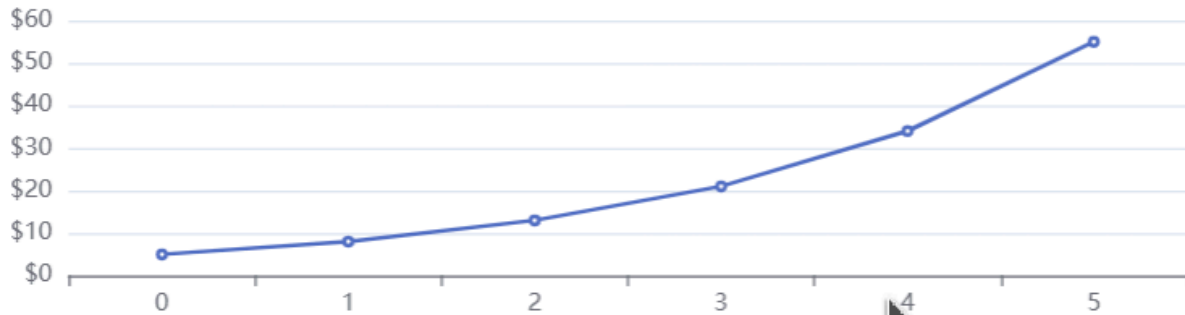
定义动态属性（即使用JavaScript表达式的内容）的示例代码：

```

1  from nicegui import ui
2
3  ui.echart({
4      'xAxis': {'type': 'category'},
5      'yAxis': {'axisLabel': {'formatter': 'value => "$" + value'}},
6      'series': [{'type': 'line', 'data': [5, 8, 13, 21, 34, 55]}],
7  })
8
9  ui.run(native=True)

```

NiceGUI



使用 `from_pycharts` 方法，可以基于 `pycharts` 对象创建图表。对于创建动态属性（即使用 JavaScript 表达式的内容），既可以像之前说的那样在属性名前加英文冒号 `:` 启用计算表达式功能，也可以使用 `pycharts.common.utils` 的 `JsCode` 方法直接传递转换后的对象，这样就不用添加英文冒号了。

示例代码如下：

```

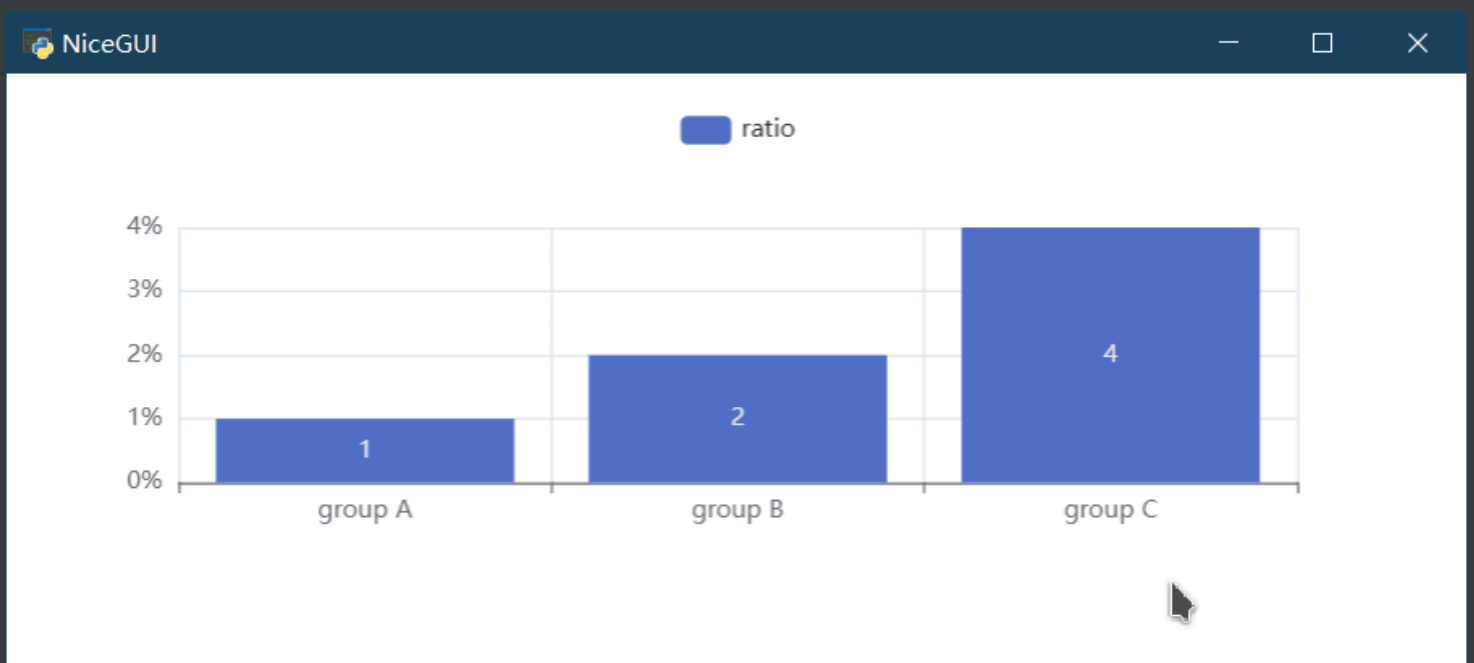
1  from nicegui import ui
2  from pycharts.charts import Bar
3  from pycharts.common.utils import JsCode
4  from pycharts.options import AxisOpts
5
6  ui.echart.from_pycharts(
7      Bar()

```

```

8     .add_xaxis(['A', 'B', 'C'])
9     .add_yaxis('ratio', [1, 2, 4])
10    .set_global_opts(
11        xaxis_opts=AxisOpts(axislabel_opts={
12            ':formatter': r'(val, idx) => `group ${val}`',
13        }),
14        yaxis_opts=AxisOpts(axislabel_opts={
15            'formatter': JsCode(r'(val, idx) => `${val}%`'),
16        }),
17    )
18 )
19 ui.run(native=True)

```



调用 `run_chart_method` 方法可以执行ECharts的实例方法（[官网文档](#)）。需要注意的是，代码中给实例方法名前面加了英文冒号 `:`，意思就是后面传给这个实例方法的参数是JavaScript表达式，需要先执行之后再传入。

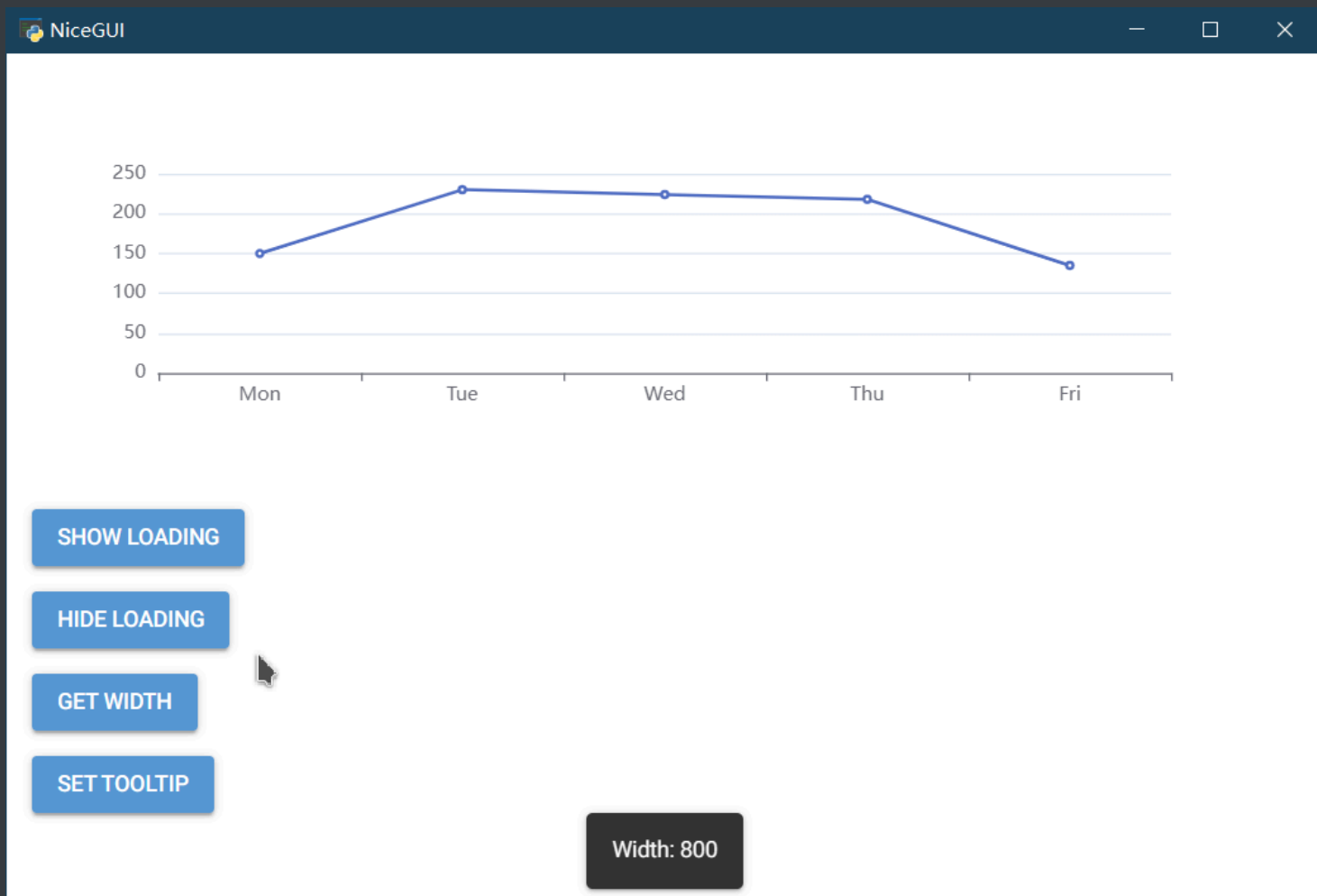
需要注意的是，因为这种方法和客户端数据相关，只能在 `ui.page` 内使用，不能用在 `auto-index` 页面。

代码示例如下：

```

1 from nicegui import ui
2

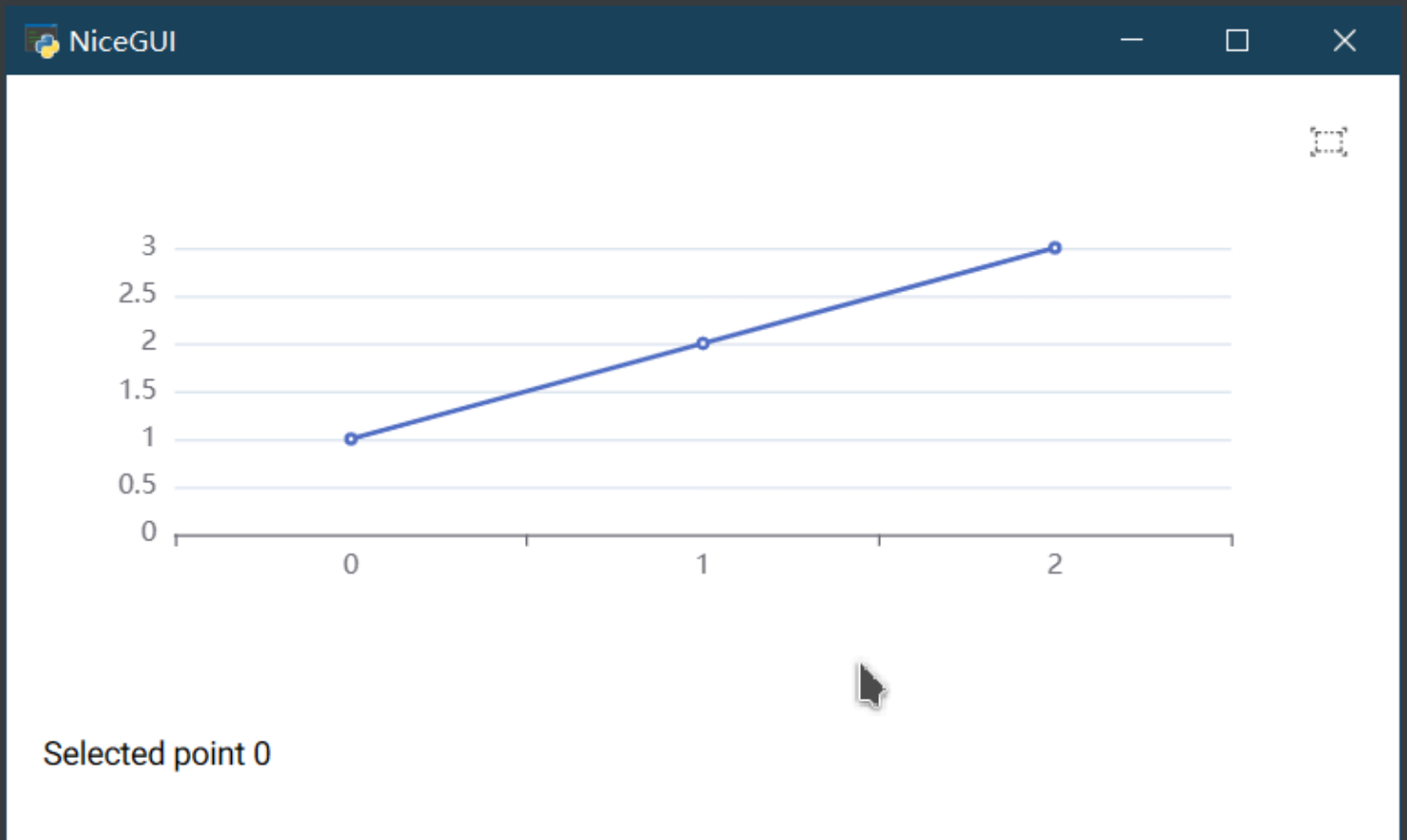
```

想要添加对ECharts的事件响应的話，可以使用 `on` 方法，订阅加了"chart:"前綴的ECharts事件名（具体事件名查询官网文档）。比如下面的代码，就是监听了ECharts的 "selectchanged" 事件（[官网文档](#)）：

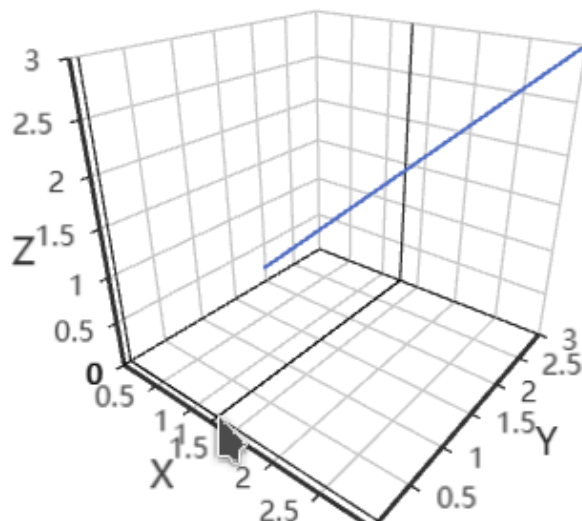
```
1 from nicegui import ui
2
3 ui.echart({
4     'toolbox': {'feature': {'brush': {'type': ['rect']}}},
5     'brush': {},
6     'xAxis': {'type': 'category'},
7     'yAxis': {'type': 'value'},
8     'series': [{'type': 'line', 'data': [1, 2, 3]}],
9 }).on('chart:selectchanged', lambda e: label.set_text(
10     f'Selected point {e.args["fromActionPayload"]["dataIndexInside"]}'
11 ))
12 label = ui.label()
13
```

```
14 ui.run(native=True)
```



一般来说，如果 `options` 参数里包含了字符串"3D"，图表会自动启用3D显示。如果没有的话，可以设置 `enable_3d` 参数为 `True` 来强制启用：

```
1 from nicegui import ui
2
3 ui.echart({
4     'xAxis3D': {},
5     'yAxis3D': {},
6     'zAxis3D': {},
7     'grid3D': {},
8     'series': [{
9         'type': 'line3D',
10        'data': [[1, 1, 1], [3, 3, 3]],
11    }],
12 })
13
14 ui.run(native=True)
```

3.14.11 ui.pyplot

`ui.pyplot` 的用法像是一个控件，但和一般的控件不太一样，它只是创建了一个上下文环境，用于绘制 [Matplotlib](#) 图形。

控件依赖的 `matplotlib` 默认不安装，如果想要使用此控件，可以在项目根目录下使用以下命令安装：

```
1 pdm add nicegui[matplotlib]
2 #或者下面这条
3 pdm add matplotlib
```

如果运行环境是全局环境或者想用 `pip` 安装，可以执行下面的命令安装：

```
1 pip install nicegui[matplotlib]
2 #或者下面这条
3 pip install matplotlib
```

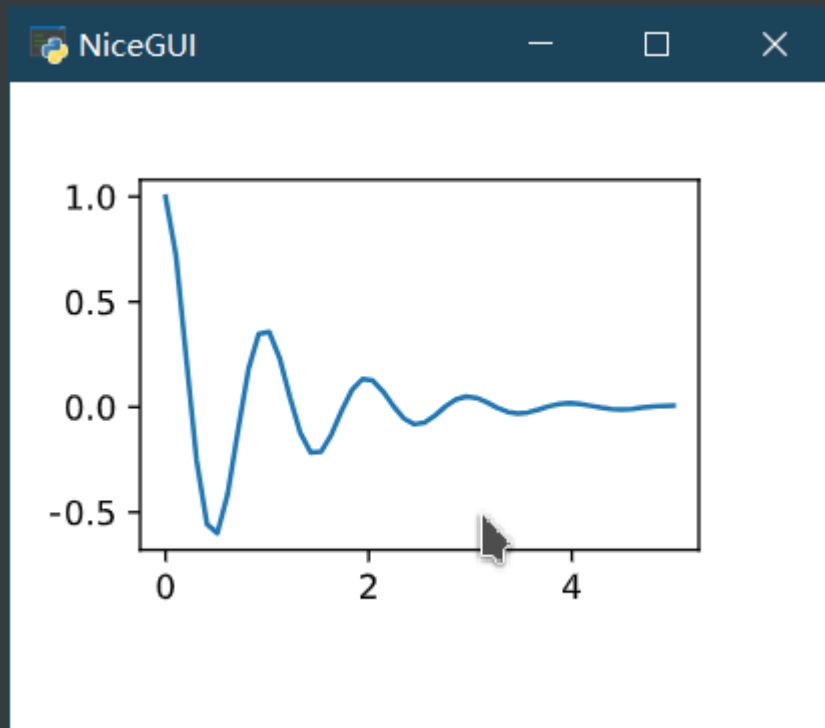
控件支持以下参数：

`close` 参数，布尔类型，表示退出上下文环境之后，是否关闭图形（`figure`，对应控件的 `fig` 属性），如果后续需要更新图形的话，此参数设置为 `False`，默认此参数为 `True`。

`kwargs` 参数，关键字参数类型，用于给 `pyplot.figure` 传入像 `figsize` 之类的参数，更多参数支持可以参考[官网文档](#)。

示例代码如下：

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3 from nicegui import ui
4
5 with ui.pyplot(figsize=(3, 2)):
6     x = np.linspace(0.0, 5.0)
7     y = np.cos(2 * np.pi * x) * np.exp(-x)
8     plt.plot(x, y, '-')
9
10 ui.run(native=True)
```



3.14.12 ui.matplotlib

`ui.matplotlib` 用法接近 `ui.pyplot`，绘图库用的也是 Matplotlib，但少了 `close` 参数，自然不能在退出上下文之后继续更新图形。

控件依赖的 `matplotlib` 默认不安装，如果想要使用此控件，可以在项目根目录下使用以下命令安装：

```
1 pdm add nicegui[matplotlib]
2 #或者下面这条
3 pdm add matplotlib
```

如果运行环境是全局环境或者想用pip安装，可以执行下面的命令安装：

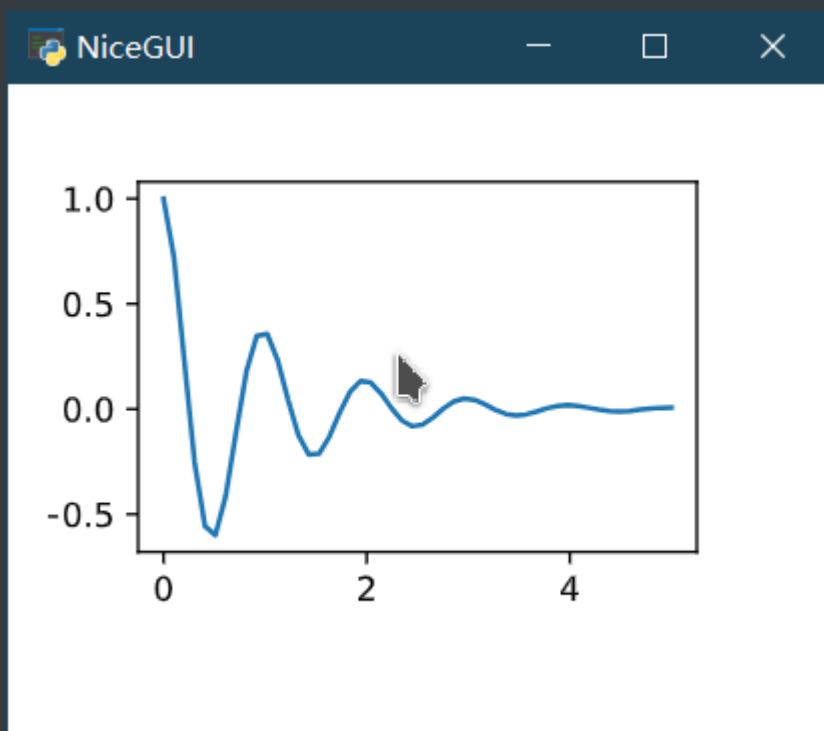
```
1 pip install nicegui[matplotlib]
2 #或者下面这条
3 pip install matplotlib
```

参数只有一个：

`kwargs` 参数，关键字参数类型，用于给 `matplotlib.figure.Figure` 传入像 `figsize` 之类的参数，更多参数支持可以参考[官网文档](#)。

示例代码如下：

```
1 import numpy as np
2 from nicegui import ui
3
4 with ui.matplotlib(figsize=(3, 2)).figure as fig:
5     x = np.linspace(0.0, 5.0)
6     y = np.cos(2 * np.pi * x) * np.exp(-x)
7     ax = fig.gca()
8     ax.plot(x, y, '-')
9
10 ui.run(native=True)
```



3.14.13 ui.line_plot

`ui.line_plot` 可以绘制折线图，使用 `pyplot` 实现，但暴露了一些相关参数，方便创建控件时优化显示效果。

将 `push` 方法与 `ui.timer` 结合，可以做到实时刷新数据。

控件支持以下参数：

`n` 参数，整数类型，图内线条的数量。

`limit` 参数，整数类型，表示每条线支持的最多几个数据点，即如果达到该数量，新的数据点会把最早的数据点顶掉。

`update_every` 参数，整数类型，表示每 `push` 多少次数据才会更新图的显示，用来节省CPU和带宽的占用。

`close` 参数，布尔类型，表示退出上下文环境之后，是否关闭图形（`figure`，对应控件的 `fig` 属性），如果后续需要更新图形的话，此参数设置为 `False`，默认此参数为 `True`。

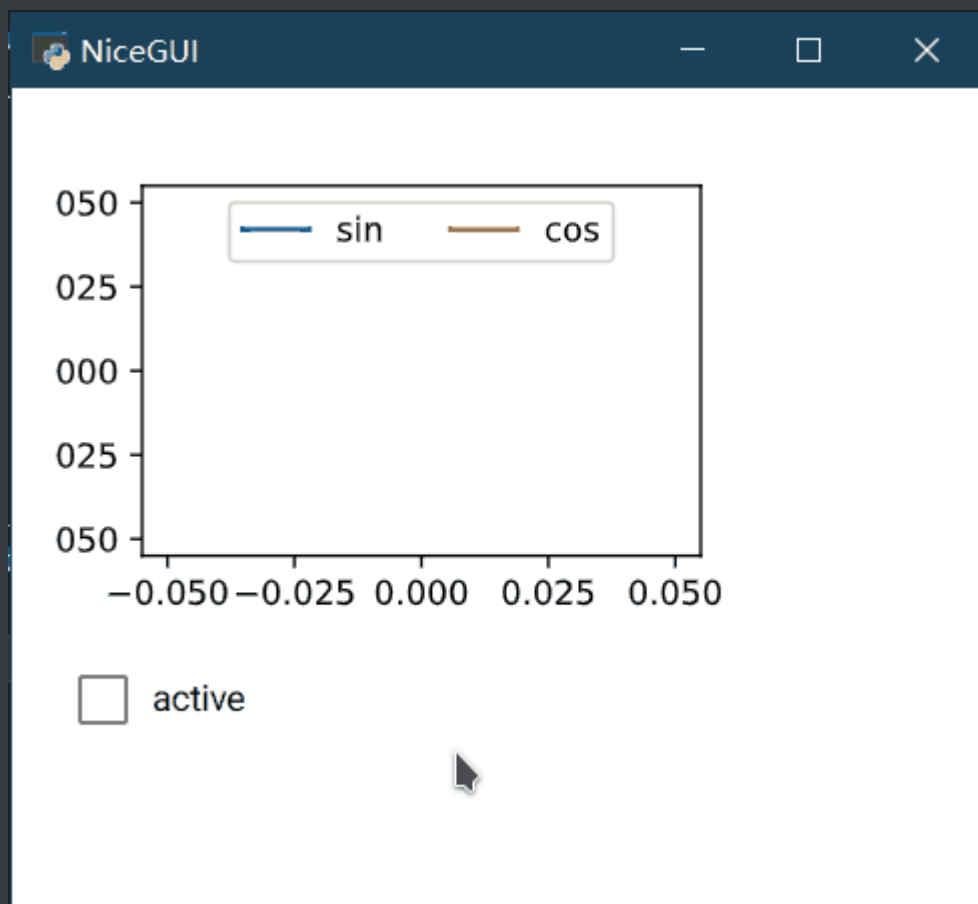
`kwargs` 参数，关键字参数类型，用于给 `pyplot.figure` 传入像 `figsize` 之类的参数，更多参数支持可以参考[官网文档](#)。

示例代码如下：

```

1 import math
2 from datetime import datetime
3 from nicegui import ui
4
5 line_plot = ui.line_plot(n=2, limit=20, figsize=(3, 2), update_every=5) \
6     .with_legend(['sin', 'cos'], loc='upper center', ncol=2)
7
8 def update_line_plot() -> None:
9     now = datetime.now()
10    x = now.timestamp()
11    y1 = math.sin(x)
12    y2 = math.cos(x)
13    line_plot.push([now], [[y1], [y2]])
14
15 line_updates = ui.timer(0.1, update_line_plot, active=False)
16 line_checkbox = ui.checkbox('active').bind_value(line_updates, 'active')
17
18 ui.run(native=True)

```



3.14.14 ui.plotly

ui.plotly 可以绘制Plotly支持的图表，功能实现源自Plotly。

控件依赖的 plotly 默认不安装，如果想要使用此控件，可以在项目根目录下使用以下命令安装：

```
1 pdm add nicegui[plotly]
2 #或者下面这条
3 pdm add plotly
```

如果运行环境是全局环境或者想用pip安装，可以执行下面的命令安装：

```
1 pip install nicegui[plotly]
2 #或者下面这条
3 pip install plotly
```

控件支持以下参数：

figure 参数，字典类型或者 plotly.graph_objects.Figure 类型（即传入该类型的对象），表示想要Plotly渲染的图形。plotly.graph_objects.Figure 类型的用法可以参考[官网文档](#)。字典类型的话，其键（key）就是 'data'（文档需要看图形类型，比如 type 是 scatter 的话，相关文档就是<https://plotly.com/javascript/reference/scatter/>）、'layout'（用法参考[官网文档](#)）、'config'（非必需，用法参考[官网文档](#)）等，具体用法可以参考[官网文档](#)。

如果想要图形的性能好，最好给 figure 参数传入字典类型。

示例代码如下：

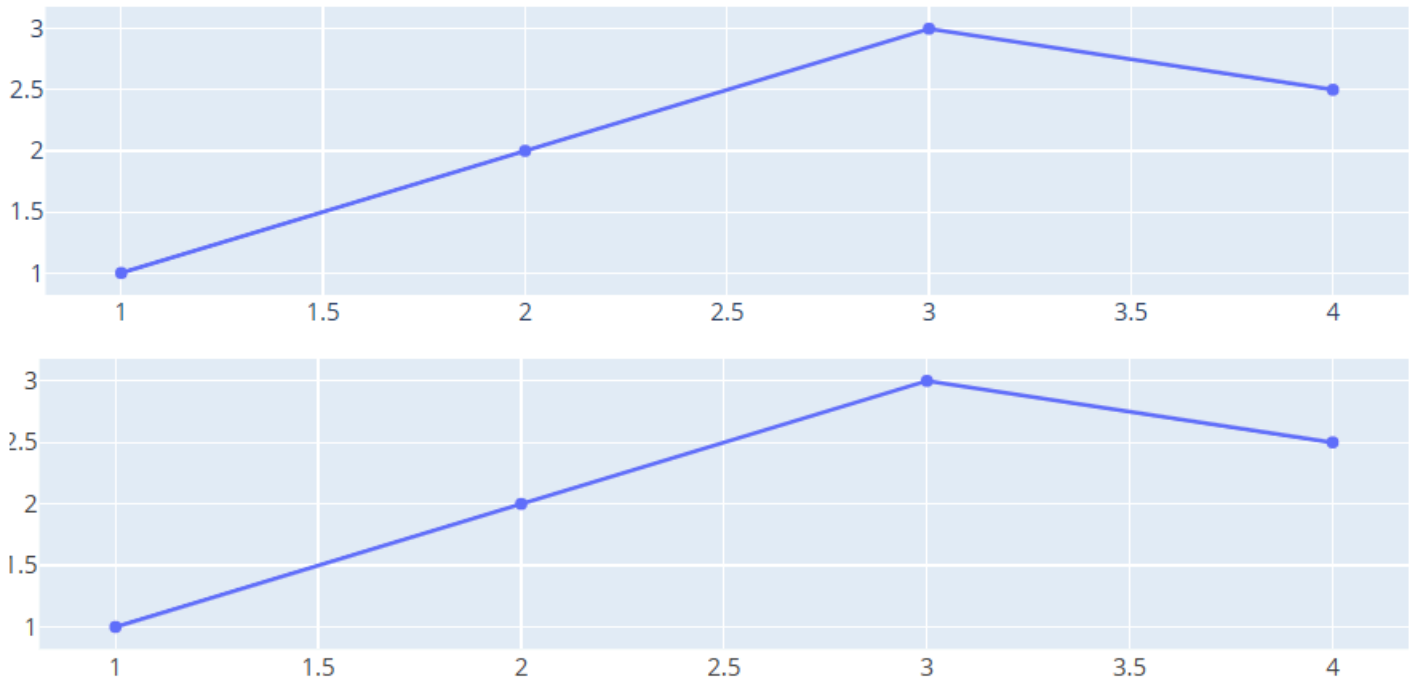
```
1 import plotly.graph_objects as go
2 from nicegui import ui
3
4 fig = go.Figure(go.Scatter(x=[1, 2, 3, 4], y=[1, 2, 3, 2.5]))
5 fig.update_layout(margin=dict(l=0, r=0, t=0, b=0))
6 ui.plotly(fig).classes('w-full h-40')
7
8 fig2 = {
9     'data': [
```

```

10     {
11         'type': 'scatter',
12         'name': 'Trace 1',
13         'x': [1, 2, 3, 4],
14         'y': [1, 2, 3, 2.5],
15         'line': {'color': '#636EFA'},
16     },
17 ],
18 'layout': {
19     'margin': {'l': 15, 'r': 0, 't': 0, 'b': 15},
20     'plot_bgcolor': '#E5ECF6',
21     'xaxis': {'gridcolor': 'white'},
22     'yaxis': {'gridcolor': 'white'},
23 },
24 }
25 ui.plotly(fig2).classes('w-full h-40')
26
27 ui.run(native=True)

```

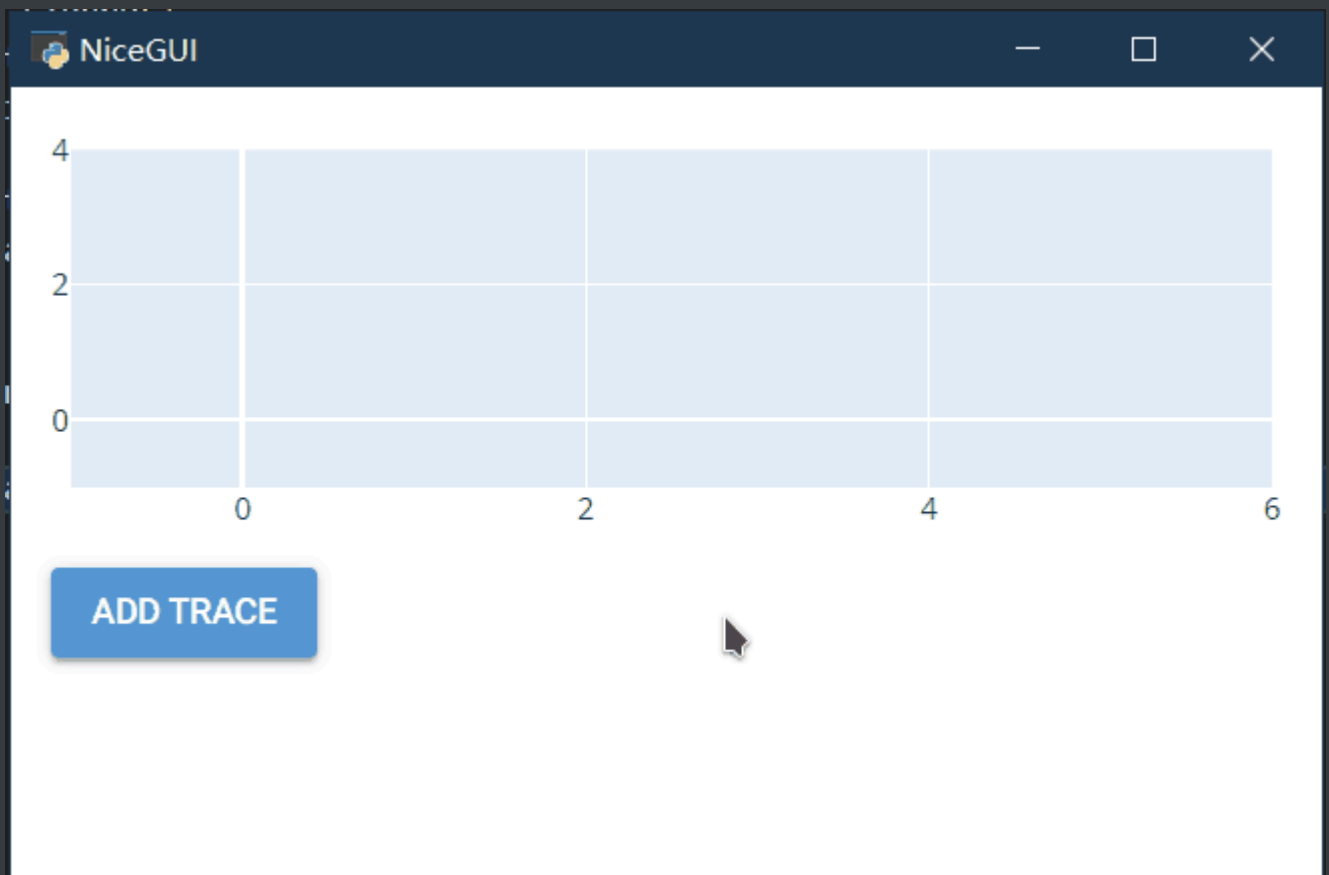
NiceGUI



使用 `plotly.graph_objects.Figure()` 的 `add_trace` 方法 (官网文档) 之后, 需要调用 `plot.update()` 或者 `ui.update(plot)` 来更新显示, 才能让图形更新。

示例代码如下:

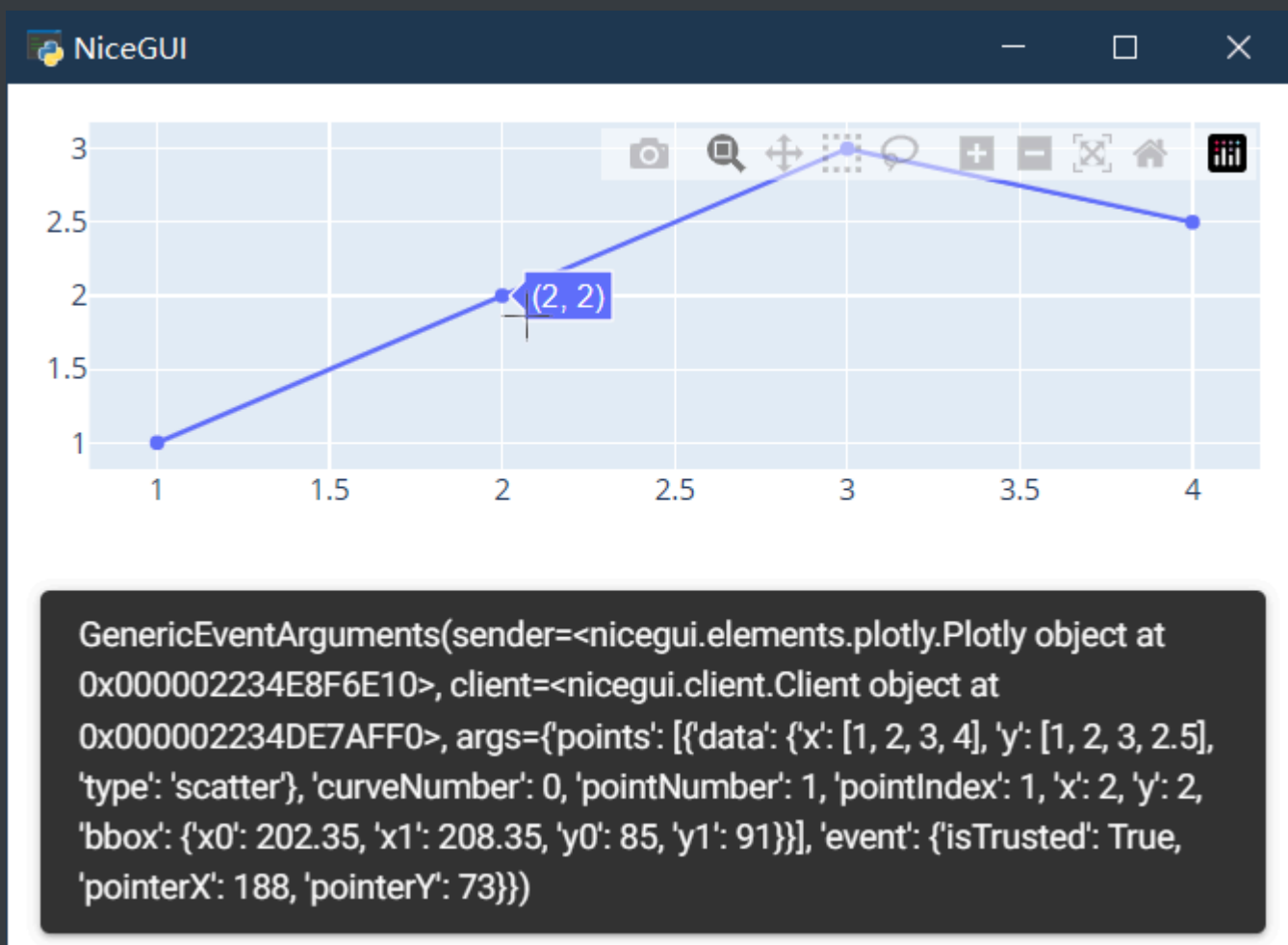
```
1 import plotly.graph_objects as go
2 from nicegui import ui
3 from random import random
4
5 fig = go.Figure()
6 fig.update_layout(margin=dict(l=0, r=0, t=0, b=0))
7 plot = ui.plotly(fig).classes('w-full h-40')
8
9 def add_trace():
10     fig.add_trace(go.Scatter(x=[1, 2, 3], y=[random(), random(), random()])))
11     plot.update()
12
13 ui.button('Add trace', on_click=add_trace)
14
15 ui.run(native=True)
```



使用 on 方法可以监听Plotly的事件 (官网文档) , 目前, ui.plotly 支持监听以下事件: "plotly_click"、"plotly_legendclick"、"plotly_selecting"、"plotly_selected"、"plotly_hover"、"plotly_unhover"、"plotly_legenddoubleclick"、"plotly_restyle"、"plotly_relayout"、"plotly_webglcontextlost"、"plotly_afterplot"、"plotly_autosize"、"plotly_deselect"、"plotly_doubleclick"、"plotly_redraw"、"plotly_animated" 。

示例代码如下:

```
1 import plotly.graph_objects as go
2 from nicegui import ui
3
4 fig = go.Figure(go.Scatter(x=[1, 2, 3, 4], y=[1, 2, 3, 2.5]))
5 fig.update_layout(margin=dict(l=0, r=0, t=0, b=0))
6 plot = ui.plotly(fig).classes('w-full h-40')
7 plot.on('plotly_click', ui.notify)
8
9 ui.run(native=True)
```

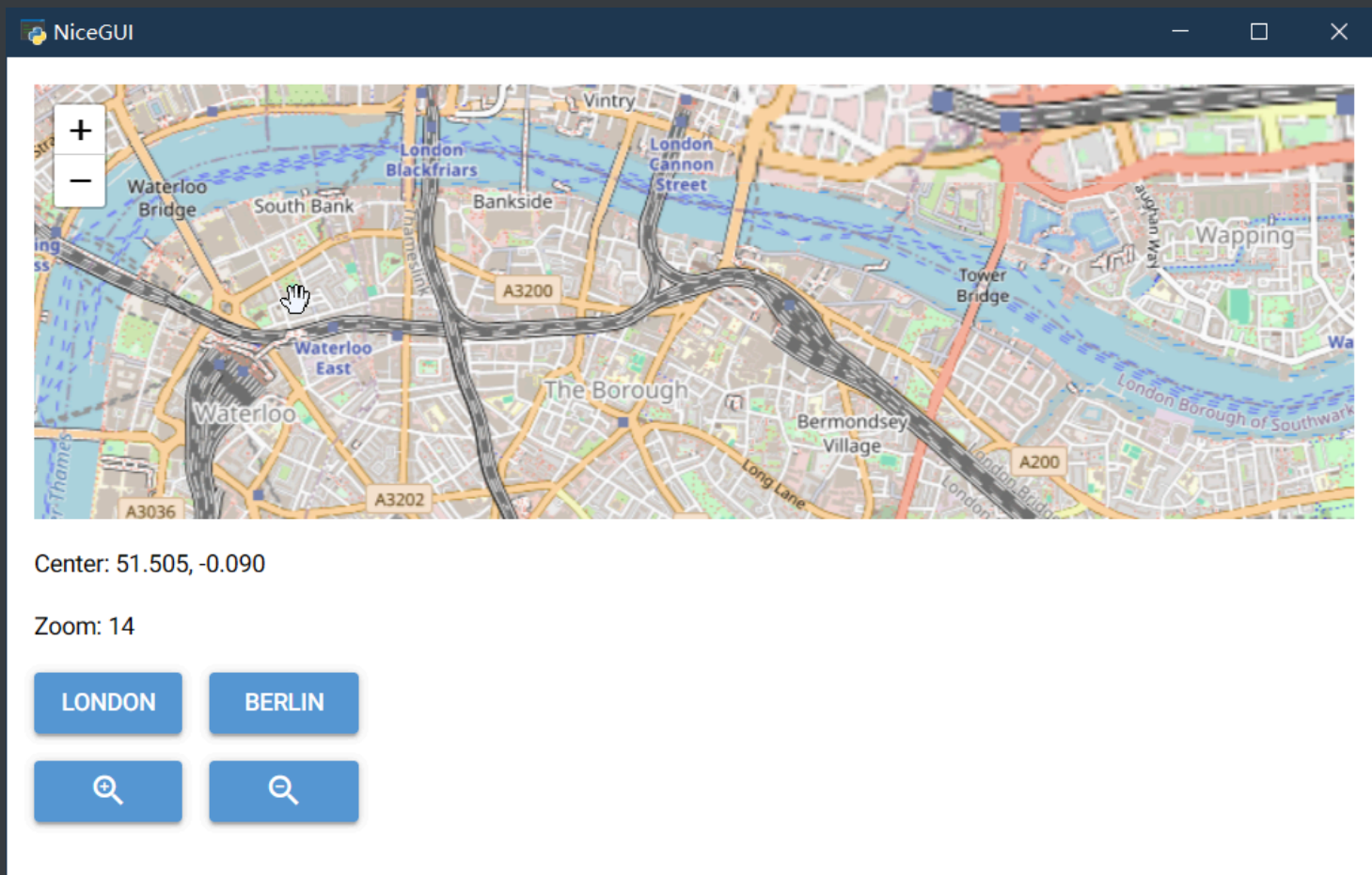


3.14.15 ui.leaflet

`ui.leaflet` 主要生成一个可以交互的地图控件，功能实现源自Leaflet ([官网文档](#))。

先看代码示例：

```
1 from nicegui import ui
2
3 m = ui.leaflet(center=(51.505, -0.09),options=dict(attributionControl=False))
4 ui.label().bind_text_from(m, 'center', lambda center: f'Center: {center[0]:.3f},
   {center[1]:.3f}')
5 ui.label().bind_text_from(m, 'zoom', lambda zoom: f'Zoom: {zoom}')
6
7 with ui.grid(columns=2):
8     ui.button('London', on_click=lambda: m.set_center((51.505, -0.090)))
9     ui.button('Berlin', on_click=lambda: m.set_center((52.520, 13.405)))
10    ui.button(icon='zoom_in', on_click=lambda: m.set_zoom(m.zoom + 1))
11    ui.button(icon='zoom_out', on_click=lambda: m.set_zoom(m.zoom - 1))
12
13 ui.run(native=True)
```



控件支持以下参数：

`center` 参数，浮点元组类型，表示地图默认中心位置的经纬度坐标。元组的第一个元素是纬度，第二个元素是经度，默认值是 $(0.0, 0.0)$ 。

`zoom` 参数，整数类型，地图的放大等级，默认为 13。

`draw_control` 参数，布尔类型或者字典类型，是否显示绘图工具栏（实现源自leaflet插件`leaflet-draw`），默认是 `False`，也可以传入定义绘图工具栏的字典（[官网文档](#)）。

`options` 参数，字典类型，传入插件的额外选项，默认为空字典，即不传入任何额外选项。

`hide_drawn_items` 参数，布尔类型，是否隐藏画在地图上元素，默认为 `False`，即不隐藏。

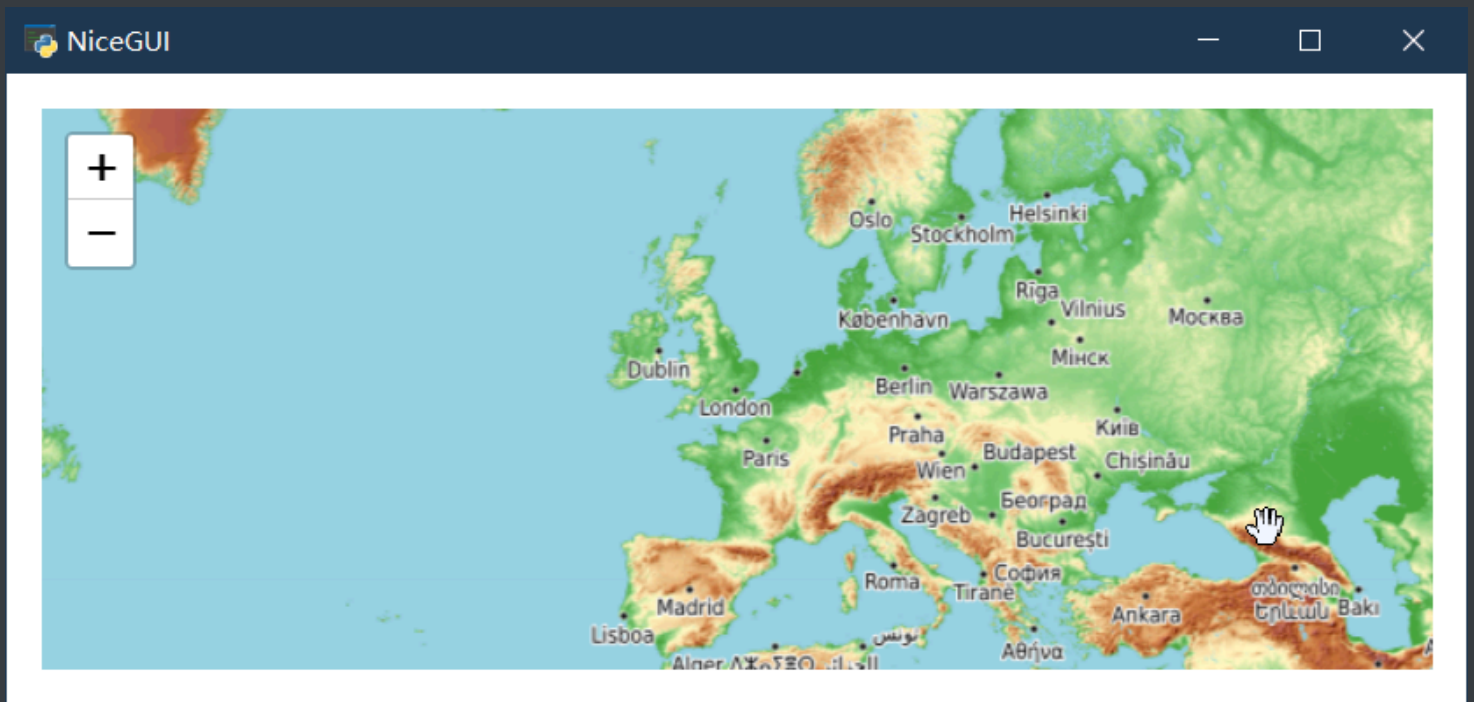
控件默认的地图风格是OpenStreetMap，但可以在[这里](#)找到其他地图风格。每次设置风格，都是将新的 `tile_layer` 图层（[官网文档](#)）叠加上去，所以需要使用 `clear_layers` 方法清除之前的地图图层。

示例代码如下：

```

1 from nicegui import ui
2
3 m = ui.leaflet(center=(51.505, -0.090),
4                 zoom=3,options=dict(attributionControl=False))
5 m.tile_layer(
6     url_template=r'https://{s}.tile.opentopomap.org/{z}/{x}/{y}.png',
7     options={
8         'maxZoom': 17,
9     },
10 )
11
12 ui.run(native=True)

```



使用 `marker` 方法可以在地图上添加一个标记。以下面的代码为例，通过响应地图的点击事件（所有事件的使用说明参考[宜网文档](#)），获取点击位置的经纬度，再将经纬度传给 `marker` 方法的 `latlng` 参数（获取到的是字典，传入的是元组，所以需要转换）。

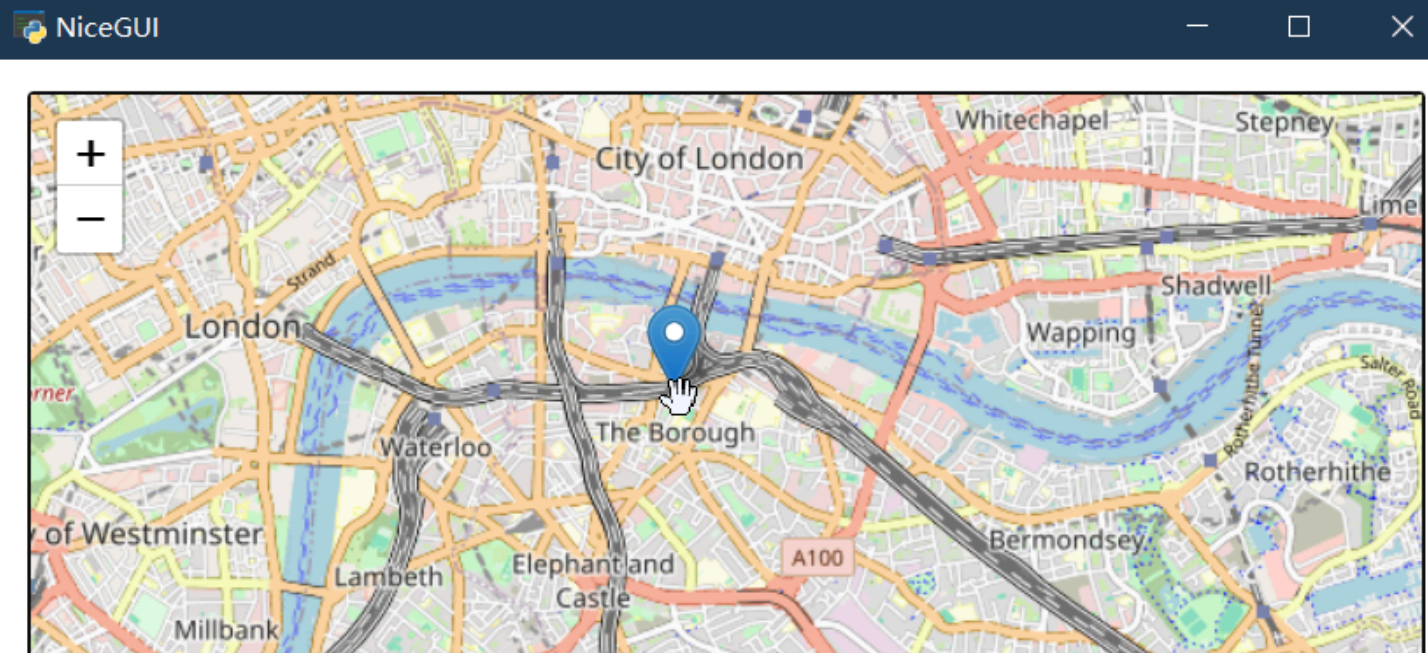
需要注意的是，这里响应地图的点击事件名是 `'map-click'`。因为地图的一些事件名和JavaScript中事件名相同，所以NiceGUI做了处理，增加了`'map-'`前缀用于区分，使得控件依然可以正常响应JavaScript中的同名事件。

示例代码如下：

```

1 from nicegui import events, ui
2
3 m = ui.leaflet(center=(51.505, -0.09),options=dict(attributionControl=False))
4
5 def handle_click(e: events.GenericEventArguments):
6     lat = e.args['latlng']['lat']
7     lng = e.args['latlng']['lng']
8     m.marker(latlng=(lat, lng))
9 m.on('map-click', handle_click)
10
11 ui.run(native=True)

```



marker 方法返回的是 marker 对象， marker 对象的 move 方法可以移动标记位置：

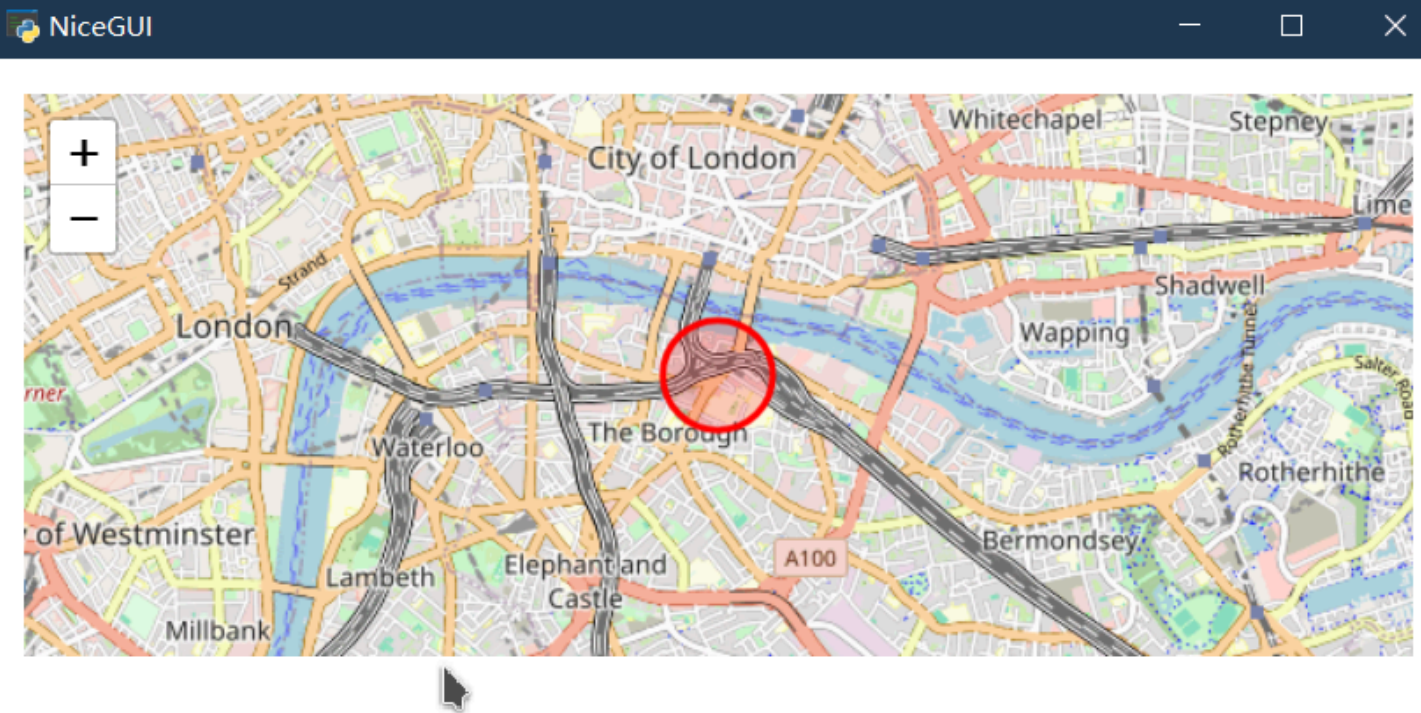
```

1 from nicegui import ui
2
3 m = ui.leaflet(center=(51.505, -0.09),options=dict(attributionControl=False))
4
5 marker = m.marker(latlng=m.center)
6 ui.button('Move marker', on_click=lambda: marker.move(51.51, -0.09))
7
8 ui.run(native=True)

```


使用 `generic_layer` 方法可以将矢量元素 (官网文档) 添加到地图中:

```
1 from nicegui import ui
2
3 m = ui.leaflet(center=(51.505, -0.09),options=dict(attributionControl=False))
4 m.generic_layer(name='circle', args=[m.center, {'color': 'red', 'radius': 300}])
5
6 ui.run(native=True)
```



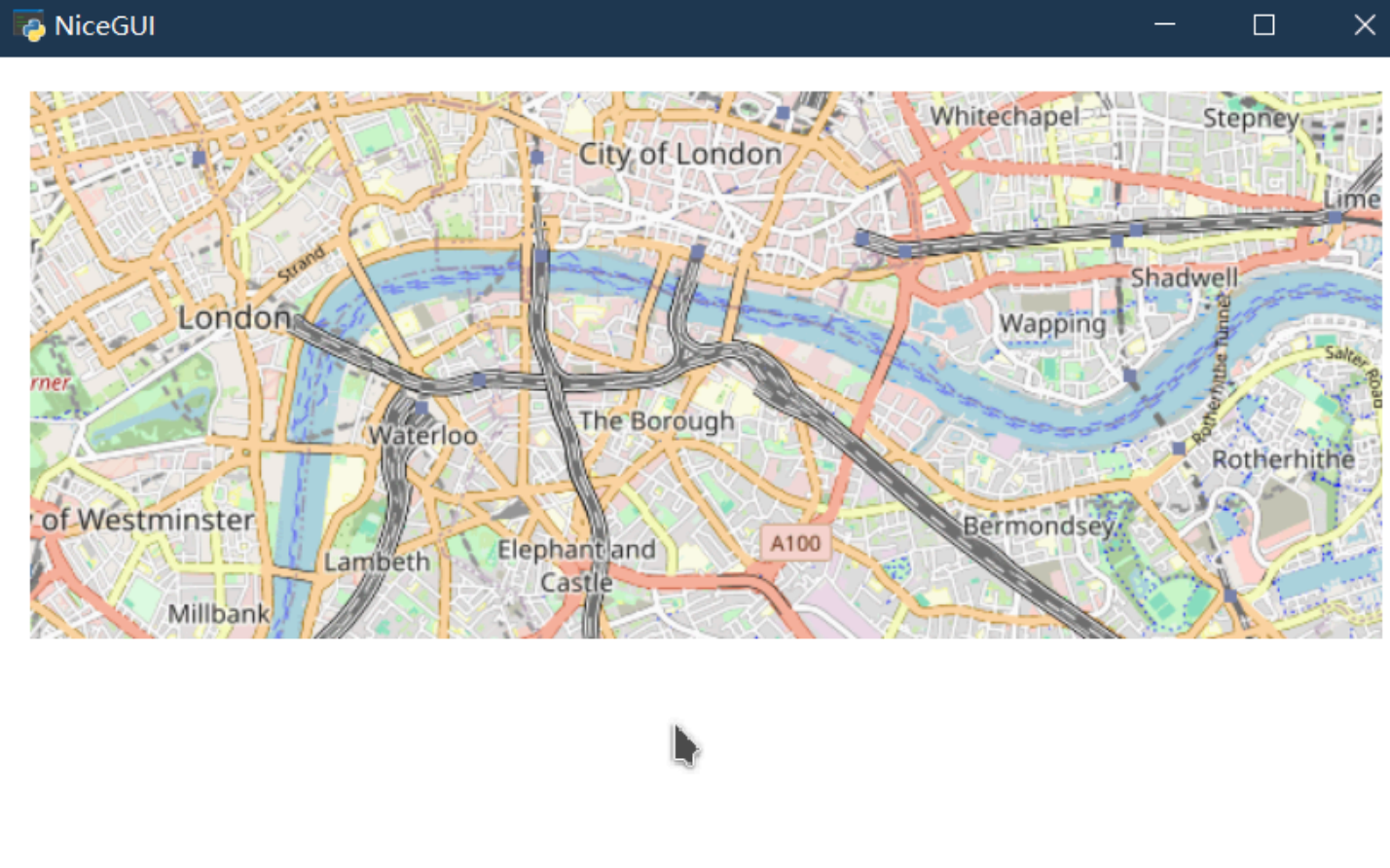
控件还支持配置项 (官网文档) , 可以实现禁止控件中的地图显示某些内容或者禁止交互, 以便于静态展示:

```
1 from nicegui import ui
2
3 options = {
4     'zoomControl': False,
5     'scrollWheelZoom': False,
6     'doubleClickZoom': False,
7     'boxZoom': False,
```

```

8     'keyboard': False,
9     'dragging': False,
10    'attributionControl': False
11 }
12 ui.leaflet(center=(51.505, -0.09), options=options)
13
14 ui.run(native=True)

```



给 `draw_control` 参数传入字典可以启用并配置绘图工具栏（[官网文档](#)），同时，可以监听‘draw:’为前缀的绘图事件（事件名参考[官网文档](#)）来响应绘图操作。

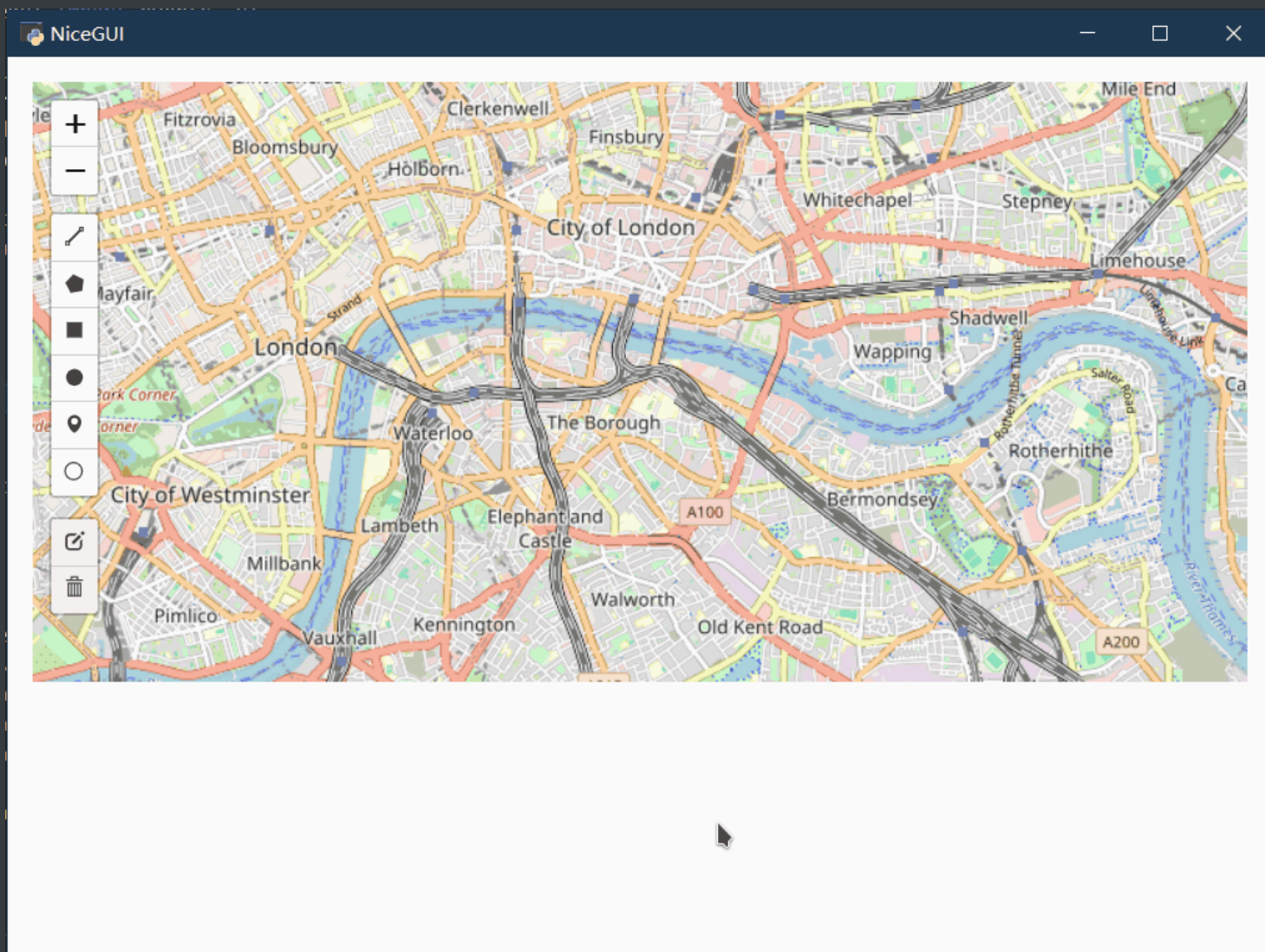
示例代码如下：

```

1 from nicegui import events, ui
2
3 def handle_draw(e: events.GenericEventArguments):
4     layer_type = e.args['layerType']
5     coords = e.args['layer'].get('_latlng') or e.args['layer'].get('_latlngs')
6     ui.notify(f'Drawn a {layer_type} at {coords}')

```

```
7
8 draw_control = {
9     'draw': {
10         'polygon': True,
11         'marker': True,
12         'circle': True,
13         'rectangle': True,
14         'polyline': True,
15         'circlemarker': True,
16     },
17     'edit': {
18         'edit': True,
19         'remove': True,
20     },
21 }
22 m = ui.leaflet(center=(51.505, -0.09),
23                 draw_control=draw_control,options=dict(attributionControl=False))
24 m.classes('h-96')
25 m.on('draw:created', handle_draw)
26 m.on('draw:edited', lambda: ui.notify('Edit completed'))
27 m.on('draw:deleted', lambda: ui.notify('Delete completed'))
28 ui.run(native=True)
```

可以响应绘图事件 'draw:created' 的同时，使用图形的点位数据（`['layer']['_latlngs']`）和自定义的样式选项（[官网文档](#)）绘制同样形状、不同样式的图形。不过，这样绘制出来的图会和默认的绘图重叠，需要将 `hide_drawn_items` 参数设置为 `True`，来隐藏默认绘图。

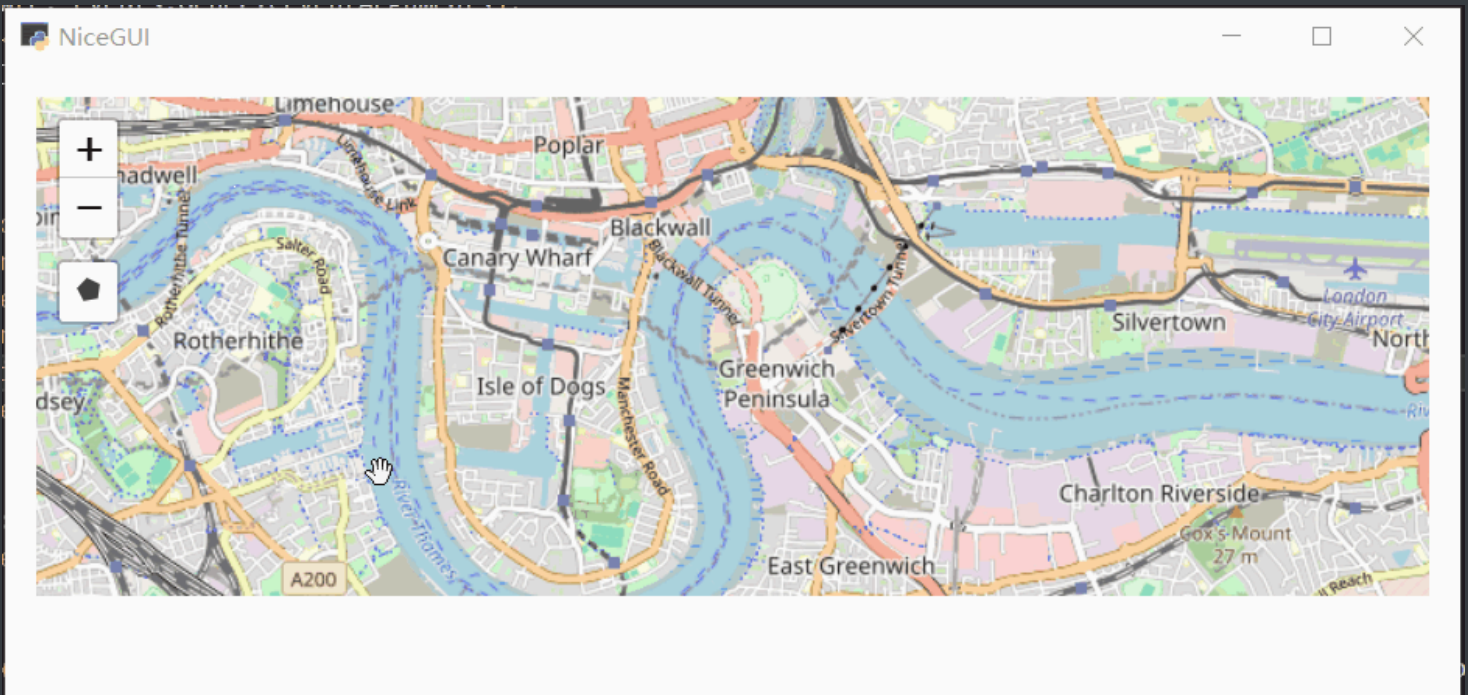
示例代码如下：

```
1 from nicegui import events, ui
2
3 def handle_draw(e: events.GenericEventArguments):
4     options = {'color': 'red', 'weight': 1}
5     m.generic_layer(name='polygon', args=[e.args['layer']['_latlngs'], options])
6
7 draw_control = {
8     'draw': {
9         'polygon': True,
10        'marker': False,
```

```

11         'circle': False,
12         'rectangle': False,
13         'polyline': False,
14         'circlemarker': False,
15     },
16     'edit': {
17         'edit': False,
18         'remove': False,
19     },
20 }
21 m = ui.leaflet(center=(51.5, 0), draw_control=draw_control,
22               hide_drawn_items=True, options=dict(attributionControl=False))
23 m.on('draw:created', handle_draw)
24 ui.run(native=True)

```



控件的 `run_map_method` 方法可以运行地图对象支持的方法（[官网文档](#)），示例代码如下：

```

1  from nicegui import ui
2
3  m = ui.leaflet(center=(51.5, 0),options=dict(attributionControl=False))
4  ui.button('Fit world', on_click=lambda: m.run_map_method('fitWorld'))
5
6  ui.run(native=True)

```

运行 `run_layer_method` 方法，可以执行指定图层的接口方法。该方法的第一个参数是图层 `id`，第二个参数是接口方法名（详见[官网文档](#)），后面的参数是传给接口参数的参数。不过，需要注意的是，该方法与客户端数据相关，只能在 `ui.page` 内执行，不能在 `auto-index` 页面内使用。

以下面的代码为例，在地图控件完成初始化之后，先设置默认 `attribution` 的前缀，然后清除现有的 `attribution`。点击 'Add Attribution' 按钮，可以调用接口方法（[官网文档](#)）增加一个 `attribution`，与先前 `options` 里设置的 `attribution` 合并，显示为两个 `attribution`。但是，`layer` 接口方法 `'getAttribution'`（[官网文档](#)）获取的 `attribution` 是 `options` 里设置的 `attribution`，因此最后弹出的通知只显示 `options` 里设置的 `attribution`。

需要额外注意的是，调用任何接口方法都要等地图完成初始化之后，可以使用异步等待控件的 `initialized` 方法，就像下面代码中的 `await m.initialized()`，就是为了确保地图确实完成了初始化，而添加的异步等待。

示例代码如下：

```

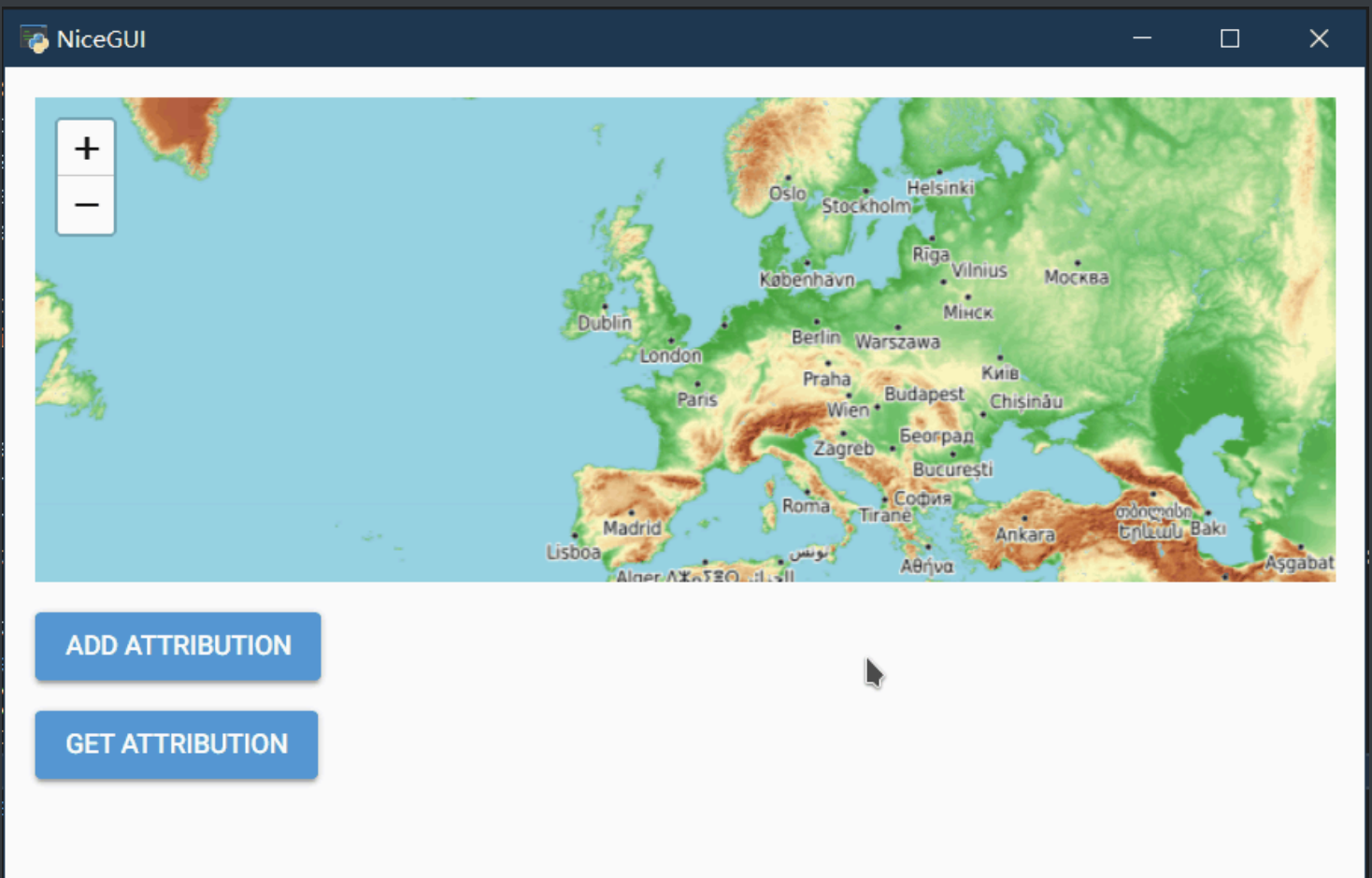
1  from nicegui import ui
2
3  @ui.page('/')
4  async def index():
5      m = ui.leaflet(center=(51.505, -0.090), zoom=3)
6      m.clear_layers()
7      layer = m.tile_layer(
8          url_template=r'https://{s}.tile.opentopomap.org/{z}/{x}/{y}.png',
9          options={
10              'maxZoom': 17,
11              'attribution': 'test1'
12          },
13      )
14      await m.initialized()

```

```

15     m.run_map_method('m => m.attributionControl.setPrefix("Psf")')
16     m.run_map_method('m => m.attributionControl.remove()')
17     ui.button('Add Attribution', on_click=lambda
:m.run_map_method(':addControl',"L.control.attribution({prefix:'Psf'}).addAttribution
('test2')"))
18
19     async def handle_layer(e):
20         result = await m.run_layer_method(layer.id, 'getAttribution')
21         ui.notify(result)
22     ui.button('Get Attribution', on_click=handle_layer)
23
24 ui.run(native=True)

```

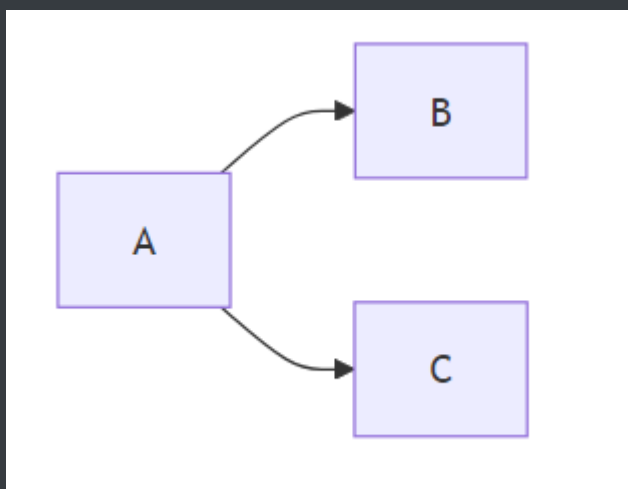


3.14.16 ui.mermaid (2025.03.23更新)

ui.mermaid 主要用于展示流程图，功能实现源自Mermaid (官网文档)。

先看代码示例：

```
1 from nicegui import ui
2
3 ui.mermaid(content='''
4 graph LR;
5     A --> B;
6     A --> C;
7 ''')
8
9 ui.run(native=True)
```



控件支持以下参数：

`content` 参数，字符串类型，表示描述流程图的字符串（语法规则参考[语法文档](#)）。

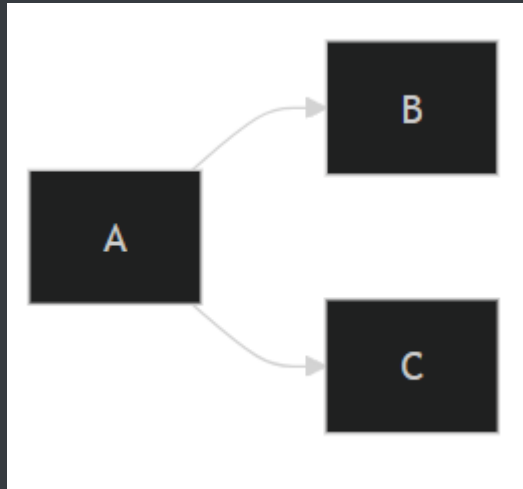
`config` 参数，字典类型，传入插件的配置（支持的配置参考[官网文档](#)），默认为 `None`，即不传入任何配置。

对于控件来说，描述流程图的字符串需要学习语法文档，在Python这边只需使用多行字符串即可。要说有点格式要求的，也就是给控件传入插件配置。比如，配置流程图的主题：


```

1  from nicegui import ui
2
3  ui.mermaid(content='''
4  graph LR;
5      A --> B;
6      A --> C;
7  ''',config={'theme':'dark'})
8
9  ui.run(native=True)

```



如果想要让Mermaid支持点击之后的交互动作，可以使用下面的语法：

```

1  #点击调用JavaScript函数
2  click taskId call callback(arguments)
3  #点击访问指定链接
4  click taskId href URL

```

注意，因为Mermaid一般是用来展示流程图，不是用来交互的，因此默认的安全等级不允许执行交互，需要设置安全等级为宽松 `{'securityLevel': 'loose'}`（完整用法参考[官网文档](#)）。

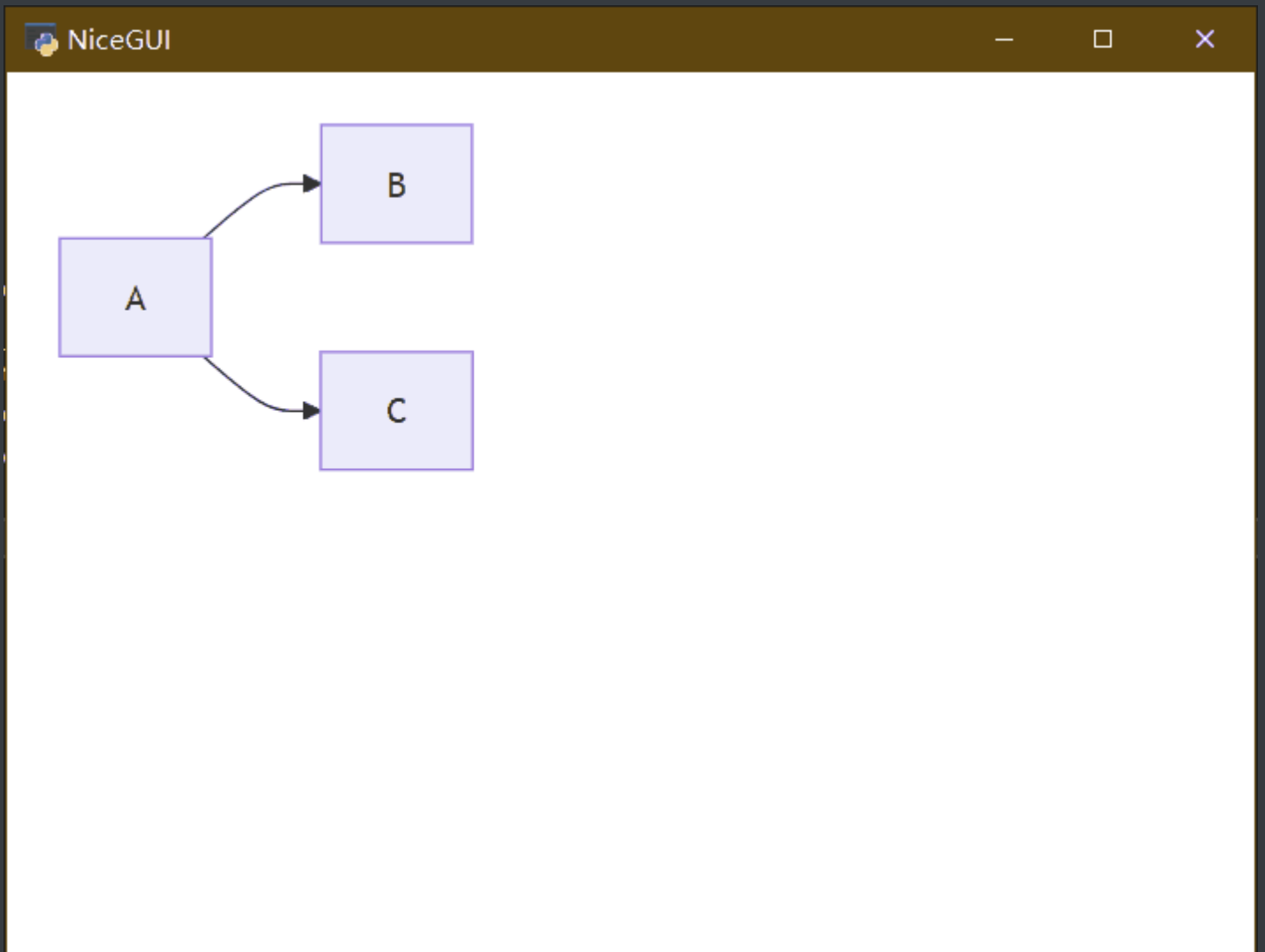
如果想要在Python中响应Mermaid的点击事件，可以在Mermaid中使用JavaScript的 `emitEvent` 方法注册一个JavaScript事件，然后在Python中使用 `ui.on` 方法监听Mermaid中注册的事件：

```

1  from nicegui import ui
2
3  ui.mermaid(content='''
4  graph LR;

```

```
5     A --> B;
6     A --> C;
7     click A call emitEvent("mermaid_click", "You clicked A!");
8     click B href "https://nicegui.io/";
9     click C call confirm("You clicked C!");
10  '', config={'securityLevel': 'loose'})
11
12  ui.on('mermaid_click', lambda e: ui.notify(e.args))
13
14  ui.run(native=True)
```

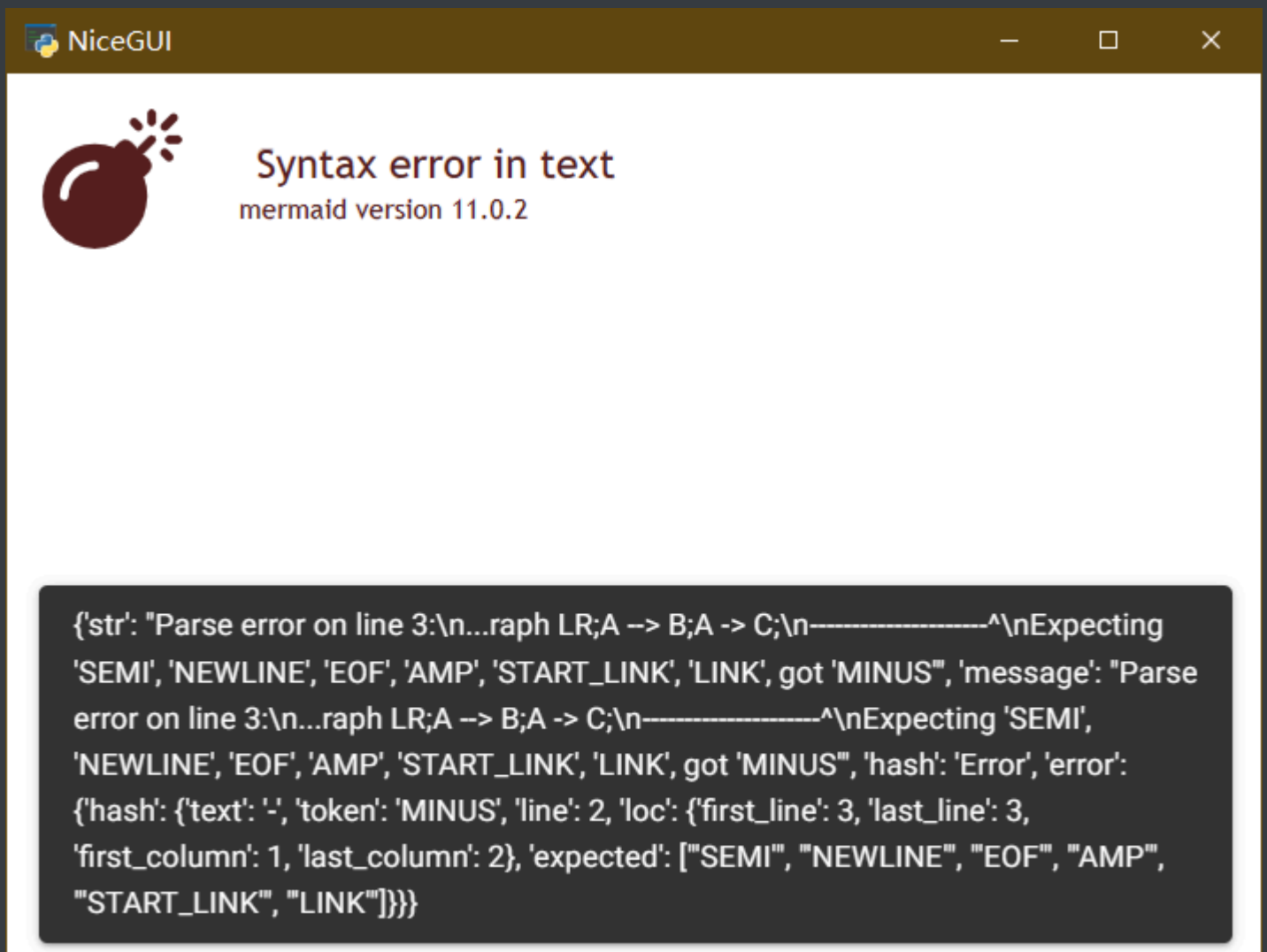


当然，控件本身也支持一个 `error` 事件，当描述字符串发生语法错误时，会触发此事件：

```

1 from nicegui import ui
2
3 ui.mermaid(content='''
4 graph LR;
5     A --> B;
6     A -> C;
7 ''', config={'securityLevel': 'loose'}).on(
8     type='error',
9     handler=lambda e: ui.notify(e.args))
10
11 ui.run(native=True)

```



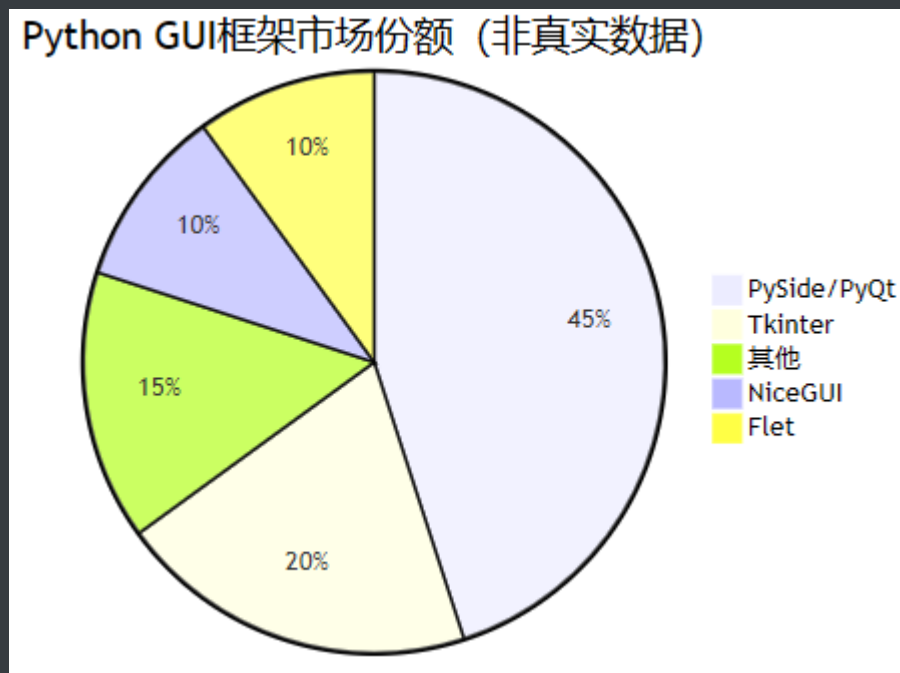
事件参数的 `args` 属性字典映射了对应JavaScript事件的以下属性（完整介绍参考[官网文档](#)）：

- `error` 属性。
- `str` 属性。

- hash 属性。
- message 属性。

当然，除了流程图，Mermaid也可以绘制其他类型的图，比如饼状图（完整用法参考[官网文档](#)）：

```
1 from nicegui import ui
2
3 ui.mermaid(content=''
4 pie title Python GUI框架市场份额（非真实数据）
5     "PySide/PyQt" : 45
6     "Tkinter" : 20
7     "NiceGUI" : 10
8     "Flet" : 10
9     "其他" : 15
10 '').classes('w-96 h-96')
11
12 ui.run(native=True)
```



3.15 多任务与异步的技巧 (2025.01.08更新)

3.15.1 异步 (2025.01.08更新)

3.15.1.1 异步支持

什么是异步？为什么要用异步？

与异步相对的是同步，平常在Python中定义、执行的函数，没有 `async` 关键字修饰，这种函数就是同步函数。同步函数的特点是，函数中所有执行的过程都是依次进行，哪怕有的操作比较费时，也要等待其执行完毕，才能响应新的代码。异步则不同，如果代码中有耗时的操作，可以使用 `await` 关键字标明耗时的操作，这样程序就不会一直等待这些操作的执行结果，可以在等待期间响应新的操作。而耗时的操作也会同时进行着，不会出现同步函数中卡死的情况。

从上面的区别可以看出，面对耗时较久或者不能立刻给出结果的操作，异步可以极大提升程序的处理效率，不容易进入卡死状态，让程序更高效。

只是看文字说明没有直观的感受，那就看一下相似代码的同步、异步的效果。

相信不少读者在初学Python的时候学过这样的操作：使用 `time.sleep` 来模拟一些代码执行的过程。虽然Python很慢，但实际执行的时候也不是执行几十秒，所以，使用 `time.sleep` 睡眠一段时间，可以让开发者明显感受到代码执行的过程，更方便调试代码。然而，在NiceGUI中，是绝对不能这样来模拟耗时操作的，比如下面示例中的操作：

```
1 import time
2 from nicegui import ui
3
4 def task():
5     ui.notify('task started')
6     time.sleep(5)
7     ui.notify('task finished')
8
9 ui.button('start task', on_click=task)
10
11 ui.run(native=True)
```

看起来示例里想要模拟一个耗时操作，让程序在耗时操作开始时、结束时分别发送一次通知。这种情况在开发中很常见，程序往往为了避免耗时操作卡住用户的其他响应，会把耗时操作放到后台，并在完成时通知用户。想法没问题，代码看上去也没问题，可实际执行的时候就有问题：程序没有按照预期显示通知，甚至还出现异常的掉线情况。

可能有的读者会怀疑代码本身有问题，那这样做个验证：注释掉 `time.sleep(5)`，看看程序能否正常执行：

```
1 import time
2 from nicegui import ui
3
4 def task():
5     ui.notify('task started')
6     #time.sleep(5)
7     ui.notify('task finished')
8
9 ui.button('start task', on_click=task)
10
11 ui.run(native=True)
```

实际上代码没有问题，注释掉休眠的代码之后，其余的代码正常执行。这是因为 `time.sleep` 是在主线程上执行的，这个休眠操作会阻塞主线程，让负责ui逻辑的主线程被休眠卡住，进而导致界面失去响应。所以，如果想要避免这样的问题，还想模拟耗时操作的话，要用异步休眠来模拟：`asyncio.sleep` 就是异步版本的 `time.sleep`。

替换休眠操作之后，代码如下：

```

1  import asyncio
2  from nicegui import ui
3
4  async def task():
5      ui.notify('task started')
6      await asyncio.sleep(5)
7      ui.notify('task finished')
8
9  ui.button('start task', on_click=task)
10
11 ui.run(native=True)

```

当然，既然是异步休眠，自然要用 `await` 关键字告诉程序“等待”这个耗时的操作。`await` 只能在异步函数中使用，因此函数在定义时，需要用 `async` 修饰。这样修改之后，按钮就可以连续点击了，也不会导致异常或者卡死界面。

实际上，NiceGUI很多控件的响应操作（比如按钮的 `on_click`）都支持异步函数，如果开发中遇到耗时较久或者不能立刻执行完毕的情况，可以先创建好异步函数，让响应操作执行这个异步函数。

3.15.1.2 `ui.clipboard` 和剪贴板

按理来说剪贴板应该早点介绍的，拖到这个时候才说有点晚。不过事出有因，不介绍异步的话，剪贴板是不能正常使用的。因为剪贴板读取的操作就是异步操作，没法立刻得到结果。

异步操作的结果不像同步结果那样可以立即获得，异步操作的返回值是一个可等待对象，必须用 `await` 修饰，等待操作完成，才能得到结果。

读取剪贴板的操作是 `ui.clipboard.read()`，本质上是NiceGUI执行了JavaScript代码。几乎所有的JavaScript代码都是异步的，因此，要想正确读取，也必须将此操作放到异步函数内，并用 `await` 修饰，比如：

```

1  async def read() -> None:
2      ui.notify(await ui.clipboard.read())

```

另外需要注意的是，不同用户的剪贴板不能互通，读写剪贴板的操作只能在确定的客户端上执行，因此读写剪贴板的操作只能在 `ui.page` 中使用。并且读写剪贴板的操作要求安全上下文（即当前网址为本地域名、ip，或者网络连接的协议是HTTPS协议）。

下面的示例代码中，`ui.page` 装饰的函数是个异步函数，其实并非必须的，只是第一次读取剪贴板的时候会弹出请求剪贴板的弹窗，如果不使用异步函数，第一次允许时会导致终端报错，但不影响读取；如果是异步函数，函数就会等待这个允许操作，终端不会报错。

```
1 from nicegui import ui
2
3 @ui.page('/')
4 async def index():
5     ui.button('Write', on_click=lambda: ui.clipboard.write('Hi!'))
6
7     async def read() -> None:
8         ui.notify(await ui.clipboard.read())
9     ui.button('Read', on_click=read)
10
11 ui.run(native=True)
```

在Python中执行读写剪贴板的操作会让服务器执行相关代码，难免给服务器添加额外的压力。这时可以使用JavaScript的接口读写剪贴板，这样的操作就是由客户端完成，可以减小服务器的压力。当然，JavaScript中同样需要异步读取，所以，JavaScript的实现会复杂一点：

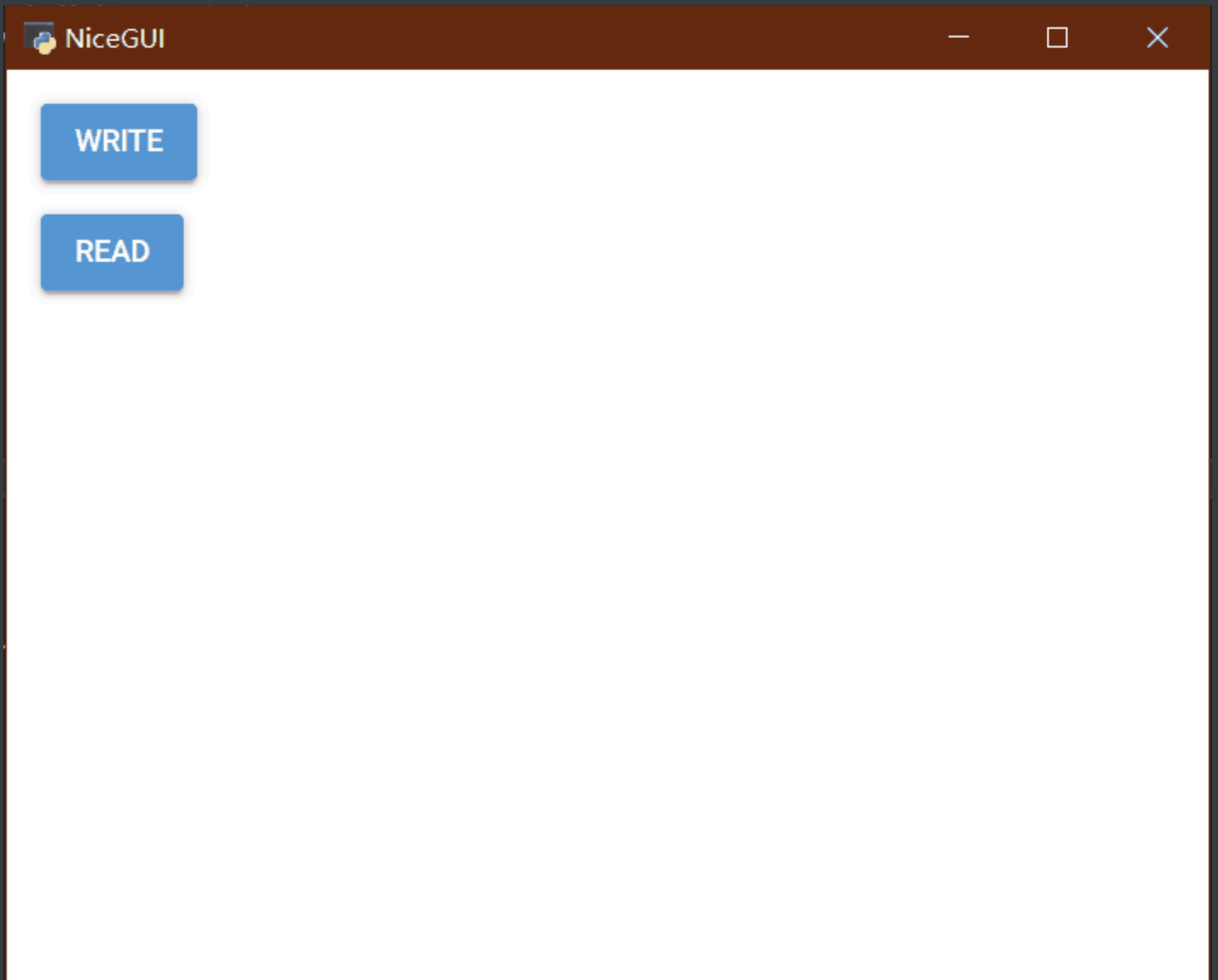
```
1 from nicegui import ui
2
3 ui.button('Write').on('click', js_handler='''
4     () => navigator.clipboard.writeText("Ho!")
5 ''')
6 ui.button('Read').on('click', js_handler='''
7     async () => emitEvent("clipboard", await navigator.clipboard.readText())
8 ''')
9 ui.on('clipboard', lambda e: ui.notify(e.args))
10
11 ui.run(native=True)
```

代码中，由JavaScript代码发射一个 "clipboard" 事件，并把读取结果通过 `args` 属性返回Python代码；Python代码监听 "clipboard" 事件，并接收返回的值，完成剪贴板的读取。

NiceGUI官方在2.10.0版本新增了从剪贴板读取图片的功能——`ui.clipboard.read_image` 方法。该方法依赖 `pillow`，想要使用该功能，需要提前安装此模块。示例代码如下：

```
1 from nicegui import ui
2
3 @ui.page('/')
4 async def index():
5     ui.button('Write', on_click=lambda: ui.clipboard.write('Hi!'))
6
7     async def read() -> None:
8         img = await ui.clipboard.read_image()
9         if img:
10             with ui.dialog() as dialog:
11                 with ui.column().classes('w-72 items-center'):
12                     ui.image(img)
13                     ui.button('关闭', on_click=dialog.close)
14             dialog.open()
15         else:
16             ui.notify(await ui.clipboard.read())
17     ui.button('Read', on_click=read)
18
19 ui.run(native=True)
```

如果当前剪贴板保存的是图片，点击"Read"按钮会弹出对话框，显示出该图片。



3.15.2 后台任务（2025.01.08更新）

前面说使用 `time.sleep` 会阻塞主线程，所以要用异步，但如果是后台任务，不在主线程上运行，那就没问题了。

NiceGUI提供了两种后台执行任务的方法，由 `run` 子模块提供：

- `run.cpu_bound` 方法，常用于占用较多CPU资源的后台任务，该方法会创建新的进程操作，让进程池的进程数扩大。
- `run.io_bound` 方法，常用于占用较多IO资源的后台任务开，因为这类后台任务不会占用太多CPU资源，因此，该方法只是创建新的线程操作，操作完线程会关闭。

下面的示例使用 `time.sleep` 模拟耗时的操作，但将其放到后台任务中，所以不会卡死主线程：

```
1 from nicegui import ui, run
```

```

2  import time
3
4  def test_task(t):
5      time.sleep(t)
6      return 0
7
8  async def sub_task():
9      #单独进程
10     result = await run.cpu_bound(test_task,5)
11     #单独线程
12     #result = await run.io_bound(test_task,5)
13     ui.notify(result)
14
15 ui.button('background task',on_click=sub_task)
16
17 ui.run(native=True)

```

3.16 版本亮点（随版本更新）

为了方便查找对应版本号添加的有趣功能，本节将版本号前置，并取消了章节序号。

2.9.0版本新增： app.timer ——app级别的定时器

NiceGUI官方在2.9.0版本新增了 app.timer 定时器，虽然用法上和 ui.timer 一样，但其归属于 app 而不是 ui，还是有所区别的。

为了理解区别，需要先运行以下示例代码：

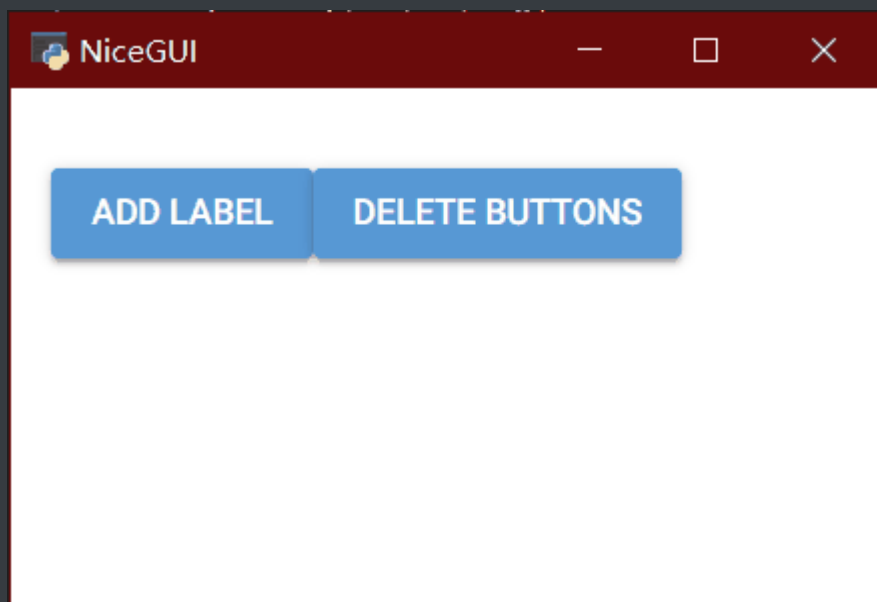
```

1  from nicegui import app, ui
2
3  counter = {'value': 0}
4  label = ui.element()
5  timer = None
6
7  with ui.element() as buttons:
8      button1 = ui.button('add label')
9      button2 = ui.button('delete buttons')
10

```



```
11 def add_label():
12     timer = ui.timer(1,lambda :counter.update(value=counter['value']+1))
13     with label:
14         ui.label().bind_text_from(counter, 'value', lambda value: f'Count: {value}')
15
16 def delete_buttons():
17     buttons.clear()
18
19 button1.on_click(add_label)
20 button2.on_click(delete_buttons)
21
22 ui.run(native=True)
```



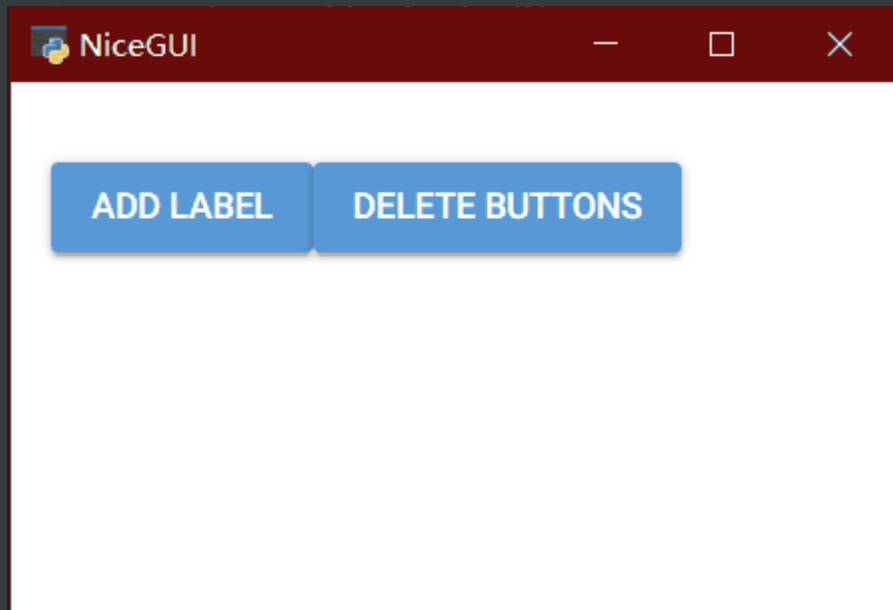
示例代码源于NiceGUI官方仓库的一个问题，这里稍微简化了一下。问题作者想要让按钮创建一个定时更新显示内容的定时器，然后用另一个按钮删掉创建定时器的按钮。就是这样听起来很简单的操作，结果在删掉按钮时，工作定时器好像被一并“删掉”了。导致删掉按钮之后，原本应该继续执行的显示更新操作随之停止了。

听起来很奇怪，像是一个问题，其实不是，一开始就没有必要让按钮创建定时器。定时器可以在按钮的响应函数之外创建，按钮只需启动（activate）、停止（deactivate）定时器即可。因为定时器（`ui.timer`）会自动关联创建定时器的UI组件，一般做法是在auto-index页创建定时器，定时器关联了auto-index页，而auto-index页一般不会被删掉（也不能删掉，会出问题），所以使用定时器不会出问题。如果是其他UI组件创建了定时器，删掉创建定时器的UI组件，同时会一并删掉定时器，这也就是问题的原因。

上面的示例代码更换成常规用法也可以，解决方法也不难，不过，NiceGUI官方还是为此增加了一个独立于UI组件的定时器——`app.timer`，既是对此问题的解决方案，也是对后续有类似需求的功能实现。

那么，上面的代码在基本不动的前提下，只需将 `ui.timer` 换成 `app.timer` 即可：

```
1 from nicegui import app, ui
2
3 counter = {'value': 0}
4 label = ui.element()
5 timer = None
6
7 with ui.element() as buttons:
8     button1 = ui.button('add label')
9     button2 = ui.button('delete buttons')
10
11 def add_label():
12     timer = app.timer(1, lambda :counter.update(value=counter['value']+1))
13     with label:
14         ui.label().bind_text_from(counter, 'value', lambda value: f'Count: {value}')
15
16 def delete_buttons():
17     buttons.clear()
18
19 button1.on_click(add_label)
20 button2.on_click(delete_buttons)
21
22 ui.run(native=True)
```



当然，有了 `app.timer` 这种独立于UI组件的定时器之后，以前那种将 `ui.timer` 放在 `ui.page` 之外，用来充当独立于页面的定时器的方法，就可以使用 `app.timer` 了：

```
1 from nicegui import app, ui
2
3 counter = {'value': 0}
4 app.timer(1.0, lambda: counter.update(value=counter['value'] + 1))
5
6 @ui.page('/')
7 def page():
8     ui.label().bind_text_from(counter, 'value', lambda value: f'Count: {value}')
9
10 ui.run()
```

两种定时器在这种情景下都不会出问题，这里替换为新的定时器更多是为了区分定时器的作用范围。

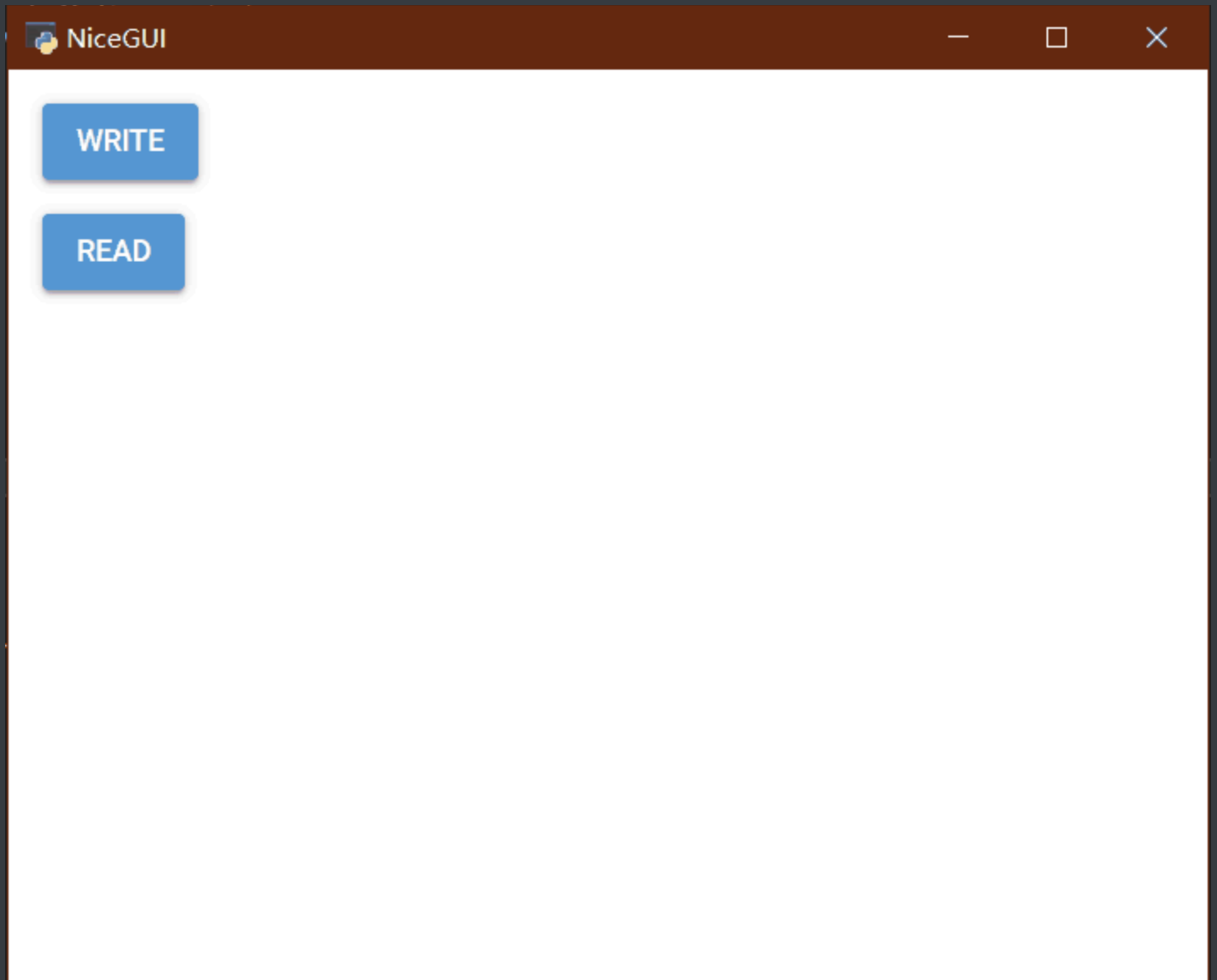
2.10.0版本新增： `ui.clipboard.read_image` ——从剪贴板读取图片

NiceGUI官方在2.10.0版本新增了从剪贴板读取图片的功能—— `ui.clipboard.read_image` 方法。该方法依赖 `pillow`，想要使用该功能，需要提前安装此模块。示例代码如下：

```
1 from nicegui import ui
2
3 @ui.page('/')
4 def page():
5     ui.clipboard.read_image()
```

```
4  async def index():
5      ui.button('Write', on_click=lambda: ui.clipboard.write('Hi!'))
6
7      async def read() -> None:
8          img = await ui.clipboard.read_image()
9          if img:
10             with ui.dialog() as dialog:
11                 with ui.column().classes('w-72 items-center'):
12                     ui.image(img)
13                     ui.button('关闭', on_click=dialog.close)
14             dialog.open()
15         else:
16             ui.notify(await ui.clipboard.read())
17     ui.button('Read', on_click=read)
18
19 ui.run(native=True)
```

如果当前剪贴板保存的是图片，点击"Read"按钮会弹出对话框，显示出该图片。



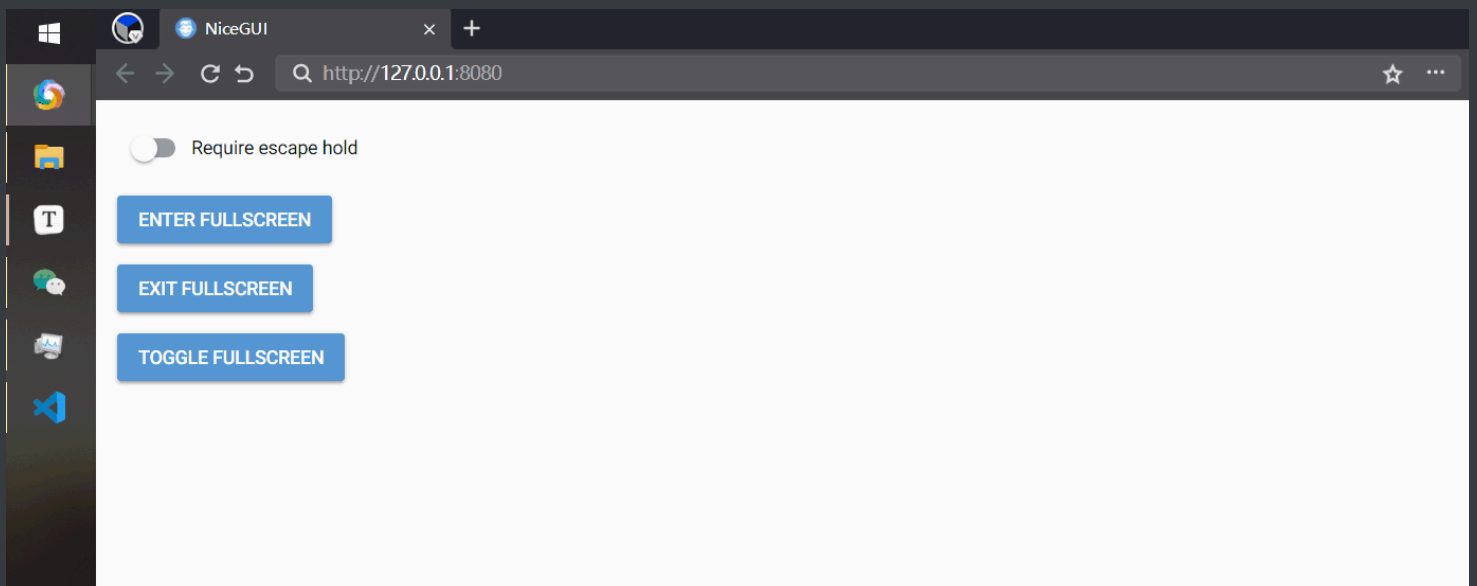
2.11.0版本新增： `ui.fullscreen` ——让浏览器全屏更简单

如果想让浏览器全屏，除了使用快捷键 `f11`，用得最多的就是使用JavaScript代码。不管是哪种方法，都不如通过NiceGUI的2.11.0版本新增的 `ui.fullscreen` 进入全屏方便。先看代码：

```

1 from nicegui import ui
2
3 fullscreen = ui.fullscreen()
4
5 ui.switch('Require escape hold').bind_value_to(fullscreen, 'require_escape_hold')
6
7 ui.button('Enter Fullscreen', on_click=fullscreen.enter)
8 ui.button('Exit Fullscreen', on_click=fullscreen.exit)
9 ui.button('Toggle Fullscreen', on_click=fullscreen.toggle)
10
11 ui.run()

```



`ui.fullscreen` 支持两个参数：

- `require_escape_hold` 参数，布尔类型，表示退出全屏时是否需要长按 `esc` 键，默认为 `False`，即短按 `esc` 键即可退出。注意，只有谷歌的Chrome浏览器和其他基于chromium开发的浏览器（比如微软的Edge浏览器和大部分国产浏览器）支持长按退出全屏的操作。另外，此参数实际上对应的是 `ui.fullscreen` 的 `require_escape_hold` 属性，如果想要动态启用、禁用长按退出全屏的功能，可以在非全屏状态下修改此属性。但是，在启用时，此功能是即时生效的；如果在启用之后进入了一次全屏，禁用之后则需要刷新页面才能生效。
- `on_value_change` 参数，可调用类型，表示当 `ui.fullscreen` 的 `value` 属性变化时，执行什么操作。`ui.fullscreen` 的 `value` 属性的值实际上就是当前是否为全屏状态的布尔值，不过并不能通过直接设置此值来切换全屏状态。因为浏览器的安全设置，想要通过此控件切换全屏状态，必须要用户手动触发，即用户点击了控件之后，此控件将此值设置为其他布尔值，才能切

换全屏状态。如果想要默认进入全屏状态，需要设置运行参数 `fullscreen`，比如 `ui.run(fullscreen=True)`。

此外，需要额外注意的是，此控件只能用于浏览器，后面提到的 `native mode` 不能通过此控件切换全屏。

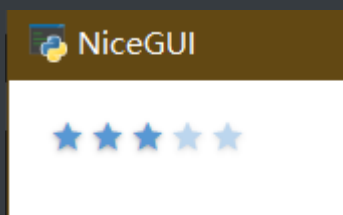
`ui.fullscreen` 支持以下方法，可以让用户通过与控件交互之后的触发动作调用这些方法，来实现对全屏状态的控制：

- `enter` 方法，进入全屏状态。
- `exit` 方法，退出全屏状态。
- `toggle` 方法，切换全屏状态。

2.12.0版本新增： `ui.rating` 和 `ui.slide_item` ——打分和滑动“解锁”

`ui.rating` 是一个打分的交互组件，实现源于Quasar的`QRating`。先看示例：

```
1 from nicegui import ui
2
3 ui.rating(value=4)
4
5 ui.run(native=True)
```



可以点击对应的星级，来确定最终的评分。

`ui.rating` 支持以下参数：

`value` 参数，浮点类型，表示初始的分值。

`max` 参数，整数类型，表示最多的分值，同时也是显示在组件中评分图标个数，默认为 5。

`icon` 参数，字符串类型，表示没有选定的评分图标，默认为 `'star'`。

`icon_selected` 参数，字符串类型，表示选定之后的评分图标，默认同 `icon` 参数。

`icon_half` 参数，字符串类型，表示半分的评分图标，默认同 `icon` 参数。

`color` 参数，字符串类型或者 `None`，表示评分图标的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让按钮变成默认颜色），默认为 `'primary'`，即和主题颜色一致。

`size` 参数，表示评分图标的大小，字符串类型，采用CSS语法的大小表示方式。

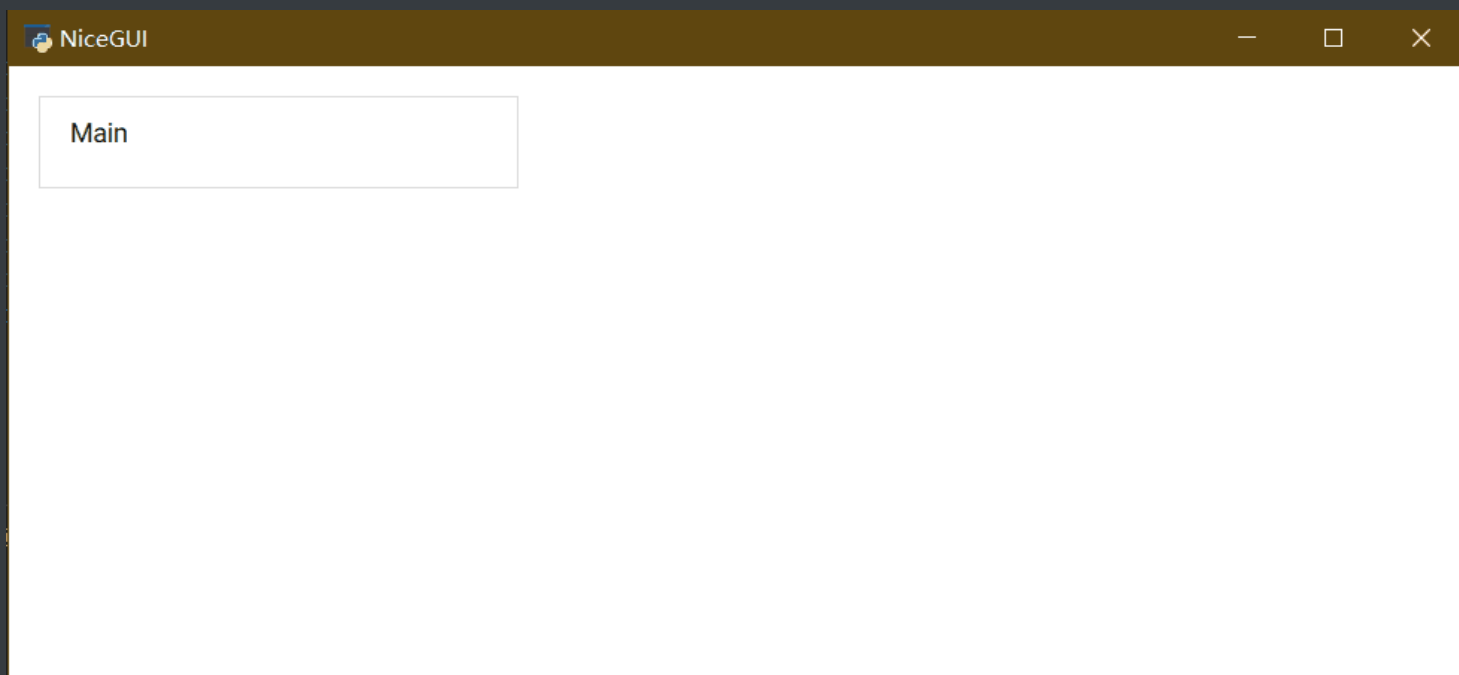
`on_change` 参数，可调用类型，表示评分改变时执行的操作，可以使用lambda表达式，也可以使用函数名。

`ui.rating` 的用法很简单，也没有需要特别注意的地方，需要重点学习的是 `ui.slide_item`。

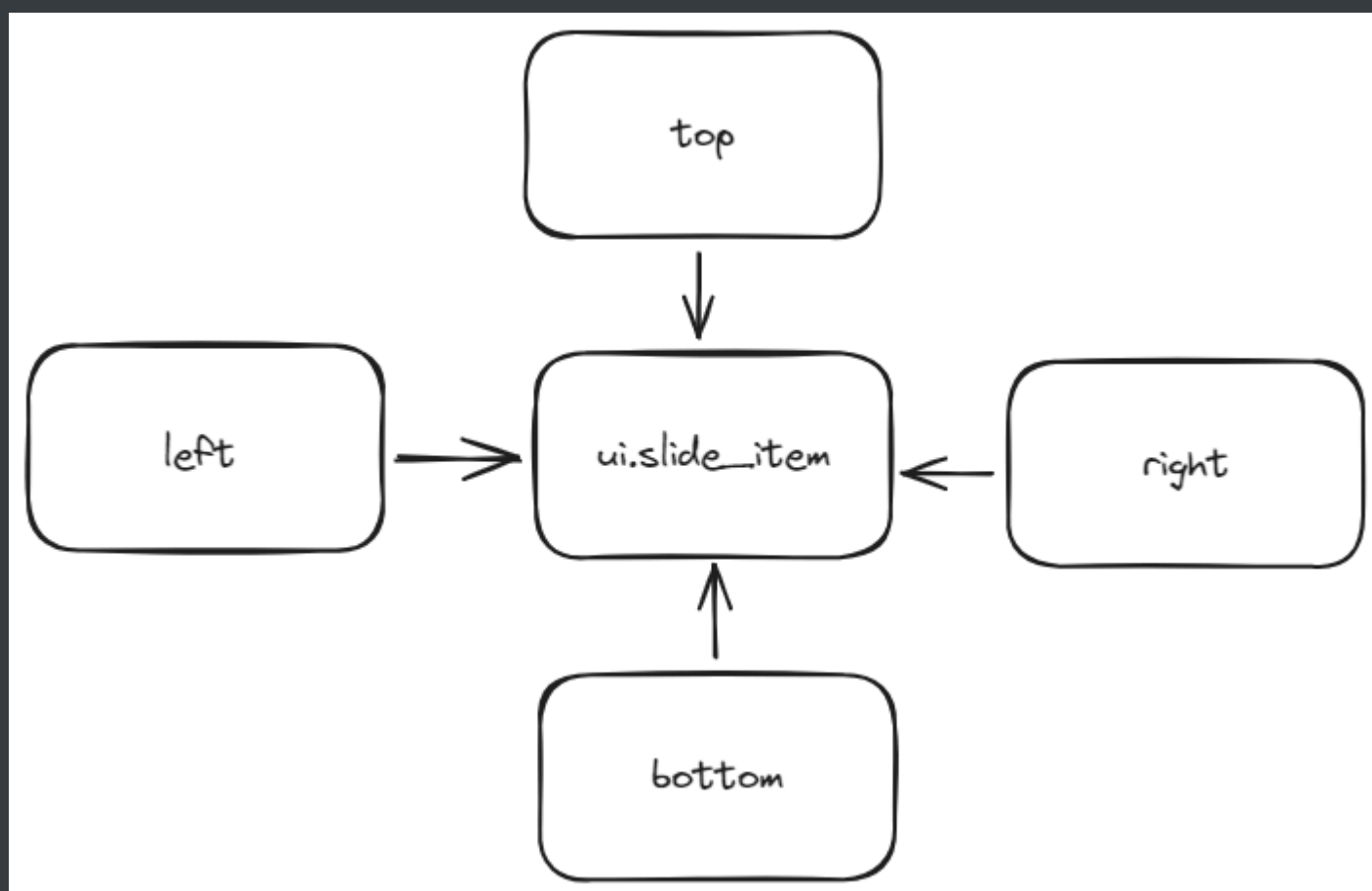
`ui.slide_item` 实现了类似滑动解锁一样操作，可以在空间有限的情况下，四向滑动，进入不同的子控件中。实现源自于Quasar的[QSlideItem](#)。

依照惯例，先看一下示例：

```
1  from nicegui import ui
2
3  with ui.list().props('bordered'), ui.slide_item(
4      text='Main',
5      on_slide=lambda e: ui.notify(f'Slide: {e.side}')
6  ).classes('w-64') as slide_item:
7      with slide_item.left(text='left'):
8          ui.button(text='reset', on_click=slide_item.reset)
9      with slide_item.right():
10         ui.button(text='reset', on_click=slide_item.reset)
11         with slide_item.top(text='top', on_slide=slide_item.reset, color='red'):
12             ...
13         with slide_item.action(side='bottom', text='bottom', on_slide=slide_item.reset,
14             color='red'):
15             ...
16  ui.run(native=True)
```

看完了示例，接下来开始介绍 `ui.slide_item` 的交互原理。实际上，`ui.slide_item` 可以看作是类似轮播图的万向非自动版。当用户滑动时，控件显示的内容会切换为来向的子控件，如下图所示，箭头表示滑动方向，用户滑动之后，对应的子控件会根据滑动方向而进入主显示区：



同样的，每个子控件也是一个容器，除了传入参数直接显示文字之外，还可以继续添加其他控件到内部。

了解了交互原理之后，下面正式介绍一下参数和方法。

`ui.slide_item` 支持以下参数：

`text` 参数，字符串类型，表示显示在控件内的文字。在实际使用时，可以不使用此参数，直接进入 default slot，自定义控件内的内容。

`on_slide` 参数，可调用类型，当滑动切换到对应的子控件时执行的操作。该操作接收一个消息参数，消息参数的字符串类型属性 `slide` 表示滑动的方向。

相比于参数的简单，`ui.slide_item` 支持的方法就复杂了一点。

`left` 方法、`right` 方法、`top` 方法、`bottom` 方法本质上是一样的，就是对应左、右、上、下四个方向上的子控件，因此参数也是一样：

`text` 参数，同 `ui.slide_item` 的 `text` 参数。

`on_slide` 参数，同 `ui.slide_item` 的 `on_slide` 参数。

`color` 参数，字符串类型或者 `None`，表示子控件的颜色，支持传入字符串类型的颜色类（Quasar、Tailwind、CSS的颜色名）或者 `None`（即让按钮变成默认颜色），默认为 `'primary'`，即和主题颜色一致。

`left` 方法、`right` 方法、`top` 方法、`bottom` 方法其实是对 `action` 方法的包装，返回的是 `action` 方法的执行结果。`action` 方法除了支持这些方法的参数之外，还支持字符串类型参数 `slide`，表示子控件对应的方向，该参数仅支持 `['left','right','top','bottom']` 中的值，参数位置在 `text` 参数前。

此外，需要注意的是，切换到对应子控件之后，需要调用 `reset` 方法才能从子控件返回：

```
1 from nicegui import ui
2
3 with ui.list().props('bordered'):
4     with ui.slide_item() as slide_item:
5         ui.item('Slide me')
6         with slide_item.left(color='blue'):
7             ui.item('Left')
8         with slide_item.right(color='purple'):
9             ui.item('Right')
10
11 ui.button('Reset', on_click=slide_item.reset)
12
13 ui.run(native=True)
```